

OXYPC INVENTORY LIFECYCLE

COMPLETE SET

Product & Architecture Handbook

All product, process and architecture documents for the OxyPC Inventory Lifecycle app, in one file.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Contents

#	Document	Page
01	**Key Findings** Security, data-safety and process gaps found while documenting the app, with what to fix first.	3
02	**Product Overview** Why the app exists, where it is going, and how success is measured.	6
03	**Personas & Journeys** The 12 roles that handle a device, their goals and frustrations, and their journeys through the app.	27
04	**Business Process & Rules** How a device moves through every stage, and the rules each team follows.	42
05	**Access & Security** Who can see and do what, and how the app protects its data.	74
06	**Technical Architecture** System design, data model, integrations, and the decisions behind them.	103
07	**Operations & Testing** Deploying, running and recovering the app, its quality targets, and how releases are tested and signed off.	164
08	**Product Requirements (PRDs)** Requirements for the top improvement opportunities, plus the template for new ones.	199



OXYPC INVENTORY LIFECYCLE

OVERVIEW

Key Findings

Security, data-safety and process gaps found while documenting the app, with what to fix first.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Key Findings

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

Writing these documents meant reading the code end to end. Along the way the review found gaps between what the business expects and what the code does today. They are listed here so they can be triaged; each links to the document with the detail and the file references.

Fix first: security

#	Finding	Why it matters	Detail
S1	Uploaded files are served at `/uploads` with no login check (`main.py` mounts `StaticFiles`)	KYC documents, payment proofs, GRN invoices and daily stock files can be downloaded by anyone on the network who knows or guesses a path	Security, Integrations
S2	Page permissions default to "allowed" for custom roles	A new custom role (including the auto-created `sub_admin`) can open every page that is not admin-only until someone restricts it	Security
S3	Default secrets ship in code and seed scripts	DB password, OxyQC agent key, admin and UAT passwords are known defaults unless changed on the server	Security
S4	Permission changes and admin password resets are not written to the audit log	No record of who widened someone's access	Security
S5	CSV exports include prices even when Pricing Visibility is off	Pricing leaks through exports	Permission Matrix

Fix first: data safety

#	Finding	Why it matters	Detail
D1	App, database and backups all sit on one server; `uploads/` is not backed up	One disk or server failure loses data and files	Runbook & DR
D2	Database dumps are made without `--clean` , but the restore script expects to replace tables; no test restore is on record	A restore may fail when it is needed	Runbook & DR
D3	No recovery targets (RPO / RTO) are set	Nobody has agreed how much data loss or downtime is acceptable	NFRs

Process rules the code does not enforce

#	Finding	Detail
P1	A tag can be sold from any stage, not only Ready for Sale. This is deliberate (director's decision, 19 Oct 2026, `services/control_engine.py::validate_sale_allowed`); only blocked tags are refused. Confirm it is still wanted, as the Sales role is described as selling Ready for Sale stock	State Machine
P2	Several stage moves accept any signed-in user and do not check the tag's current stage (L1/L2 Mark Complete, Stress Test fail/complete, Back to Inventory)	State Machine

#	Finding	Detail
P3	The three lists of stages and allowed moves in the code disagree with each other	State Machine, Data Model
P4	Grades differ by screen: Final QC offers A/B/C, the device model A–E plus scrap, CRM A/B/C/D/Mix/As-Is/Scrap	PRD-03
P5	The `returned` stage is never used; a return moves the tag straight to Stock In	State Machine

Roles that do not exist in the code yet

Production Manager, Final QC Inspector, Accounts and TRC Manager have no built-in role. Production work runs under `inventory_manager`, and the Accounts pages refuse a custom Accounts role because `routers/accounts.py` checks a fixed list (`FINANCE_ROLES`). Fifteen templates still check role names directly instead of using `role_allowed()`. See Permission Matrix.

Features expected but not built

#	Expected	Today	Detail
F1	E-way bill generation	Only e-way bill numbers/files for goods coming in are stored	PRD-04, Integrations
F2	GST invoice generation	A printed HTML tax invoice, one tag per invoice, HSN fixed at 8471, GST in floating point, free-text invoice number	PRD-04
F3	Rentals	No code found	Roadmap
F4	Staging server	Auto-deploy pushes `main` straight to production	Deployment
F5	Automated test run	172 test files, no CI job runs them	NFRs

Housekeeping

- `docs/schema.dbml` (29 Apr 2026) is missing 58 of the 147 tables in `models/`; the approved diagram was never saved back. Data Model
- WhatsApp was removed, but `services/care_service.py` still calls it, so Care offer sends fail. Integrations
- `.github/workflows/deploy.yml` still restarts the pre-Docker service. Deployment
- Dashboard: "Revenue This Month" ignores the overrides "Profit This Month" uses; "Active lots" counts trashed lots; the P&L date filter changes nothing on screen. KPI Definitions
- The `CLAUDE.md` note on `is_trashed` is out of date: the columns are now NOT NULL. ADRs



OXYPC INVENTORY LIFECYCLE

PRODUCT

Product Overview

Why the app exists, where it is going, and how success is measured.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Product Overview

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

In this document: Part A — Product Vision · Part B — Roadmap · Part C — KPI Definitions

PART A

Product Vision

Scope of this document: why the app exists, who it serves, what "good" looks like, and what is in and out of scope. Every statement about current behaviour is taken from the code on branch `staging-docs`. Anything not in the code yet is marked **Proposed**.

---

1. The problem

OxyPC Computers / Bluemonk buys used laptops, desktops and TFT monitors in lots, refurbishes them and sells them. One device passes through up to ten teams before it is sold:

```
Buy lot → GRN → IQC → Stock In → (Production) → L1/L2 or L3/L4 Repair → Stress Test (QC)
       → Cosmetic (Cleaning, Putty, Dry Sanding, Masking, Painting, Water Sanding)
       → Final QC (grade) → Ready to Sale → Sold → (Return / Credit Note / Replacement)
```

The stages are real code, not a diagram: `models/device.py::DeviceStage` has 24 stages, and every move is checked against the `AllowedTransition` table by `services/control_engine.py::validate_transition`.

What still hurts, as seen in the code and the agreed opportunities:

#	Pain	Evidence in code today
P1	Engineers cannot see where their part request stands, and "Part Not Available" (PNA) is tracked apart from the request.	`models/part_request.py::PartRequest.status` and `models/pna_part.py::DevicePNAPart` are separate tables with no link. Status is shown only on Device Detail (`templates/devices/detail.html`). No notification on handover (`routers/part_requests.py`).
P2	Fail reasons do not travel with the tag. Each fail overwrites the last one.	`Device.stress_notes` is cleared on Complete; `Device.repair_notes` is overwritten by a Cosmetic fail; `Device.fqc_failure_reason` keeps one value (`models/device.py` , `routers/cosmetic.py::cosmetic_fail_assign`). History lives only as free text in `StageMovement.notes` .
P3	The cosmetic chain moves one tag at a time.	`routers/cosmetic.py::advance_stage` takes one barcode. `bulk_assign` only reassigns (it advances only Cosmetic Received → Cleaning).
P4	Grade does not drive the price.	Final QC sets `Device.grade` (A/B/C in `templates/cosmetic/final_qc.html`). Ready to Sale min price = unit cost + other rate + margin (`services/ready_pricing.py::tag_min_price`) — grade is not an input. The CRM `GradePriceMatrix` (`models/crm.py`) is not used outside `routers/crm_price_matrix.py` .

#	Pain	Evidence in code today
P5	Harvested parts are not used first.	Part Master keeps New and Harvest rows apart (`routers/spare_parts.py::parts_list` , `SparePart.source`). Handover uses whatever `part_id` the request names (`routers/part_requests.py::handover_part`).
P6	Invoice and e-way bill are not produced from the sale.	Invoice is a per-device HTML print (`routers/invoices.py::print_invoice`), invoice number is typed or parsed from an uploaded PDF (`routers/sales.py::parse_invoice_pdf`). E-way bill exists only on inbound lots / GRN / Parts GRN (`models/lot.py::e_way_bill` , `models/parts_grn.py::eway_bill_*`), never on a sale.
P7	No single TRC view.	The TRC Manager has to open Production Manager (`/trc-production`), Part Master (`/spare-parts`), WorkID Status (`/workid-status`), Daily Stock (`/reports/daily-stock`) and Physical Audits (`/audits`) separately.

2. Vision statement

Every device, every hour: one tag, one truth. Anyone at OxyPC can scan a tag and see where it is, why it is there, what it is waiting for, what it cost and what it should sell for — and the next person in the chain is told without being asked.

3. Target users — the 12 personas

App role keys come from `models/user.py::UserRole` and `ROLE_LABELS` . Roles are free text, so custom roles (for example `trc_manager`) are allowed and governed by the Module Permission matrix (`auth/dependencies.py::require_roles` , `models/role_permissions.py`).

#	Persona	App role (today)	Main job in the app	Main pages
1	IQC Inspector	`iqc_inspector` ("IQC Handler")	Inspect incoming devices, record specs and parts required	`/iqc` , OxyQC agent
2	Inventory Manager	`inventory_manager`	Lots, GRN, Stock In, stage moves, flags, credit-note table	`/lots` , `/grn` , `/devices` , `/transfers`
3	Production Manager	custom role / matrix (`production_manager` module)	Allocate tags to engineers, buckets, PNA summary	`/trc-production`
4	Repair Engineer L1/L2, L3/L4	`l1_engineer` , `l2_engineer` (legacy), `l3_engineer`	Repair, raise part requests, mark PNA, hand to Stress	`/repair/l1` , `/repair/l3l4` , Device Detail
5	QC Inspector (Stress Test, OS install, diagnose)	`qc_inspector` ("QC Handler")	Stress test, pass to Cosmetic or fail to L1/L2	`/stress` , `/qc`
6	Cosmetic Lead	`cosmetic_manager`	Run Cleaning → Putty → Dry Sanding → Masking → Painting → Water Sanding	`/cosmetic/*`
7	Final QC Inspector	permission `cosmetic_finalqc`	Manual diagnose, grade, pass to Ready or fail back by reason	`/cosmetic/final_qc`
8	Spare Parts Manager	`spare_parts_manager` ("Parts Manager")	Hand over, Not In Stock, Procure, Parts GRN (As New), Harvest	`/spare-parts` , `/parts-grn`

#	Persona	App role (today)	Main job in the app	Main pages
9	Sales Executive	`sales`, `sales_manager`, `telecaller`	Ready for Sale, block tags, sell, challan, gate pass, returns	`/sales/ready`, `/block-tags`, `/sales`, `/gate-pass`, `/returns`
10	Accounts	custom role / matrix	Payments, invoice, sales list, daily stock	`/accounts/*`, `/sales`, `/reports/daily-stock`
11	Managers / Admin	`admin`, `sales_manager`	Dashboard, reports, P&L, user tickets	`/dashboard`, `/reports/*`, `/tickets`
12	TRC Manager	custom `trc_manager`	Oversee inventory, parts, production; warranty extension; daily stock; physical audits	`/trc-production`, `/spare-parts`, `/extended-warranty`, `/audits`

Note: Rentals are listed as a Sales Executive task, but no rental feature was found in `routers/`, `models/` or `templates/`. See Open questions.

4. Value proposition

For	Who need to	The app gives them	Unlike
Shop-floor teams (IQC, Repair, QC, Cosmetic, Final QC)	Know what to work on next and why	A role queue per stage with WorkIDs, tag history and (proposed) the full fail and part story on the tag	Paper job cards and WhatsApp messages
Spare Parts	Fill requests fast and use stock we already own	Request → Handover → Verify flow, Parts GRN, Harvest stock; (proposed) PNA status and harvest-first	Separate spreadsheets for new and harvested parts
Sales and Accounts	Sell at the right price and bill correctly	Ready-to-Sale pricing, block tags, sale with GST invoice, challan, gate pass, credit notes; (proposed) grade price and e-way bill from the sale	Re-keying sale data into a billing tool
Managers, TRC Manager	See flow, cost and profit	Overall Dashboard, Business P&L, Lot P&L, Daily Stock, Stock Aging; (proposed) one TRC view	End-of-day calls to each team lead

5. Goals and success measures

Targets are measurable. Where the app does not yet store or show the figure, the baseline is "to be measured". Targets marked *(proposed)* need sign-off from the owner named in 05_KPI_Definitions.md.

Goal	Success measure	Baseline	Target	By
G1 Faster flow	Median days from Stock In to Ready to Sale (from `stage_movements`)	baseline: to be measured	−20% vs baseline *(proposed)*	Q2 2027
G1 Faster flow	Tags older than 30 days in any one stage (stage ageing)	baseline: to be measured	−50% vs baseline *(proposed)*	Q2 2027
G2 Parts never block silently	Median PNA wait time (mark → clear, `device_pna_parts`)	baseline: to be measured	≤ 5 working days *(proposed)*	Q1 2027
G2 Parts never block silently	Parts fill rate: requests handed over within 1 working day	baseline: to be measured	≥ 85% *(proposed)*	Q1 2027

Goal	Success measure	Baseline	Target	By
G2 Parts never block silently	Share of handed-over parts that came from Harvest stock	baseline: to be measured	+25% vs baseline *(proposed)*	Q2 2027
G3 Right first time	First-pass yield at Stress Test and at Final QC	baseline: to be measured	≥ 85% at Final QC *(proposed)*	Q2 2027
G3 Right first time	Tags failed at Final QC with a fail reason that is still visible at the next stage	0% for Stress notes after Complete (cleared in code)	100%	Q4 2026
G4 Price by grade	Ready-to-Sale tags with a grade-based suggested price	0% (grade not used in `services/ready_pricing.py`)	100% of graded tags	Q1 2027
G5 Clean billing	Sales with an invoice number recorded	baseline: to be measured (`sales.invoice_no` null rate)	100%	Q1 2027
G5 Clean billing	Sales above the e-way bill threshold with an e-way bill number	0% (no field on `sales`)	100%	Q2 2027
G6 Trust the numbers	Tags at `sold` with no Sale row ("orphans", `routers/reports.py::_sold_orphan_rows`)	baseline: to be measured	0	Q4 2026
G6 Trust the numbers	Write operations covered by the audit log	~35% (`docs/audits/2026-04-29-claude-md-full-audit.md`, Section 5)	≥ 90%	Q2 2027
G7 Fast pages	Dashboard loads over 1 s (logged by `routers/dashboard.py::dashboard`)	baseline: to be measured from server log	< 5% of loads	Q4 2026
G8 Adoption	Personas with signed UAT for their journey (06)	0 of 12 for this release	12 of 12	Each release

6. Scope

In scope (this vision, next three quarters)

Area	In scope
Device lifecycle	GRN → IQC → Repair → Stress → Cosmetic → Final QC → Ready → Sold → Returns
Parts	Part requests, PNA, Parts GRN (As New), Harvest, warranty part cycle
Sales	Ready to Sale pricing, block tags, sale, invoice, delivery challan, gate pass, returns, credit notes, warranty extension
Management	Overall Dashboard, Business P&L, Lot P&L, Daily Stock, Stock Aging, Receivables, Overdue, Physical Audits, Tickets
Proposals	The seven agreed opportunities (PRD-01 to PRD-04 plus batch cosmetic moves, harvest-first, one TRC view)

Out of scope (for now)

Area	Why
CRM sourcing / telecalling / dealer calling redesign	Separate product line in the same app; not part of the agreed opportunities
Reliance FieldOps (`/fieldops`)	Standalone offline app; touches no OxyPC tables (`README.md`)
Trade Partner portal and external partner bidding	Not part of the refurbishment flow being improved
Full accounting ledger (GL, TDS, bank reconciliation)	Accounts uses an external tool; the app records payments and invoices only
Multi-tenant SaaS	Mentioned in BRD v6 objectives; not in this horizon
Rentals	Not found in code; needs discovery first (see Open questions)

7. Guiding principles

1. **The tag is the unit of truth.** Every fact about a device hangs off its tag and stays in its history. Never overwrite a fact that explains a delay.
2. **Server decides.** Stage moves go through `validate_transition`; prices and totals are computed server-side in `Decimal` (project skill `server-side-financial-snapshot`).
3. **Tell the next person.** When work changes hands, the receiver gets a queue entry and a notification (`services/notifications.py`), not a phone call.
4. **One number, one source.** A tile and its drill-down use the same query (the pattern in `routers/stock.py::_prod_tile_criteria` and `services/business_pl.py`).
5. **Permissions from the matrix.** Gate buttons with `role_allowed()` / the Module Permission matrix, never hard-coded role lists (project skill `custom-role-permissions`).
6. **Same look everywhere.** New screens use the Design System: summary tiles → filter bar → alert → tabs → Global Table card; success colour `#0F7A37` (`qa_docs/design_system/design_system.html`).
7. **Audit before change.** Every write that matters is in `audit_logs` (`services/audit_engine.py::audit`); no schema change without dbdiagram approval (`CLAUDE.md`).

8. Risks

#	Risk	Likelihood	Impact	Mitigation
R1	Schema changes needed by PRDs wait on dbdiagram approval	Medium	Delays Now items	Batch the schema asks into one review per quarter; keep changes additive (`db_validator.py` auto-provisions additive columns)
R2	No dedicated staging server exists in code; autodeploy goes straight from `origin/main` to production (`deploy/oxypc-autodeploy.sh`)	High	Untested changes reach users	Stand up staging before the first UAT cycle (06); deploy only after sign-off
R3	Dashboard numbers drift from report numbers	Medium	Managers lose trust	Shared computations (`services/business_pl.py`); KPI owners sign 05
R4	Grade-based price makes Sales feel second-guessed	Medium	Low adoption of PRD-03	Suggest, do not block; keep manual override with reason

#	Risk	Likelihood	Impact	Mitigation
R5	E-way bill / e-invoice rules change or need a GSP account	Medium	PRD-04 rework	Phase 1 captures numbers only; API integration only after Accounts confirms provider
R6	Test coverage is thin for business rules (audit Apr-29: testing 3.0/10)	High	Regressions	Regression checklist in 06; grow `tests/` (173 files today) per PRD
R7	Free-text roles mean a new custom role silently lacks a module	Medium	Blocked users at go-live	UAT entry criterion: permission matrix reviewed per persona
R8	Audit trail covers ~35% of writes (Apr-29 audit)	High	Disputes cannot be traced	Every PRD adds audit actions for its writes

9. Open questions

#	Question	Owner
Q1	Where are rentals handled today? No code found.	Sales lead
Q2	Is the Production Manager a custom role in production, or does `inventory_manager` do this job?	TRC Manager
Q3	Who is Accounts in the app (custom role name)?	Admin
Q4	Do we need e-invoice (IRN) as well as e-way bill?	Accounts
Q5	Grade scale: Final QC offers A/B/C, `DeviceGrade` has A–E + scrap, CRM uses A/B/C/D/Mix/As-Is/Scrap. Which is the sales grade?	Final QC lead + Sales lead

Sources read

`CLAUDE.md`, `TASKS.md`, `README.md`, `docs/audits/2026-04-28-as-is-audit.md`, `docs/audits/2026-04-29-claude-md-full-audit.md`, `docs/generate_brd_v6.js`, `models/device.py`, `models/user.py`, `models/part_request.py`, `models/pna_part.py`, `models/crm.py`, `models/sales.py`, `routers/part_requests.py`, `routers/cosmetic.py`, `routers/stress_api.py`, `routers/sales.py`, `routers/invoices.py`, `routers/spare_parts.py`, `services/ready_pricing.py`, `deploy/oxypc-autodeploy.sh`, `qa_docs/design_system/design_system.html`.

PART B

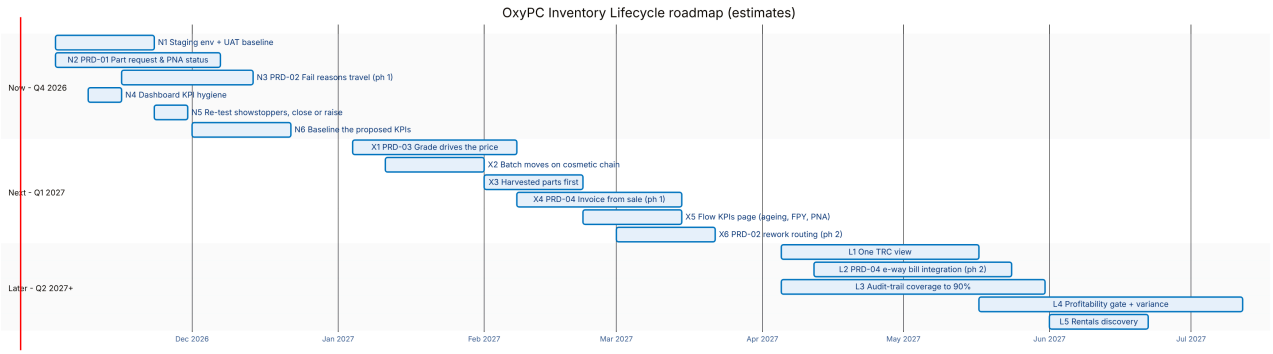
Roadmap

Horizons: **Now = Q4 2026** (Oct–Dec 2026) · **Next = Q1 2027** (Jan–Mar 2027) · **Later = Q2 2027 and beyond**. Built from the seven agreed improvement opportunities, `TASKS.md`, the open findings in `docs/audits/*.md`, the open gaps in `QA_UAT_REPORT.md` and the "Showstoppers" sheet of `docs/OxyPC_UAT_TestPlan.xlsx`. Effort is an **estimate**: S = up to 1 week, M = 2–4 weeks, L = 5+ weeks of one developer, including tests and UAT fixes.

1. Inputs and what they told us

Input	What it says today
Agreed opportunities	Part-request & PNA status · Fail reasons travel with the tag · Batch moves on the cosmetic chain · Grade drives the price · Harvested parts first · Invoice & e-way bill from the sale · One TRC view
`TASKS.md`	Every listed backlog item is ticked `[x]`. No open feature tasks remain in that file. "Current Sprint Focus" in `CLAUDE.md` says: next move to device/sale page enhancements.
QA defect log (`seed_qa_*.py`, QA Dashboard `/qa/defects`)	All seeded defects (BUG-001…BUG-213) are `Closed`. Live open defects must be read from the QA Dashboard on the target server before each planning cycle.
`QA_UAT_REPORT.md` (28 Mar 2026)	Still marked Open: G-08 WhatsApp "Not Connected" state, G-09 cosmetic routing check, G-11 dealer follow-up data, G-12 pagination on Telecalling and Market.
`docs/OxyPC_UAT_TestPlan.xlsx` (27 Apr 2026, Showstoppers)	P0 CSRF 403 expected / 307 seen; P1 no HTTPS; P1 backup automation unconfirmed; P2 default admin password; four route status mismatches (307 vs 302). Several look like test-harness redirects; they must be re-run on staging, not assumed fixed.
Audits (Apr 2026)	Open themes: audit trail ~35% of writes; no profitability gate / variance tracking; `expected_sale_value` never set (auto-scrap never fires); testing 3.0/10.
Code review for this document	Dashboard P&L From/To filter feeds only values the page no longer shows; `work_queue_devices` and `today_followups` are computed but not rendered; "active lots" counts all lots (`routers/dashboard.py::dashboard`).

2. Roadmap at a glance



3. Now — Q4 2026

Theme: **make blockers visible and keep the story on the tag.**

ID	Item	Outcome	Personas	Dependency	Effort (estimate)
N1	Staging environment + UAT baseline	A staging server that mirrors production, seeded with anonymised data, used for every UAT in 06. Today `deploy/oxypc-autodeploy.sh` deploys `origin/main` straight to production.	All; Admin	Server / VM; copy of prod DB with PII masked	S
N2	**PRD-01 Part Request & PNA Status**	Engineers see each request's status (Requested, Handed Over, Not In Stock, Sent for Sourcing, PNA) on their queue and get notified on handover. PNA links to the request.	Repair Engineer, Spare Parts Manager, Production Manager, TRC Manager	dbdiagram approval for `part_requests` ↔ `device_pna_parts` link	M
N3	**PRD-02 Fail Reasons phase 1** — keep every fail on the tag	A fail-history panel on Device Detail and on the receiving queue; stop clearing / overwriting `stress_notes`, `repair_notes`, `fqc_failure_reason` without a history row.	QC Inspector, Cosmetic Lead, Final QC Inspector, Repair Engineer	New `device_fail_events` table (dbdiagram approval)	M
N4	Dashboard KPI hygiene	Remove or wire the P&L From/To filter (it changes nothing visible today); fix "active lots" label (counts all lots); drop unused queries (`work_queue_devices` , `today_followups`); make "Revenue This Month" use the same override-aware source as "Profit This Month". See 05 §6.	Managers/Admin, Sales, Inventory Manager	KPI owners agree definitions	S
N5	Re-test 27-Apr showstoppers on staging	Each Showstopper row in `docs/OxyPC_UAT_TestPlan.xlsx` is either closed with evidence or raised as a defect in the QA Dashboard.	Admin, App Development Team	N1	S
N6	Baseline the proposed KPIs	Measured baselines for stage ageing, first-pass yield, PNA wait, parts fill rate, grade mix, sales cycle time (SQL in 05 §5), so Next-quarter targets are real.	Managers/Admin, TRC Manager	Read access to prod DB copy	S

4. Next — Q1 2027

Theme: **move faster, price smarter, bill from the sale.**

ID	Item	Outcome	Personas	Dependency	Effort (estimate)
X1	**PRD-03 Grade drives the price**	Final QC grade + device type + model give a suggested min/max selling price on Ready to Sale; Sales can override with a reason.	Final QC Inspector, Sales Executive, Managers/Admin	Single agreed grade scale (Vision Q5); price table approval	M (estimate)

ID	Item	Outcome	Personas	Dependency	Effort (estimate)
X2	Batch moves on the cosmetic chain	Cosmetic Lead selects many tags and advances them one stage in one action (today `routers/cosmetic.py::advance_stage` is one tag; `bulk_assign` only reassigns). Per-tag result alert.	Cosmetic Lead	None (no schema change expected)	M
X3	Harvested parts first	When a request is handed over, Part Master suggests matching Harvest stock before New stock; report the harvest share.	Spare Parts Manager, Repair Engineer	PRD-01 (request status)	M
X4	**PRD-04 phase 1** — invoice from the sale	One invoice per sale event (all tags of a multi-tag sale on one document), server-generated invoice number series, e-way bill number and date captured on the sale.	Sales Executive, Accounts	dbdiagram approval (`sale_invoices`); Accounts confirms numbering	M
X5	Flow KPIs page	Stage ageing, first-pass yield per QC stage, PNA wait time, parts fill rate, grade mix, sales cycle time on one management page using the Global Table.	Managers/Admin, TRC Manager, Production Manager	N6 baselines; N3 for clean fail data	M
X6	**PRD-02 phase 2** — rework routing	Stress and Cosmetic fails choose a reason category (as Final QC does) and route by rule; returned tags show "Rework #n".	QC Inspector, Cosmetic Lead, Final QC Inspector, Production Manager	N3	S

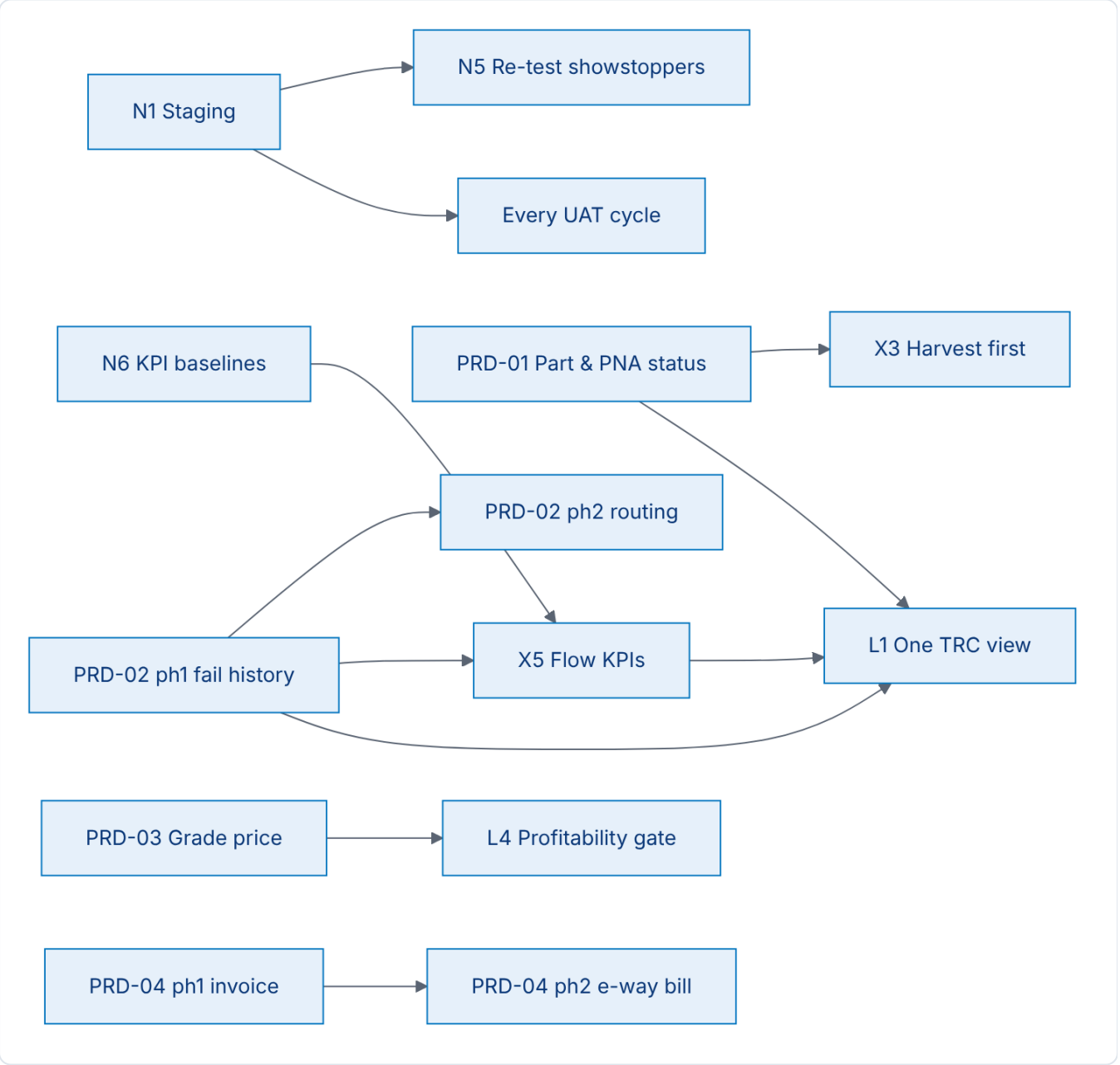
5. Later — Q2 2027 and beyond

Theme: **one view for the TRC and numbers you can defend.**

ID	Item	Outcome	Personas	Dependency	Effort (estimate)
L1	One TRC view	A single TRC Manager page: tags by stage and age, PNA list, open part requests, Final QC fails, today's daily-stock opening/closing, last physical audit result. Reuses existing queries.	TRC Manager, Production Manager	X5, PRD-01, PRD-02	L
L2	**PRD-04 phase 2** — e-way bill (and e-invoice if needed) via a GST provider	E-way bill generated from the sale, number written back, printed on challan and gate pass.	Accounts, Sales Executive	GSP account and credentials; Accounts decision on e-invoice (Vision Q4)	L
L3	Audit-trail coverage to ≥ 90%	IQC registration, user/role changes, spare-part movements, CRM deal edits all audited (gaps listed in `docs/audits/2026-04-29-claude-md-full-audit.md` §5).	Managers/Admin	None	L
L4	Profitability gate + variance tracking	Lot buy decision shows expected margin from grade mix; actual vs estimate per lot after sale (audit "Commercial 3.5/10").	Managers/Admin, Inventory Manager	X1 grade price table	L

ID	Item	Outcome	Personas	Dependency	Effort (estimate)
L5	Rentals discovery	Decide whether rentals belong in this app; if yes, write PRD. No rental code exists today.	Sales Executive, Accounts	Business input	S
L6	Remaining low gaps	G-08, G-11, G-12 from `QA_UAT_REPORT.md` (WhatsApp state, dealer follow-ups data, Telecalling/Market pagination).	Sales	None	S

6. Dependencies map



7. How we decide what moves

Rule	Detail
Blockers first	Items that stop a tag moving (parts, fails) beat items that make it look nicer.
Measure before target	A KPI target is set only after N6 baseline exists.
One schema review per quarter	Collect all schema asks for the quarter into one dbdiagram review (`CLAUDE.md` : no schema change without approval).
Release gate	Nothing ships to production without staging UAT sign-off for the affected personas (06).

8. Open questions

#	Question	Owner
1	Is there a live open-defect list beyond the seeded (closed) ones? Pull from `/qa/defects` on production before sprint planning.	QA lead
2	Who owns the GST provider account for e-way bill (L2)?	Accounts
3	Should batch cosmetic moves (X2) also allow "Skip Cosmetic → Final QC" in bulk? Single-tag skip exists in `advance_stage` .	Cosmetic Lead
4	Is the TRC view (L1) a new page or a tab on `/trc-production` ?	TRC Manager

PART C

KPI Definitions

Every figure below is read from the code on branch `staging-docs`. The formula column cites `file::function`. **Target** is "to be agreed" unless the business has already set one. **Owner** is the persona who answers for the number. Section 5 lists **proposed** KPIs that the app does not show yet.

1. Shared rules (apply to every dashboard figure unless stated)

Rule	Detail	Source
Trashed devices	Excluded: `Device.is_trashed == False`.	`routers/dashboard.py::dashboard`
Test entity	With no Entity filter, the External Partner test entity is excluded (NULL-safe).	`models/master.py::EXTERNAL_PARTNER_TEST_ENTITY`, `routers/dashboard.py::dashboard` (`_ent`)
Filters	Entity (multi), Device Type (multi, Master Data `device_type`), Zone (latest `DeviceLocationLog`, fallback `Device.location_id`).	`routers/dashboard.py::dashboard`
Refresh	Counts are cached for **30 seconds** when no filter is active (`_AGG_TTL = 30`). Filtered views run live. Page has no auto-refresh; figures update on reload.	`routers/dashboard.py` (`_AGG_CACHE`)
Who sees the page	Gated by the Module Permission matrix; every role that can open `/dashboard` sees the same analytics. Weekly Sales Price chart needs `can_view_pricing`.	`routers/dashboard.py`, `templates/dashboard.html`, `templates_config.py::_can_view_pricing`
Stage groups	**Cosmetic** = `cosmetic_received, cleaning, putty, dry_sanding, masking, painting, water_sanding, cosmetic_completed` (`models/device.py::COSMETIC_STAGES`). **Final QC** = `final_qc, final_qc_pass_hold, final_qc_fail_hold`. **L1/L2** = `l1, l2` (l2 legacy). **L3/L4** = `l3`. **Stress/QC** = `qc_check`. **Production** = `trc_production`.	`routers/dashboard.py` (`STEP_STAGES`)
Financial Year	Indian FY, April–March, picked by its start year (2025 = "2025-26").	`services/business_pl.py::fy_label`
Admin overrides	Business P&L month values can be overridden by admin; overrides flow into FY figures.	`routers/reports.py::save_business_pl_override`, `models/business_pl_override.py`

2. Overall Dashboard tiles (`/dashboard`)

Template: `templates/dashboard.html` and `templates/dashboard/_admin_analytics.html`. Computation: `routers/dashboard.py::dashboard` unless stated.

2.1 Device Stage Pipeline card (10 tiles)

Each tile counts devices passing the shared rules plus the stage clause below. Tiles link to the page whose count should match.

#	Tile	Definition	Formula / query logic	Owner	Target	Refresh
D1	IQC	Tags waiting for or in IQC	`count(Device) where current_stage = 'iqc'` (`PIPELINE_STEPS`)	IQC Inspector	to be agreed	30 s cache
D2	Stock In	Tags passed IQC, not yet allocated	`current_stage = 'stock_in'`	Inventory Manager	to be agreed	30 s
D3	Production	Tags with the Production Manager for allocation	`current_stage = 'trc_production'`	Production Manager	to be agreed	30 s
D4	L1/L2	Tags in L1/L2 repair	`current_stage IN ('l1','l2')`	Repair Engineer L1/L2	to be agreed	30 s
D5	L3/L4	Tags in L3/L4 repair	`current_stage = 'l3'` . Note: the linked page `/repair/l3l4` counts open `L3L4-%` WorkOrders and may differ (comment in `dashboard`).	Repair Engineer L3/L4	to be agreed	30 s
D6	Stress/QC	Tags in Stress Test	`current_stage = 'qc_check'`	QC Inspector	to be agreed	30 s
D7	Cosmetic	Tags anywhere on the paint line	`current_stage IN COSMETIC_STAGES`	Cosmetic Lead	to be agreed	30 s
D8	Final QC	Tags in Final QC incl. Pass/Fail Hold	`current_stage IN FINAL_QC_STAGES`	Final QC Inspector	to be agreed	30 s
D9	Ready to Sale	Sellable stock	`current_stage = 'ready_to_sale'`	Sales Executive	to be agreed	30 s
D10	Sold	All-time sold tags	`current_stage = 'sold'`	Sales Executive	n/a (cumulative)	30 s
D11	GRN (computed, not shown as a tile)	Tags mapped to a GRN	`grn_number IS NOT NULL AND grn_number <> '' AND is_active` (`GRN_WHERE`)	Inventory Manager	n/a	30 s
D12	Returned / Scrapped / Scrap for Sale (footer line)	Dead-end stage counts, shown apart from totals	`stage_counts['returned']` , `['scrapped']` , `['scrap_for_sale']` from `GROUP BY current_stage`	Inventory Manager	to be agreed	30 s
D13	Split Entity / Split Zone (toggles)	Same tile counts broken down by `Device.entity` or by Zone of resolved location	One grouped query: stage × entity × resolved location (`_pipe_count_by_entity` , `_zone_label`)	Managers/Admin	n/a	30 s

2.2 Financial Summary card (Financial Year)

All five figures come from the same functions as Business P&L, so the card always equals that report for the chosen FY.

#	Tile	Definition	Formula / query logic	Owner	Target	Refresh
F1	Total Investment	Device cost of tags **sold** in the FY	Σ `Device.device_price` joined to `Sale` by `sold_at` month, Apr–Mar, with Device COGS overrides — `services/business_pl.py::compute_year_device_cogs` via `compute_fy_totals`	Managers/Admin (Accounts)	to be agreed	30 s, keyed by FY
F2	Parts Spent	Parts changed on tags sold in the FY	Σ `PartRequest.qty_handed_over × SparePart.unit_price` where `PartRequest.status = 'received'`, joined to `Sale.sold_at` month; overrides applied — `services/business_pl.py::compute_year_parts_labour_cogs`	Spare Parts Manager	to be agreed	30 s
F3	Labour Spent	Flat labour per worked-on sold tag	`count(distinct Sale.device_id)` for sold tags with any `StageMovement.to_stage IN LABOUR_STAGES` (L1, L2, L3, Stress, all Cosmetic, all Final QC) × CostConfig `repair_labour_rate` (default ₹150); overrides applied — same function	Production Manager	to be agreed	30 s
F4	Total Revenue	Sales value in the FY	Σ `Sale.sale_price` by `sold_at` month, overrides applied — `services/business_pl.py::compute_year_revenue` via `compute_fy_revenue`	Sales Executive / Managers	to be agreed	30 s
F5	Net Profit	Gross profit for the FY	F4 − F1 − F2 − F3 (`fy_net_profit` in `dashboard`)	Managers/Admin	to be agreed	30 s

2.3 Category quick-view tiles (Laptops, Desktops, TFT Monitor, Tablet, Mini PC, Server)

#	Tile part	Definition	Formula / query logic	Owner	Target	Refresh
C1	Big number	Active stock of this category	Σ counts by `Device.sub_category` excluding stages `grn, sold, returned, scrapped, scrap_for_sale` (`CATEGORY_EXCLUDED_STAGES`)	Inventory Manager	to be agreed	30 s
C2	In Ready	Sellable units of this category	`category_counts[cat]['ready_to_sale']` (`laptops_available` etc.)	Sales Executive	to be agreed	30 s
C3	In TRC	Units still being worked on	`max(C1 − C2, 0)` (computed in template)	Production Manager	to be agreed	30 s

2.4 Summary stat cards

#	Tile	Definition	Formula / query logic	Owner	Target	Refresh
S1	Total Inventory	All active stock in the building	Σ `stage_counts` excluding `CATEGORY_EXCLUDED_STAGES` (`total_devices`)	Inventory Manager	to be agreed	30 s
S2	In Repair	Work-in-progress between Stock In and Ready	Pipeline D3 + D4 + D5 + D6 + D7 + D8 (`in_repair_count`)	Production Manager	to be agreed	30 s
S3	Revenue This Month	Sales value since the 1st of this month	Σ `Sale.sale_price` where `date(sold_at) >= first of month` (`_compute_financials`). **No overrides, not net of returns.**	Sales Executive	to be agreed	30 s

#	Tile	Definition	Formula / query logic	Owner	Target	Refresh
S4	Profit This Month	Gross profit this calendar month	Revenue − Device − Parts − Labour for the month, overrides applied — `services/business_pl.py::compute_month_profit`	Managers/Admin	to be agreed	30 s

2.5 Charts

#	Chart	Definition	Formula / query logic	Owner	Target	Refresh
G1	Stage Distribution (pie)	Work-in-progress mix	Pipeline counts for IQC, Stock In, Production, L1/L2, L3/L4, Stress, Cosmetic, Final QC (`admin_charts['stage_pie_*']`)	Production Manager	n/a	30 s
G2	Inventory by Category & Stage (bar)	Stock by category per pipeline step (Sold excluded)	`chart_groups` × `category_counts`	Inventory Manager	n/a	30 s
G3	Weekly Sourcing Price — Buyer PO / Seller PO	Value of dealer orders vs supplier POs per ISO week, last 8 weeks	Σ `DealerOrder.total_amount` by `order_date` ; Σ `CRMPurchaseOrder.total_amount` by `created_at` (`_weekly_series`)	Managers/Admin	n/a	30 s
G4	Weekly Sales Price — Ready to Sale / Sold (pricing-gated)	Tags entering Ready per week (count) vs sold value per week (₹)	count `StageMovement.to_stage = 'ready_to_sale'` ; Σ `Sale.sale_price` by `sold_at`	Sales Executive	n/a	30 s
G5	Weekly: Products in IQC / GRN / In Stock	Inflow per week	count `StageMovement.to_stage IN ('iqc','grn','stock_in')` by `moved_at`	IQC Inspector, Inventory Manager	n/a	30 s
G6	Weekly: Spare Parts — Sourcing Request / GRN / Harvest	Parts demand and inflow per week	count `PartSourcingRequest.created_at` ; count `PartsGRNLineItem` by `created_at` split on `is_harvest`	Spare Parts Manager	n/a	30 s

2.6 Role "My Work Queue" banners (top of page, by role)

#	Banner figure	Role(s)	Formula	Owner	Target	Refresh
Q1	devices awaiting IQC	`iqc_inspector`	`stage_counts['iqc']`	IQC Inspector	to be agreed	30 s
Q2	devices in L1 repair queue	`l1_engineer`	`stage_counts['l1']`	Repair Engineer	to be agreed	30 s
Q3	devices in L2 queue	`l2_engineer`	`stage_counts['l2']` (legacy stage)	Repair Engineer	n/a	30 s
Q4	devices in L3 queue	`l3_engineer`	`stage_counts['l3']`	Repair Engineer L3/L4	to be agreed	30 s
Q5	devices pending QC check	`qc_inspector`	`stage_counts['qc_check']`	QC Inspector	to be agreed	30 s

#	Banner figure	Role(s)	Formula	Owner	Target	Refresh
Q6	ready to sell / sold today / month revenue	`sales`, `sales_manager`, `telecaller`	`stage_counts['ready_to_sale']`; `count(Sale) where date(sold_at)=today`; Σ `sale_price` this month	Sales Executive	to be agreed	live (sold today) / 30 s
Q7	dealer outstanding / overdue orders	sales roles	Σ `DealerOrder.due_amount` where status in pending/confirmed/delivered; count where `due_amount > 0 AND payment_due_date < now`	Accounts	to be agreed	live
Q8	low stock alerts / parts used today / stock value	`spare_parts_manager`	count `SparePart.qty_in_stock <= min_stock_alert`; Σ `SparePartConsumption.qty_used` today; Σ `qty_in_stock × unit_price`	Spare Parts Manager	to be agreed	live
Q9	devices in Stock-In / "active lots"	`inventory_manager`	`stage_counts['stock_in']`; `count(Lot)` — **all lots, including trashed**	Inventory Manager	to be agreed	live

3. Key reports (`/reports/*`, `routers/reports.py`)

Router gate: `_REPORT_ROLES` (inventory_manager, qc_inspector, sales; admin and custom roles via matrix). P&L reports add `_require_financials` (inventory_manager, sales_manager). Receivables add `_require_receivables`.

#	Report / KPI	Definition	Formula / query logic	Owner	Target	Refresh
R1	Lot P&L — Revenue (net)	Sales value per lot net of customer returns	Σ `Sale.sale_price` per lot − Σ `coalesce(Return.refund_amount, Sale.sale_price)` for non-rejected returns — `routers/reports.py::lot_pl_report`	Managers/Admin	to be agreed	on load
R2	Lot P&L — Total Cost / Profit / Margin %	Whole-lot economics	Cost = `Lot.buying_price` + Σ `SparePartConsumption.total_cost` (by lot) + Σ `RepairAttempt.cost`; Profit = R1 − Cost; Margin = Profit / R1 × 100 — `lot_pl_report`	Managers/Admin	Margin ≥ `GradePriceMatrix.min_margin_pct` default 15% *(not enforced; to be agreed)*	on load
R3	Lot P&L — Cost / Profit on Sold Units	Fair profit for a part-sold lot	Sold buying cost (`device_price`, else lot price ÷ devices) + parts + labour on sold tags; Profit = R1 − that — `lot_pl_report`	Managers/Admin	to be agreed	on load
R4	Business P&L — Revenue, Device/Parts/Labour COGS, Gross Profit, Gross Margin %	FY by month	Monthly lists from `services/business_pl.py` (§2.2); GM % = (Revenue − COGS) / Revenue × 100 — `routers/reports.py::business_pl`	Managers/Admin, Accounts	to be agreed	on load
R5	Business P&L — Sales count / Avg Sale Price	Volume and price per FY	`count(Sale) where sold_at in FY`; Revenue ÷ count — `business_pl`	Sales Executive	to be agreed	on load

#	Report / KPI	Definition	Formula / query logic	Owner	Target	Refresh
R6	Business P&L — Inventory value	Cost of stock on hand	Σ `Device.device_price` where stage not in (sold, scrapped) — `business_pl`	Inventory Manager	to be agreed	on load
R7	Business P&L — Lots Sold	Lots fully resolved	count lots with ≥1 active device and none still in process (each active device sold, scrapped or replaced) — `business_pl`	Inventory Manager	to be agreed	on load
R8	Daily Stock — Opening / Closing / Net per stage	Stock at 00:00 of day vs 00:00 next day	Latest `StageMovement.to_stage` per device at the boundary (DISTINCT ON), excluding `sold` and `grn` stages and devices trashed before the boundary — `routers/reports.py::daily_stock_report`, `_stock_as_of`	TRC Manager, Accounts	Closing = Opening + In − Out (reconciles)	on load; files auto-generated daily 9 PM (`services/daily_stock_reports.py`)
R9	Daily Stock — Sold today	Tags in stock at opening that are sold by close	`_sold_in_day`	Accounts	to be agreed	on load
R10	Stock Aging	Units and cost by age bracket and stage	Age = today − `Device.created_at` (**device age, not stage age**); brackets 0–7, 8–30, 31–60, 61–90, 90+; cost = `device_price`; excludes sold/scrapped/returned — `routers/reports.py::stock_aging_report`	Inventory Manager	90+ days bucket: to be agreed	on load
R11	Overdue (stuck tags)	Tags not updated for N days	`Device.updated_at <= now − min_days` (default 3), stage not sold/scrapped — `routers/reports.py::overdue_report`	Production Manager	to be agreed	on load
R12	Receivables ageing	Dealer dues by days overdue	`DealerOrder.due_amount > 0`, status pending/confirmed/delivered, bucketed current / 1–30 / 31–60 / 61–90 / 90+ from `payment_due_date` — `routers/reports.py::receivables_report`	Accounts	90+ bucket: to be agreed	on load
R13	Sales Report	Sales list in range (default 90 days) incl. orphan "sold without Sale" rows	`Sale` joined to `Device`, `Lot`; plus `_sold_orphan_rows` — `routers/reports.py::sales_report`	Sales Executive, Accounts	Orphans = 0	on load
R14	Stage Movement log	Every stage move	`StageMovement` joined to `Device`, newest first — `routers/reports.py::stage_movement_report`	Managers/Admin	n/a	on load

4. Other operational tiles used as KPIs

#	KPI	Definition	Formula / query logic	Owner	Target	Refresh
O1	Production tiles: Tags at You / in L1/L2 / L3/L4 / Stress / Cosmetic / Final QC / Tags Returned	Work by stage for the Production Manager	Shared clauses in `routers/stock.py::_prod_tile_criteria` (active, not trashed); "Tags Returned" = `tag_return_status = 'Return for Repair'` in pipeline stages	Production Manager	to be agreed	on load
O2	PNA tags (Production)	L1/L2 tags with ≥1 active PNA part	`count(distinct Device)` join `DevicePNAPart.is_active` where stage in (l1, l2) — `routers/stock.py::trc_production_list`	Production Manager	to be agreed	on load
O3	Part Master: Below Min / Out of Stock / Stock Value / New vs Harvest	Parts health	`qty_in_stock <= min_stock_alert` ; live qty (stock − consumed − sold) ≤ 0; Σ `unit_price × qty_in_stock` ; count by `SparePart.source` — `routers/spare_parts.py::parts_list`	Spare Parts Manager	to be agreed	on load
O4	Open sourcing requests	Parts sent for procurement	`count(PartSourcingRequest) where status = 'open'` — `parts_list`	Spare Parts Manager	to be agreed	on load
O5	WorkID Status: total PNA	WorkIDs whose tag has active PNA	`routers/workid_status.py` (`total_pna`)	TRC Manager	to be agreed	on load

5. Proposed KPIs (not in the app yet)

All six are **proposed**. Data already exists in the tables named; no new capture is needed except where stated.

#	KPI (proposed)	Definition	Formula / query logic (proposed)	Owner	Target	Refresh
P1	Stage ageing	Days a tag has been in its **current** stage; median and count > 7 / 30 days per stage	For each device, latest `StageMovement` with `to_stage = Device.current_stage` and `exited_at IS NULL` : `now − moved_at` . Fallback `Device.updated_at` where no row. Group by stage.	Production Manager, TRC Manager	baseline: to be measured; then to be agreed	daily (9 PM with Daily Report)
P2	First-pass yield (FPY) per QC stage	Share of tags that pass a QC stage at the first attempt	**Stress:** tags whose first exit from `qc_check` goes forward (to a cosmetic stage) ÷ all tags that exited `qc_check` in period. **Final QC:** tags whose first decision is `final_qc_pass_hold` ÷ tags with any Final QC decision. From ordered `StageMovement` rows per device.	QC Inspector, Final QC Inspector	baseline: to be measured; proposed ≥ 85% at Final QC	daily

#	KPI (proposed)	Definition	Formula / query logic (proposed)	Owner	Target	Refresh
P3	PNA wait time	Time a part stays PNA on a tag	`DevicePNAPart.cleared_at − marked_at` for cleared rows (median, P90); open rows aged `now − marked_at`. Note: re-marking reuses the row, so only the latest cycle is kept (`models/pna_part.py`).	Spare Parts Manager	baseline: to be measured; proposed median ≤ 5 working days	daily
P4	Parts fill rate	Share of part requests handed over within 1 working day	`PartRequest` with `status IN ('handed_over','received')` and `actioned_at − created_at ≤ 1 working day` ÷ all requests raised in period (excl. cancelled). Caveat: `actioned_at` is overwritten by the later Verify step (`validate_receiving`) — PRD-01 adds a dedicated `handed_over_at`.	Spare Parts Manager	baseline: to be measured; proposed ≥ 85%	daily
P5	Grade mix	Share of Final-QC-passed tags by grade, by lot and month	count `Device.grade` for tags with a `StageMovement.to_stage = 'final_qc_pass_hold'` in period ÷ total	Final QC Inspector, Managers/Admin	to be agreed	daily
P6	Sales cycle time	Days from Ready to Sale to Sold	`Sale.sold_at − (first StageMovement.to_stage = 'ready_to_sale' for the device)` ; median by category and grade	Sales Executive	baseline: to be measured; then to be agreed	daily

UX (proposed): one "Flow KPIs" page, summary tiles on top, one Global Table per KPI with drill-down to tags (Roadmap X5).

6. Findings to fix (from reading the code)

#	Finding	Where	Suggested fix	Roadmap
K1	P&L From/To filter changes only values the page no longer shows (`total_revenue`, `total_investment`, `overall_profit`, `lot_pl`). The Lot P&L table macro `lot_pl_table()` is defined but never rendered.	`routers/dashboard.py`, `templates/dashboard.html`	Remove the filter or render the table	N4
K2	"Revenue This Month" ignores admin overrides; "Profit This Month" uses them. The two can disagree.	`_compute_financials` vs `compute_month_profit`	Use `compute_year_revenue(...)[month]` for S3	N4
K3	"active lots" counts every lot, including trashed.	`user_queue['lot_count']`	Filter `Lot.is_trashed.isnot(True)` or relabel	N4
K4	`work_queue_devices` and `today_followups` are queried on every load but not displayed.	`dashboard`	Remove or render	N4
K5	Stock Aging measures device age from creation, not time in stage.	`stock_aging_report`	Add P1 Stage ageing; keep R10 as "device age"	X5

#	Finding	Where	Suggested fix	Roadmap
K6	Dashboard Lot P&L (`_compute_lot_pl`) and `/reports/lot-pl` use different cost rules (dashboard adds cosmetic rate × cleaning entries and labour fallback; report nets returns).	both	Retire the dashboard copy (K1) or share one function	N4

7. Sign-off of definitions

KPI group	Owner to sign	Signed (name / date)
Pipeline and stock (D, C, S1–S2, R6–R11, O1–O2)	Inventory Manager + Production Manager	
Financial (F, S3–S4, R1–R5, R12)	Managers/Admin + Accounts	
Parts (F2, Q8, O3–O4, P3–P4)	Spare Parts Manager	
Quality (P2, P5)	Final QC Inspector + QC Inspector	
Sales (Q6–Q7, R13, P6)	Sales Executive lead	
TRC (O5, P1)	TRC Manager	



OXYPC INVENTORY LIFECYCLE

PRODUCT

Personas & Journeys

The 12 roles that handle a device, their goals and frustrations, and their journeys through the app.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Personas & Journeys

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

In this document: Part A — The 12 personas · Part B — End-to-end device journey · Part C — Persona profiles and journeys

PART A

The 12 personas

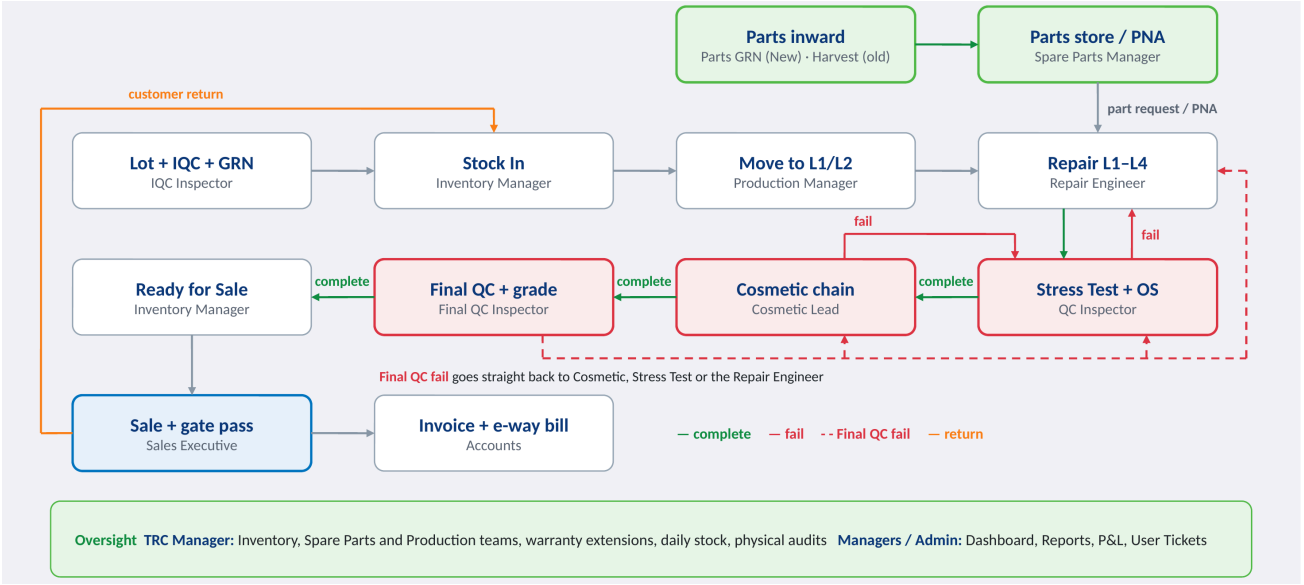
Twelve personas cover every role that handles a device, from lot intake to sale and oversight. Each is named by role only. App roles marked `custom role` have no built-in role in the code and are set up through the Permission Matrix (see Access & Security).

#	Persona	App role	What they do
1	**IQC Inspector**	`iqc_inspector`	Brings every purchased lot into the system: creates the lot, inspects each device in IQC, and maps it to a GRN.
2	**Inventory Manager**	`inventory_manager`	Owns stock outside the repair line: Stock In, floor movement, scrap, Ready for Sale, and holding tags for staff.
3	**Production Manager**	`custom role`	Runs the repair line from L1/L2 to Final QC: plans lots, moves tags to engineers, reassigns work, and reports team productivity.
4	**Repair Engineer (L1/L2, L3/L4)**	`l1 / l2 / l3_engineer`	Repairs devices: diagnoses each tag, requests parts, records PNA when a part isn't available, and scraps tags beyond repair.
5	**QC Inspector (Stress Test)**	`qc_inspector`	Tests every repaired device: runs the stress test, installs the OS, runs system diagnostics, then completes or fails it.
6	**Cosmetic Lead**	`cosmetic_manager`	Runs the cosmetic chain: Cleaning, Putty, Dry Sanding, Masking, Painting and Water Sanding, then completes or fails each tag.
7	**Final QC Inspector**	`qc_inspector / custom`	The last check before sale: diagnoses each device by hand and assigns its grade, or fails it back to the stage that needs rework.
8	**Spare Parts Manager**	`spare_parts_manager`	Runs the parts store: books new parts through Parts GRN and harvested parts from scrap, and fulfils engineers' part requests.
9	**Sales Executive**	`sales / sales_manager / telecaller`	Sells Ready for Sale stock: blocks tags, submits and records sales, raises delivery challans, and handles returns and rentals.
10	**Accounts**	`custom role`	Closes the money loop: generates invoices and e-way bills, records payments, and checks the sales list and daily stock.
11	**Managers / Admin**	`admin / Sub Admin`	Runs the business from the numbers: dashboard, reports and P&L, and resolves the tickets users raise.
12	**TRC Manager**	`custom role`	Leads the TRC: oversees the Inventory, Spare Parts and Production teams, approves warranty extensions, and owns daily stock and audits.

PART B

End-to-end device journey

A device passes three quality checks before it can be sold. Stress Test, the Cosmetic chain and Final QC each complete a tag forward or fail it back; Final QC can fail straight back to Cosmetic, Stress Test or the Repair Engineer. Parts come in new through Parts GRN or harvested from old devices. Customer returns come back to Stock In.



PART C

Persona profiles and journeys

One page per persona: profile, goals, frustrations, and the journey through the app, with the module used and the pain point or improvement idea at each step.

1. IQC Inspector

Brings every purchased lot into the system: creates the lot, inspects each device in IQC, and maps it to a GRN.

1

Team: Intake
App role: `iqc_inspector`
Where they work: Intake bench with a barcode scanner; pallets of 50–200 devices.
Main modules: GRN, Lot Overview, IQC Line Items

GOALS

- Inspect the whole lot the day it arrives
- Record condition accurately so prices are fair
- No device without a tag

FRUSTRATIONS

- Retyping details for identical models
- Bulk-upload errors that are hard to see
- Not knowing which invoice a lot belongs to

User journey

Step	What they do	Where in the app	Pain point / idea
1. Create lot	Records supplier, invoice and quantity	`Lot Overview`	Invoice details in one place
2. Inspect	Checks parts and condition of each device	`IQC Line Items`	Repetitive entry for same models
3. Add assets & tags	Adds assets; assigns Tag Numbers	`IQC Line Items`	Clear bulk-upload error table
4. Map to GRN	Maps inspected tags to the GRN	`GRN`	One step moves tags to Stock In
5. Hand off	Tags land in Stock In for Inventory	`Inventory Manager`	Confirm the whole lot moved

2. Inventory Manager

Owns stock outside the repair line: Stock In, floor movement, scrap, Ready for Sale, and holding tags for staff.

2

Team: Inventory
App role: `inventory_manager`
Where they work: Desk with the dashboard open; walks the racks when a count looks wrong.
Main modules: Stock In, Floor Movement, Ready for Sale, Block Tags, Extended Warranty

GOALS

- Stock In moves to production without delay
- Every tag has a known location
- Ready for Sale stock is accurate for sales

FRUSTRATIONS

- Tags sitting in Stock In unnoticed
- Floor moves made without a log
- Blocked tags nobody releases

User journey

Step	What they do	Where in the app	Pain point / idea
1. Receive Stock In	Checks tags arriving from GRN	`Stock In`	Counts match the GRN
2. Move to Production	Sends tags to the Production Manager	`Stock In → Production`	Bulk move in one step
3. Floor movement	Moves tags between locations and entities	`Floor Movement`	Every move logged
4. Block for staff	Holds tags for any staff member's deal	`Block Tags`	Blocks expire or get released
5. Ready for Sale	Receives graded tags from Final QC	`Ready for Sale`	Grade visible on every tag
6. Scrap & warranty	Scraps dead tags; extends warranties	`Scrap, Extended Warranty`	Reason recorded for each

3. Production Manager

Runs the repair line from L1/L2 to Final QC: plans lots, moves tags to engineers, reassigns work, and reports team productivity.

3

Team: Production
App role: `custom role`
Where they work: On the floor with engineers; checks progress on a laptop through the day.
Main modules: Production Manager, Lot Estimation, WorkID Status, Reports

GOALS

- Every lot repaired within its budget
- No tag stuck between stages
- Fair, visible productivity per engineer

FRUSTRATIONS

- Parts budgets guessed, not estimated
- Learning late that a tag is waiting
- Reassigning work when someone is absent

User journey

Step	What they do	Where in the app	Pain point / idea
1. Estimate lot	Works out parts needed and repair budget	`Lot Estimation`	Budget set before repair starts
2. Move to L1/L2	Sends tags from production to engineers	`Production Manager`	Bulk move with assignee
3. Monitor line	Watches tags from L1/L2 through Final QC	`Production Manager`	One view of every stage
4. Reassign	Moves work to another engineer	`Production Manager`	Keeps history of who worked
5. Productivity	Reports output per engineer and team	`WorkID Status, Reports`	Ageing per WorkID, CSV export

4. Repair Engineer (L1/L2, L3/L4)

Repairs devices: diagnoses each tag, requests parts, records PNA when a part isn't available, and scraps tags beyond repair.

4

Team: Workshop
App role: `l1 / l2 / l3_engineer`
Where they work: Shared bench PC between repairs, so every extra click costs bench time.
Main modules: L1/L2 Repair, L3/L4 Repair, Part Request, PNA

GOALS

- Finish more devices per shift
- Get parts without walking to stores
- Credit for every job

FRUSTRATIONS

- Part requests with no status
- Waiting on PNA with no update
- Devices failed back with no clear reason

User journey

Step	What they do	Where in the app	Pain point / idea
1. Pick work	Opens assigned tags; starts a WorkID	`L1/L2 Repair`	Queue in priority order
2. Diagnose & repair	Finds the fault and fixes it	`L1/L2 Repair`	IQC report and history at hand
3. Part request	Raises a request for the parts needed	`Part Request`	One row per part, with status
4. PNA	Submits Part Not Available when stock is out	`PNA Submission`	Alert when the part arrives
5. Escalate	Sends hard cases up to L3/L4	`L3/L4 Repair`	Reason travels with the tag
6. Complete or scrap	Passes to Stress Test, or marks scrap	`L1/L2, L3/L4`	Scrap reason required

5. QC Inspector (Stress Test)

Tests every repaired device: runs the stress test, installs the OS, runs system diagnostics, then completes or fails it.

5

Team: Quality
App role: `qc_inspector`
Where they work: Test bench with several devices running tests and OS installs at once.
Main modules: Stress Test, OS Installation, System Diagnose

GOALS

- No faulty device moves forward
- Clear fail reasons for engineers
- OS installed and checked on every device

FRUSTRATIONS

- Repeating the checklist by hand
- Devices returning with the same fault
- Waiting on long tests with no queue view

User journey

Step	What they do	Where in the app	Pain point / idea
1. Receive	Picks up tags completed in repair	`Stress Test`	Queue with filters like L1/L2
2. Stress test	Runs the stress checklist	`Stress Test`	Report saved with the tag
3. OS install	Installs and activates the OS	`Stress Test`	OS status recorded
4. Diagnose	Runs system diagnostics	`Stress Test`	Results attached to the tag
5. Complete or fail	Moves forward to Cosmetic, or fails back	`Stress Test`	Fail reasons go with the tag

6. Cosmetic Lead

Runs the cosmetic chain: Cleaning, Putty, Dry Sanding, Masking, Painting and Water Sanding, then completes or fails each tag.

6

Team: Cosmetic
App role: `cosmetic_manager`
Where they work: Leads a crew of workers moving tags between stations all day.
Main modules: Cosmetic Received, Cosmetic Chain, Devices Passed

GOALS

- Batches move between stations quickly
- Fair credit for each worker
- Finish quality that passes Final QC

FRUSTRATIONS

- Moving tags one by one
- Credit to whoever clicked, not who worked
- Failed tags with no reason

User journey

Step	What they do	Where in the app	Pain point / idea
1. Cleaning	Cleans the body and screen	`Cosmetic Chain`	Batch assign to one worker
2. Putty	Fills dents and scratches	`Cosmetic Chain`	Mover credited automatically
3. Dry sanding	Smooths the putty	`Cosmetic Chain`	Selections kept while searching
4. Masking	Masks ports, logos and screens	`Cosmetic Chain`	Tag Number shown first
5. Painting	Paints the body	`Cosmetic Chain`	Station time visible
6. Water sanding	Final finish; complete or fail	`Cosmetic Chain`	Complete → Final QC, or fail back

7. Final QC Inspector

The last check before sale: diagnoses each device by hand and assigns its grade, or fails it back to the stage that needs rework.

7

Team: Final QC
App role: `qc_inspector / custom`
Where they work: Final QC desk, checking each device in person before it reaches Ready for Sale.
Main modules: Final QC, Grading, Search Tags

GOALS

- Only sale-ready devices pass
- Grades that match the device
- Failed tags reach the right stage

FRUSTRATIONS

- Grade rules that differ between people
- Missing part and repair history
- Deciding which stage a fail goes back to

User journey

Step	What they do	Where in the app	Pain point / idea
1. Receive	Picks up tags completed in Cosmetic	`Final QC`	Full history on one screen
2. Manual diagnose	Checks the system by hand	`Final QC`	Checklist by model
3. Assign grade	Grades the device	`Final QC`	Grade rules shown beside the field
4. Fail back	Sends to L1/L2, Stress or Cosmetic	`Final QC`	Pick the stage and the reason
5. Complete	Moves to Ready for Sale (Inventory)	`Final QC → Ready for Sale`	Grade travels with the tag

8. Spare Parts Manager

Runs the parts store: books new parts through Parts GRN and harvested parts from scrap, and fulfils engineers' part requests.

8

Team: Stores
App role: `spare_parts_manager`
Where they work: Between the store counter and the racks; engineers come here when a repair stalls.
Main modules: Parts Manager, Parts GRN, Part Requests

GOALS

- Fill every request the same day
- Use harvested parts before buying new
- Exact stock per part and shelf

FRUSTRATIONS

- Requests bundling several parts in one row
- Harvested parts not booked
- PNA requests with no follow-up

User journey

Step	What they do	Where in the app	Pain point / idea
1. Parts GRN	Books As New parts from vendors	`Parts GRN`	Price and qty captured once
2. Harvest inward	Books As Harvest parts from scrapped tags	`Parts Manager`	Harvest linked to its source tag
3. Fulfil requests	Works through engineers' requests	`Parts Manager`	One row per part
4. Issue	Issues parts; consumption recorded	`Parts Consumption`	Date filter covers consumption
5. Close PNA	Fills PNA requests when parts arrive	`Parts Manager`	Alert to the engineer

9. Sales Executive

Sells Ready for Sale stock: blocks tags, submits and records sales, raises delivery challans, and handles returns and rentals.

9

Team: Sales
App role: `sales / sales_manager / telecaller`
Where they work: Calls with dealers and customers, working from the Ready for Sale list.
Main modules: Ready for Sale, Block Tags, Sales, Gate Pass, Returns, Rentals

GOALS

- Answer 'do you have this model?' in seconds
- Never sell a held tag
- Returns resolved the right way, fast

FRUSTRATIONS

- Stock that looks free but is blocked
- Retyping sale details for the challan
- Choosing the right return type

User journey

Step	What they do	Where in the app	Pain point / idea
1. Check stock	Filters Ready for Sale by model and grade	`Ready for Sale`	Fast filters and exports
2. Block	Holds tags for a deal	`Block Tags`	Blocked tags clearly marked
3. Sell	Submits the sale; marks items sold	`Sales, Sales Report`	The only way into Sold
4. Dispatch	Raises the delivery challan / gate pass	`Gate Pass`	Challan from the sale record
5. Returns	Return, paid repair, replacement, or credit note	`Returns, Credit Note`	Type decides where the tag goes
6. Rentals	Rents out devices and tracks them	`Rentals`	Due dates always visible

10. Accounts

Closes the money loop: generates invoices and e-way bills, records payments, and checks the sales list and daily stock.

10

Team: Finance
App role: `custom role`
Where they work: Accounts office, working from sales, bank statements and the app.
Main modules: Invoices, E-way Bill, Payments, Sales List, Daily Stock

GOALS

- Invoice and e-way bill out with every dispatch
- Every sale matched to a payment
- Daily stock that matches the books

FRUSTRATIONS

- Retyping sale details into invoices
- E-way bills generated outside the app
- Payments with no sale reference

User journey

Step	What they do	Where in the app	Pain point / idea
1. Invoice	Generates the invoice for each sale	`Invoices`	Built from the sale record
2. E-way bill	Generates the e-way bill for dispatch	`E-way Bill Generator`	Ready before the vehicle leaves
3. Payments	Records customer and supplier payments	`Payments`	Outstanding per customer
4. Sales list	Checks sales against invoices and payments	`Sales List`	Paid / due on every sale
5. Daily stock	Reviews opening and closing stock	`Daily Stock`	Matches the books

11. Managers / Admin

Runs the business from the numbers: dashboard, reports and P&L, and resolves the tickets users raise.

11

Team: Management
App role: `admin / Sub Admin`
Where they work: Reads dashboards and reports daily; reviews P&L weekly.
Main modules: Dashboard, Reports, P&L, User Tickets

GOALS

- Stock, cash and profit by lot at any time
- Slow stock and margin leaks spotted early
- User problems resolved quickly

FRUSTRATIONS

- Dashboard numbers that disagree with pages
- Reports exported and reworked in Excel
- Tickets with no owner

User journey

Step	What they do	Where in the app	Pain point / idea
1. Dashboard	Stock by stage, money and weekly trends	`Overall Dashboard`	Tiles match their pages
2. Reports	Sales, stock and team reports	`Reports`	Excel export for meetings
3. P&L	Profit by financial year and by lot	`Business P&L, Lot P&L`	Lot margin at a glance
4. User tickets	Reviews and resolves raised tickets	`Ticket Queue`	Owner and status on each

12. TRC Manager

Leads the TRC: oversees the Inventory, Spare Parts and Production teams, approves warranty extensions, and owns daily stock and audits.

12

Team: TRC
App role: `custom role`
Where they work: Moves between teams through the day; reviews numbers each morning and evening.
Main modules: Inventory, Spare Parts, Production, Extended Warranty, Daily Stock, Physical Audits

GOALS

- Teams hand off without delays
- Daily stock and audits always clean
- Warranty extensions handled fairly

FRUSTRATIONS

- Problems surfacing only at audit
- Three teams' status in three places
- Warranty requests with no history

User journey

Step	What they do	Where in the app	Pain point / idea
1. Daily stock	Reviews opening and closing stock by stage	`Daily Stock`	Exceptions flagged first
2. Inventory team	Checks Stock In, floor moves, Ready for Sale	`Inventory Manager`	Ageing per stage
3. Spare Parts team	Checks requests, PNA and parts stock	`Parts Manager`	Open PNA count
4. Production team	Checks the repair line and productivity	`Production Manager`	Bottleneck stage highlighted
5. Warranty	Approves warranty extensions	`Extended Warranty`	Device history on the request
6. Audits	Runs physical and zone audits	`Physical Audits`	Gaps listed by location



OXYPC INVENTORY LIFECYCLE

PROCESS

Business Process & Rules

How a device moves through every stage, and the rules each team follows.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Business Process & Rules

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

In this document: Part A — Device Lifecycle State Machine · Part B — Business Rules & SOPs

PART A

Device Lifecycle State Machine

1. Purpose

This document describes how one device (one **Tag Number**, stored as `devices.barcode`) moves through the OxyPC Inventory Lifecycle app. It is written from the code, not from the business view. Where the two differ, the gap is listed in section 9.

Use it to:

- see every stage value the app stores in `devices.current_stage`;
- find which screen and endpoint moves a tag from one stage to another;
- check the guards (what must be true) and side effects (WorkIDs, part requests, audit rows) of each move;
- understand the smaller state machines that run beside the main one (repair status, cosmetic chain, Final QC holds, part requests, PNA, returns, credit notes, block tags).

Related: Business Rules & SOPs · Role & Permission Matrix.

1.1 How stage moves are recorded

Record	What it holds	Where
`devices.current_stage`	The one current stage (enum `DeviceStage`)	`models/device.py::Device`
`stage_movements`	One row per move: from, to, who, when, note. `exited_at` is stamped on the previous open row when the tag leaves a stage	`models/device.py::StageMovement`
`work_orders`	One row per assignment to a person (the **WorkID**). Status `pending` → `in_progress` → `completed`	`models/work_order.py::WorkOrder`
`audit_logs`	Business audit rows written by `services/audit_engine.py::audit` (not every move writes one)	`services/audit_engine.py`
`allowed_transitions`	Admin-editable table of legal moves. Only some endpoints check it (see 1.2)	`models/stage_control.py`, `routers/stage_control.py`

WorkID formats (from `routers/transfers.py::_gen_work_id` and `routers/repair.py::_gen_prefixed_work_id`):

WorkID	Meaning	`work_orders.stage` code
12-digit number	Normal assignment (pick, bucket assign, cosmetic move, Final QC decision, Stress result)	`l1`, `qc`, `strs`, `recv`, `clean`, `putty`, `drysd`, `mask`, `paint`, `water`, `comp`, `fqc`, `asgn`, or first 5 letters of the stage

WorkID	Meaning	`work_orders.stage` code
`L3L4-0000001`	L3/L4 repair request	`l3`
`STRS-0000001`	Hand-off from L1/L2 Mark Complete to Stress Test	`qc`
`L1L2-…`	Legacy L1/L2 WorkIDs (still closed by `_close_l1l2_work_orders`)	`l1` / `l2`

1.2 The Control Engine

`services/control_engine.py` has four checks:

Function	Rule	Used by
`assert_device_in_stage`	Tag must be in the expected stage, else HTTP 409 "STAGE CONFLICT"	`routers/repair.py::start_repair`, `complete_repair`; `routers/qc.py::qc_submit`
`validate_transition`	Move must exist in `allowed_transitions`, else HTTP 403. Admin bypasses	`routers/repair.py::start_repair`, `complete_repair`, `move_device`; `routers/qc.py::qc_submit`; `routers/grn.py::grn_bulk_move_to_trc`; `routers/iqc.py::iqc_bulk_apply_grade_type`; `routers/bulk_upload.py::approve_duplicate_move`; `routers/api_v1/devices.py::move_device_stage`
`validate_sale_allowed`	Refuses a sale only when `devices.is_blocked` is true. Since 19 Oct 2026 a sale is allowed **from any stage**	`routers/sales.py::create_sale`
`validate_repair_level`	L2 needs an L1 RepairJob; L3 needs an L2 RepairJob	`routers/repair.py::start_repair`

Most dedicated workflow screens (L1/L2 Mark Complete, Stress Test, Cosmetic, Final QC, buckets, returns) **do not** call `validate_transition`. Their own code decides the next stage.

2. Stage reference

Stage values come from `models/device.py::DeviceStage`. Display names come from `STAGE_LABELS`. Badge colours come from `STAGE_COLORS` (Bootstrap colour names; `teal` and `purple` are custom).

Owner role = the business persona that works the stage. The code role in brackets is the built-in `UserRole` that the code actually checks.

#	Code value	Display name	Colour	Owner role (code role)	Entry action(s)	Exit action(s)
1	`grn`	GRN Receipt	warning	IQC Inspector (any signed-in user)	Register tag on lot: `routers/stock.py::register_device_barcode`, `register_bulk_csv`	Advance GRN to IQC → `iqc`

#	Code value	Display name	Colour	Owner role (code role)	Entry action(s)	Exit action(s)
2	`iqc`	IQC	secondary	IQC Inspector (`iqc_inspector`)	IQC form without GRN no. (`routers/iqc.py::iqc_create`); OxyQC agent (`routers/iqc_api.py::submit_iqc`); GRN Records "Sending for Selling"; return approved; External Tag return creates new tag	Map GRN / Validate GRN → `stock_in` ; Customise move
3	`stock_in`	Stock In	info	Inventory Manager (`inventory_manager`)	GRN mapped/validated; IQC form with GRN no.; bucket create; Back to Inventory from repair; return submitted; manual moves	Move to Production → `trc_production` ; Move to L1/L2 → `l1` ; Customise / manual moves
4	`trc_production`	TRC Production	info	Production Manager (`inventory_manager` , or custom role `production_manager`)	Move to TRC / Move bucket to Production; Final QC Fail (Hardware, no engineer found); Change Engineer	Assign / Bulk Assign / Change Engineer → `l1` , `qc_check` , `cosmetic_received`
5	`l1`	L1/L2 Repair	warning	Repair Engineer L1/L2 (`l1_engineer` , `l2_engineer`)	Move to L1/L2; bucket / device assign; Stress fail; Cosmetic fail; Final QC fail (Hardware); L3/L4 complete or scrap decision	Request to L3/L4 → `l3` ; Mark Complete → `qc_check` ; Back to Inventory → `stock_in` or `scrap_for_sale` ; Scrap → `scrapped`
6	`l2`	L2 Repair	warning	Legacy — no page	None today (retired; kept for old rows)	Legacy `complete_repair` moves
7	`l3`	L3/L4 Repair	danger	Repair Engineer L3/L4 (`l3_engineer`)	Request to L3/L4 from L1/L2; Change Engineer	L3/L4 Complete or Scrap → back to `l1`
8	`qc_check`	Stress Test	primary	QC Inspector (`qc_inspector`)	L1/L2 Mark Complete; bucket assign "Stress Test"; Final QC fail (Software); Change Engineer	Complete → `cosmetic_received` ; Fail → `l1`
9	`cosmetic_received`	Cosmetic Received	teal	Cosmetic Lead (`cosmetic_manager`)	Stress Complete; bucket assign "Cosmetic Repair"; Final QC fail (Cosmetic); Send to Cosmetic	Move → `cleaning` ; Skip → `final_qc` ; Fail → `l1`
10	`cleaning`	Cleaning	teal	Cosmetic Lead / stage user	Move from Cosmetic Received; bulk Assign	Move → `putty` ; Skip → `final_qc` ; Fail → `l1`
11	`putty`	Putty	teal	Cosmetic Lead / stage user	Move from Cleaning	Move → `dry_sanding` ; Skip → `final_qc`
12	`dry_sanding`	Dry Sanding	teal	Cosmetic Lead / stage user	Move from Putty	Move → `masking` ; Skip → `final_qc`
13	`masking`	Masking	teal	Cosmetic Lead / stage user	Move from Dry Sanding	Move → `painting` ; Skip → `final_qc`

#	Code value	Display name	Colour	Owner role (code role)	Entry action(s)	Exit action(s)
14	`painting`	Painting	teal	Cosmetic Lead / stage user	Move from Masking	Move → `water_sanding`; Skip → `final_qc`
15	`water_sanding`	Water Sanding	teal	Cosmetic Lead / stage user	Move from Painting	Move → `cosmetic_completed`; Skip → `final_qc`; Fail → `l1`
16	`cosmetic_completed`	Cosmetic Completed	teal	Cosmetic Lead	Move from Water Sanding	Move → `final_qc`; Fail → `l1`
17	`final_qc`	Final QC	purple	Final QC Inspector (any role with `cosmetic_finalqc` edit permission)	From cosmetic chain; Edit (revert) from a hold; Change Engineer	Pass → `final_qc_pass_hold`; Fail → `final_qc_fail_hold`
18	`final_qc_pass_hold`	Final QC — Pass Hold	success	Final QC Inspector → Inventory Manager	Final QC Pass	Move to Inventory → `ready_to_sale`; Edit → `final_qc`
19	`final_qc_fail_hold`	Final QC — Fail Hold	warning	Final QC Inspector	Final QC Fail	Move / Bulk Move → `l1`, `trc_production`, `qc_check` or `cosmetic_received`; Edit → `final_qc`
20	`ready_to_sale`	Ready to Sale	success	Inventory Manager / Sales Executive	Move to Inventory from Pass Hold; ReGRN of a sold tag; Trade Partner test devices; Customise	Sale → `sold`
21	`sold`	Sold	dark	Sales Executive (`sales`, `sales_manager`, `telecaller`)	New Sale / New Tag Sale / Sell from Block; Return Replace Now; Return Stock Complete	Return submitted → `stock_in`; ReGRN → `ready_to_sale`; Sale deleted → previous stage
22	`returned`	Returned	warning	—	**No code path sets this stage today** (see Q2)	Allowed transitions list `iqc`, `scrapped`, `stock_in`
23	`scrapped`	Scrapped	danger	Inventory Manager / Production Manager	Repair "Scrap" or cost auto-scrap (`services/cost_engine.py::auto_scrap_device`); return approved with action Scrap	"Move to Inventory" → `scrap_for_sale`
24	`scrap_for_sale`	Scrap for Sale	dark	Inventory Manager (Tags Scrapped page)	Back to Inventory after L3/L4 scrap; Scrap Products "Move to Inventory"; Customise	None in code (scrap sale does not change stage, see Q6)

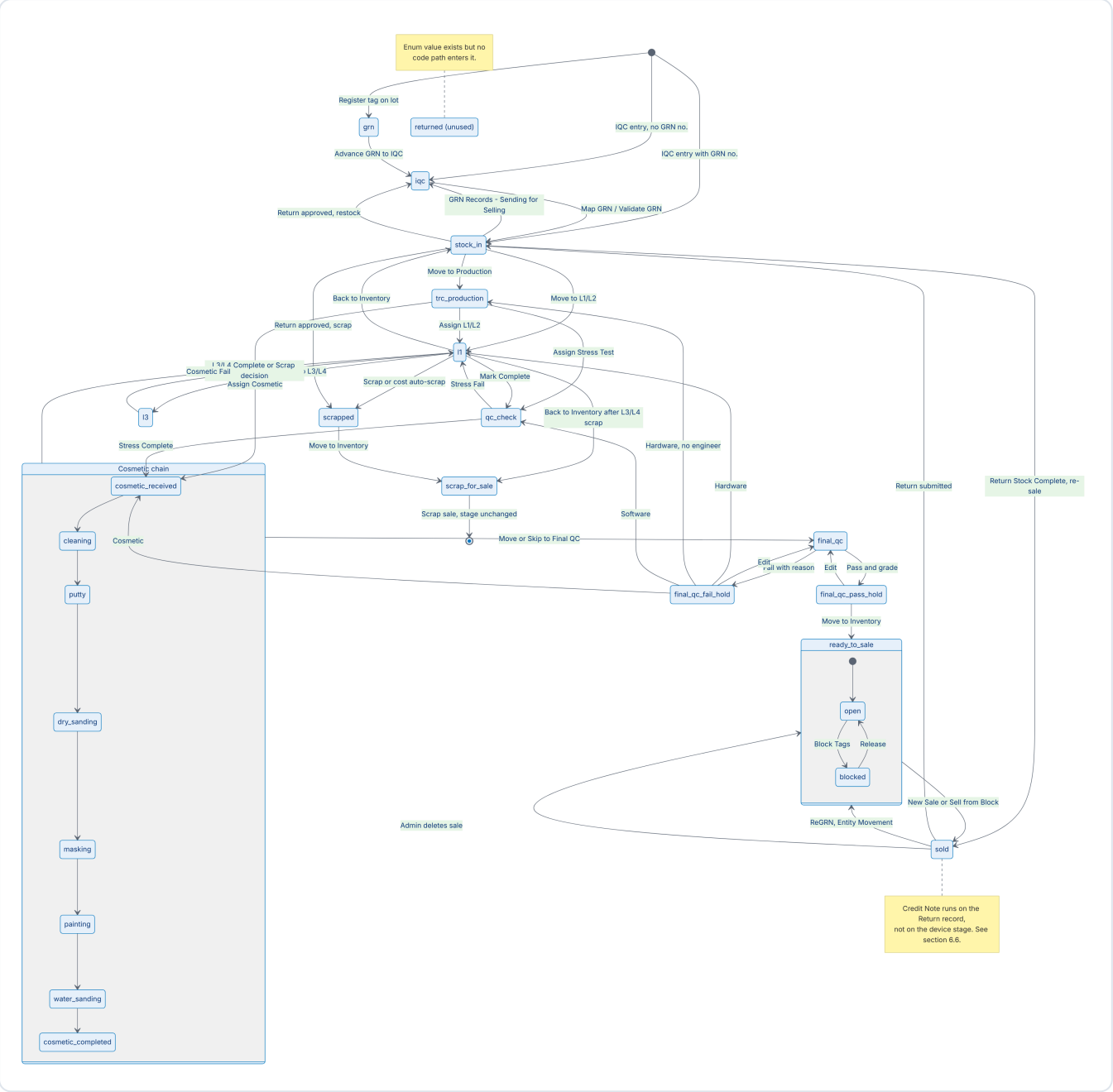
Notes:

- `DROPDOWN_STAGES` (all stages except `l2` and `sold`) feeds every generic "move to any stage" dropdown (`models/device.py`).

- `COSMETIC_STAGES` groups stages 9–16 for dashboard counts.
- `stage_sort_expr` sorts stages in the enum order above.
- `is_blocked`, `return_status`, `tag_return_status`, `final_qc_status`, `l1l2_status` and `l34_status` are **flags beside the stage**, not stages.

3. Lifecycle diagram

The diagram shows the main paths. Generic admin/Customise moves (any stage to any stage) are left out to keep it readable; see transition T40–T43.



4. Transitions

"Role" shows the code guard first, then the business persona. "Any user" means the endpoint only checks that someone is signed in (`get_current_user`).

4.1 Intake

#	From	To	Trigger / screen	Role	Guards	Side effects
T01	(new)	`grn`	Lot → Register Device / Bulk CSV (`routers/stock.py::register_device_barcode` , `register_bulk_csv`)	Any user · IQC Inspector	Lot exists	Device row created
T02	`grn`	`iqc`	Lot → Advance GRN to IQC (`routers/stock.py::advance_grn_to_iqc`)	Any user · IQC Inspector	Only tags at `grn` in that lot	**No StageMovement, no audit** (Q12)
T03	(new)	`iqc` or `stock_in`	IQC form (`routers/iqc.py::iqc_create`)	`admin` , `inventory_manager` , `iqc_inspector` , `sales_manager` · IQC Inspector	Barcode must be new; lot must resolve (blank lot → "Unassigned" lot via `utils/lot_helpers.get_or_create_unassigned_lot`). GRN no. given → `stock_in` , else `iqc`	IQCInspection row; StageMovement "IQC Entry" (+ second row to Stock In if GRN given); `device_price` from lot line item or lot average; audit `DEVICE_IQC_REGISTERED`
T04	(new)	`iqc`	OxyQC agent submit (`routers/iqc_api.py::submit_iqc`), API (`routers/api_v1/iqc.py::register_device`)	API / agent	—	Device + inspection rows
T05	`iqc`	`stock_in`	GRN Post-IQC → Map this GRN (`routers/grn.py::grn_map`)	`admin` , `inventory_manager` · Inventory Manager (business: IQC Inspector)	Only tags at `iqc` move; others just get the GRN no.	`grn_number` set; blank entity → "OxyPC Computers"; StageMovement; default location; GRN Stocked memory; audit `GRN_MAPPED`
T06	`iqc`	`stock_in`	GRN Import → Validate (`routers/grn.py::grn_validate`)	`admin` , `inventory_manager`	GRN linked to a lot; tags of that lot at `iqc`	GRN marked validated; auto-map as T05. No audit row
T07	any (allowed)	`iqc` / `stock_in`	GRN Records → Pending for TRC bulk move (`routers/grn.py::grn_bulk_move_to_trc`)	`admin` , `inventory_manager`	`validate_transition` (admin bypass); failed tags skipped	"Selling" → `iqc` and GRN cleared; "Sold to OxyPC" → `stock_in` ; audit `GRN_BULK_MOVE_TO_TRC`

4.2 Stock and production

#	From	To	Trigger / screen	Role	Guards	Side effects
T08	any	`stock_in`	Create bucket (`routers/buckets.py::create_bucket`)	`admin`, `inventory_manager` · Inventory Manager	Active, not trashed. **No stage guard**	Bucket created or reused by name; location set. **No StageMovement** (Q12)
T09	`stock_in`	`trc_production`	Move to TRC (`routers/stock.py::move_to_trc`)	`admin`, `inventory_manager`, `qc_inspector`, `sales_manager` · Inventory Manager	Tag must be at `stock_in`	StageMovement
T10	`stock_in` (or any, for customer-return buckets)	`trc_production`	Bucket → Move to Production / Assign to Production (`routers/buckets.py::move_bucket_to_trc`, `assign_bucket_to_production` → `_move_bucket_devices_to_trc`)	`admin`, `inventory_manager`	Only `stock_in` tags, unless `bucket.is_customer_return`	Bucket status `trc_pending`, `assigned_to_production`; StageMovement per tag
T11	`stock_in`	`l1`	Move to L1/L2, single or bulk (`routers/stock.py::move_to_l1l2`, `bulk_move_to_l1l2` → `_move_device_to_l1l2`)	Stock roles · Inventory Manager	Tag must be at `stock_in`	Stale open `l1` WorkOrders closed; repair status reset to New (`reset_repair_cycle_on_entry`); lands **unassigned** (Pick This)
T12	any	`l1` / `qc_check` / `cosmetic_received`	Production Manager → Assign bucket / Assign device / Bulk Assign (`routers/buckets.py::assign_bucket`, `assign_device`, `bulk_assign_devices_l1l2` → `_apply_department_move`)	`admin`, `inventory_manager` (+ custom `production_manager`) · Production Manager	Department must map in `DEPT_TO_STAGE` ("L1/L2 Repair", "Stress Test", "Cosmetic Repair")	StockTransfer (`department_assignment`); open WorkOrders closed (`services/work_order_close.py::close_open_work_orders`); new WorkID or reuse of open unassigned STRS- (`reuse_open_stress_work_order`); notification; status reset on L1/L2 entry
T13	any	`l1`	Bucket → Assign to Engineer (`routers/buckets.py::assign_bucket_to_engineer`)	`admin`, `inventory_manager`	—	StageMovement; status reset; bucket `validated`

#	From	To	Trigger / screen	Role	Guards	Side effects
T14	any active	`l1`, `l3`, `qc_check`, `cosmetic_received`, `final_qc`, `stock_in`, `trc_production`	Change Engineer (`routers/stock.py::change_engineer`)	Stock roles · Production Manager	Target in `CHANGE_ENGINEER_STAGES`; user active. **No source-stage guard**	Close open WorkOrders, new WorkID (or STRS- reuse); notification; audit `change_engineer`
T15	any	`stock_in`	Move to Stock (`routers/stock.py::move_to_stock`)	Stock roles	**No stage guard**	StageMovement; flag cleared by `_clear_flag_on_stock_in`

4.3 Repair (L1/L2 and L3/L4)

#	From	To	Trigger / screen	Role	Guards	Side effects
T16	`l1`	`l1`	Pick This (`routers/repair.py::l1_pick`)	Permission `repair_l1` add · Repair Engineer	Tag at `l1`; picking own tag is a no-op	All open WorkOrders closed; new 12-digit WorkID; same-stage StageMovement; previous engineer notified; audit `L1L2_PICKED`
T17	`l1`	`l1`	Start Repair (`routers/repair.py::l1l2_start`)	Any user	—	`l1l2_status` = "Repair Started"; RepairJob opened; audit
T18	`l1`	`l3`	Request to L3/L4 (`routers/repair.py::request_l3l4`)	Any user · Repair Engineer L1/L2	L3/L4 engineer must exist	`l1l2_status` = "Requested to L3/L4"; optional `repair_notes`; new `L3L4-` WorkID (L1/L2 WorkID left open); notification; audit `REQUEST_L3L4`
T19	`l3`	`l3`	L3/L4 Pick / Start (`routers/repair.py::l3l4_pick`, `l3l4_start`)	`repair_l3` enable (pick) / any user (start)	Tag at `l3` (pick)	Pick: close open WorkIDs, new `L3L4-` WorkID. Start: `l34_status` = "Repair Started"
T20	`l3`	`l1`	L3/L4 Mark Complete (`routers/repair.py::l3l4_complete`)	Any user · Repair Engineer L3/L4	—	`L3L4-` WorkOrders completed; `l34_status` = "Completed"; `l1l2_status` = "Returned from L3/L4"; activity + notes; audit
T21	`l3`	`l1`	L3/L4 Scrap (`routers/repair.py::l3l4_scrap`)	Any user · Repair Engineer L3/L4	Type must be "Normal Scrap" or "Replacement Scrap"	`l34_status` = scrap type; L1/L2 page then shows "Back to Inventory"

#	From	To	Trigger / screen	Role	Guards	Side effects
T22	`l1` (any)	`scrap_for_sale`	Back to Inventory after scrap decision (`routers/repair.py::back_to_production`)	Any user · Repair Engineer	**No stage or status guard**	`grade` = scrap; RepairJobs closed; audit `BACK_TO_INVENTORY`. Tag shows on Tags Scrapped page
T23	`l1` (any)	`qc_check`	L1/L2 Mark Complete (`routers/repair.py::l1l2_complete_to_stress`)	Any user · Repair Engineer	**Blocked if any active PNA part**; no stage guard	Open part requests: `handed_over` → `received` (cost counted), `requested` → `cancelled` ; L1/L2 WorkOrders and RepairJobs closed; `l1l2_status` = "Completed"; new `STRS-` WorkID (engineer optional); audits `PART_AUTO_VERIFIED_ON_COMPLETE` , `PART_AUTO_CANCELLED_ON_COMPLETE` , `L1L2_COMPLETE_TO_STRESS`
T24	`l1`	`l1` / `qc_check` / `stock_in` / `scrapped`	Legacy side panel "Complete Job" (`routers/repair.py::complete_repair`)	Permission `repair_<stage>` edit	`assert_device_in_stage` ; `validate_transition` on some branches	Final status decides: Scrap or cost auto-scrap → `scrapped` ; Escalate → stays `l1` ; PNA → no move; Back to Inventory → `stock_in` ; Completed/OK → `qc_check` (no STRS- WorkID). Writes RepairAttempt, parts consumption, `REPAIR_COMPLETE` audit
T25	`stock_in` / `l1`	`l1`	Legacy Start Repair (`routers/repair.py::start_repair`)	Permission `repair_l1` add	Stage check, repair-level check, transition check	RepairJob opened

4.4 Stress Test, cosmetic and Final QC

#	From	To	Trigger / screen	Role	Guards	Side effects
T26	`qc_check` (any)	`l1`	Stress Test → Fail (`routers/stress_api.py::stress_fail_assign`)	Any user · QC Inspector	Engineer must have role `l1_engineer` or `l2_engineer` . **No stage guard**	`stress_notes` = failed checklist items + note; status reset; checklist result + PDF saved; completed `strs` WorkID for the tester; new `l1` WorkID for engineer; notification; audit `STRESS_FAIL_ASSIGNED`
T27	`qc_check` (any)	`cosmetic_received`	Stress Test → Complete (`routers/stress_api.py::stress_complete_to_paint`)	Any user · QC Inspector	Optional engineer must be `qc_inspector` , `inventory_manager` , `sales_manager` or `cosmetic_manager`	`stress_notes` cleared; checklist + PDF; completed `strs` WorkID; new `recv` WorkID (may be unassigned); audit `STRESS_COMPLETE_TO_PAINT`

#	From	To	Trigger / screen	Role	Guards	Side effects
T28	`qc_check`	`cosmetic_received` or `l1` / `l2` / `l3`	Legacy QC form (`routers/qc.py::qc_submit`)	`qc_check` add permission	`assert_device_in_stage` ; `validate_transition`	QCCheck row with scores; pass sets grade; 3rd fail notes "recommend scrap"
T29	any	`cosmetic_received`	Send to Cosmetic (`routers/cosmetic.py::send_to_cosmetic`)	Cosmetic roles + `cosmetic_received` add	**No stage guard**	StageMovement only
T30	cosmetic stage *n*	stage *n*+1	Cosmetic page → Move (`routers/cosmetic.py::advance_stage`)	Cosmetic roles + per-stage edit permission (`PERM_MODULE_BY_STAGE`) · Cosmetic Lead	Stage must be in `NEXT_COSMETIC` . Assignee required unless landing on `final_qc` or `cosmetic_completed` ; Flow Data can auto-pick (`_resolve_flow_next_user`)	Origin stage's pending WorkOrder completed; new WorkID for assignee; notification. **No audit row**
T31	any cosmetic stage	`final_qc`	Skip cosmetic (`advance_stage` with `target=final_qc`)	As T30	Tag in `COSMETIC_PIPELINE`	`bucket_id` cleared on arrival at Final QC
T32	`cosmetic_received`	`cleaning`	Bulk Assign (`routers/cosmetic.py::bulk_assign`)	`admin` or `cosmetic_manager` only	Tag in `BULK_ASSIGN_STAGES`	New WorkID per tag. Other cosmetic stages: reassignment only, no move
T33	`cosmetic_received` , `cleaning` , `water_sanding` , `cosmetic_completed`	`l1`	Cosmetic → Fail (`routers/cosmetic.py::cosmetic_fail_assign`)	Per-stage edit permission · Cosmetic Lead	Only these 4 stages; engineer must be `l1_engineer` / `l2_engineer`	`repair_notes` = " Failed — ..."; status reset; `l1` WorkID; notification; audit `COSMETIC_FAIL_ASSIGNED`
T34	`final_qc`	`final_qc_pass_hold`	Final QC → Pass (`routers/cosmetic.py::advance_stage`)	`cosmetic_finalqc` edit · Final QC Inspector	—	Completed `fqc` WorkID for the inspector; grade set; touch type; spec corrections; verified RAM/HDD changes written (`apply_spec_changes_on_pass`); **price finalised once** (base + parts consumed + changed parts); bucket by name
T35	`final_qc`	`final_qc_fail_hold`	Final QC → Fail (`advance_stage`)	`cosmetic_finalqc` edit	—	`final_qc_status` = fail; `fqc_failure_reason` , `fqc_final_notes` ; bucket fail reason/engineer recorded
T36	Pass/Fail Hold	`final_qc`	Edit (`routers/cosmetic.py::fqc_edit_revert`)	`cosmetic_finalqc` edit	Tag must be in a hold	Decision fields cleared

#	From	To	Trigger / screen	Role	Guards	Side effects
T37	`final_qc_pass_hold`	`ready_to_sale`	Devices Passed → Move to Inventory (`routers/cosmetic.py::fqc_move_passed`)	`cosmetic_finalqc` edit · Final QC Inspector → Inventory Manager	Tag must be at Pass Hold	StageMovement; default location
T38	`final_qc_fail_hold`	`l1` / `trc_production` / `qc_check` / `cosmetic_received`	Devices Failed → Move / Bulk Move (`routers/cosmetic.py::fqc_move_failed`)	`cosmetic_finalqc` edit	Tag at Fail Hold and reason is Hardware, Software or Cosmetic	Routed by `FAIL_REASON_TO_DEPT` ; to last engineer at that step (`_resolve_fail_engineer`). Hardware with no engineer → `trc_production` . Uses `_apply_department_move` (T12 side effects)

4.5 Sales, returns and scrap

#	From	To	Trigger / screen	Role	Guards	Side effects
T39	**any stage**	`sold`	New Sale / New Tag Sale / Ready to Sale bulk / Sell from Block (`routers/sales.py::create_sale`)	`admin` , `sales` , `sales_manager` , `telecaller` + `sales` add · Sales Executive	Tag entity = seller's `User.entity` (seller must have one); not blocked unless sold from its own block (`validate_sale_allowed`)	Sale row `SALE-nnnn` ; company snapshot; warranty expiry; sale invoice no. replaces tag invoice no.; block item released with note "Sold — SALE-..."; below-cost warning (not blocking); audit `SALE_CREATED` ; event `SALE_COMPLETED`
T40	any	any	Customise modal → Move to Stage (`routers/iqc.py::iqc_bulk_apply_grade_type`)	IQC roles + `iqc` or `devices` edit	`sold` refused. `validate_transition` skipped for admin and when posted from `/devices` or `/stock`	Status reset on L1/L2 entry; L1/L2 or L3/L4 WorkOrders closed on exit; spec changes past Final QC
T41	any	any (allowed)	Manual move (`routers/repair.py::move_device`)	Any user	`validate_transition` (admin bypass). **Does not refuse `sold`**	Closes `L3L4-` WorkOrders when leaving `l3` ; audit `STAGE_MOVED`
T42	any	any (allowed)	API v1 (`routers/api_v1/devices.py::move_device_stage`)	API key	Transition must exist	StageMovement by `api_key:<name>`
T43	any	any (allowed)	Bulk upload duplicate → Approve Move (`routers/bulk_upload.py::approve_duplicate_move`)	Bulk upload roles	`validate_transition`	audit `STAGE_MOVED`

#	From	To	Trigger / screen	Role	Guards	Side effects
T44	`sold` (or any with a sale)	`stock_in`	Return New → Internal Tag (`routers/sales.py::process_return`)	Sales roles + `returns` add · Sales Executive	A sale must exist (or tag at `sold` → placeholder sale); one return per sale; reason, condition, received-by required	Return row `pending` ; `return_status` = true; `tag_return_status` = "Return for Repair" or "Replaced by "; audit `RETURN_SUBMITTED`
T45	`ready_to_sale`	`sold`	Return New → Replace Now (`process_return`)	As T44	Replacement tag must be `ready_to_sale` and not the same tag	₹0 Sale with original customer; `replaced` = "Replaced with "; customer invoice no. copied; audit `RETURN_REPLACEMENT_SOLD`
T46	any	`iqc` / `scrapped`	Pending Returns → Approve (`routers/sales.py::approve_return`)	`admin` , `sales_manager` only	Return must be `pending`	Restock → `iqc` ; Scrap → `scrapped` ; audit `RETURN_APPROVED`
T47	—	—	Pending Returns → Reject (`reject_return`)	`admin` , `sales_manager`	Return must be `pending`	Status `rejected` . **Tag stays at `stock_in`** (Q2)
T48	(new)	`iqc`	Return New → External Tag (`process_external_return`)	As T44	Reason, condition, received-by required	New device in lot `EXT-RETURNS` if tag unknown; placeholder ₹0 sale; pending Return with paid-repair amount
T49	any (returned tag)	`sold`	Return Stock → Complete (`routers/stock.py::return_stock_complete`)	Stock roles · Inventory Manager	A return and its original sale must exist	New Sale with original customer and warranty days; `return_status` = false; audit; event `SALE_COMPLETED`
T50	`sold`	previous stage	Sales list → Delete Selected (`routers/sales.py::bulk_delete_sales`)	`admin` only	Referenced sales (credit notes, receipts, care) skipped	Audit `SALE_DELETED` before delete; tag returns to the stage it was sold from
T51	`sold`	`ready_to_sale`	Entity Movement → ReGRN (`routers/entity_movement.py`)	`entity_movement` permission	Tag not trashed	New GRN/invoice/entity; location; audit `REGRN_TAG`
T52	any	`scrapped`	Cost engine auto-scrap (`services/cost_engine.py::auto_scrap_device` , from `complete_repair`)	System	Total cost ≥ expected sale value	AuditLog `AUTO_SCRAP`

#	From	To	Trigger / screen	Role	Guards	Side effects
T53	any	`scrap_for_sale`	Scrap Products from Repair Line → Move to Inventory (`routers/stock.py::scrap_move_to_inventory`)	Stock roles	**No stage guard**	StageMovement only
T54	`ready_to_sale`	`ready_to_sale` (blocked)	Block Tags → New Block (`routers/block_tags.py::create_block`)	`block_tags` add · Inventory Manager, Sales Executive	Not already blocked. Ready-to-Sale check is in the search box only (Q11)	`is_blocked` = true; BlockRecord `DDMMYYYY-CONTAINER-0001` ; `assigned_to_user_id` = Block For user. **No audit**
T55	blocked	open	Block Tags → Release / Bulk Release (`_release_items`)	`block_tags` edit	Item still `blocked`	`is_blocked` = false; item `released`

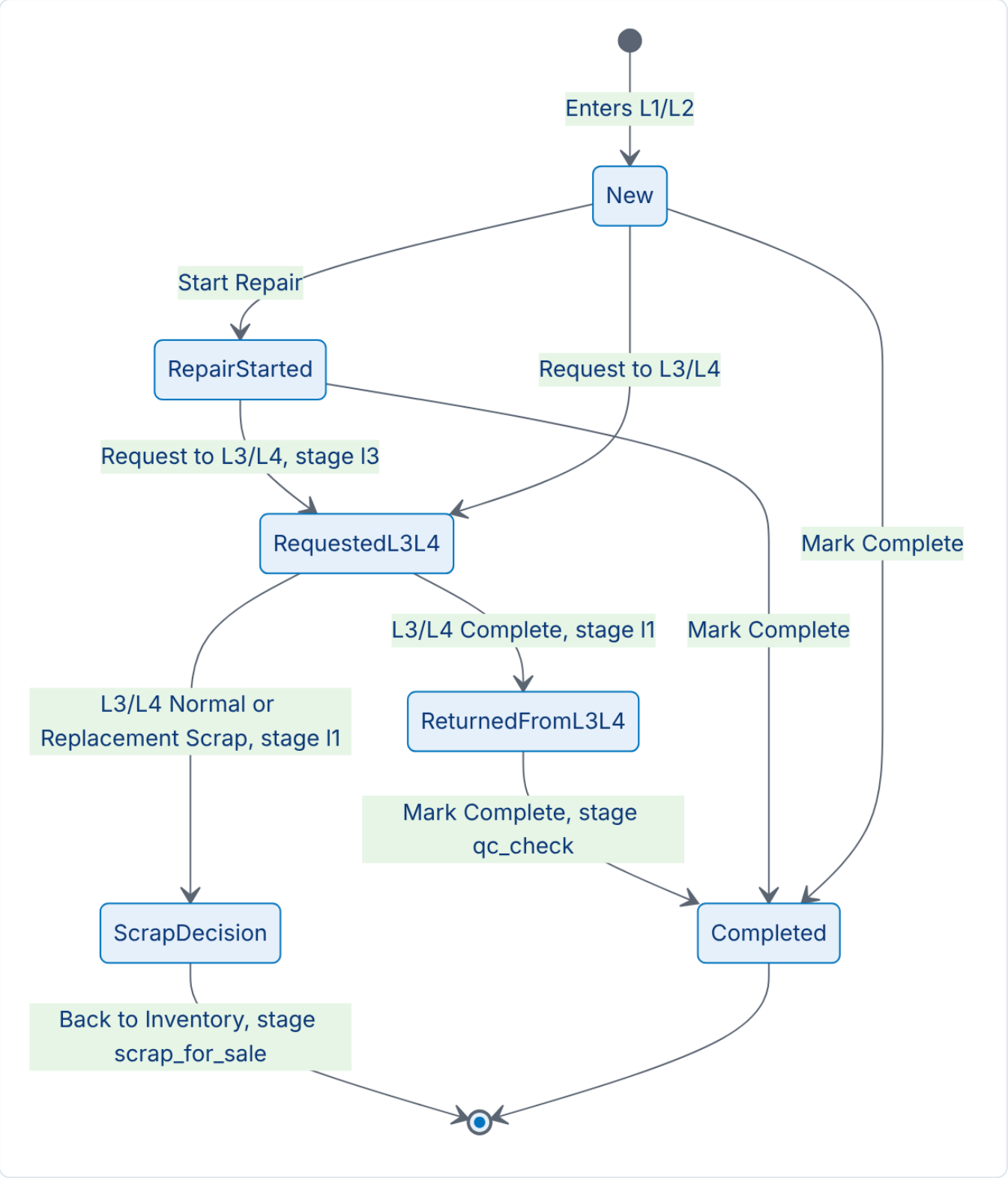
5. Flags and status fields beside the stage

Field	Values	Set by	Cleared by
`is_blocked`	true / false	`block_tags.py::create_block`	`_release_items` ; sale from block
`flag_*` (Urgent flag)	4 required fields	`stock.py::stock_flag_save`	Remove action; automatically on entry to `stock_in` (`models/device.py::_clear_flag_on_stock_in`)
`return_status`	true / false	Any return or credit note	`return_stock_complete` ; last Return deleted
`tag_return_status`	"Return for Repair", "Replaced by ", "Return for Credit Note"	`process_return` , `process_credit_note`	Last Return deleted
`final_qc_status`	pass / fail	Final QC decision	Edit (revert)
`grade`	A, B, C, D, E, scrap (`DeviceGrade`)	IQC form, Customise, Final QC Pass, Back to Inventory (scrap)	—
`scrap_verified`	true / false	`scrap.py::verify_scrap`	—
`sub_lot_number`	free text	GRN "Yes As-Is", transfers As-Is Lot, Customise	Customise Stock Type Finish Good / In Progress

6. Sub-state machines

6.1 L1/L2 and L3/L4 repair status

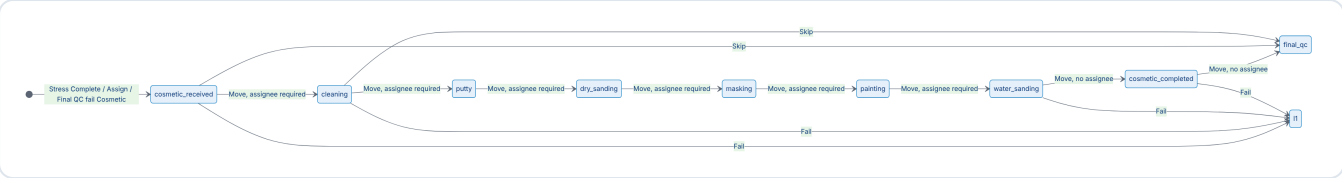
`devices.l1l2_status` and `devices.l34_status` (`models/device.py`). Both reset when a tag enters `l1` / `l2` from any other stage (`reset_repair_cycle_on_entry`).



`l34_status` values: blank → "Repair Started" → "Completed" or "Normal Scrap" / "Replacement Scrap".

6.2 Cosmetic chain

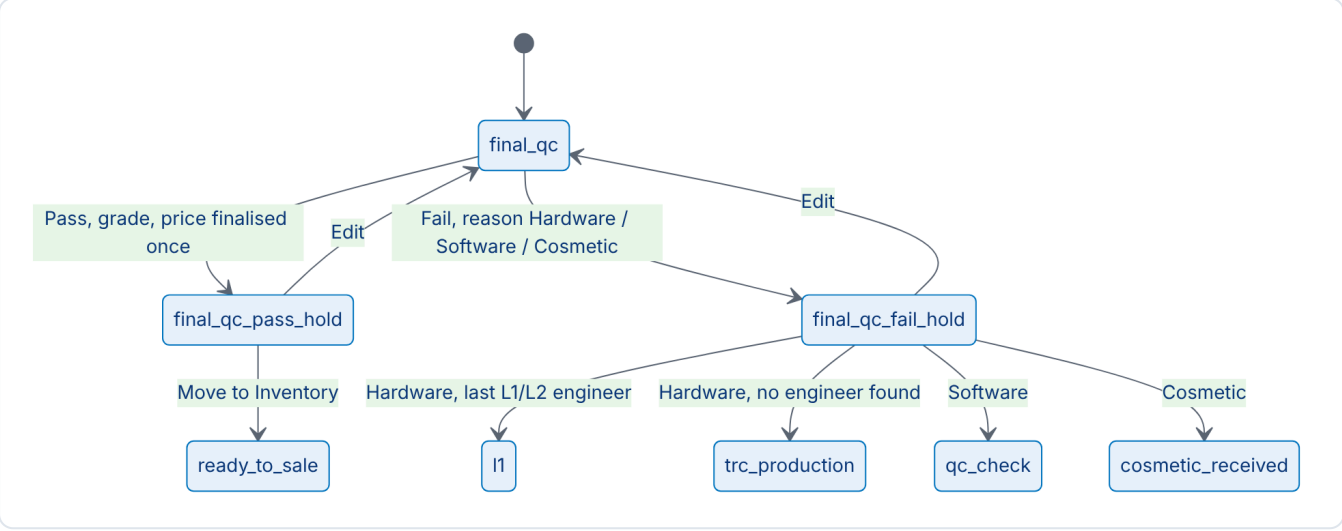
Order is fixed in `routers/cosmetic.py::COSMETIC_PIPELINE` and `NEXT_COSMETIC`.



Stage	WorkID code	Permission module	Fail allowed	Bulk Assign
Cosmetic Received	`recv`	`cosmetic_received`	Yes	Yes (also moves to Cleaning)
Cleaning	`clean`	`cosmetic_cleaning`	Yes	Yes (reassign only)
Putty	`putty`	`cosmetic_putty`	No	Yes (reassign only)
Dry Sanding	`drysd`	`cosmetic_dry_sanding`	No	Yes (reassign only)
Masking	`mask`	`cosmetic_masking`	No	Yes (reassign only)
Painting	`paint`	`cosmetic_painting`	No	Yes (reassign only)
Water Sanding	`water`	`cosmetic_water_sanding`	Yes	Yes (reassign only)
Cosmetic Completed	`comp`	`cosmetic_completed`	Yes	No

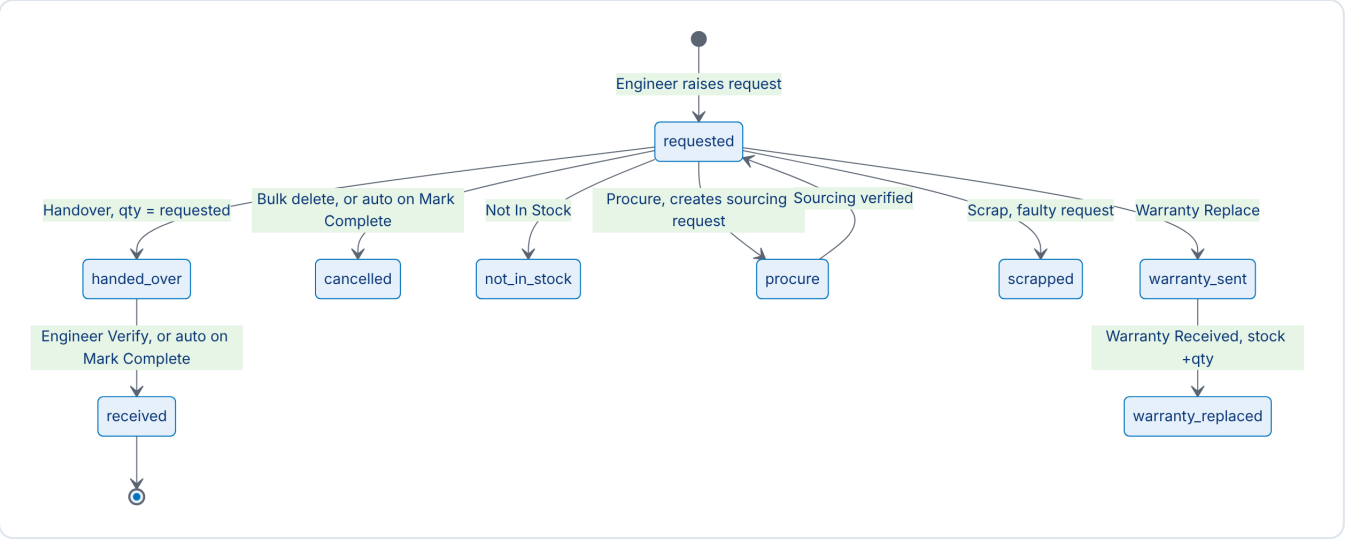
Skip to Final QC is allowed from any cosmetic stage (`advance_stage`, `target=final_qc`). Flow Data rows (`models/cosmetic_flow.py::CosmeticFlowRow`) name one user per stage; when the mover sits in a row, the next user is picked automatically.

6.3 Final QC holds



6.4 Part request and PNA

`part_requests.status` (`models/part_request.py::PartRequest`). Actions in `routers/part_requests.py` and `routers/repair.py`.



Action	Endpoint	Role	Guard
Raise (single / multi / bulk)	`create_part_request`, `create_part_requests_batch`, `repair.py::bulk_part_request`	`admin`, `inventory_manager`, `l1/l2/l3_engineer` (bulk: any user)	Bulk skips tags with a live request for the same part
Handover	`handover_part`	`admin`, `spare_parts_manager`	None on current status (single); bulk only from `requested`
Verify receiving	`validate_receiving`	Any user	None
Not In Stock / Procure / Scrap / Warranty Replace	`not_in_stock`, `procure_part`, `scrap_part_request`, `warranty_replace`	`admin`, `spare_parts_manager`	None on current status
Warranty Received	`warranty_received`	`admin`, `spare_parts_manager`	Must be `warranty_sent`

Sourcing request (`PartSourcingRequest.status`): `open` → `closed` (Sales Manager, `close_sourcing`) → verified (Spare Parts Manager, `verify_sourcing`, needs `closed`; adds stock via `SparePartsLedger` IN). Production requests from Part Estimation (`source = production`) are confirmed instead (`confirm_production_sourcing`).

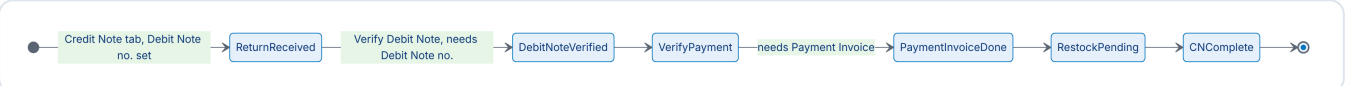
PNA (`models/pna_part.py::DevicePNAPart`): one row per tag + part. Mark sets `is_active` = true; clear sets false and keeps the row (`mark_part_pna`, `clear_all_pna`; any signed-in user). While any PNA row is active, L1/L2 Mark Complete is refused (T23).

6.5 Return approval

`returns.approval_status`: `pending` → `approved` (tag to `iqc` or `scrapped`) or `rejected` (no stage change). Only `admin` and `sales_manager` (`routers/sales.py::MANAGER_ROLES`).

6.6 Credit note

`returns.cn_stage` (`models/sales.py::CN_STAGES`). It never changes the device stage.



- Entry: `routers/sales.py::process_credit_note` (sales roles + `returns` add). It creates a Return and placeholder ₹0 sale if the tag has none. Re-submitting never moves a record back.
- Advance: one step at a time, `credit_note_advance` or `credit_note_bulk_verify` (`credit_note` edit permission), rules in `_try_advance_return`.

6.7 Block item, WorkOrder and delivery

Machine	States	Source
Block item	`blocked` → `released` (by Release or by sale from block)	`models/block_tags.py::BlockItemStatus`
WorkOrder	`pending` → `in_progress` (L3/L4 start) → `completed`	`models/work_order.py`
Sale delivery	`pending` → `dispatched` (Gate Pass exit) → `delivered`	`models/sales.py::Sale.delivery_status`, `routers/gate_pass.py::confirm_exit`
Return Stock repair cost	Verify records repair + labour cost, Complete re-sells	`routers/stock.py::return_stock_verify`, `return_stock_complete`

7. Invariants

These hold in the code today, with the exceptions named.

#	Invariant	Enforced by	Known exceptions
I1	A tag reaches `sold` only with a Sale row	`create_sale`, `process_return` (Replace Now), `return_stock_complete`; `sold` is not in `DROPDOWN_STAGES`; Customise refuses `sold`	`repair.py::move_device` and `api_v1/devices.py::move_device_stage` accept `sold` if `ready_to_sale → sold` is in `allowed_transitions` (Q3). `process_return` makes a placeholder sale for old orphans
I2	A blocked tag cannot be sold, except from its own block	`control_engine.validate_sale_allowed`, `create_sale::_check_sellable`	—
I3	A seller can only sell tags of their own entity	`create_sale::_check_sellable`	`return_stock_complete` and Replace Now do not check entity
I4	One return per sale	`process_return`	External Tag and Credit Note create their own placeholder sale
I5	Every stage move writes a StageMovement	Each endpoint	`stock.py::advance_grn_to_iqc`, `buckets.py::create_bucket` (Q12)
I6	Entry into L1/L2 from another stage starts a fresh repair cycle	`models/device.py::reset_repair_cycle_on_entry`; Stress/Cosmetic fail set it directly	`repair.py::move_device` does not call it
I7	A tag with active PNA parts cannot be Marked Complete in L1/L2	`l1l2_complete_to_stress`	Legacy `complete_repair` and Customise moves do not check PNA
I8	The Final QC price is finalised once per tag	`advance_stage` checks for an earlier "price finalized" StageMovement	—
I9	Entering `stock_in` clears the urgent flag	SQLAlchemy listener `_clear_flag_on_stock_in`	Bulk `update()` statements bypass ORM listeners

#	Invariant	Enforced by	Known exceptions
I10	Credit Note stages only move forward, one at a time	`_try_advance_return`	Delete removes the record
I11	Leaving L1/L2 closes the L1/L2 WorkIDs	`_close_l1l2_work_orders`, `close_open_work_orders`	Stress Complete/Fail do not close the open `STRS-` WorkID (Q10)

8. Allowed-transitions table

The live table is edited by admins on `/stage-control` (`routers/stage_control.py::add_transition`, `remove_transition`). Three seed lists exist and differ:

Source	Includes cosmetic_received / cosmetic_completed / putty?	Notes
`models/stage_control.py::DEFAULT_TRANSITIONS`	No	Has `qc_check → cleaning`, `final_qc → cleaning`
`upgrade_db.py` (CORRECT_TRANSITIONS)	No	Smaller list
`db_validator.py::CANONICAL_TRANSITIONS`	cosmetic_received and cosmetic_completed yes; putty no	Has fail moves to `l1`

None of the three includes `putty`, `final_qc_pass_hold`, `final_qc_fail_hold`, `scrap_for_sale` or `grn`.

9. Open questions / gaps

#	Gap	Code fact	Business view	Suggested owner
Q1	Sale from any stage	`create_sale` sells from any stage, including `sold`, `scrapped` and in-repair stages (decision dated 19 Oct 2026 in `control_engine.validate_sale_allowed`). A sold tag can be sold again	Sales Executive sells **Ready for Sale** items	Director / Sales
Q2	`returned` stage unused; reject leaves tag in Stock In	`process_return` moves the tag to `stock_in` at once; approve moves to `iqc` / `scrapped`; reject leaves it at `stock_in` though the code comment says "Device stays sold"	Returns: return, paid repair, replacement, credit note	Sales Manager
Q3	Transition rules are weak	Three seed lists differ (section 8); most workflow screens skip `validate_transition`; Manual Move and API can set `sold`	Clear, enforced stage flow	Architect
Q4	Roles in code ≠ 12 business roles	No built-in Production Manager, Final QC Inspector, Accounts or TRC Manager role. Production Manager = `inventory_manager` (`routers/buckets.py::production_manager`). Final QC = anyone with `cosmetic_finalqc` edit. Custom roles work through the permission matrix (`auth/dependencies.py::require_roles`)	12 distinct roles	Admin
Q5	Many moves have no stage guard and only need a login	`l1l2_complete_to_stress`, `back_to_production`, `l3l4_complete`, `l3l4_scrap`, `stress_fail_assign`, `stress_complete_to_paint`, `move_to_stock`, `send_to_cosmetic`, `scrap_move_to_inventory`	Only the owner role moves a tag forward	Architect

#	Gap	Code fact	Business view	Suggested owner
Q6	Scrap sale does not move tags	`scrap_for_sale.py::sell_scrap_for_sale` marks the batch sold; tags stay at `scrap_for_sale`. Two scrap stages (`scrapped` , `scrap_for_sale`)	Tags scrapped handled by Inventory Manager	Inventory
Q7	Customise bypass depends on a form field	`iqc_bulk_apply_grade_type` skips transition checks when the posted `return_to` is `/devices` or `/stock`	—	Architect
Q8	IQC Inspector cannot run GRN in code	`routers/grn.py` allows only `admin` , `inventory_manager`	IQC Inspector owns GRN, Lot, IQC	Admin
Q9	Two grade scales	`QCCheck.grade` A/B/C/D/S (`models/qc.py`) vs `DeviceGrade` A–E/scrap	Final QC assigns grading	Final QC
Q10	Open `STRS-` WorkID after Stress	Stress Complete/Fail create new WorkIDs but do not close the `STRS-` one raised by Mark Complete	Productivity reporting	Production
Q11	Block create trusts the client	`create_block` does not re-check `ready_to_sale` ; no audit on block/release; model comment mentions a "Done" status the enum lacks	Block tags for any staff	Inventory / Sales
Q12	Silent stage moves	`advance_grn_to_iqc` and `create_bucket` change stage without StageMovement; `create_bucket` moves any stage to `stock_in`	Full history	Architect
Q13	Rentals	No rental flow found in sales, dispatch or gate pass	Sales Executive handles rentals	Sales

PART B

Business Rules & SOPs

1. Purpose

This document lists the business rules the OxyPC Inventory Lifecycle app enforces today, and a short daily routine (SOP) for each of the 12 business roles.

- Every rule cites the file and function it comes from. If a rule is not in the code, it is not in this list.
- "Role" is the business persona. Where the code checks a different role, the code role is shown in brackets.
- Where the business view and the code disagree, the gap is listed in section 4.

Stage names and transitions are in the Device Lifecycle State Machine.

1.1 Business roles and the code roles behind them

#	Business role	Built-in code role(s) today	Note
1	IQC Inspector	`iqc_inspector` ("IQC Handler")	Cannot open `/grn` unless the permission matrix grants it
2	Inventory Manager	`inventory_manager`	Also acts as Production Manager in code
3	Production Manager	`inventory_manager`, or a custom role `production_manager`	No built-in role (`routers/buckets.py::production_manager`)
4	Repair Engineer	`l1_engineer` ("L1/L2 Engineer"), `l2_engineer`, `l3_engineer` ("L3/L4 Engineer")	
5	QC Inspector	`qc_inspector` ("QC Handler")	
6	Cosmetic Lead	`cosmetic_manager`	Single-stage custom roles also exist
7	Final QC Inspector	Any role with `cosmetic_finalqc` edit permission	No built-in role
8	Spare Parts Manager	`spare_parts_manager` ("Parts Manager")	
9	Sales Executive	`sales` ("Sourcing Sales"), `telecaller`, `sales_manager`	
10	Accounts	None; Accounts pages allow `admin`, `inventory_manager`, `sales_manager`	`routers/accounts.py::FINANCE_ROLES`
11	Managers / Admin	`admin`	Admin passes every role check
12	TRC Manager	Custom role `trc_manager`	Sees the full repair queue (`routers/repair.py::FULL_QUEUE_ROLES`)

Custom (admin-created) roles pass any non-admin-only `require_roles` gate and are then limited by the Module Permission matrix (`auth/dependencies.py::require_roles`).

2. Business rules

2.1 Intake: Lot, GRN and IQC

ID	Rule	Source	Role
BR-01	A tag number (barcode) must be unique; IQC refuses a barcode that already exists.	`routers/iqc.py::iqc_create`	IQC Inspector
BR-02	An IQC entry with no lot is filed under the system "Unassigned" lot.	`routers/iqc.py::iqc_create`, `utils/lot_helpers.py::get_or_create_unassigned_lot`	IQC Inspector
BR-03	An IQC entry that carries a GRN number goes straight to Stock In; without one it stays at IQC.	`routers/iqc.py::iqc_create`	IQC Inspector
BR-04	The device price defaults to the lot line item's unit price, else the lot average (buying price ÷ qty).	`routers/iqc.py::iqc_create`	IQC Inspector
BR-05	Tags registered on a lot start at GRN Receipt and move to IQC with "Advance GRN to IQC".	`routers/stock.py::register_device_barcode`, `advance_grn_to_iqc`	IQC Inspector
BR-06	Mapping a GRN moves only tags at IQC to Stock In; tags in other stages just receive the GRN number.	`routers/grn.py::grn_map`	Inventory Manager (code: `admin`, `inventory_manager`)
BR-07	A tag with no entity gets "OxyPC Computers" when its GRN is mapped; an existing entity is never overwritten.	`routers/grn.py::grn_map`	Inventory Manager
BR-08	Validating a Plant GRN auto-maps every IQC-stage tag of the linked lot to Stock In.	`routers/grn.py::grn_validate`	Inventory Manager
BR-09	A lot GRN records expected vs received quantity and warns on a mismatch; the GRN number is auto-generated if blank.	`routers/grn.py::submit_grn`	Inventory Manager
BR-10	"Pending for TRC" bulk move sends tags back to IQC with the GRN cleared ("Sending for Selling") or to Stock In ("Sold to OxyPC"); tags whose move is not an allowed transition are skipped.	`routers/grn.py::grn_bulk_move_to_trc`	Inventory Manager
BR-11	"Yes As-Is" writes one Sub-Lot Number (and optional Assigned To) on every tag mapped to that GRN.	`routers/grn.py::grn_as_is_lot`	Inventory Manager

2.2 Stock and floor movement

ID	Rule	Source	Role
BR-12	Only tags at Stock In can be moved to TRC Production or to L1/L2.	`routers/stock.py::move_to_trc`, `_move_device_to_l1l2`	Inventory Manager
BR-13	A tag moved to L1/L2 from Stock In lands unassigned (Pick This), and any stale L1 WorkID is closed first.	`routers/stock.py::_move_device_to_l1l2`	Inventory Manager
BR-14	"Move to Production" on a bucket moves only its Stock In tags, except customer-return buckets, which move all their tags.	`routers/buckets.py::_move_bucket_devices_to_trc`	Inventory Manager
BR-15	A bucket name is reused case-insensitively; creating a bucket sets its tags to Stock In.	`routers/buckets.py::create_bucket`	Inventory Manager

ID	Rule	Source	Role
BR-16	An urgent flag needs all four fields: Authorized By, Request Date, Target Date and Reason.	`routers/stock.py::stock_flag_save`	Inventory Manager
BR-17	The urgent flag clears itself whenever a tag moves into Stock In.	`models/device.py::_clear_flag_on_stock_in`	System
BR-18	A transfer (Move Item / Bucket / Lot) records the assignment and location only; it never changes the stage.	`routers/transfers.py::create_transfer`, `_move_devices_bulk`	Inventory Manager
BR-19	Every transfer needs an employee to assign to and creates an `asgn` WorkID plus a notification.	`routers/transfers.py::create_transfer`	Inventory Manager
BR-20	A transfer of type Ready for Sale (shown as "Finish Good") or As-Is Lot stamps the employee as the tag's Assigned To.	`routers/transfers.py::create_transfer`	Inventory Manager
BR-21	A transfer of type Scrap for Sale creates one Scrap ID batch per submission.	`routers/transfers.py::_maybe_create_scrap_batch`	Inventory Manager
BR-22	A new Location ID resets the tag's Bin Location and writes a location log row.	`routers/transfers.py::create_transfer`	Inventory Manager
BR-23	The Customise modal can never set a tag to Sold; selling must go through New Sale.	`routers/iqc.py::iqc_bulk_apply_grade_type`	Inventory Manager
BR-24	Customise stage moves skip the allowed-transition check for admin and when used from All Inventory or Inventory Manager pages.	`routers/iqc.py::iqc_bulk_apply_grade_type`	Inventory Manager
BR-25	ReGRN (Entity Movement) moves a sold tag back to Ready to Sale under the new GRN, invoice and entity.	`routers/entity_movement.py` (change entities)	Inventory Manager / TRC Manager

2.3 Production and repair

ID	Rule	Source	Role
BR-26	Production assignment has three departments: L1/L2 Repair → L1/L2, Stress Test → Stress Test, Cosmetic Repair → Cosmetic Received.	`routers/buckets.py::DEPT_TO_STAGE`, `_apply_department_move`	Production Manager
BR-27	Reassigning a tag closes the previous person's open WorkID and opens a new one at the same moment.	`services/work_order_close.py::close_open_work_orders`	Production Manager
BR-28	Assigning a Stress Test engineer reuses the open, unassigned STRS- WorkID instead of creating a second one.	`services/work_order_close.py::reuse_open_stress_work_order`	Production Manager
BR-29	Change Engineer can move a tag from any stage to L1/L2, L3/L4, Stress Test, Cosmetic, Final QC, Stock In or TRC Production and assign any active user.	`routers/stock.py::change_engineer`, `CHANGE_ENGINEER_STAGES`	Production Manager
BR-30	Any tag entering L1/L2 from another stage starts a fresh repair cycle (status New, L3/L4 status cleared).	`models/device.py::reset_repair_cycle_on_entry`	System
BR-31	"Pick This" lets the clicking engineer take a tag from anyone; all open WorkIDs close and a new one opens.	`routers/repair.py::l1_pick`, `l3l4_pick`	Repair Engineer
BR-32	Engineers see their own tags plus the unclaimed pool; admin, TRC Manager, Inventory Manager and Production Manager see the full queue.	`routers/repair.py::FULL_QUEUE_ROLES`	Repair Engineer / supervisors

ID	Rule	Source	Role
BR-33	Request to L3/L4 moves the tag to L3/L4 and creates an L3L4- WorkID for the chosen engineer; the L1/L2 WorkID stays open.	`routers/repair.py::request_l3l4`	Repair Engineer (L1/L2)
BR-34	L3/L4 Complete and L3/L4 Scrap both send the tag back to L1/L2 for the next action.	`routers/repair.py::l3l4_complete`, `l3l4_scrap`	Repair Engineer (L3/L4)
BR-35	L3/L4 scrap must be "Normal Scrap" or "Replacement Scrap".	`routers/repair.py::l3l4_scrap`	Repair Engineer (L3/L4)
BR-36	After an L3/L4 scrap decision, "Back to Inventory" sets grade Scrap and moves the tag to Scrap for Sale (Tags Scrapped page).	`routers/repair.py::back_to_production`	Repair Engineer
BR-37	L1/L2 Mark Complete is refused while the tag has any active PNA part.	`routers/repair.py::l1l2_complete_to_stress`	Repair Engineer
BR-38	Mark Complete auto-verifies handed-over parts (cost counted) and auto-cancels parts never handed over.	`routers/repair.py::_mark_complete_open_parts`, `l1l2_complete_to_stress`	Repair Engineer
BR-39	Mark Complete always moves the tag to Stress Test and raises an STRS- WorkID; the Stress engineer is optional.	`routers/repair.py::l1l2_complete_to_stress`	Repair Engineer
BR-40	In the legacy Complete Job panel, a repair whose total cost reaches the expected sale value is auto-scrapped; at 70% the engineer is warned.	`services/cost_engine.py::check_scrap_decision`, `routers/repair.py::complete_repair`	System
BR-41	L2 repair needs an earlier L1 attempt and L3 needs an L2 attempt (legacy Start Repair).	`services/control_engine.py::validate_repair_level`	Repair Engineer
BR-42	The L2 stage is retired; every escalation or fail lands in L1/L2.	`routers/repair.py::complete_repair`, `routers/stress_api.py::stress_fail_assign`	System
BR-43	Part Estimation costs a lot's IQC-required parts; "Download & Attach Estimate" raises a whole-lot production sourcing request.	`routers/part_estimation.py` (module docstring)	Production Manager

2.4 QC (Stress Test)

ID	Rule	Source	Role
BR-44	The Stress Test checklist has 27 fixed items; only Touch Screen may be marked N/A.	`models/stress.py::STRESS_CHECKLIST_ITEMS`, `STRESS_OPTIONAL_ITEMS`	QC Inspector
BR-45	Stress Fail must assign an L1 or L2 engineer and sends the tag to L1/L2 with the failed items saved as Stress Notes.	`routers/stress_api.py::stress_fail_assign`	QC Inspector
BR-46	Stress Complete sends the tag to Cosmetic Received and clears old Stress Notes; an assignee is optional.	`routers/stress_api.py::stress_complete_to_paint`	QC Inspector
BR-47	Each Stress result gives the tester a completed WorkID and saves one checklist report (with PDF) per visit.	`routers/stress_api.py::_record_checklist_result`, `stress_fail_assign`	QC Inspector
BR-48	On the legacy QC form, a score of 70/100 or more passes, and the 3rd fail is noted "recommend scrap".	`routers/qc.py::qc_submit`, `QC_FAIL_SCRAP_THRESHOLD`	QC Inspector

2.5 Cosmetic

ID	Rule	Source	Role
BR-49	The cosmetic chain runs in this order: Cosmetic Received, Cleaning, Putty, Dry Sanding, Masking, Painting, Water Sanding, Cosmetic Completed, Final QC.	`routers/cosmetic.py::COSMETIC_PIPELINE`, `NEXT_COSMETIC`	Cosmetic Lead
BR-50	Every cosmetic Move needs an assignee, except a move that lands on Cosmetic Completed or Final QC.	`routers/cosmetic.py::advance_stage`	Cosmetic Lead
BR-51	If the mover appears in a saved Flow Data row, the next stage's user is assigned automatically.	`routers/cosmetic.py::_resolve_flow_next_user`	Cosmetic Lead
BR-52	A tag can skip the rest of the cosmetic chain and go straight to Final QC from any cosmetic stage.	`routers/cosmetic.py::advance_stage`	Cosmetic Lead
BR-53	Cosmetic Fail is allowed only at Cosmetic Received, Cleaning, Water Sanding and Cosmetic Completed, and must assign an L1/L2 engineer.	`routers/cosmetic.py::cosmetic_fail_assign`	Cosmetic Lead
BR-54	Bulk Assign is for admin and Cosmetic Manager only; at Cosmetic Received it also moves tags to Cleaning.	`routers/cosmetic.py::bulk_assign`	Cosmetic Lead
BR-55	Each cosmetic page needs its own edit permission (for example `cosmetic_painting`).	`routers/cosmetic.py::PERM_MODULE_BY_STAGE`	Cosmetic Lead
BR-56	Only admin and Cosmetic Manager can create or edit Flow Data rows.	`models/cosmetic_flow.py::CosmeticFlowRow`, `routers/cosmetic.py::flow_data_allowed`	Cosmetic Lead

2.6 Final QC and grading

ID	Rule	Source	Role
BR-57	Final QC decisions need edit permission on `cosmetic_finalqc`.	`routers/cosmetic.py::advance_stage`, `fqc_move_passed`, `fqc_move_failed`	Final QC Inspector
BR-58	Final QC Pass sets the grade (A, B, C, D, E or Scrap) and holds the tag in "Pass Hold".	`routers/cosmetic.py::advance_stage`, `models/device.py::DeviceGrade`	Final QC Inspector
BR-59	On the first Final QC Pass, the device price becomes buying price + parts consumed + changed parts received; later passes do not add again.	`routers/cosmetic.py::advance_stage`	System
BR-60	Verified RAM and hard-drive changes are written onto the tag's spec at Final QC Pass.	`services/parts_changed.py::apply_spec_changes_on_pass`	System
BR-61	Final QC Fail needs a reason; the tag waits in "Fail Hold" until someone clicks Move.	`routers/cosmetic.py::advance_stage`	Final QC Inspector
BR-62	The fail reason routes the tag: Hardware → L1/L2, Software → Stress Test, Cosmetic → Cosmetic Received, each to the last person who worked that step.	`routers/cosmetic.py::FAIL_REASON_TO_DEPT`, `_resolve_fail_engineer`, `fqc_move_failed`	Final QC Inspector
BR-63	A Hardware fail with no engineer found goes to TRC Production (Tag Number Allocation).	`routers/cosmetic.py::fqc_move_failed`	Final QC Inspector
BR-64	"Move to Inventory" moves Pass Hold tags to Ready to Sale.	`routers/cosmetic.py::fqc_move_passed`	Final QC Inspector → Inventory Manager

ID	Rule	Source	Role
BR-65	"Edit" returns a Pass Hold or Fail Hold tag to Final QC and clears the decision.	`routers/cosmetic.py::fqc_edit_revert`	Final QC Inspector
BR-66	Touch Type is saved on both Pass and Fail; a blank value shows as Non-Touch.	`routers/cosmetic.py::advance_stage`, `models/device.py::touch_type`	Final QC Inspector

2.7 Spare parts

ID	Rule	Source	Role
BR-67	Engineers raise part requests as New, Replace or Downgrade; the request is linked to the tag's open WorkID.	`routers/part_requests.py::create_part_request`	Repair Engineer
BR-68	Bulk Part Request raises one request per L1/L2 tag that needs the part and skips tags that already have a live request for it.	`routers/repair.py::bulk_part_request`	Repair Engineer
BR-69	The handover quantity always equals the requested quantity; it cannot be edited.	`routers/part_requests.py::handover_part`, `bulk_request_action`	Spare Parts Manager
BR-70	Bulk Handover acts only on requests still at "requested".	`routers/part_requests.py::bulk_request_action`	Spare Parts Manager
BR-71	Deleting a part request is a soft close ("cancelled"); handed-over or received requests cannot be cancelled.	`routers/part_requests.py::bulk_request_action`	Spare Parts Manager
BR-72	"Procure" creates a sourcing request that the Sales Manager closes in the CRM.	`routers/part_requests.py::procure_part`, `close_sourcing`	Spare Parts Manager / Sales Manager
BR-73	A sourcing request can be verified only after its deal is closed; verifying adds the sourced quantity to stock and reopens the part request.	`routers/part_requests.py::verify_sourcing`	Spare Parts Manager
BR-74	A faulty part under supplier warranty goes Warranty Sent → Warranty Replaced, and the replacement goes back into stock.	`routers/part_requests.py::warranty_replace`, `warranty_received`	Spare Parts Manager
BR-75	Any signed-in user can mark a part PNA on a tag; clearing keeps the history.	`routers/part_requests.py::mark_part_pna`, `models/pna_part.py`	Repair Engineer
BR-76	Parts GRN (As New) records PO, invoice, e-way bill and vehicle details and adds line quantities to Part Master stock.	`routers/parts_grn.py::grn_create`	Spare Parts Manager
BR-77	A harvest part (from an old device) is recorded as an internal Parts GRN ("Harvest") and added to Part Master with source "harvest".	`routers/parts_grn.py::harvest_part`	Spare Parts Manager

2.8 Sales and dispatch

ID	Rule	Source	Role
BR-78	A seller can only sell tags whose entity matches their own profile entity; a seller with no entity cannot sell.	`routers/sales.py::create_sale` (`_check_sellable`)	Sales Executive
BR-79	A blocked tag cannot be sold, except through its own block's Sell button.	`services/control_engine.py::validate_sale_allowed`, `routers/sales.py::create_sale`	Sales Executive

ID	Rule	Source	Role
BR-80	Since 19 Oct 2026 a tag can be sold from any stage.	`services/control_engine.py::validate_sale_allowed`	Sales Executive
BR-81	A sale below the device's total cost is allowed but shows a below-cost warning.	`services/cost_engine.py::below_cost_warning_for`	Sales Executive
BR-82	Sale numbers come from one sequence (SALE-0001…) and are drawn only for tags that will actually sell.	`routers/sales.py::create_sale`	System
BR-83	The selling company is matched to the tag's entity and snapshotted on the sale, so invoices never change later.	`routers/sales.py::create_sale`, `routers/invoices.py::_sale_company_snapshot`	System
BR-84	A sale's invoice number replaces the tag's purchase invoice number.	`routers/sales.py::create_sale`	Sales Executive
BR-85	Warranty expiry = sale date + the warranty type's days.	`routers/sales.py::create_sale`	Sales Executive
BR-86	Only admin can delete sales or change a sale date; referenced sales are skipped and the tag returns to the stage it was sold from.	`routers/sales.py::bulk_delete_sales`, `change_sale_date`	Admin
BR-87	Block Tags hold Ready to Sale tags for a staff member or customer; the Block ID is `DDMMYYYY-CONTAINER-0001`, counting per date and container.	`routers/block_tags.py::create_block`, `_next_block_id`, `search_tags`	Inventory Manager, Sales Executive
BR-88	Setting a Ready to Sale price needs the "Set Price" pricing permission.	`routers/sales.py::_role_can_set_price`	Sales Executive / Manager
BR-89	The Gate Pass lists every sale with its Delivery Challan; confirming exit marks the sale "dispatched" and skips ones already dispatched.	`routers/gate_pass.py::confirm_exit`, `bulk_confirm_exit`	Sales Executive / Inventory Manager
BR-90	A telecaller's dispatch request is approved by the Sales Manager on Ready to Dispatch.	`routers/dispatch.py::approve_dispatch`	Sales Executive / Sales Manager

2.9 Returns and credit notes

ID	Rule	Source	Role
BR-91	An internal return needs a sale on file, a reason, a condition and a "Received By" person.	`routers/sales.py::process_return`	Sales Executive
BR-92	Only one return is allowed per sale.	`routers/sales.py::process_return`	Sales Executive
BR-93	A returned tag moves to Stock In at once; the return waits for manager approval.	`routers/sales.py::process_return`	Sales Executive
BR-94	Replace Now sells a Ready to Sale tag at ₹0 to the same customer; it cannot be the returned tag.	`routers/sales.py::process_return`	Sales Executive
BR-95	Only admin or Sales Manager can approve or reject a return; approval sends the tag to IQC (restock) or Scrapped.	`routers/sales.py::approve_return`, `reject_return`, `MANAGER_ROLES`	Managers
BR-96	Warranty status at return is computed by the server from the sale, never typed in.	`routers/sales.py::process_return`	System
BR-97	An External Tag return creates the tag if unknown (lot EXT-RETURNS) and a ₹0 placeholder sale, and records the paid-repair amount.	`routers/sales.py::process_external_return`	Sales Executive

ID	Rule	Source	Role
BR-98	Return Stock "Verify" records repair and labour cost; "Complete" re-sells the tag to the original customer under a new sale number.	`routers/stock.py::return_stock_verify` , `return_stock_complete`	Inventory Manager
BR-99	A credit note needs a Debit Note number and a "Received By" person, and marks the tag "Return for Credit Note".	`routers/sales.py::process_credit_note`	Sales Executive
BR-100	Credit notes move forward one step at a time through six stages; "Payment Invoice Done" needs a payment invoice number.	`routers/sales.py::_try_advance_return` , `models/sales.py::CN_STAGES`	Accounts / Sales Manager (`credit_note` edit)

2.10 Accounts

ID	Rule	Source	Role
BR-101	Supplier payments and customer receipts can be added only by admin, Inventory Manager or Sales Manager, and the amount must be above zero.	`routers/accounts.py::create_supplier_payment` , `FINANCE_ROLES`	Accounts
BR-102	The Tax Invoice and Delivery Challan print from the sale's frozen company snapshot.	`routers/invoices.py` (`/print` , `/waybill`)	Accounts
BR-103	Lot P&L and other financial reports need the financial report roles.	`routers/reports.py::_require_financials`	Accounts / Managers
BR-104	Daily Stock reports are available as overall, tag, location and daily workbook exports.	`routers/reports.py::daily_stock_report` and export routes	Accounts / TRC Manager

2.11 Warranty

ID	Rule	Source	Role
BR-105	A warranty can be set only on a tag that has a sale; the expiry is counted from the sale date.	`routers/extended_warranty.py::extended_warranty_set`	Inventory Manager / TRC Manager
BR-106	An extension date must be later than the current Warranty Stops date; the added days are kept as a running total and noted on the sale.	`routers/extended_warranty.py::extended_warranty_update`	Inventory Manager / TRC Manager

2.12 Scrap

ID	Rule	Source	Role
BR-107	The Tags Scrapped page lists every tag at Scrap for Sale, whatever its grade.	`routers/scrap.py::scrap_products`	Inventory Manager
BR-108	Verifying a scrapped tag's quantity enables its Sell button.	`routers/scrap.py::verify_scrap`	Inventory Manager
BR-109	A Replacement Scrap records which tag replaced it.	`routers/scrap.py::set_scrap_replacement` , `routers/stock.py::scrap_replace_with_another`	Inventory Manager

2.13 Audits and traceability

ID	Rule	Source	Role
BR-110	A physical audit records the stage and location seen against the system values; a tag can be audited only once per report date.	`routers/physical_audits.py::submit_audit`, `_already_audited`	TRC Manager
BR-111	Every stage move should write a StageMovement row; the previous open row gets an exit time.	`models/device.py::StageMovement`, all stage-move endpoints	System
BR-112	Only admin can edit the allowed-transitions table.	`routers/stage_control.py::add_transition`, `remove_transition`	Admin
BR-113	Each WorkID gets a Completed Date when the tag leaves that person's step.	`services/work_order_close.py`, `routers/repair.py::_close_l1l2_work_orders`	System

3. Standard operating procedures

Short daily routines. Screens are named as they appear in the app.

3.1 IQC Inspector

1. Open the lot (or create it) and register incoming tags, or use IQC → New.
2. Fill the IQC form: tag number, lot, specs, grade and location. Add the GRN number if it is known.
3. Run OxyQC on the device if the agent is used, and check the result reached IQC.
4. If tags were registered at GRN Receipt, click **Advance GRN to IQC**.
5. Before leaving, check the IQC list for tags without a GRN number and hand them to the Inventory Manager for mapping.

3.2 Inventory Manager

1. Map or validate GRNs on **GRN Post-IQC** so IQC tags move to Stock In.
2. Review **Inventory Manager (Stock)**: bucket new tags, set location, and flag urgent tags (all four flag fields).
3. Move tags to production: **Move to Production** (bucket) or **Move to L1/L2** (single or bulk). Only Stock In tags move.
4. Use **Transfers** for floor moves and assignments (Finish Good, As-Is Lot, Scrap for Sale).
5. On **Final QC → Devices Passed**, click **Move to Inventory** so tags reach Ready to Sale.
6. Work the **Return Stock** table: verify repair and labour cost, then Complete or rebucket.
7. Review **Tags Scrapped**: verify quantity and record replacements.
8. Create or release **Block Tags** when asked; set or extend warranties on **Extended Warranty**.

3.3 Production Manager

1. Open **TRC Production** and read the tiles (At You, L1/L2, L3/L4, Stress, Cosmetic, Final QC, Tags Returned).
2. Assign each bucket or tag to a department and engineer (L1/L2 Repair, Stress Test, Cosmetic Repair).
3. Clear the **Tag Number Allocation** queue, including Final QC Hardware fails with no engineer.
4. Use **Change Engineer** for absent staff; the old WorkID closes and a new one opens.
5. Review **PNA summary** and stuck tags; chase parts with the Spare Parts Manager.

6. For new lots, run **Part Estimation** and attach the estimate to raise the parts request.
7. Check **WorkID Status** for team output at the end of the day.

3.4 Repair Engineer (L1/L2 and L3/L4)

1. Open **L1/L2 Repair** (or **L3/L4**). Click **Pick This** on your next tag.
2. Click **Start Repair**.
3. Raise part requests from the tag; mark parts PNA if they cannot be sourced.
4. When a part is handed over, **Verify** it on the device page.
5. If deeper repair is needed, **Request to L3/L4** with a repair note.
6. L3/L4: finish with **Mark Complete** (choose the activity) or **Scrap** (Normal or Replacement).
7. L1/L2: clear all PNA marks, then **Mark Complete** to send the tag to Stress Test. After a scrap decision, use **Back to Inventory**.

3.5 QC Inspector (Stress Test)

1. Open **Stress Test** and take the next tag.
2. Install the OS and run the diagnostic or the stress runner if used.
3. Tick each of the 27 checklist items as Pass, Fail (or N/A for Touch Screen).
4. All pass: click **Complete**. The tag goes to Cosmetic Received.
5. Any fail: click **Fail**, pick an L1/L2 engineer and add a note. The tag goes back to L1/L2.

3.6 Cosmetic Lead

1. Open **Cosmetic & Paint** and check each stage tab.
2. On Cosmetic Received, **Bulk Assign** the day's tags to the Cleaning team (or Move one at a time).
3. Each stage worker clicks **Move** when done and picks the next person (or Flow Data picks them).
4. Use **Skip to Final QC** for tags that need no further cosmetic work.
5. If a tag has a hardware problem at Received, Cleaning, Water Sanding or Completed, click **Fail** and assign an L1/L2 engineer.
6. Keep Flow Data rows up to date when teams change.

3.7 Final QC Inspector

1. Open **Final QC** and take the next tag.
2. Run the manual diagnosis. Correct specs if needed (or use Revalidate IQC).
3. Pass: choose the grade, touch type and bucket name, add pass notes, submit.
4. Fail: choose the reason (Hardware, Software or Cosmetic), add notes, submit.
5. On **Devices Failed**, click **Move** or **Bulk Move** to send each tag back.
6. On **Devices Passed**, click **Move to Inventory** to hand tags to the Inventory Manager.

3.8 Spare Parts Manager

1. Open **Part Master → Requests**. Hand over parts in stock (bulk handover for many).
2. Mark **Not In Stock** or **Procure** for parts you cannot supply.
3. On **Sourcing Requests**, verify closed deals and confirm production requests.
4. Record new stock on **Parts GRN** (As New) with PO, invoice and e-way bill.

5. Record parts taken from old devices with **Add Harvest Part**.
6. Handle faulty parts: Scrap, or send under warranty and receive the replacement.

3.9 Sales Executive

1. Review **Ready to Sale**; check prices and blocked tags.
2. Block tags for a customer or colleague on **Block Tags** when asked.
3. Record sales on **New Sale** / **New Tag Sale**, or **Sell** from a block. Check entity warnings.
4. Print the invoice and Delivery Challan; confirm exit on **Gate Pass**.
5. Log customer returns on **Return New**: Internal Tag, External Tag (paid repair) or Credit Note. Use Replace Now for replacements.
6. Raise dispatch requests (telecallers) and review the sales report.

3.10 Accounts

1. Record **Customer Receipts** against sales and **Supplier Payments** against lots or POs.
2. Upload invoices on POs and quotes; complete payments.
3. Advance **Credit Notes** through Debit Note Verified, Verify Payment and Payment Invoice Done.
4. Check the **Sales** list and the **Daily Stock** report.

3.11 Managers / Admin

1. Read the **Dashboard** and key reports (Lot P&L, Sales, Stage Movement).
2. Approve or reject **Pending Returns**.
3. Handle **Tickets** raised by users.
4. Maintain users, roles, Master Data and the Module Permission matrix.
5. Admin only: correct sales (delete or change date) and edit **Stage Control** transitions.

3.12 TRC Manager

1. Review TRC Production, Inventory and Part Master to see the whole floor.
2. Check the **Daily Stock** report.
3. Run **Physical Audits** for the day's stages and locations; resolve mismatches.
4. Set or extend warranties on **Extended Warranty** when approved.
5. Escalate stuck tags (PNA, long WorkIDs) to the Production and Spare Parts Managers.

4. Open questions

#	Question	Code fact	Business view
OQ1	Should sales be limited to Ready to Sale?	Any stage can be sold, including a tag already Sold (`services/control_engine.py::validate_sale_allowed` , `routers/sales.py::create_sale`)	Sales Executive sells Ready for Sale items
OQ2	What happens to a rejected return?	The tag is already in Stock In and stays there; the `returned` stage is never used (`routers/sales.py::process_return` , `reject_return`)	Return, paid repair, replacement or credit note

#	Question	Code fact	Business view
OQ3	Who are Production Manager, Final QC Inspector, Accounts and TRC Manager in the system?	No built-in roles; Production Manager = `inventory_manager` ; Accounts pages hard-code `admin` , `inventory_manager` , `sales_manager` , so a custom Accounts role is refused (`routers/accounts.py::FINANCE_ROLES`)	12 distinct roles
OQ4	Should IQC Inspectors run GRN?	`/grn` allows only `admin` and `inventory_manager` (`routers/grn.py`)	IQC Inspector owns GRN, Lot, IQC
OQ5	Should stage moves check the current stage and the user's role?	Many moves accept any signed-in user and any current stage (for example `routers/repair.py::l1l2_complete_to_stress` , `routers/stress_api.py::stress_fail_assign` , `routers/stock.py::move_to_stock`)	Only the owning role moves a tag forward
OQ6	How are e-way bills generated for sales?	E-way bill fields exist only on inbound Parts GRN and lot GRN; no sale e-way bill flow found	Accounts generates e-way bills
OQ7	Where are rentals handled?	No rental flow found in sales, dispatch or gate pass	Sales Executive handles rentals
OQ8	Should a scrap sale move tags out of Scrap for Sale?	Selling a Scrap for Sale batch marks the batch sold only (`routers/scrap_for_sale.py::sell_scrap_for_sale`)	Inventory Manager handles scrapped tags
OQ9	Should harvest parts link to the source tag?	Harvest records a free-text lot number, not the device (`routers/parts_grn.py::harvest_part`)	Harvest parts come from old devices
OQ10	Which grade scale is official?	Device grade A–E/Scrap; legacy QC check grade A/B/C/D/S (`models/qc.py`)	Final QC assigns grading
OQ11	Should Block Tags be audited and stage-checked on the server?	No audit on create/release; Ready to Sale is checked only in the search box (`routers/block_tags.py::create_block`)	Block tags for any staff
OQ12	Should part request actions check the current status?	Handover, Not In Stock, Procure and Scrap do not check status on single rows (`routers/part_requests.py`)	Spare Parts Manager actions requests
OQ13	Is dispatch approval required before a telecaller sells?	The rule is in the dispatch module text; `create_sale` does not check it	—



OXYPC INVENTORY LIFECYCLE

ARCHITECTURE

Access & Security

Who can see and do what, and how the app protects its data.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Access & Security

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

In this document: Part A — Role & Permission Matrix · Part B — Security & Access Design

PART A

Role & Permission Matrix

This document shows who can open which page and who does what in the OxyPC Inventory Lifecycle app. It is built from the code, not from the database. The live Permission Matrix in production can narrow (or widen) what is shown here. Every row cites the file that enforces it. Anything the code cannot prove is listed under **Open questions**.

---

1. How permissions work

The app has three layers that stack on top of each other. **Built-in roles** are the 12 values of `UserRole` (`models/user.py:13-25`). Each router has a hard-coded allow-list of built-in roles (`require_roles(...)` in `auth/dependencies.py:175-198`, or an in-handler check such as `if current_user.role not in FINANCE_ROLES` in `routers/accounts.py:31,64`). Admin always passes. **Custom roles** are free-text role names an admin creates (`CustomRole`, `models/role_permissions.py:37-45`; users store the role as text via `RoleType`, `models/user.py:53-65`). A custom role passes every `require_roles` gate that is not admin-only (`auth/dependencies.py:188-189`), but it does **not** pass in-handler checks that compare against built-in roles. `sub_admin` is seeded as a custom role on every start-up (`main.py:582-597`); `trc_manager` exists in the production database (CLAUDE.md) but is not created by code. **The per-page Permission Matrix** (Admin Settings → Master Data → Permission Matrix, `routers/master.py:841-895`) stores one row per role and module (`RoleModulePermission`, `models/role_permissions.py:19-34`). It drives the sidebar (`has_perm()` in `templates/base.html`) and module-gated routes (`require_module_perm`, `auth/dependencies.py:208-232`). It is **permissive by default**: a role or module with no row is allowed (`has_perm`, `models/role_permissions.py:221-240`), except the default-off modules (`DEFAULT_DISABLED_MODULES`, `models/role_permissions.py:215-218`). Only the **Enable** tick is enforced; the Add/Edit/Upload ticks are display-only (`models/role_permissions.py:225-230`). An explicit Enable row can also *widen* a built-in role past a router allow-list, including admin-only pages (`_matrix_grants_path`, `auth/dependencies.py:147-172, 190-196`). A separate tab, **Role Additional Permissions**, holds app-wide switches (Add Lot/IQC/GRN, Edit Devices/IQC/Lot/GRN, Pricing Visibility, Set Price, Upload/Download/Export/Print) — also permissive by default (`has_additional_perm`, `models/role_permissions.py:177-186`).

Practical rule: a page is reachable when **the route gate lets the role in** (hard-coded list, custom-role pass-through, or explicit matrix grant) **and** no matrix row disables it. The sidebar link is shown by `has_perm()` alone, so a link can appear for a built-in role that the route then refuses with 403.

2. Persona → role mapping

#	Persona (business)	Role value in code	Label shown in app	Notes
1	IQC Inspector	`iqc_inspector`	IQC Handler	`models/user.py:16,31`
2	Inventory Manager	`inventory_manager`	Inventory Manager	`models/user.py:15,30`
3	Production Manager	**None.** `inventory_manager` acts as Production Manager	—	"production_manager is NOT a role in this system — it is a module/page name" (`routers/repair.py:47-52`). The string is listed in `FULL_QUEUE_ROLES` in case a custom role of that name is created (`routers/repair.py:59`).
4	Repair Engineer L1/L2	`l1_engineer` (`l2_engineer` is legacy)	L1/L2 Engineer, L2 Engineer	`models/user.py:17-18,32-33` . L2 pages are retired (`routers/repair.py:40-43`).
4	Repair Engineer L3/L4	`l3_engineer`	L3/L4 Engineer	`models/user.py:19,34`
5	QC Inspector (Stress Test, OS install, diagnose)	`qc_inspector`	QC Handler	`models/user.py:20,35`
6	Cosmetic Lead	`cosmetic_manager`	Cosmetic Manager	`models/user.py:25,40` . Missing from `_ROLE_DISPLAY` in `templates_config.py:261-273` and from `ROLE_PERMISSIONS` in `auth/dependencies.py:16-41` .
7	Final QC Inspector	**None.** Final QC page admits `qc_inspector` and `cosmetic_manager`	—	`routers/cosmetic.py:40-41`
8	Spare Parts Manager	`spare_parts_manager`	Parts Manager	`models/user.py:22,37`
9	Sales Executive	`sales` , `sales_manager` , `telecaller`	Sourcing Sales, Sales Manager, Telecaller Sales	`models/user.py:21,23-24`

#	Persona (business)	Role value in code	Label shown in app	Notes
10	Accounts	**None.** Finance pages admit `admin`, `inventory_manager`, `sales_manager` only	—	`routers/accounts.py:31`
11	Managers / Admin	`admin` (plus custom `sub_admin`)	Admin / Sub Admin	`models/user.py:14`; `main.py:582-597`
12	TRC Manager	custom role `trc_manager`	TRC Manager (prettified)	Not seeded by code; referenced in `routers/repair.py:59` and `models/role_permissions.py:248-269`.

3. Page access matrix (code defaults)

3.1 How to read the table

Cells

Cell	Meaning
F	Full: can open the page and use its actions.
V	View: can open the page; actions are refused or not offered.
—	Refused (403 or redirect) by code. For a built-in role, an explicit matrix grant can still open it (`auth/dependencies.py:190-196`).
C	Configurable only: off by default; admin must enable it in the matrix.
U	Link hidden in the sidebar, but the URL itself opens for any signed-in user.
a/b	Engineer column: L1/L2 / L3/L4.
a/b/c	Sales column: `sales` / `sales_manager` / `telecaller`.

Gate type (the "Gate" column says where every cell in that row comes from)

Gate	Source of the cell values
H	Hard-coded role check in the router (`require_roles` or an in-handler role list).

Gate	Source of the cell values
O	Open to any signed-in user (`get_current_user` only).
M	Module permission (`require_module_perm` / `has_perm`); permissive until an admin unticks it. Cells show the default **F**.
S	Seeded default: a migration script wrote `can_enable=False` rows for every non-admin role.
D	Default-disabled in code (`DEFAULT_DISABLED_MODULES`). Configurable only.
A	Admin-only route; any role can be granted it with an explicit matrix row.

Columns. Prod Mgr uses `inventory_manager` values (see §2). Final QC, Accounts and TRC Mgr have no built-in role, so their columns show what **a custom role** gets from code before any matrix row narrows it. For `trc_manager` the production matrix is known to be configured (`models/role_permissions.py:248-269` mentions "~100 OTHER modules"), so its real access may be narrower.

3.2 Matrix

Sidebar section › Page (label in `templates/base.html`)	Gate	Source	IQC	Inv Mgr	Prod Mgr	Repair Eng	QC	Cosmetic	Final QC	Spares	Sales S/M/T	Accounts	Admin	TRC Mgr
Admin Dashboard	O	`routers/dashboard.py` (`/dashboard`); role widgets `:534-604`	V	V	V	V	V	V	V	V	V/V/V	V	F	V
Inventory Search	O view, H edit	`routers/devices.py:39-40` , `:2099`	V	F	F	V	V	V	F	V	V/V/V	F	F	F
TRC Dashboard	H	`routers/dispatch.py:31-32`	—	F	F	—	—	—	F	—	F/F/F	F	F	F
ATTENDANCE › My Attendance	O	`routers/attendance.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
ATTENDANCE › Attendance Report	O (wider scope for privileged)	`routers/attendance.py:23-24,418-428`	V	F	F	V	V	V	V	V	V/V/V	V	F	V
INTAKE › GRN with Invoice	H	`routers/grn.py:32` ; Add/Edit GRN switches `:964,1103`	—	F	F	—	—	—	F	—	—/—/—	F	F	F

Sidebar section › Page (label in `templates/base.html`)	Gate	Source	IQC	Inv Mgr	Prod Mgr	Repair Eng	QC	Cosmetic	Final QC	Spares	Sales S/M/T	Accounts	Admin	TRC Mgr
INTAKE › GRN post IQC	H	`routers/grn.py:32`	—	F	F	—	—	—	F	—	—/—/—	F	F	F
INTAKE › Lot Overview	H	`routers/stock.py:47-48` ; Add/Edit Lot `:433,550`	—	F	F	—	F	—	F	—	—/F/—	F	F	F
INTAKE › IQC Line Items	O list + new IQC; H export/bulk edit	`routers/iqc.py:1405-1422` (open entry); `:170-171` , `:927`	F	F	F	V[1]	V[1]	V[1]	F	V[1]	V[1]/F/V[1]	F	F	F
INTAKE › GRN Records	H	`routers/grn.py:32`	—	F	F	—	—	—	F	—	—/—/—	F	F	F
INTAKE › Entity Movement	M	`routers/entity_movement.py:32`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
INVENTORY › Stock Inwards	H	`routers/stock.py:47-48`	—	F	F	—	F	—	F	—	—/F/—	F	F	F
INVENTORY › Scrap Products	H	`routers/scrap.py:20-21`	—	F	F	—/F	—	—	F	—	F/F/—	F	F	F
INVENTORY › Inventory Request	H view/raise; approve open	`routers/dispatch.py:31-37`	—	F	F	—	—	—	F	—	F/F/F	F	F	F
INVENTORY › Model Requested	O	`routers/model_requested.py`	V	V	V	V	V	V	V	V	V/V/V	V	F	V
INVENTORY › Production Manager	H page; H bucket assign	`routers/stock.py:47-48` (`/trc-production`); `routers/buckets.py:26`	—	F	F	—	V	—	F	—	—/V/—	F	F	F
INVENTORY › Extended Warranty	M	`routers/extended_warranty.py:37`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
INVENTORY › Part Estimation	M	`routers/part_estimation.py:45`	F	F	F	F	F	F	F	F	F/F/F	F	F	F

Sidebar section › Page (label in `templates/base.html`)	Gate	Source	IQC	Inv Mgr	Prod Mgr	Repair Eng	QC	Cosmetic	Final QC	Spares	Sales S/M/T	Accounts	Admin	TRC Mgr
INVENTORY › Move Device	O list; H create	`routers/transfers.py:28-29`	V	F	F	V	F	V	F	V	V/F/V	F	F	F
REPAIR › L1 Repair	O page; M actions	`routers/repair.py:966-969`; `:229,1369,1453`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
REPAIR › L3/L4 Repair	O page; M actions	`routers/repair.py:687-690`; `:826`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
REPAIR › Stress Test	H	`routers/qc.py:28-29`; `qc_check` add	—	—	—	F/F	F	—	F	—	—/F/—	F	F	F
REPAIR › Cosmetic & Paint / Cosmetic Stage	H	`routers/cosmetic.py:40-41`; stage roles `:221-240`	—	F	F	—	F	F	F	—	—/F/—	F	F	F
REPAIR › Final QC	H	`routers/cosmetic.py:40-41`; `cosmetic_finalqc`	—	F	F	—	F	F	F	—	—/F/—	F	F	F
REPAIR › WorkID Status	O	`routers/workid_status.py`	V	V	V	V	V	V	V	V	V/V/V	V	F	V
STORE MANAGER › Parts Dashboard / Parts Sale Request	H	`routers/part_sales.py:40-41,263,577`	—	—	—	—	—	—	F	F	F/F/—	F	F	F
STORE MANAGER › Parts Manager	H	`routers/spare_parts.py:32-33`	—	—	—	—	F	—	F	F	—/F/—	F	F	F
STORE MANAGER › Parts GRN	H	`routers/parts_grn.py:26`	—	—	—	—	—	—	F	F	—/—/—	F	F	F
Inventory Locations › Location Map	O	`routers/inventory_location.py` (`/locations/dashboard`)	F	F	F	F	F	F	F	F	F/F/F	F	F	F
Inventory Locations › Physical Audits	O	`routers/physical_audits.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F

Sidebar section › Page (label in `templates/base.html`)	Gate	Source	IQC	Inv Mgr	Prod Mgr	Repair Eng	QC	Cosmetic	Final QC	Spares	Sales S/M/T	Accounts	Admin	TRC Mgr
Inventory Locations › Gap Alerts	O	`routers/inventory_location.py` (`/locations/gaps`)	V	V	V	V	V	V	V	V	V/V/V	V	F	V
Inventory Locations › Zone Audit	H create/close; O scan	`routers/inventory_location.py` (`/locations/audit*`)	V	F	F	V	V	V	F	V	V/V/V	F	F	F
Inventory Locations › Manage Locations	H	`routers/inventory_location.py` (`/locations/master*`)	—	F	F	—	—	—	F	—	—/—/—	F	F	F
Inventory Locations › Trash	H	`routers/trash.py:27`	—	F	F	—	—	—	F	—	—/—/—	F	F	F
CRM › Procure Dashboard	O view; H actions	`routers/procure_dashboard.py:31,48`	V	V	V	V	V	V	V	V	V/F/V	V	F	V
CRM › Contact Leads	O	`routers/crm_contacts.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
CRM › Sourcing Deals	O	`routers/crm_sourcing.py:33,68`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
CRM › Sales Opportunities / Quotes	O	`routers/crm_sales.py:58` ; `routers/crm_quotes.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
CRM › Price Matrix	O view; H edit (in-handler)	`routers/crm_price_matrix.py:25,55,67`	V	F	F	V	V	V	V	V	V/F/V	V	F	V
CRM › Purchase Orders	H (in-handler)	`routers/crm_purchase_orders.py:26-29,58,87`	—	F	F	—	—	—	—	—	V/F/V	—	F	—
CRM › CRM Analytics	O	`routers/crm_reports.py`	V	V	V	V	V	V	V	V	V/V/V	V	F	V

Sidebar section › Page (label in `templates/base.html`)	Gate	Source	IQC	Inv Mgr	Prod Mgr	Repair Eng	QC	Cosmetic	Final QC	Spares	Sales S/M/T	Accounts	Admin	TRC Mgr
CRM › Assign Social Leads	O	`routers/crm_assign_leads.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
EXTERNAL PARTNER › Partner Accounts, Listings Manager, Bids Created, Manage Lots, Bookings Queue, My Desk, Portal Settings	S	`migrate_seed_trade_partner_split_perms.py` ; `routers/partner_admin.py`	—	—	—	—	—	—	—	—	—/—/—	—	F	—
EXTERNAL PARTNER › Margin Floors	S + sidebar admin-only	`templates/base.html:407`	—	—	—	—	—	—	—	—	—/—/—	—	F	—
EXTERNAL PARTNER › All Deals	M (not in the seed list)	`routers/deals.py:23`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
CUSTOMER CARE › Ticket Queue, Device Pairings, Provision Device, Offers, Pilot Dashboard	M	`routers/care_admin.py` (`care_support`)	F	F	F	F	F	F	F	F	F/F/F	F	F	F
SALES & CRM › TeleSales Dashboard	M	`routers/telesales_dashboard.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
SALES & CRM › Dealers	H	`routers/dealers.py:73-77` ; manage `:80-100`	—	—	—	—	—	—	F	—	F/F/F	F	F	F
SALES & CRM › Telecalling	O (own records unless manager)	`routers/telecalling.py:79,234,448`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
SALES & CRM › Quotations	M	`routers/quotations_list.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
SALES & CRM › Model Requests	O list; H raise/fulfil	`routers/model_requests.py` (`REQUEST_ROLES` , `FULFIL_ROLES`)	V	F	F	V	V	V	F	V	F/F/F	F	F	F

Sidebar section › Page (label in `templates/base.html`)	Gate	Source	IQC	Inv Mgr	Prod Mgr	Repair Eng	QC	Cosmetic	Final QC	Spares	Sales S/M/T	Accounts	Admin	TRC Mgr
SALES & CRM › Assign Dealer Leads	M	`routers/assign_dealer_leads.py:26`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
SELLING › Ready to Sale	O view; H sell; Set Price switch	`routers/sales.py:79,85,1395` ; `:718-742,940,980`	V	V	V	V	V	V	F	V	F/F/F	F	F	F
SELLING › Ready to Sale Parts	H	`routers/part_sales.py:40-41,128`	—	—	—	—	—	—	F	F	F/F/—	F	F	F
SELLING › Scrap for Sale	H	`routers/scrap_for_sale.py:33`	—	F	F	—	—	—	F	—	—/F/—	F	F	F
SELLING › Sales List	H; hard delete admin-only	`routers/sales.py:79` ; `:1780-1805`	—	—	—	—	—	—	F	—	F/F/F	F	F	F
SELLING › Spare Part Sales	H	`routers/part_sales.py:40-41`	—	—	—	—	—	—	F	F	F/F/—	F	F	F
SELLING › Scrap Sale Report	D	`models/role_permissions.py:215-218`	C	C	C	C	C	C	C	C	C/C/C	C	F	C
SELLING › Block Tags	M	`routers/block_tags.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
SELLING › Gate Pass	H	`routers/gate_pass.py:38-39`	—	F	F	—	—	—	F	—	F/F/—	F	F	F
SELLING › Credit Note	M	`routers/sales.py:3230-3479`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
SELLING › Returns	H; approve/reject in-handler	`routers/sales.py:79,2554-2681` ; `:3521,3545-3553`	—	—	—	—	—	—	F²	—	F²/F/F²	F²	F	F²
FINANCE › Accounts, Supplier Payments, Customer Receipts	H (in-handler)	`routers/accounts.py:31,64-65,109`	—	F	F	—	—	—	—	—	—/F/—	—	F	—
FINANCE › Partner Payments	H	`routers/partner_payments.py:28`	—	—	—	—	—	—	F	—	—/F/—	F	F	F

Sidebar section › Page (label in `templates/base.html`)	Gate	Source	IQC	Inv Mgr	Prod Mgr	Repair Eng	QC	Cosmetic	Final QC	Spares	Sales S/M/T	Accounts	Admin	TRC Mgr
Terms & Conditions	A	`routers/terms_conditions.py:16` ; `templates/base.html:565`	C	C	C	C	C	C	C	C	C/C/C	C	F	C
Company Setting	H (in-handler, admin)	`routers/settings.py:55,93,142` ; `templates/base.html:570-574`	—	—	—	—	—	—	—	—	—/—/—	—	F	—
Cost Config	M	`routers/admin.py:545,563`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
PRODUCT MASTER › Product Master, Pricing Calculator, Hardware, Spare Parts, Product Pricing	D	`models/role_permissions.py:215-218` ; `routers/product_master.py`	C	C	C	C	C	C	C	C	C/C/C	C	F	C
REPORTS › Business P&L, Lot P&L	H (two gates)	`routers/reports.py:43-47,60-63,66-69`	—	F	F	—	—	—	F	—	—/—[3]/—	F	F	F
REPORTS › Sales Report, Daily Stock, Stage Log	H (router gate)	`routers/reports.py:43-47,66-69`	—	F	F	—	F	—	F	—	F/—[3]/—	F	F	F
ADMIN › Users, Master Data, Sidebar Config, Module Page Titles, Group Config	A	`routers/admin.py:100` ; `routers/master.py:25` ; `routers/sidebar_config.py` ; `routers/landing_pages.py` ; `routers/attendance_group_config.py:23`	C	C	C	C	C	C	C	C	C/C/C	C	F	C
ADMIN › Move Device Internal	O route; link admin/explicit	`routers/repair.py:1772,1790` ; `templates/base.html:691`	U	U	U	U	U	U	U	U	U/U/U	U	F	U

Sidebar section › Page (label in `templates/base.html`)	Gate	Source	IQC	Inv Mgr	Prod Mgr	Repair Eng	QC	Cosmetic	Final QC	Spares	Sales S/M/T	Accounts	Admin	TRC Mgr
ADMIN › QA Dashboard	O view, H edit; link admin-only	`routers/qa_uat.py:54-56` ; `templates/base.html:719-723`	U	U	U	U	U	U	U	U	U/U/U	U	F	U
Application Settings › Stage Control, Stage Audit Log	A	`routers/stage_control.py:23`	C	C	C	C	C	C	C	C	C/C/C	C	F	C
Application Settings › Aging Tracker	O	`routers/stage_control.py` (`/stage-control/aging`)	V	V	V	V	V	V	V	V	V/V/V	V	F	V
Application Settings › System Audit Log	A	`routers/admin.py:444-451`	C	C	C	C	C	C	C	C	C/C/C	C	F	C
Application Settings › Stock Aging, Overdue Devices	H (router gate)	`routers/reports.py:43-47`	—	F	F	—	F	—	F	—	F/—[3]/—	F	F	F
Application Settings › Receivables	H (two gates)	`routers/reports.py:50-53`	—	F	F	—	—	—	F	—	—/—[3]/—	F	F	F
Application Settings › Market Intel	O	`routers/market.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F
Application Settings › Parts Purchased, Parts Tracking	H	`routers/spare_parts.py:32-33`	—	—	—	—	F	—	F	F	—/F/—	F	F	F
Application Settings › Parts Consumption	O	`routers/spare_parts.py` (`/spare-parts/consume`)	F	F	F	F	F	F	F	F	F/F/F	F	F	F
NOTIFICATIONS › Alerts; SUPPORT › Tickets Raised, Manuals	O	`templates/base.html:821-841` ; `routers/notifications.py` , `routers/tickets.py` , `routers/manuals.py`	F	F	F	F	F	F	F	F	F/F/F	F	F	F

Footnotes

1. **V**[1] — the role can open the IQC list and register a new IQC device (open to every signed-in user, `routers/iqc.py:1412-1422`), but cannot export or bulk-edit grades (`routers/iqc.py:170-171,927`).
2. **F**[2] — can view and raise returns; only `admin` and `sales_manager` can approve or reject (`routers/sales.py:3521,3553`). Custom roles are refused at that step because the check compares built-in roles.
3. **—**[3] — `sales_manager` is in the financial gate (`routers/reports.py:60-63`) and the receivables gate (`:50-53`) but **not** in the router-level gate (`:43-47`) that every `/reports` page also passes. By code reading, a `sales_manager` with no explicit matrix row for a reports module gets 403. QA test TC-254-1 expects 200 for `sales_manager` (`seed_qa_july16_batch.py:247-250`), so production likely relies on a matrix grant. See Open questions.

What the table does **not** show:

- **Sidebar vs route.** For every **H** row, the sidebar link still appears for a built-in role that has no matrix row, because `has_perm()` is permissive (`models/role_permissions.py:234-238`). The user then sees the 403 page (`main.py:381-383`).
- **Global Visibility.** An admin can hide any module from everyone, admin included (`_MODULE_HIDDEN_CACHE`, `models/role_permissions.py:151-166`; `templates_config.py:225-233`).
- **WA Audit Log / WhatsApp.** The WhatsApp router is no longer registered (`main.py`, "WhatsApp integration removed"), so these links lead nowhere.
- **Add/Edit/Upload ticks.** They are saved but do not restrict anything (`models/role_permissions.py:225-230`; only "Enable" is offered, `routers/master.py:303-305`).

4. Sensitive data rules

Data	Rule in code	Where it is enforced	Gaps found
Buying price, sale price, cost, margin (screens)	**Pricing Visibility** switch per role (`can_view_pricing`, default on for every role). Admin always sees prices.	Column `RoleAdditionalPermission.can_view_pricing` (`models/role_permissions.py:83`); check `can_view_pricing()` (`:189-193`); saved by `routers/master.py:946-986`. Used in 21 templates (e.g. `templates/sales/list.html:171`, `templates/lots/list.html:81-117`, `templates/iqc/list.html:195-223`) and in server JSON (`routers/sales.py:218,2112`; `routers/devices.py:527,1190`).	Not applied to CSV exports: `/sales/export` (`routers/sales.py:2331`), `/sales/export-selected` (`:1953`), `/reports/export/sales` (`routers/reports.py:1121`), `/reports/export/lot-pl` (`:870`).

Data	Rule in code	Where it is enforced	Gaps found
Setting a selling price	**Set Price** switch per role (default on).	`routers/sales.py:718-742,940,980` (server-side 403).	—
Company and lot P&L	Only `admin`, `inventory_manager`, `sales_manager` by the financial gate. The page also hides content when Pricing Visibility is off.	`routers/reports.py:55-63` ("Flagged in the 2026-07-15 RBAC audit"); `templates/reports/lot_pl.html:5-8`; QA defect BUG-212 (`seed_qa_july16_batch.py:348-352`).	Custom roles pass the financial gate (`auth/dependencies.py:188-189`).
Supplier payments, customer receipts	Only `admin`, `inventory_manager`, `sales_manager`.	`routers/accounts.py:31,64-65` (in-handler).	A custom "Accounts" role cannot open these pages.
Cost Config (cost rates)	Module `cost_config`, permissive by default.	`routers/admin.py:545,563`.	Open to every role until an admin unticks it.
Margin floors (partner portal)	Admin-only link; module seeded off.	`templates/base.html:407`; `migrate_seed_trade_partner_split_perms.py`.	—
Exports, downloads, uploads, printing	App-wide switches exist (File Upload/Download/Export/Print).	Stored and cached only (`routers/master.py:313-325,898-940`).	**No route or template reads them.** Exports are limited only by the page's own gate and a 5,000-row cap (`routers/reports.py:40`).
Per-user permissions (`UserPermission`)	Admin can tick per-user grants such as "View Device Buying / Cost Data".	Stored by `routers/admin.py:375-430`; model `models/user.py:133-143`.	**Not enforced anywhere** (only `routers/admin.py` reads the table).
Hard delete of sales	Admin only; audited with old values.	`routers/sales.py:1780-1805`.	—

5. RACI for key business events

R (Responsible) comes from code: the roles the routes let perform the action. **A, C, I** are **proposed** from the persona list; the code does not model accountability or notifications for these events. Please confirm.

Event	R — who code lets do it	A (proposed)	C (proposed)	I (proposed)	Audit record written
Lot intake (GRN with invoice, create lot)	GRN: `admin`, `inventory_manager` (`routers/grn.py:32`). Create lot: `admin`, `inventory_manager`, `qc_inspector`, `sales_manager` + Add Lot switch (`routers/stock.py:47,433`). Custom roles pass both.	Inventory Manager	Accounts (invoice)	Production Manager, IQC	`GRN_SUBMITTED`, `GRN_CREATED_MANUAL`, `LOT_CREATED` (`services/audit_engine.py` callers in `routers/grn.py`, `routers/stock.py`)
IQC registration	Any signed-in user + Add IQC switch (`routers/iqc.py:1412-1422`).	IQC Inspector	Inventory Manager	Production Manager	`DEVICE_IQC_REGISTERED` (`routers/iqc.py:1648`)
Repair (L1/L2, L3/L4)	Any signed-in user whose matrix does not disable `repair_l1` / `repair_l3` (`routers/repair.py:229,826,1369,1453`). Part requests: `admin`, `inventory_manager`, L1/L2/L3 engineers (`routers/part_requests.py:29`); hand-over: `admin`, `spare_parts_manager`.	Production Manager	Spare Parts Manager	QC Inspector	`REPAIR_STARTED`, `REPAIR_COMPLETE`, `REPAIR_PNA`, `STAGE_MOVED`
QC pass / fail (Stress Test)	`admin`, `qc_inspector`, `sales_manager`, L1/L2/L3 engineers (`routers/qc.py:28-29`) + `qc_check` module.	QC Inspector	Repair Engineer	Production Manager	`QC_PASS` / `QC_FAIL` with score and grade (`routers/qc.py:321-325`); `STRESS_FAIL_ASSIGNED` (`routers/stress_api.py:643`)
Grading	At IQC entry (any signed-in user); bulk grade/type: `admin`, `inventory_manager`, `iqc_inspector`, `sales_manager` + Edit Devices/IQC switch (`routers/iqc.py:170,927`); Final QC edit: cosmetic gate (`routers/cosmetic.py:40`).	QC / Final QC Inspector	Cosmetic Lead	Sales	`IQC_BULK_GRADE_TYPE_APPLIED` (`routers/iqc.py:1166`)
Sale	`admin`, `sales`, `sales_manager`, `telecaller` + `sales` module (`routers/sales.py:79,1395`).	Sales Manager	Accounts	Inventory Manager	`SALE_CREATED`; event `SALE_COMPLETED`
Gate pass (exit)	`admin`, `sales_manager`, `sales`, `inventory_manager` (`routers/gate_pass.py:38-39`).	Inventory Manager	Sales	Accounts	`GATE_EXIT_CONFIRMED`, `GATE_EXIT_BULK_CONFIRMED` (`routers/gate_pass.py:364,411`)
Return / credit note	Raise: sales roles + `returns` module (`routers/sales.py:2665-2681`). Approve/reject: `admin`, `sales_manager` (`:3545-3553`). Credit-note stages: `credit_note` module, open by default (`:3335-3341`).	Sales Manager	Accounts, QC	Inventory Manager	`RETURN_SUBMITTED`, `RETURN_APPROVED`, `RETURN_REJECTED`, `CREDIT_NOTE_STAGE_ADVANCED`
Warranty extension	`extended_warranty` module, open by default (`routers/extended_warranty.py:37`).	Sales Manager	Accounts	Customer Care	`WARRANTY_SET`, `WARRANTY_EXTENDED` (`routers/extended_warranty.py:118,159`)

Event	R — who code lets do it	A (proposed)	C (proposed)	I (proposed)	Audit record written
Stock audit (physical / zone)	Physical Audits: any signed-in user (`routers/physical_audits.py`). Zone audit create/close: `admin` , `inventory_manager` ; scan: any (`routers/inventory_location.py`).	Inventory Manager	Production Manager	Admin	Stored as audit records (`models/audit_record.py`); `STOCK_VALIDATE` for stock validation
System audit review	System Audit Log: admin, or explicit grant (`routers/admin.py:444-451`).	Admin	—	Director	Read-only view of `audit_logs`

6. Open questions

1. **Production matrix.** What rows does the live `role_module_permissions` table hold per role? This document shows code defaults only. An export of that table would let us replace defaults with real values.
2. **Missing roles.** Should Production Manager, Final QC Inspector and Accounts become built-in roles or custom roles? Today a custom "Accounts" role is refused by `routers/accounts.py:64-65` , and a custom role gets almost every non-admin page by default (`auth/dependencies.py:188-189`).
3. `sub_admin` **default.** `main.py:585-587` says a fresh `sub_admin` "sees nothing extra". By code reading it gets every permissive module and passes every non-admin `require_roles` gate. Is that intended?
4. `sales_manager` **and reports.** Should `sales_manager` be added to the router-level reports gate (`routers/reports.py:43-47`), or is the matrix grant the intended route?
5. **Admin-equivalent grants.** An explicit grant of `admin_master` or `admin_users` opens the Permission Matrix or User Management (via `routers/landing_pages.py:176-177` and `auth/dependencies.py:166-168`). Who may hold these?
6. **Repair pages.** `stage_allowed()` (`routers/repair.py:76-85`) is defined but not used, so L1 and L3/L4 pages are open to every role. Is that intended?
7. **Pricing on exports.** Should Pricing Visibility also strip price columns from CSV exports?
8. **Export/Download/Print switches.** Should these be enforced, or removed from the UI?
9. **Cost Config and Credit Note defaults.** Should these sensitive modules start off (add to `DEFAULT_DISABLED_MODULES`)?
10. `trade_partner_all_deals` **.** It is missing from the seed list in `migrate_seed_trade_partner_split_perms.py` , so All Deals is open by default. Intended?
11. **RACI.** Please confirm the proposed A / C / I columns in §5.

PART B

Security & Access Design

This document explains how the OxyPC Inventory Lifecycle app signs people in, decides what they may do, protects forms, limits abuse, records what happened, and stores secrets and files. It also lists the security risks found while reading the code. Every claim cites a file. Anything the code cannot prove is listed under **Open questions**. For the page-by-page access table, see Role & Permission Matrix.

1. Summary

Area	How it works today	Main source
Staff login	Username + password, bcrypt, JWT in an `httponly` cookie	`routers/auth.py:54-122` ; `auth/dependencies.py:44-90`
Session length	24 hours (1,440 min), fixed in code; can be extended from the page	`config.py:29-33` ; `routers/auth.py:208-234`
Authorisation	Hard-coded role lists + custom roles + per-page Permission Matrix (cached in memory)	`auth/dependencies.py:175-303` ; `models/role_permissions.py`
CSRF	Double-submit cookie (`csrf_token`), `SameSite=strict` cookies	`auth/dependencies.py:306-343`
Rate limiting	SlowAPI: 2,000/min default; 20/min login per IP; 5 failed logins lock an account for 15 min	`limiter.py:74` ; `routers/auth.py:37-38,55`
Audit trail	`audit_logs` table written by `audit()` ; `login_logs` for logins; `event_log` for webhook events	`services/audit_engine.py` ; `models/engines.py:107-125` ; `models/user.py:120-130` ; `models/event_log.py`
Partner portal	Separate dealer login and JWT cookie with `typ: partner`	`auth/partner_auth.py`
Machine access	Hashed, scoped API keys (`/api/v1`), a static OxyQC key, device tokens for the Care Agent	`auth/api_key.py` ; `routers/iqc_api.py:39-40` ; `auth/care_auth.py`

2. Authentication

2.1 Staff login

Step	What happens	Source
1. Open login page	Server issues a fresh `csrf_token` cookie for the login form.	`routers/auth.py:41-51`
2. Submit	`POST /auth/login` checks CSRF and the per-IP limit (20/min).	`routers/auth.py:54-62`
3. Lockout check	If the user has 5 or more `login_failed` rows in the last 15 minutes, login is refused.	`routers/auth.py:37-38,75-88`
4. Password check	bcrypt `checkpw`, run in a thread so it does not block other users.	`auth/dependencies.py:57-83`
5. Failure	A `login_failed` row is written (only for a real username). The message is the same for unknown user and wrong password.	`routers/auth.py:90-97`
6. Disabled account	`status = False` users are refused.	`routers/auth.py:98-99`
7. Success	JWT created with `sub` (username) and `role`; `last_login` updated; `login` row written with IP.	`routers/auth.py:101-108`
8. Cookies	`access_token` (`httponly`, `SameSite=strict`), `csrf_token` (readable by JS), `session_expires` (readable by JS for the warning popup). `Secure` only if `COOKIE_SECURE` is on.	`routers/auth.py:110-122`
9. Landing	Admin → `/dashboard`; everyone else → `/attendance`.	`routers/auth.py:22-33`

2.2 Passwords

Item	Value	Source
Hashing	bcrypt with a per-hash salt (`gensalt()`); input capped at 72 bytes to keep old hashes valid	`auth/dependencies.py:44-65`
Cost	About 410 ms per check at cost 12 (comment)	`auth/dependencies.py:68-77`
Self-service change	Needs current password; minimum 6 characters	`routers/auth.py:174-205`

Item	Value	Source
Admin create / reset	No length or strength check; reset is not written to the audit log	`routers/admin.py:190-236,359-372`

2.3 Session / token

Item	Value	Source
Token type	JWT, HS256, signed with `SECRET_KEY`	`auth/dependencies.py:14,86-90`
Lifetime	1,440 minutes. Env/config values are deliberately ignored.	`config.py:29-33`
Refresh token	`REFRESH_TOKEN_EXPIRE_DAYS` is defined but not used by any route.	`config.py:34` (no other reference in the code)
Each request	Cookie decoded; user re-read from the database; disabled users are sent to login. Role changes therefore apply on the next request.	`auth/dependencies.py:93-109`
Extend session	`GET /auth/extend-session` re-issues all three cookies for another 24 h while the current token is valid.	`routers/auth.py:208-234`
Logout	Deletes the three cookies. The JWT itself stays valid until it expires (no server-side revocation list). Logout is not logged.	`routers/auth.py:125-145`
Back-button safety	HTML responses get `Cache-Control: no-store`.	`main.py:100-109`

3. Authorisation layers

A request passes up to five checks. Admin skips almost all of them.

#	Layer	What it does	Source
1	`get_current_user`	Valid JWT cookie + active user, else redirect to `/auth/login`.	`auth/dependencies.py:93-109`
2	`require_roles(*roles)`	Admin passes. Listed built-in roles pass. **Custom roles pass any gate that is not admin-only.** Built-in roles not listed pass only with an explicit matrix grant for the path's module.	`auth/dependencies.py:175-198`

#	Layer	What it does	Source
3	In-handler role checks	Some routers compare `current_user.role` against a tuple of built-in roles inside the handler (custom roles fail these). Examples: Finance (`routers/accounts.py:31,64`), Purchase Orders (`routers/crm_purchase_orders.py:26-29`), Price Matrix edit (`routers/crm_price_matrix.py:25`), Returns approval (`routers/sales.py:3521,3553`), Company Setting (`routers/settings.py:55`).	as listed
4	Module permission	`require_module_perm` / `has_perm` check the matrix **Enable** bit. No row = allowed (except default-off modules).	`auth/dependencies.py:208-256` ; `models/role_permissions.py:215-240`
5	Additional permission	`require_additional_perm` checks app-wide switches such as Add Lot, Edit Devices, Set Price. No row = allowed.	`auth/dependencies.py:259-303` ; `models/role_permissions.py:177-200`

3.1 Template layer

Helper	Purpose	Source
`has_perm` , `any_perm`	Show a sidebar item or section when the module is enabled.	`templates_config.py:113-153,247-249`
`has_explicit_perm`	Show admin items only for admin or an explicit grant (Sub Admin tab).	`templates_config.py:123-129`
`role_allowed(role, *values)`	Mirror of `require_roles` for buttons in templates, so custom roles see the same buttons the backend allows. CLAUDE.md asks all new role-gated buttons to use it.	`templates_config.py:174-190`
`can_view_pricing` , `can_set_price`	Hide price columns / Set Price buttons.	`templates_config.py:132-149`
`module_hidden`	Global Visibility: hide a module for everyone, admin included.	`templates_config.py:225-233`

3.2 Permission cache

Item	Behaviour	Source
Storage	Module-level dicts `_PERM_CACHE` , `_ADDITIONAL_PERM_CACHE` (and label/visibility caches)	`models/role_permissions.py:102-166`

Item	Behaviour	Source
Load	On start-up from `role_module_permissions` and `role_additional_permissions`	`main.py:570-578` ; `routers/master.py:1110-1131`
Update	Save handlers update the cache of the worker that served the save	`routers/master.py:880-895,932-940`
Other workers	Every worker reloads all caches every 60 s	`main.py:789-829`
Worker count	`main.py:863` and `Dockerfile:71-73` run 1 worker; `deploy/oxypc.service:28` runs 4 workers. With 4, a matrix change can take up to 60 s to reach every worker.	as listed
`sub_admin` seed	Custom role `sub_admin` is created at start-up if missing	`main.py:582-597`

3.3 403 handling

Item	Behaviour	Source
Page requests	`error.html` with title "Access Denied" and the hint "Ask an admin to enable it for your role in Master Data → Module Permissions".	`main.py:325-326,355-365,381-383`
API / AJAX	JSON `{error, code, solution}` (detected by `/api/` path, `Accept`, or `X-Requested-With`).	`main.py:340-352`
Auth redirects	302 from dependencies is turned into **303** to `/auth/login` to avoid POST redirect loops.	`main.py:367-378,386-398`
Debug detail	Exception text is shown only if `OXYPC_DEBUG=1`.	`main.py:26,428,488`

4. CSRF

Item	Behaviour	Source
Pattern	Double-submit: form field `csrf_token` (or header `X-CSRF-Token`) must equal the `csrf_token` cookie. GET/HEAD/OPTIONS/TRACE skip the check.	`auth/dependencies.py:306-343`
Failure	Redirect to login (treated as a timed-out page), not a 403.	`auth/dependencies.py:323-343`

Item	Behaviour	Source
Where applied	Router-level `dependencies=[Depends(verify_csrf)]` on most routers (e.g. `routers/admin.py:99`, `routers/sales.py:67`), or per route (e.g. `routers/accounts.py:192`).	as listed
Logout	Deliberately not CSRF-checked, so logout cannot fail.	`routers/auth.py:125-139`
Partner portal	Parallel cookie `partner_csrf_token`.	`auth/partner_auth.py:76-91`
Cookie-auth POSTs without CSRF (found by scan)	`routers/qa_uat.py` (10 POSTs), `routers/api_v1/api_keys.py`, `routers/api_v1/webhooks.py`, `routers/api_v1/telecalling.py` (`/assignments`, `/calls`), `routers/partner_admin.py` (`/bids/{bid_id}/generate-po`). All cookies are `SameSite=strict`, which blocks normal cross-site posts.	as listed; cookies `routers/auth.py:115-121`

5. Rate limiting and lockout

Limit	Value	Key	Source
Default, all routes	2,000 / minute	Hash of the session cookie, else client IP	`limiter.py:32-49,74`
Staff login	20 / minute	Client IP only (cookie ignored)	`routers/auth.py:55`; `limiter.py:52-60`
Partner login	5 / minute + account lockout	Client IP	`routers/partner_portal.py:74-75,107`
Care Agent pairing	10 / minute	Client IP	`routers/care_api.py:48-49`
Care tickets / diagnostics	10 / 20 per minute	Default key	`routers/care_api.py:193-194,281-282`
Account lockout	5 failures in 15 min	Per username	`routers/auth.py:37-38,75-88`
Client IP	Right-most `X-Forwarded-For` entry (cannot be forged by the client)	—	`limiter.py:11-29`
Storage	In memory (no storage back-end configured) — per worker process	—	`limiter.py:74`

Note: the comment in `main.py:34` says login is "5/min"; the code says 20/min (`routers/auth.py:55`).

6. Audit trail and event log

Store	What is recorded	Written by	Retention	Source
`audit_logs`	user, action, table, record id, old/new JSON, notes, IP, time	`audit()` (caller must commit) — 217 distinct action names across routers and services	No pruning code found	`services/audit_engine.py:23-58`; `models/engines.py:107-125`
`audit_logs` (DB trigger)	Changes to `telecalling_records`, `telecalling_assignments`	Trigger `fn_audit_central`	Same table	`alembic/versions/20260514_1000_telecalling_mobile_pwa.py:44-49,168-210`
`login_logs`	`login`, `login_failed`, IP	Login route	No pruning code found	`models/user.py:120-130`; `routers/auth.py:94-107`
`event_log`	Event type, payload, webhook attempts and last status	Webhook dispatcher, for the 7 subscribed event types	No pruning code found	`models/event_log.py`; `services/webhook_dispatcher.py:94-125`; `main.py:558-567`
Care audit	Pairing and device events	`care_audit()`	Unknown	`routers/care_api.py`
Physical stock audits	Counts per stage/location and per tag	Physical Audits page	— (business data, not a security log)	`models/audit_record.py`
Error log	Full tracebacks for 5xx	Exception handlers	File grows without rotation in code	`main.py:112-119,433-489`

Examples of audited actions: `USER_CREATED`, `USER_DELETED` (`routers/admin.py:231,352`), `SALE_CREATED`, `SALE_DELETED`, `QC_PASS` / `QC_FAIL` (`routers/qc.py:321-325`), `GATE_EXIT_CONFIRMED` (`routers/gate_pass.py:364`), `WARRANTY_EXTENDED` (`routers/extended_warranty.py:159`), `MANUAL_BACKUP`, `DELETE_BACKUP` (`routers/admin.py:703,731`).

Not audited (no `audit()` call found): Permission Matrix, Additional Permissions, Pricing Visibility and Sub Admin saves (`routers/master.py`), Stage Control transitions (`routers/stage_control.py`), Sidebar Config (`routers/sidebar_config.py`), admin password reset (`routers/admin.py:359-372`), logout.

"Append-only" is stated in the model docstring (`models/engines.py:109-110`). No database rule, trigger or `REVOKE` protecting `audit_logs` was found.

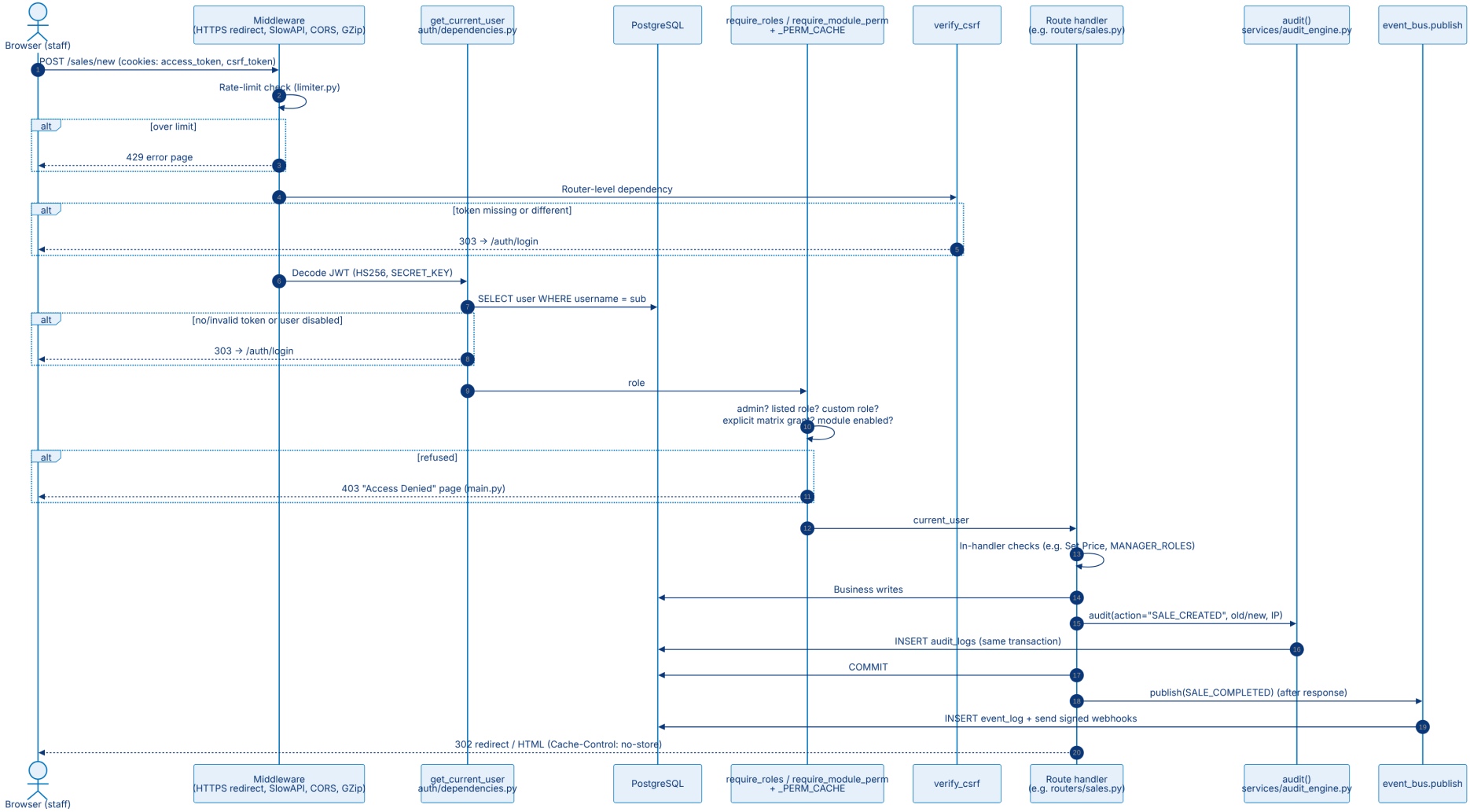
7. Partner portal, APIs and agents

Channel	Who	Authentication	Authorisation / scoping	Source
Partner portal `/partner/*`	Dealers	Phone + password (bcrypt); `partner_token` JWT cookie, 12 h; claim `typ: partner`; `pv` (password version) kills old tokens after reset	Every query scoped to the logged-in dealer (e.g. `PartnerBid.dealer_id == dealer.id`); dealer must be `portal_enabled` and `active` on every request	`auth/partner_auth.py:22-73`; `routers/partner_portal.py:74-138,1099-1126`
Partner admin `/trade-partner/*`	Staff	Staff session	Module permissions `trade_partner_*`, seeded off for all non-admin roles	`routers/partner_admin.py:63,231-446`; `migrate_seed_trade_partner_split_perms.py`
Ecosystem API `/api/v1/*`	Other apps	`Authorization: Bearer <key>`; key stored as SHA-256; revocable (`is_active`, `deleted_at`); no expiry field	Per-key scopes (`devices:read`, `iqc:write`, ...)	`auth/api_key.py:122-212`; `models/api_key.py:13-41`
API key and webhook admin	Admin	Staff session	`require_roles(admin)`	`routers/api_v1/api_keys.py:24`; `routers/api_v1/webhooks.py:24`
Mobile telecalling `/api/v1/telecalling/*`	Staff	Staff session	Legacy `ROLE_PERMISSIONS` strings (`tc.*`); custom roles and `cosmetic_manager` have none	`routers/api_v1/telecalling.py:23-39`; `auth/dependencies.py:16-41`
Outbound webhooks	—	HMAC-SHA256 header `X-OxyPC-Signature`; 10 s timeout	Subscribed event types only	`services/webhook_dispatcher.py:25-81`
OxyQC desktop app `/iqc/api/*`	IQC PCs	Static header `X-OxyQC-Key`, default `oxyqc-default-key-change-me`, compared with `!=`	Full access to its endpoints, incl. user list and device submit	`config.py:45`; `routers/iqc_api.py:39-41,88-105,257-261`
Care Agent `/care/api/v1/*`	Customer laptops	One-time provisioning token (single use, expiring, atomic) → long-lived device token (stored hashed)	Every query scoped to the token's pairing	`auth/care_auth.py:1-75`; `routers/care_api.py:48-114`
CORS	Browser apps on listed origins	`allow_credentials=True`; origins from `OXYPC_ALLOWED_ORIGINS` (default localhost ports)	—	`config.py:49-56`; `main.py:43-52`
FieldOps `/fieldops`	Separate app	Own database and accounts	Out of scope here	`main.py:497-513`; `routers/fieldops.py`

8. Data protection

Topic	What the code does	Source
Secrets	Read from env vars first, then `config.ini`, then code defaults. `.env` and `config.ini` are git-ignored.	`config.py:1-56`; `.gitignore:2-5`
Code defaults	DB URL with password `oxypc123`; OxyQC key `oxyqc-default-key-change-me`; random `SECRET_KEY` written to `config.ini` on first run.	`config.py:27-28,45,58-69`; `main.py:28-30`
Default accounts	`setup_db.py` falls back to admin password `oxypc@admin123`; `seed_uat_users.py` holds fixed UAT passwords for every role, including admin.	`setup_db.py:41-50`; `seed_uat_users.py`
HTTPS	Redirect to HTTPS only when a proxy sends `X-Forwarded-Proto`. `Secure` cookie flag off unless `OXYPC_COOKIE_SECURE=1`. LAN install runs on plain HTTP (`192.168.1.100:8000`, CLAUDE.md).	`main.py:54-80`; `config.py:39-43`
Uploads	Stored under `uploads/` (CRM files, accounts documents, GRN invoices, quotes, partner photos and bid documents, daily stock CSVs). The folder is mounted at `/uploads` **with no login check**.	`main.py:133-136`; `routers/accounts.py:45-56`; `routers/grn.py:34,626-631`; `routers/crm_quotes.py:608`; `routers/partner_admin.py:549,1653,2241`; `services/daily_stock_reports.py:4,28`
Upload names	Accounts / CRM: random UUID names, but any file extension is kept. GRN: `<grn_number>_<original name>`. Daily stock: `daily_stock_reports/<YYYY-MM-DD>/overall.csv` (predictable).	as above
Private uploads	Partner payment proofs are stored outside `uploads/` and served through a logged-in route.	`services/partner_service.py:24`; `routers/partner_admin.py:545-548`
`.gitignore` notes	`uploads/*` described as "CRM lists, IQC photos, KYC docs — PII".	`.gitignore:32-34`
Backups	`pg_dump` → gzip in `backups/` on the same host; not encrypted. Admin can list, download, run and delete backups (`oxypc_YYYYMMDD_HHMMSS.sql.gz`, path-traversal guarded). Production wrapper keeps all backups unless `OXYPC_BACKUP_PRUNE=1`; `scripts/backup.sh` keeps 30 days. Off-site copy is recommended but not automated.	`scripts/backup_db.py:26-27,88-95`; `routers/admin.py:609-735`; `deploy/oxypc-backup.sh:7-15`; `scripts/backup.sh:32,61-62`; `docs/backup-runbook.md`

9. Request flow



FastAPI resolves router-level and route-level dependencies before the handler; the order between CSRF and auth shown above is illustrative. Sources: `main.py:34-109`, `auth/dependencies.py:93-343`, `routers/sales.py:67,79,1372-1400`, `services/audit_engine.py`, `services/event_bus.py:53-78`.

10. Known risks and recommendations

Severity is our judgement from reading the code. "Verify" means a live test should confirm it before acting.

#	Risk	Evidence	Severity	Recommendation
R1	**Uploaded files are public.** `/uploads` is served with no login. It holds accounts documents, CRM files, GRN invoices, quotes, partner bid POs and daily stock CSVs at predictable paths.	`main.py:133-136` ; `services/daily_stock_reports.py:4,28` ; `routers/grn.py:628`	High	Serve these files through a logged-in route with a permission check (as partner proofs already are). Keep only public marketing photos on a static mount.
R2	**Uploads keep any file type** and are served from the app's own origin. An uploaded `.html` file could run script in the app's origin (where `csrf_token` is readable).	`routers/accounts.py:45-56` ; `routers/crm_contacts.py:80-90`	High (verify)	Allow-list extensions and content types; serve downloads with `Content-Disposition: attachment` .
R3	**Permissive defaults.** A role or module with no matrix row is allowed. A new custom role (including the auto-seeded `sub_admin`) passes every non-admin `require_roles` gate. Sensitive modules such as Cost Config and Credit Note are open by default.	`models/role_permissions.py:234-238` ; `auth/dependencies.py:188-189` ; `main.py:582-597` ; `routers/admin.py:545`	High	Make custom roles deny-by-default; add sensitive modules to `DEFAULT_DISABLED_MODULES` ; require a matrix save when creating a role.
R4	**Admin-equivalent grants.** An explicit `admin_master` or `admin_users` grant passes the admin-only gate on the Permission Matrix and User Management, so the holder can grant itself more or create an `admin` user. The comment in `auth/dependencies.py:117-120` says `/admin/master` is not a matrix module, but `routers/landing_pages.py:177` maps it.	`auth/dependencies.py:147-172,190-196` ; `routers/landing_pages.py:176-177` ; `routers/admin.py:190-236` (role is free text)	High (verify)	Keep `/admin/master` and `/admin/users` strictly admin, or block a non-admin from granting itself or assigning `admin` .
R5	**Permission changes are not audited**, and `audit_logs` is append-only only by convention. Audit IP uses the left-most `X-Forwarded-For` value, which a client can set.	`routers/master.py` (no `audit()`); `models/engines.py:109-110` ; `services/audit_engine.py:40-43`	Medium	Audit every matrix/role/pricing save and password reset; `REVOKE UPDATE, DELETE` on `audit_logs` for the app user; use the same IP logic as `limiter.py:11-29` .

#	Risk	Evidence	Severity	Recommendation
R6	**Pricing leaks through exports.** Pricing Visibility hides screens but not CSV exports. Export/Download/Print/Upload switches and per-user `UserPermission` grants are stored but never enforced.	`routers/sales.py:1953,2331` ; `routers/reports.py:870,1121` ; `routers/master.py:313-325` ; `routers/admin.py:375-430`	Medium	Apply `can_view_pricing` in export builders; enforce or remove the unused switches.
R7	**Weak defaults in config and seed scripts**: default DB password, default OxyQC key (compared without constant time), default admin password, UAT passwords in the repo; `Secure` cookie flag off by default; LAN runs on HTTP.	`config.py:27,41-45` ; `routers/iqc_api.py:39-40` ; `setup_db.py:41-50` ; `seed_uat_users.py`	Medium	Refuse to start with default secrets in production; use `hmac.compare_digest` ; enable HTTPS and `OXYPC_COOKIE_SECURE=1` ; disable UAT accounts in production.
R8	**Sessions cannot be revoked.** 24 h JWT, extendable from the page; logout only clears cookies; password change does not end other sessions. Password rules: 6 characters on self-change, none on admin create/reset.	`config.py:33` ; `routers/auth.py:125-145,194,208-234` ; `routers/admin.py:359-372`	Medium	Add a token version per user (as the partner portal does with `pv` , `auth/partner_auth.py:31,70`); raise the password policy.
R9	**Code and matrix disagree.** In-handler role tuples refuse custom roles that `require_roles` lets in (Finance, Purchase Orders, Price Matrix edit, Returns approval). `stage_allowed()` is dead code, so repair pages are open to all. `sales_manager` is missing from the reports router gate.	`routers/accounts.py:31` ; `routers/crm_purchase_orders.py:26-29` ; `routers/repair.py:76-85` ; `routers/reports.py:43-47`	Medium	Replace in-handler tuples with `require_roles` / `role_allowed` ; remove or wire `stage_allowed()` ; align the reports gate.
R10	**Multi-worker side effects.** With 4 workers, the in-memory limiter allows about 4× the stated limits and permission changes take up to 60 s to apply everywhere.	`deploy/oxypc.service:28` ; `limiter.py:74` ; `main.py:789-829`	Low	Use a shared limiter store (e.g. Redis) or keep 1 worker; refresh caches on save across workers.
R11	**CSRF gaps** on a few cookie-authenticated POSTs.	§4 table	Low (SameSite=strict mitigates)	Add `verify_csrf` to `routers/qa_uat.py` , `routers/api_v1/*` cookie routes and `generate-po` .
R12	**Lockout can be abused.** Five wrong passwords lock a known username for 15 minutes.	`routers/auth.py:75-88`	Low	Lock per username + IP, or add a CAPTCHA after failures.

11. Open questions

1. Is production actually on HTTPS, and is `OXYPC_COOKIE_SECURE=1` set? (`deploy/nginx-oxypc.conf:6` suggests Let's Encrypt for `erp.oxypc.in`; the LAN server runs HTTP.)
2. Are the default `SECRET_KEY`, DB password and OxyQC key replaced in every environment? Are UAT accounts present in production?
3. Which deployment is live: 1 worker (`Dockerfile:73`) or 4 workers (`deploy/oxypc.service:28`)?
4. Is there a retention rule for `audit_logs`, `login_logs`, `event_log` and `errors.log`? None exists in code.
5. Does the nginx front end block `/uploads` for non-public folders? The configs in `deploy/` do not mention `/uploads`.
6. Are backups copied off the server and encrypted? `docs/backup-runbook.md` recommends it; no script does it.
7. Is the `admin_master` / `admin_users` grant (R4) used in production, and by whom?
8. Should `audit_logs` be protected at the database level?
9. With `forwarded_allow_ips="*"` (`main.py:867`), what does `request.client.host` hold behind the proxy? It feeds `login_logs.ip_address` (`routers/auth.py:95,106`).



OXYPC INVENTORY LIFECYCLE

ARCHITECTURE

Technical Architecture

System design, data model, integrations, and the decisions behind them.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Technical Architecture

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

In this document: Part A — Solution Architecture · Part B — Data Model & Dictionary · Part C — Integrations · Part D — Architecture Decision Records

PART A

Solution Architecture

This document describes how the OxyPC Inventory Lifecycle app is built: who and what talks to it, what is inside it, how a request flows through it, and the patterns every new feature should follow. Every claim cites a file or commit. Anything not proven by the code is listed under **Open questions** at the end.

---

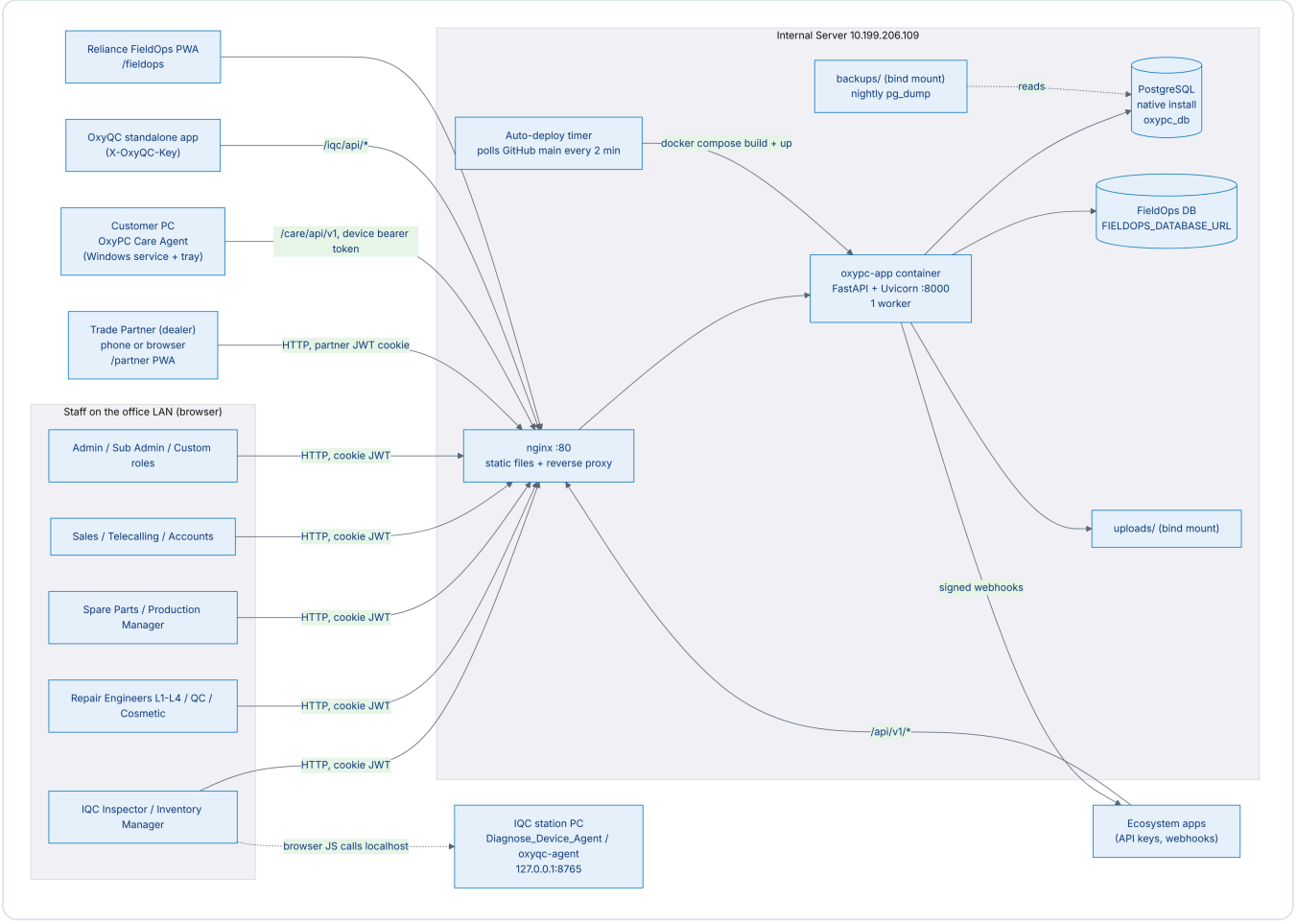
1. Summary

Item	Value	Source
App name	OxyPC Inventory Lifecycle	`config.py` (`APP_NAME`), commit `e5538cf` (rebrand)
Style	Server-rendered web app (FastAPI + Jinja2), one Python process	`main.py` , `templates_config.py`
Database	PostgreSQL, async access through SQLAlchemy 2 + asyncpg	`database.py` , `requirements.txt`
Production	Linux "Internal Server" at `http://10.199.206.109` , app in a Docker container behind nginx	`deploy/nginx-oxypc-internal.conf` , `docs/superpowers/specs/2026-10-01-docker-containerization-design.md` §9
Staging	`http://localhost:8001` on staging machines	`qa_docs/design_system/design_system.html` (Environments row, release table)
Users	30+ concurrent LAN users	`CLAUDE.md`
Size of code base	81 router modules + 9 API v1 modules, 57 model modules, 27 services, 242 Jinja templates, 172 test files	file counts in `routers/` , `routers/api_v1/` , `models/` , `services/` , `templates/` , `tests/`

---

2. System context

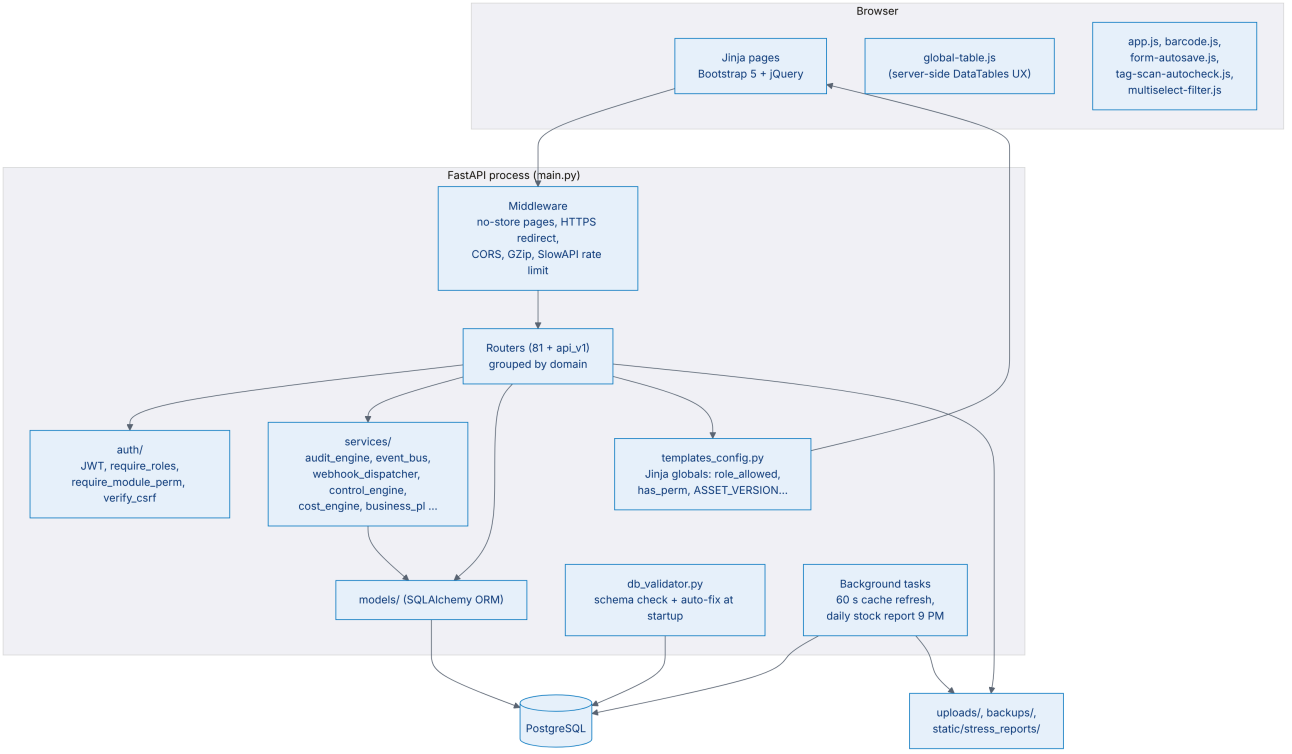
The diagram shows the people and systems around the app. Production nginx serves `/static/` straight from disk and passes everything else to the app (`deploy/nginx-oxypc-internal.conf`).



Actor / system	How it connects	Auth	Source
Staff (all roles)	Browser → nginx → app	`access_token` JWT cookie + CSRF double-submit cookie	`auth/dependencies.py` (`get_current_user` , `verify_csrf`)
Trade Partner (dealer)	`/partner` routes, installable PWA	Separate partner JWT cookie; staff sessions cannot open these routes	`routers/partner_portal.py` header, `auth/partner_auth.py` , `static/partner/manifest.json` , `static/partner/sw.js`
IQC station agent (`oxyqc-agent`)	The IQC page's browser JS calls `http://127.0.0.1:8765/diagnose` on the same PC; the agent is never on the LAN	None (loopback only)	`oxyqc-agent/README_AGENT.txt`
OxyQC standalone app	Machine API `/iqc/api/*`	Static key in `X-OxyQC-Key` header	`routers/iqc_api.py` header, `config.py` (`OXYQC_API_KEY`)
Customer Care Agent	`/care/api/v1`	Device-bound bearer token, scoped to one pairing	`routers/care_api.py` header, `auth/care_auth.py` , `care_agent_windows/README.md`
Ecosystem apps	`/api/v1/*` JSON API; outbound HMAC-signed webhooks	API keys with scopes; CORS allow-list	`routers/api_v1/` , `auth/api_key.py` , `services/webhook_dispatcher.py` , `config.py` (`ALLOWED_ORIGINS`)
Reliance FieldOps	`/fieldops` , offline-first PWA	Own users and own database	`routers/fieldops.py` header, `fieldops_db.py` , `README.md`

Actor / system	How it connects	Auth	Source
Telecalling mobile	Mobile PWA with offline queue	Staff login	`static/m/telecalling/` (`manifest.json` , `sw.js` , `offline-queue.js`)
WhatsApp service	**Disabled.** Router not registered	n/a	`main.py` lines 266-267

3. Container view



3.1 Routers grouped by domain

The app has one router file per feature area (`CLAUDE.md` "Architecture"). All are registered in `main.py` lines 234-316.

Domain	Router files (in `routers/`)
Inbound and IQC	`grn.py` , `iqc.py` , `iqc_api.py` , `bulk_upload.py` , `stock.py` , `parts_grn.py` , `physical_audits.py` , `inventory_location.py` , `block_tags.py` , `entity_movement.py` , `transfers.py`
Production, repair and QC	`devices.py` , `repair.py` , `qc.py` , `cosmetic.py` , `buckets.py` , `stage_control.py` , `workid_status.py` , `part_requests.py` , `part_estimation.py` , `spare_parts.py` , `procure_dashboard.py` , `product_master.py` , `scrap.py` , `scrap_for_sale.py` , `stress_api.py`
Sales and dispatch	`sales.py` , `dispatch.py` , `gate_pass.py` , `invoices.py` , `quotations_list.py` , `dealer_quotations.py` , `part_sales.py` , `extended_warranty.py` , `market.py` , `terms_conditions.py`

Domain	Router files (in `routers/`)
CRM, dealers, telecalling	`crm_dashboard.py`, `crm_contacts.py`, `crm_sourcing.py`, `crm_sales.py`, `crm_quotes.py`, `crm_activities.py`, `crm_price_matrix.py`, `crm_purchase_orders.py`, `crm_reports.py`, `crm_assign_leads.py`, `deals.py`, `accounts.py`, `dealers.py`, `assign_dealer_leads.py`, `telecalling.py`, `m_telecalling.py`, `telesales_dashboard.py`, `model_requests.py`, `model_requested.py`
Finance and reports	`partner_payments.py`, `reports.py`, `dashboard.py`
External parties	`partner_portal.py`, `partner_admin.py`, `care_api.py`, `care_internal.py`, `care_admin.py`, `fieldops.py`, `api.py`, `api_v1/*` (api_keys, devices, health, iqc, lots, sales, spare_parts, telecalling, webhooks)
Platform and admin	`auth.py`, `admin.py`, `master.py`, `settings.py`, `company_settings.py`, `sidebar_config.py`, `landing_pages.py`, `trash.py`, `notifications.py`, `manuals.py`, `health.py`, `qa_uat.py`, `attendance.py`, `attendance_group_config.py`, `tickets.py`
Disabled	`whatsapp.py` (import and registration commented out, `main.py` lines 176, 266-267)

Note: `routers/repair.py.bak` is a stray backup file in the router folder.

3.2 Services

Service	Job	Source
`audit_engine.audit()`	Writes an `AuditLog` row for a significant action. Does not commit; the caller commits. Captures user, action, table, record id, old/new values, IP	`services/audit_engine.py`
`event_bus`	In-process pub/sub. Routes call `publish()` after commit; handlers run as FastAPI background tasks after the response	`services/event_bus.py`
`webhook_dispatcher`	Subscribed at startup to 7 event types; sends HMAC-signed webhooks	`services/webhook_dispatcher.py`, `main.py` startup
`control_engine`	Stage-transition rules (cached)	`services/control_engine.py`, `main.py` (`invalidate_transitions_cache`)
`daily_stock_reports`	Generates Daily Stock exports every day at 9 PM	`services/daily_stock_reports.py`, `main.py` startup
Others	Costing, P&L, pricing, part estimation, warranty/care, GRN stocking, invoice parsing, PO PDF	`services/*.py`

3.3 Background jobs (inside the app process)

There is no separate scheduler or worker. Jobs are `asyncio` tasks started in `startup_event()` (`main.py`).

Job	Interval	Purpose	Source
Cache refresh	Every 60 s	Reloads permissions, sidebar labels, page titles, breadcrumbs, master dropdowns, module visibility; clears transition cache	`main.py` `_refresh_all_caches_periodically`
Daily stock report	Checks every few minutes; generates at 9 PM	Writes Daily Stock export files	`services/daily_stock_reports.py` `scheduler_loop`
Event handlers	Per request	Webhook delivery after response	`services/event_bus.py`

Job	Interval	Purpose	Source
One-time backfills	Each startup (idempotent)	GRN-Lot links, GRN entity, tag backfills, size-unit spacing, Product Master seeding, QA release tracking	`main.py` startup blocks

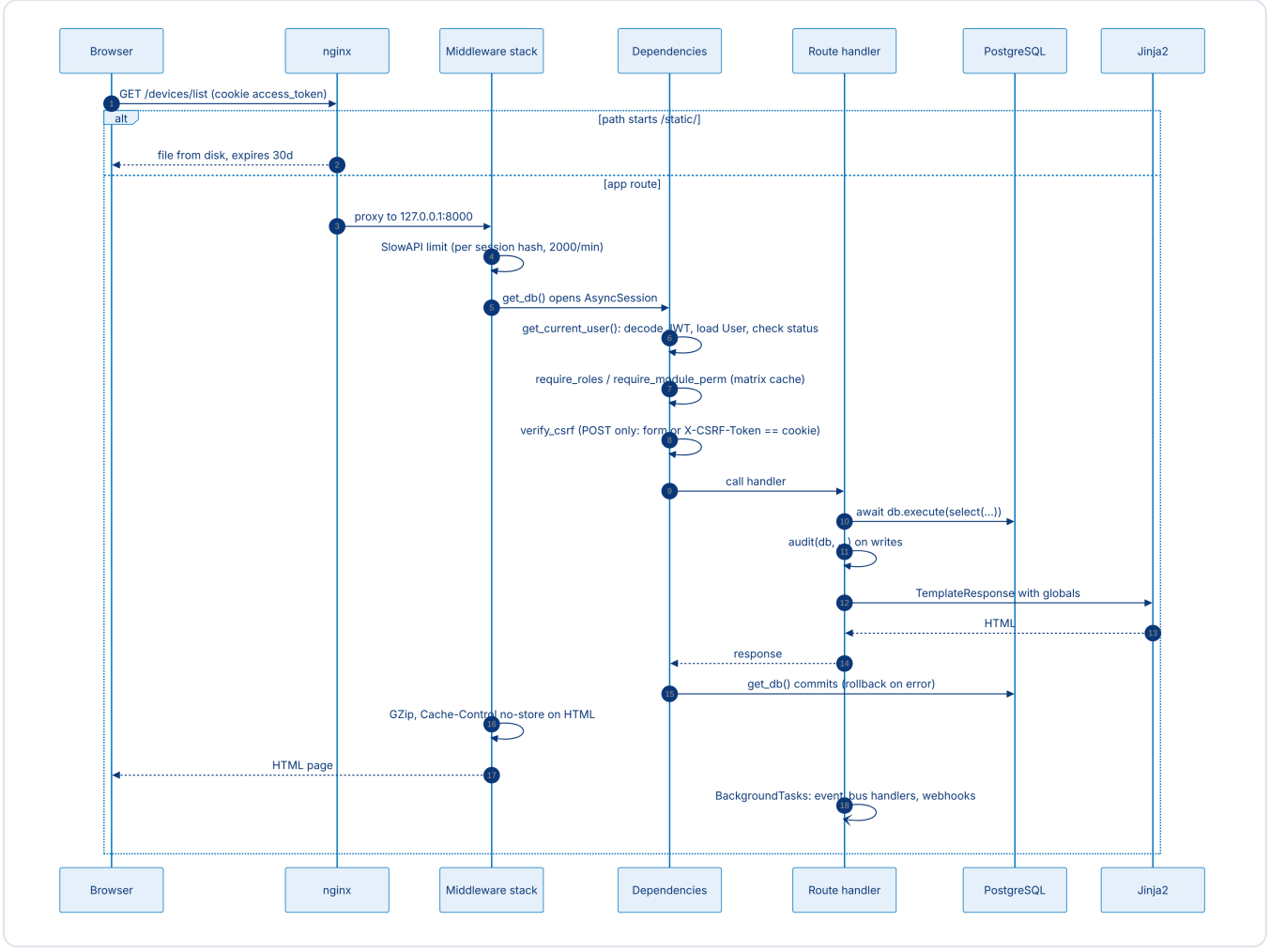
Host-level jobs (outside the app) are covered in 06 Deployment: nightly backup timer and auto-deploy timer.

3.4 Front end

Part	Detail	Source
Templates	242 Jinja2 files; all pages extend `base.html`	`templates/`, `qa_docs/design_system/design_system.html` checklist
Jinja globals	`role_allowed`, `has_perm`, `has_explicit_perm`, `any_perm`, `can_view_pricing`, `can_set_price`, `module_hidden`, `sidebar_label`, `master_options`, `ASSET_VERSION` and more	`templates_config.py` lines 43-330
Global Table	One JS module that layers the standard heavy-table UX on a server-side DataTables instance (sticky first column, one-line headers, 12 rows default, cell clamping, scan-to-select)	`static/js/global-table.js` header, commit `ae43e3f`
Cache busting	`?v={{ ASSET_VERSION }}` from asset mtimes	`templates_config.py` `_VERSIONED_ASSETS`, `tests/test_asset_version_cache_busting.py`
App zoom	Whole UI at 90 % (`--app-zoom: 0.9`); use `var(--full-vh)` not `100vh`	`static/css/app.css` lines 10-20
Vendor libraries	Served locally, no CDN	`static/vendor/`, design system checklist "No CDN links"

4. Request lifecycle

A normal staff page request (for example opening a list page, then submitting a form):



Step	What happens	Source
Rate limit	Bucket is the hashed session token, or client IP before login. Default 2000/min. Login is 20/min per IP	`limiter.py`, `routers/auth.py` line 55
HTTPS redirect	Only acts if `X-Forwarded-Proto` is present. The LAN nginx deliberately omits it	`main.py` `_force_https`, `deploy/nginx-oxypc-internal.conf`
Auth	JWT in `access_token` cookie, HS algorithm with `OXYPC_SECRET_KEY`; missing/invalid → 302 to `/auth/login`	`auth/dependencies.py` lines 86-109
Authorisation	Built-in roles pass if listed; custom roles pass non-admin gates; otherwise the Permission Matrix can grant the path	`auth/dependencies.py` `require_roles`, `_matrix_grants_path`
CSRF	Double-submit cookie on every non-GET; missing cookie → redirect to login (treated as expiry)	`auth/dependencies.py` `verify_csrf`
Transaction	One session per request; commit after handler, rollback on exception	`database.py` `get_db`
Errors	JSON for API/AJAX callers, HTML modal page otherwise; 500s written to `errors.log`	`main.py` `_wants_json`, `_render_error`, `generic_exception_handler`
Caching	HTML gets `Cache-Control: no-store`; static files are cached and cache-busted	`main.py` `_no_store_pages`

5. Tech stack

Versions come from `requirements.txt` (pinned with `==` unless noted) and from file headers in `static/vendor/`.

Layer	Technology	Version	Source
Language	Python	3.12 in the Docker image; 3.10.12 on the server host; Windows scripts point to 3.13/3.14; Windows bundle 3.11.9	`Dockerfile`, docker spec §9, `run_server.ps1`, `install_service.bat`, `prepare_bundle.bat`
Web framework	FastAPI	0.115.6	`requirements.txt`
ASGI server	uvicorn[standard]	0.32.1	`requirements.txt`
ORM	SQLAlchemy[asyncio]	2.0.36	`requirements.txt`
DB driver	asyncpg	0.30.0	`requirements.txt`
Migrations	Alembic	1.14.0 (36 revisions)	`requirements.txt`, `alembic/versions/`
Database	PostgreSQL	17 on the Internal Server; 16.6 in the Windows bundle	`Dockerfile` comment (pg_dump 17), `prepare_bundle.bat`
Templates	Jinja2	3.1.4	`requirements.txt`
Auth	python-jose[cryptography] 3.3.0, bcrypt 5.0.0	pinned	`requirements.txt` (with bcrypt/passlib note)
Rate limit	slowapi	0.1.9	`requirements.txt`
Config	python-dotenv	1.0.1	`requirements.txt`
Files/exports	openpyxl ≥3.1.0, fpdf2 ≥2.7.9, reportlab ≥4.0.0, python-barcode ≥0.15.1, python-pptx ≥1.0.0, pypdf, pdfplumber (unpinned)	floor or unpinned	`requirements.txt`
Tests	pytest 8.3.4, pytest-asyncio 0.24.0, httpx 0.28.1	pinned	`requirements.txt`, `pytest.ini`
CSS/JS	Bootstrap	5.3.2	`static/vendor/bootstrap/css/bootstrap.min.css` header
Icons	Bootstrap Icons	1.11.3	`static/vendor/bootstrap-icons/bootstrap-icons.css` header
Tables	DataTables	1.13.7	`static/vendor/datatables/js/jquery.dataTables.min.js` header
DOM	jQuery	3.7.1	`static/vendor/jquery/jquery-3.7.1.min.js`
Charts	Chart.js	4.4.0	`static/vendor/chartjs/chart.umd.min.js` header
Select boxes	Tom Select	2.3.1	`static/vendor/tom-select/js/tom-select.complete.min.js` header

Layer	Technology	Version	Source
Container	Docker Engine 29.8.0, Compose v5.5.1	as found on server	docker spec §9
Reverse proxy	nginx	not recorded	`deploy/nginx-oxypc-internal.conf`
Packaging (legacy)	PyInstaller ≥6.10.0, Inno Setup 6		`requirements.txt`, `build_all.bat`, `oxypc_setup.iss`

6. Key design patterns

Each pattern below is a rule for new code. The project skills in `.claude/skills/` enforce several of them.

#	Pattern	Rule	Where it lives
P1	Async SQLAlchemy	`await db.execute(select(...))` then `.scalar_one_or_none()` or `.scalars().all()`. One session per request from `get_db`	`CLAUDE.md`, `database.py`
P2	Soft delete	Rows are flagged `is_trashed` instead of deleted; lists filter them out; a Trash page restores them. On devices, lots, spare parts, parts GRN, scrap, CRM tables the column is now `NOT NULL DEFAULT false`	`models/device.py` l.248, `models/lot.py` l.47, `models/spare_parts.py` l.48, `routers/trash.py`, migrations `20260503_0900_soft_delete_devices.py`, `20260526_1400_add_trash_to_lots_devices.py`
P3	Audit before delete	Call `audit()` with the old values **before** `db.delete()`; the row is gone after commit. 242 `await audit(` calls in routers/services	`CLAUDE.md` Critical Rules, `services/audit_engine.py`
P4	`role_allowed()` for UI gating	Never hard-code a role list in a template. `role_allowed()` mirrors `require_roles` so custom roles (e.g. `trc_manager`) see the same buttons	`templates_config.py` `_role_allowed`, `.claude/skills/custom-role-permissions`
P5	Permission Matrix	Admin grants modules per role in Master Data; nav and backend both read the cached matrix	`auth/dependencies.py` `_matrix_grants_path`, `routers/master.py`
P6	AJAX partial refresh	Submit by fetch, parse the returned HTML with `DOMParser`, swap only the changed region, then re-bind handlers	`.claude/skills/ajax-partial-refresh`, commit `684fa27`
P7	Global Table	Heavy lists use server-side DataTables plus `global-table.js` (13 templates use `serverSide: true`)	`static/js/global-table.js`, e.g. `templates/devices/list.html`
P8	Server-side money	Totals calculated server-side with `Decimal`; priced records are snapshotted as columns	`.claude/skills/server-side-financial-snapshot`
P9	Schema provisioning at startup	`db_validator.validate_and_fix()` creates missing tables/columns before the app serves; aborts on unfixable issues	`db_validator.py`, `main.py` startup
P10	In-memory caches	Permissions, transitions, labels, dropdowns cached per process and refreshed every 60 s	`main.py` startup
P11	Error UX	One error page with a message and a "what to do" line; JSON for AJAX	`main.py` `_ERROR_INFO`

Drift found in code (for follow-up):

- `CLAUDE.md` says `is_trashed` is nullable and must be filtered with `isnot(True)`. The models now declare it `nullable=False`, and 126 filters use `== False` against 10 using `isnot(True)` (grep of `routers/`, `services/`). Both work on a NOT NULL column; the rule text is out of date.
- Only 4 templates call `role_allowed(`; 15 places still compare `current_user.role.value` directly (e.g. `templates/crm/contacts/list.html` l.52, `templates/dealers/list.html` l.54).

7. Folder structure map

Path	What it holds
`main.py`	App factory, middleware, router registration, error handlers, startup jobs
`config.py`	Settings from env vars, then `config.ini`, then defaults
`database.py`	Async engine, pool settings, `get_db`
`db_validator.py`	Schema validator and auto-fixer
`limiter.py`	SlowAPI limiter and key functions
`templates_config.py`	Shared `Jinja2Templates` and all Jinja globals
`auth/`	Staff, partner, care-agent and API-key auth
`routers/`	One module per feature area; `routers/api_v1/` for the JSON API
`models/`	SQLAlchemy models (57 modules)
`schemas/`	Pydantic schemas for the API
`services/`	Business engines (audit, events, cost, P&L, pricing, care ...)
`utils/`	Helpers (timezone, master data, grades, warranty, CSV decode ...)
`templates/`	Jinja pages by module; `base.html`, `error.html`
`static/`	`css/app.css`, `js/*.js`, `vendor/`, `m/telecalling/` PWA, `partner/` PWA, `manuals/`
`fieldops_app/`	Reliance FieldOps PWA, served behind login by `routers/fieldops.py`
`alembic/`	36 migration revisions
`migrate_*.py`, `backfill_*.py`, `seed_*.py`, `verify_*.py`	51 / 8 / 9 / 12 one-off scripts at repo root
`scripts/`	Backup/restore, Windows service install
`deploy/`	Linux: systemd units, nginx vhosts, auto-deploy and backup scripts, `DEPLOY.md`
`Dockerfile`, `docker-compose.yml`, `.dockerignore`	Container build used in production
`*.bat`, `*.ps1`, `*.iss`, `launcher.py`, `installer.py`	Windows / legacy run and installer tooling
`oxyqc-agent/`	IQC station hardware agent (EXE + source, PXE)
`care_agent_windows/`	Customer Care Agent source (service + tray)

Path	What it holds
`wa-service/`	Node WhatsApp service (integration disabled)
`tests/`	172 pytest files
`docs/` , `qa_docs/`	BRDs, plans/specs, audits, design system
`.claude/skills/`	7 project workflow skills
`.github/workflows/`	`deploy.yml` (self-hosted deploy), `build-agent.yml` (agent EXE)

Open questions

1. Is `.github/workflows/deploy.yml` still active? It rsyncs to `/opt/oxypc` and runs `systemctl restart oxypc`, but the Docker cutover disabled `oxypc.service` (docker spec §9). If a self-hosted runner is still registered it may conflict with the Docker auto-deploy.
2. Is there a public deployment at `app.oxypc.com` (named in `README.md` for `/fieldops`) or on Railway (mentioned in `main.py`, `limiter.py`, `deploy/oxypc-autodeploy.timer`)? Its host, owner and data are not in the repo.
3. Which ecosystem apps actually receive webhooks today, and which `OXYPC_ALLOWED_ORIGINS` are set in production?
4. Is the OxyQC standalone app (described in `CLAUDE.md` as offline-capable) still in use, or has it been replaced by the browser IQC page plus `oxyqc-agent`?
5. Is the Care Agent deployed to any customer yet? Its README says "not a shipped product".
6. nginx version on the Internal Server is not recorded.
7. Should `routers/repair.py.bak` and `wa-service/` be removed from the tree?

PART B

Data Model & Dictionary

This document describes how the OxyPC Inventory Lifecycle app stores its data. It is written from the SQLAlchemy models in `models/`, which are the real source of truth. `docs/schema.dbml` and `docs/databasetablesoxypcapp.pdf` are older snapshots (see section 6).

Every claim names the file it comes from. Where the code does not say something, this document says "not found" rather than guessing.

---

1. At a glance

Item	Value	Source
Database	PostgreSQL, async SQLAlchemy (`create_async_engine`)	`database.py`
ORM base	`database.Base` (`DeclarativeBase`)	`database.py`
Tables defined in `models/`	147 (`__tablename__` count)	`models/*.py`
Tables in `docs/schema.dbml`	89 `Table` blocks (file header still says "Tables: 57", generated 2026-04-29)	`docs/schema.dbml`
Separate FieldOps tables (own metadata, no FKs into OxyPC)	`fieldops_users` , `fieldops_records` , `fieldops_admin_audit`	`fieldops_db.py`
Primary keys	UUID (`UUID(as_uuid=True)` , `default=uuid.uuid4`) almost everywhere	`models/*.py`
Connection pool	`pool_size` 8, `max_overflow` 4 per worker (env `OXYPC_DB_POOL_SIZE` / `OXYPC_DB_MAX_OVERFLOW`)	`database.py`
Session pattern	one `AsyncSession` per request; commit on success, rollback on error	`database.py` `get_db()`

How the schema changes

The schema is changed in three different ways. All three are in use, which is the main reason the docs drift.

Mechanism	What it does	Source
Alembic	36 revisions, from `20260331_1600_..._initial_baseline.py` to `20261013_1200_dealers_sub_category_website_cid.py`	`alembic/versions/`
`db_validator.py` at startup	Creates missing tables (`create_all(checkfirst=True)`), adds missing columns (`ALTER TABLE ... ADD COLUMN IF NOT EXISTS`), seeds `stage_master` / `allowed_transitions`	`db_validator.py` (called from `main.py` startup)
One-shot `migrate_*.py` scripts	52 scripts at repo root, run by hand (e.g. `migrate_credit_note_workflow.py` , `migrate_work_orders_unique_constraint.py` , `migrate_perf_indexes.py`)	repo root

Many model comments say a column is "additive; auto-provisioned by db_validator" (for example `devices.entity` , `devices.stress_notes` , `spare_parts.crate_number`). Those columns have no Alembic revision.

2. Domains

Domain	What it covers	Main tables	Model files
Intake	Buying a lot, receiving it against an invoice (GRN), first inspection (IQC), creating the tag	`lots`, `lot_line_items`, `grn_imports`, `grn_lots`, `grn_stocked_tags`, `devices`, `iqc_inspections`, `crm_sourcing_deals`, `crm_purchase_orders`	`lot.py`, `grn_import.py`, `device.py`, `iqc_inspection.py`, `crm.py`
Inventory	Where each tag is, which stage it is in, how it moves	`devices`, `stage_movements`, `storage_locations`, `device_location_logs`, `buckets`, `stock_transfers`, `block_records`, `block_record_items`, `audit_records`, `audit_record_items`, `tag_number_movements`, `stage_master`, `allowed_transitions`, `device_aging`	`device.py`, `location.py`, `bucket.py`, `stock_transfer.py`, `block_tags.py`, `audit_record.py`, `stage_control.py`, `engines.py`
Repair	L1/L2 and L3/L4 work, work IDs, repair cost	`work_orders`, `repair_jobs`, `repair_attempts`, `device_costing`	`work_order.py`, `repair.py`, `engines.py`
QC	Stress test, legacy QC scoring, Final QC result	`stress_test_results`, `qc_checks`, final QC columns on `devices`, fail columns on `buckets`	`stress.py`, `qc.py`, `device.py`, `bucket.py`
Cosmetic	Paint line stages and who works each stage	`cosmetic_flow_rows`, cosmetic values of `devices.current_stage`	`cosmetic_flow.py`, `device.py`
Parts	Spare part master, stock, requests, PNA, parts GRN, part sales, estimates	`spare_parts`, `spare_parts_purchases`, `spare_parts_consumption`, `spare_parts_ledger`, `ram_tracking`, `part_requests`, `part_sourcing_requests`, `device_pna_parts`, `parts_grn`, `parts_grn_line_items`, `part_sale_requests`, `part_sales`, `part_estimates` (+3 line tables)	`spare_parts.py`, `engines.py`, `part_request.py`, `pna_part.py`, `parts_grn.py`, `part_sales.py`, `part_estimate.py`
Sales & CRM	Selling a tag, returns and credit notes, dispatch requests, warranty, contacts, deals, quotes, dealers, telecalling	`sales`, `returns`, `telecaller_dispatch_requests`, `companies`, `care_warranties`, `crm_contacts`, `crm_sales_opportunities`, `crm_quotes`, `dealers`, `dealer_orders`, `dealer_credit_notes`, `telecalling_records`	`sales.py`, `dispatch_request.py`, `company.py`, `care.py`, `crm.py`, `dealers.py`, `telecalling.py`
Finance	Money in and out, P&L inputs	`supplier_payments`, `customer_receipts`, payment columns on `crm_purchase_orders` / `crm_quotes`, `partner_bid_payments`, `device_costing`, `cost_config`, `business_pl_overrides`	`crm.py`, `partner_payment.py`, `engines.py`, `cost_config.py`, `business_pl_override.py`
Partner	Trade Partner (dealer) portal: listings, bookings, bids, payments	`partner_listings`, `partner_listing_devices`, `partner_bookings`, `partner_payment_proofs`, `partner_bids`, `partner_bid_documents`, `partner_bid_payments`, `lot_dealer_visibility`, `lot_booking_requests`, `partner_settings`, `partner_login_logs`	`partner.py`, `partner_payment.py`, `dealers.py`

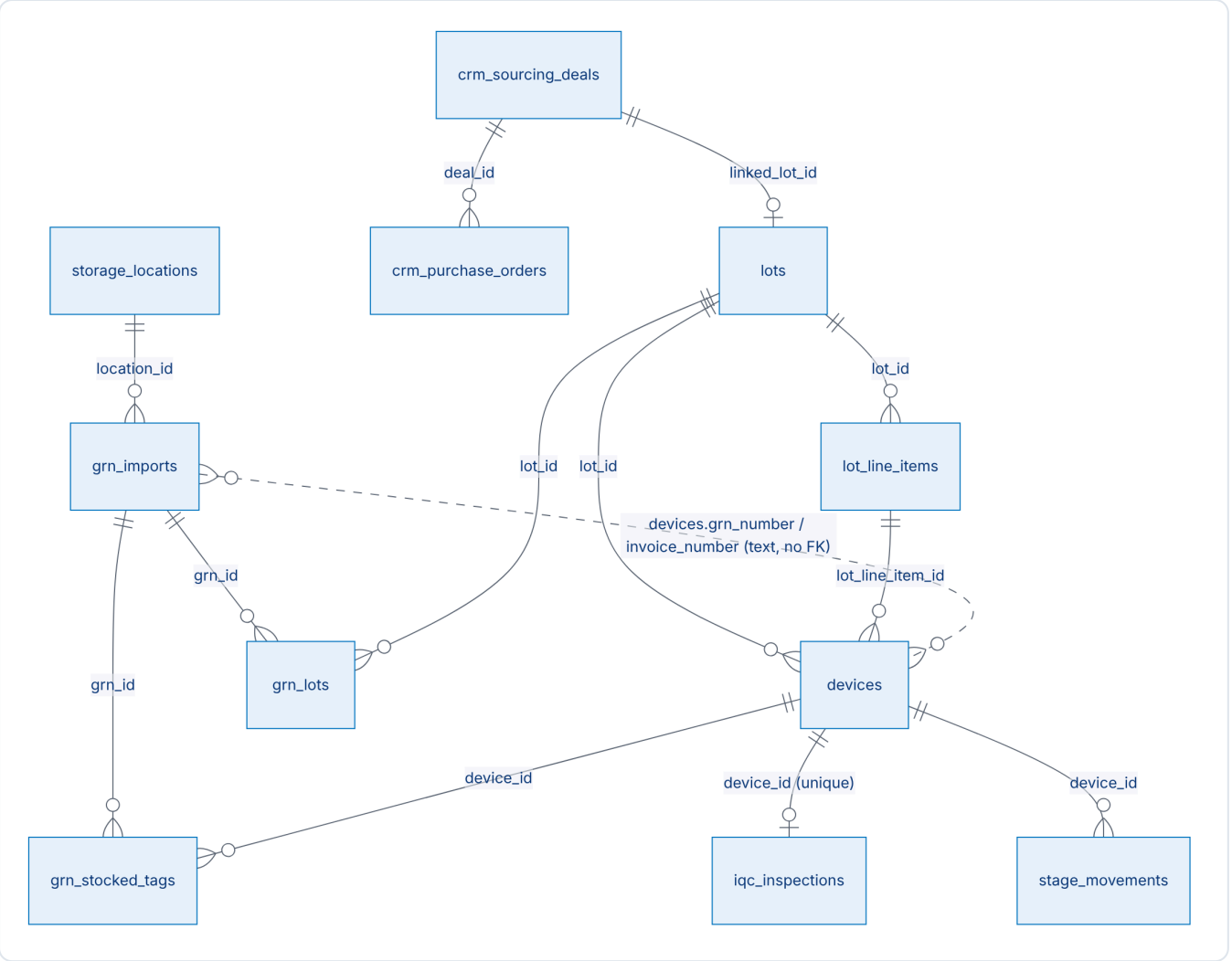
Domain	What it covers	Main tables	Model files
Admin / Audit	Users, roles, permissions, audit trail, events, API keys, settings	`users`, `login_logs`, `user_permissions`, `role_module_permissions`, `custom_roles`, `role_additional_permissions`, `audit_logs`, `event_log`, `webhooks`, `api_keys`, `app_settings`, `master_data`, `notifications`	`user.py`, `role_permissions.py`, `engines.py`, `event_log.py`, `webhook.py`, `api_key.py`, `settings.py`, `master.py`, `notification.py`

Not covered in detail here: QA/UAT tracking (`models/qa_uat.py`), Product Master / pricing (`models/product_master.py`), attendance (`models/attendance*.py`), Assign Leads (`crm_leads*` in `models/crm.py`), Customer Care tickets beyond warranty (`models/care.py`), and FieldOps (`fieldops_db.py`).

3. Entity-relationship diagrams

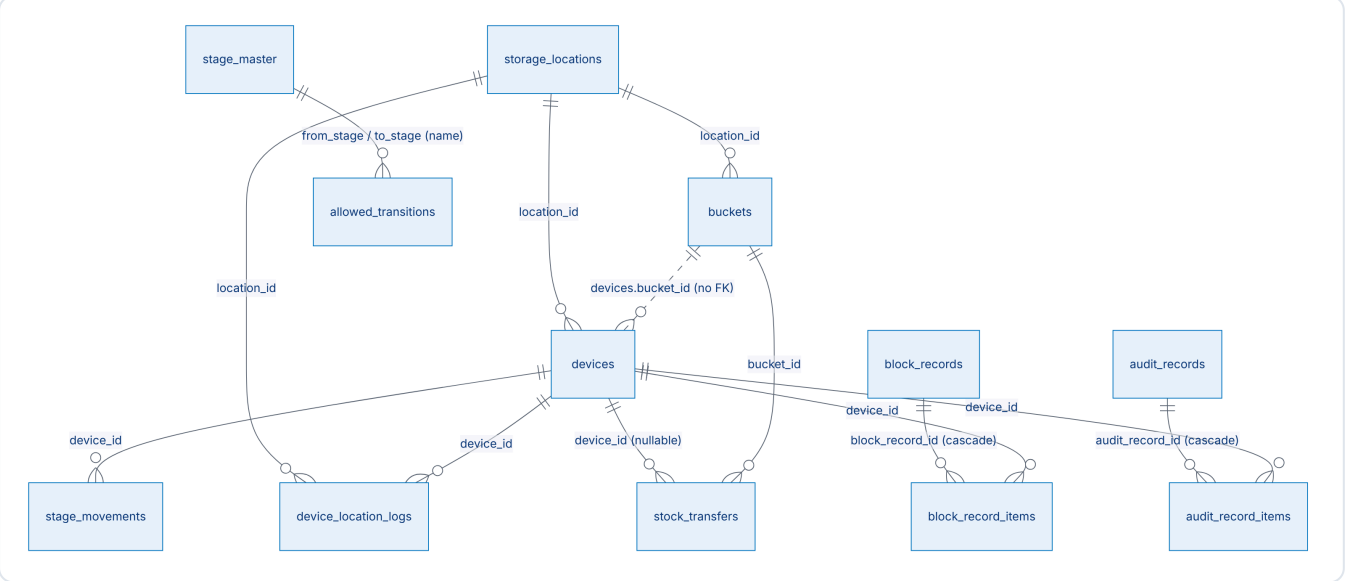
Solid lines are real foreign keys in the model. Dotted lines are links the code relies on but that have **no** foreign key (matched by a text value or an untyped UUID). Columns are left out to keep the diagrams readable; see section 5 for them.

3.1 Intake



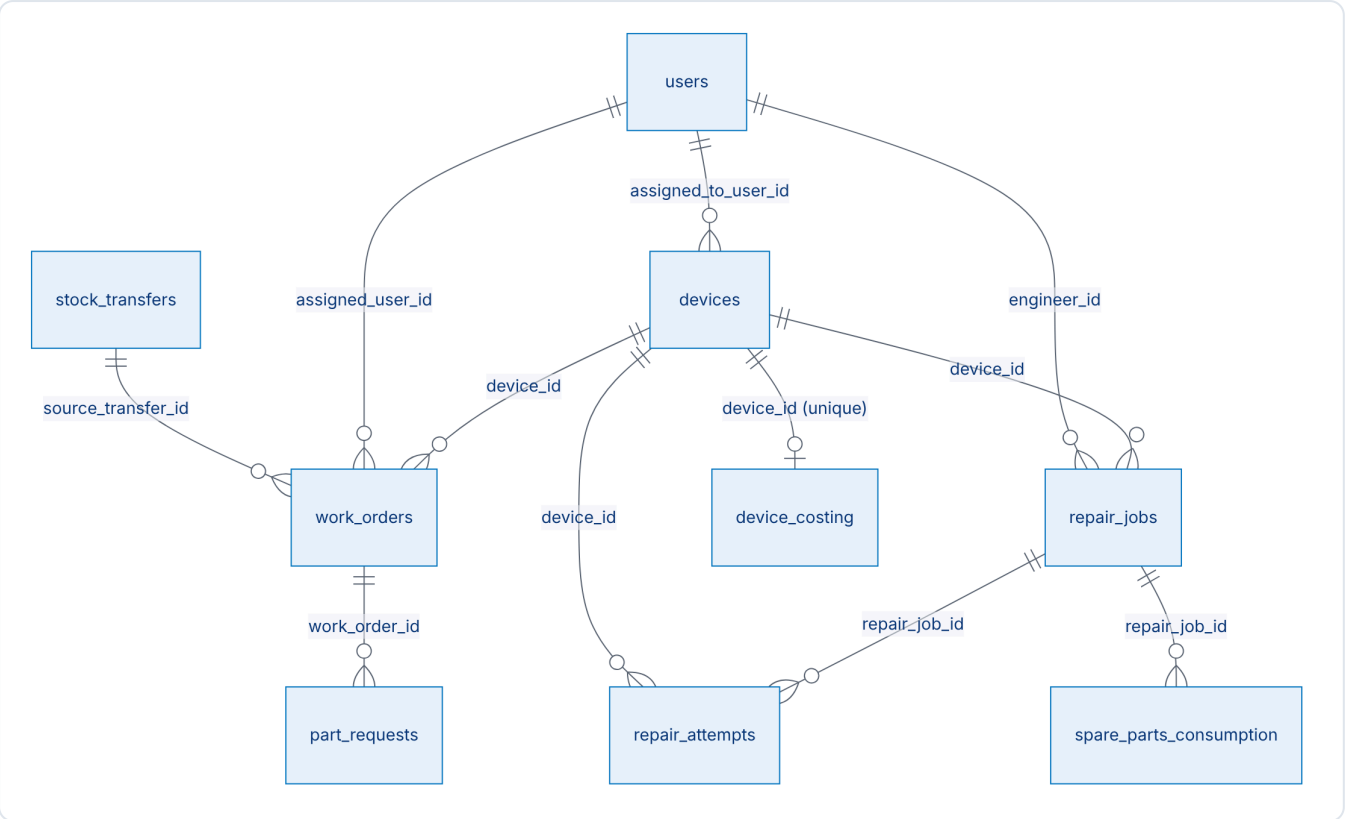
Sources: `models/crm.py`, `models/lot.py`, `models/grn_import.py`, `models/device.py`, `models/iqc_inspection.py`.

3.2 Inventory



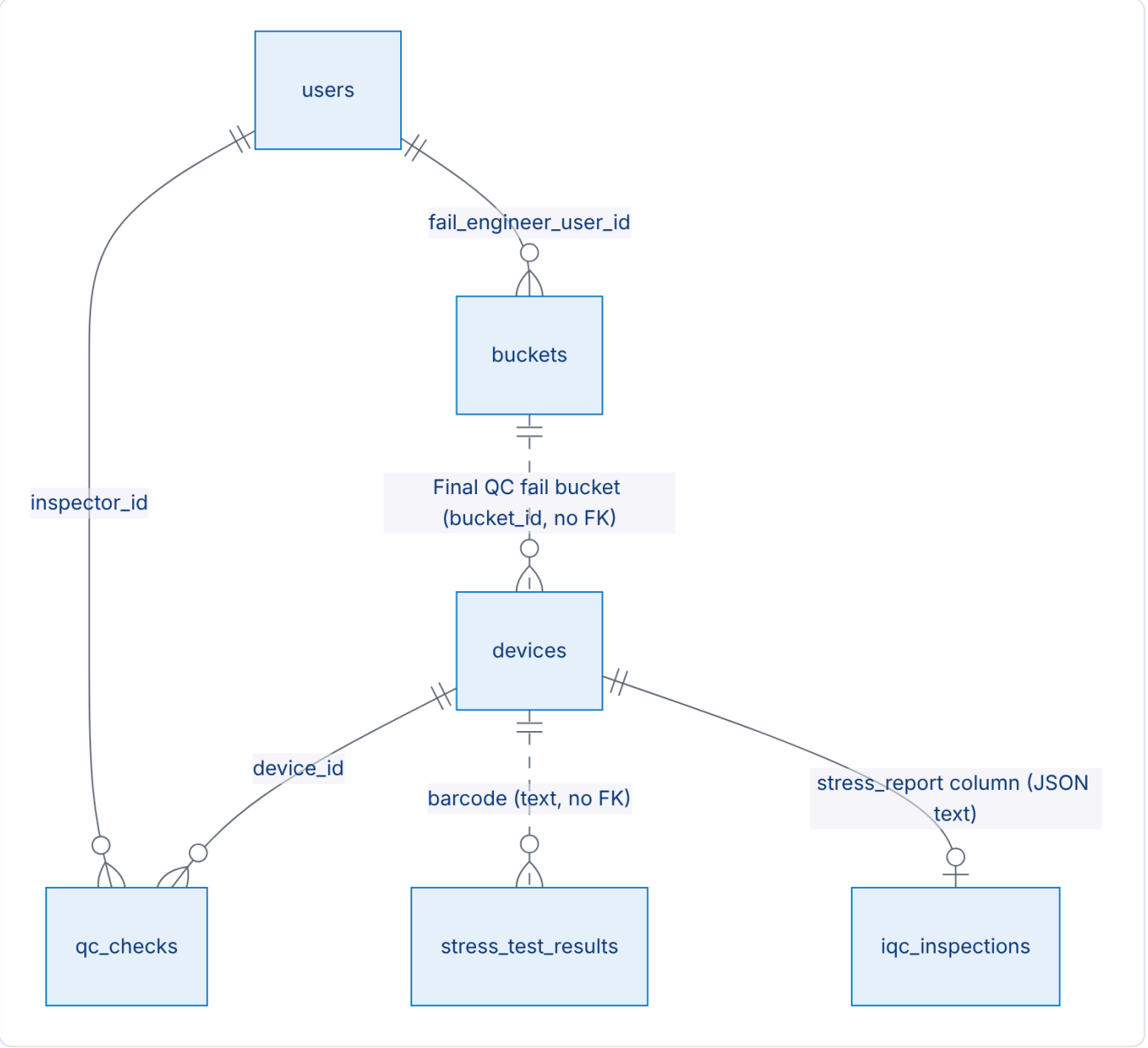
Sources: `models/device.py`, `models/location.py`, `models/bucket.py`, `models/stock_transfer.py`, `models/block_tags.py`, `models/audit_record.py`, `models/stage_control.py`.

3.3 Repair



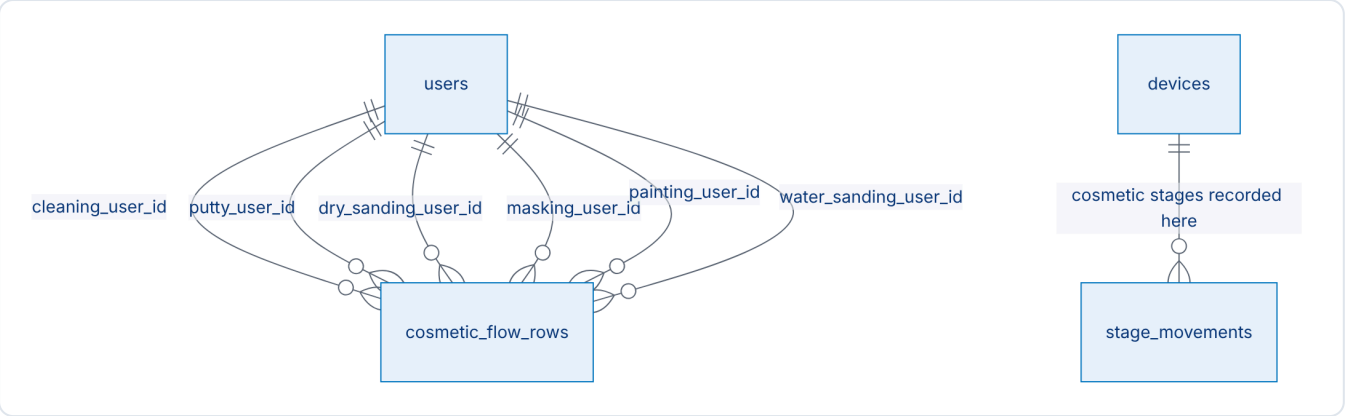
Sources: `models/work_order.py`, `models/repair.py`, `models/engines.py`, `models/spare_parts.py`, `models/part_request.py`.

3.4 QC and Final QC



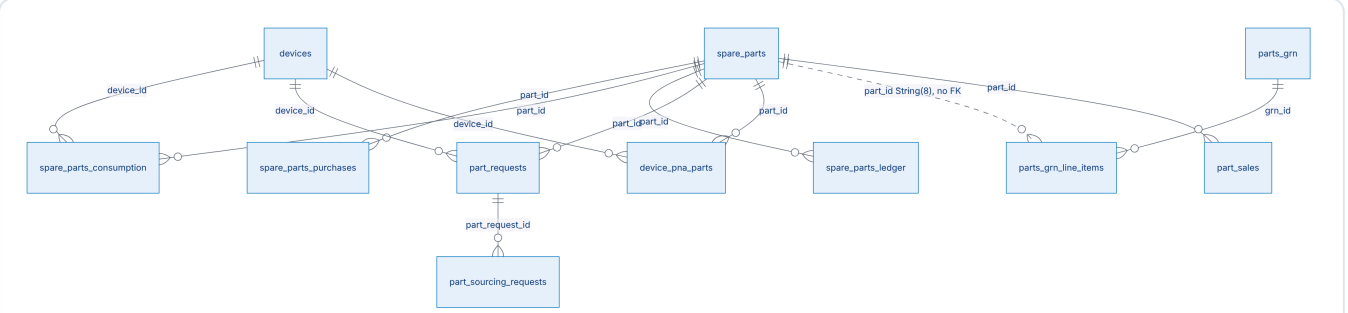
Final QC itself has no table. The result lives on `devices` (`final_qc_status`, `grade`, `fqc_*`, `touch_type`) and the stage moves to `final_qc_pass_hold` / `final_qc_fail_hold` (`models/device.py`). Sources: `models/qc.py`, `models/stress.py`, `models/bucket.py`.

3.5 Cosmetic



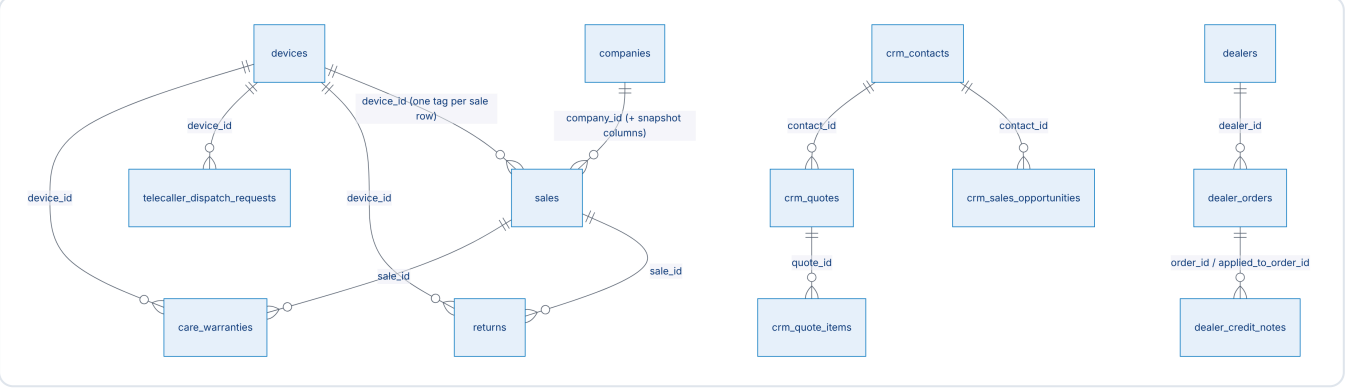
`cosmetic_flow_rows` is "purely a definition table today — nothing reads these rows yet to drive behaviour" (`models/cosmetic_flow.py` docstring). Progress through the paint line is tracked only through `devices.current_stage` and `stage_movements` (`COSMETIC_STAGES` in `models/device.py`).

3.6 Parts



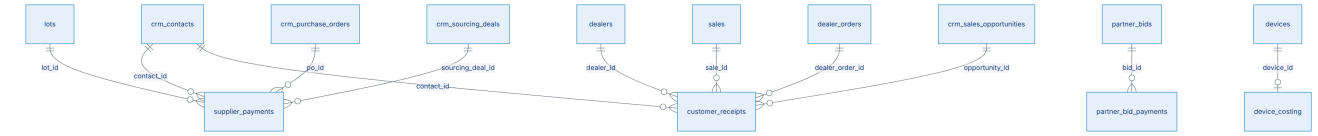
Sources: `models/spare_parts.py` , `models/engines.py` , `models/part_request.py` , `models/pna_part.py` , `models/parts_grn.py` , `models/part_sales.py` .

3.7 Sales, CRM and warranty



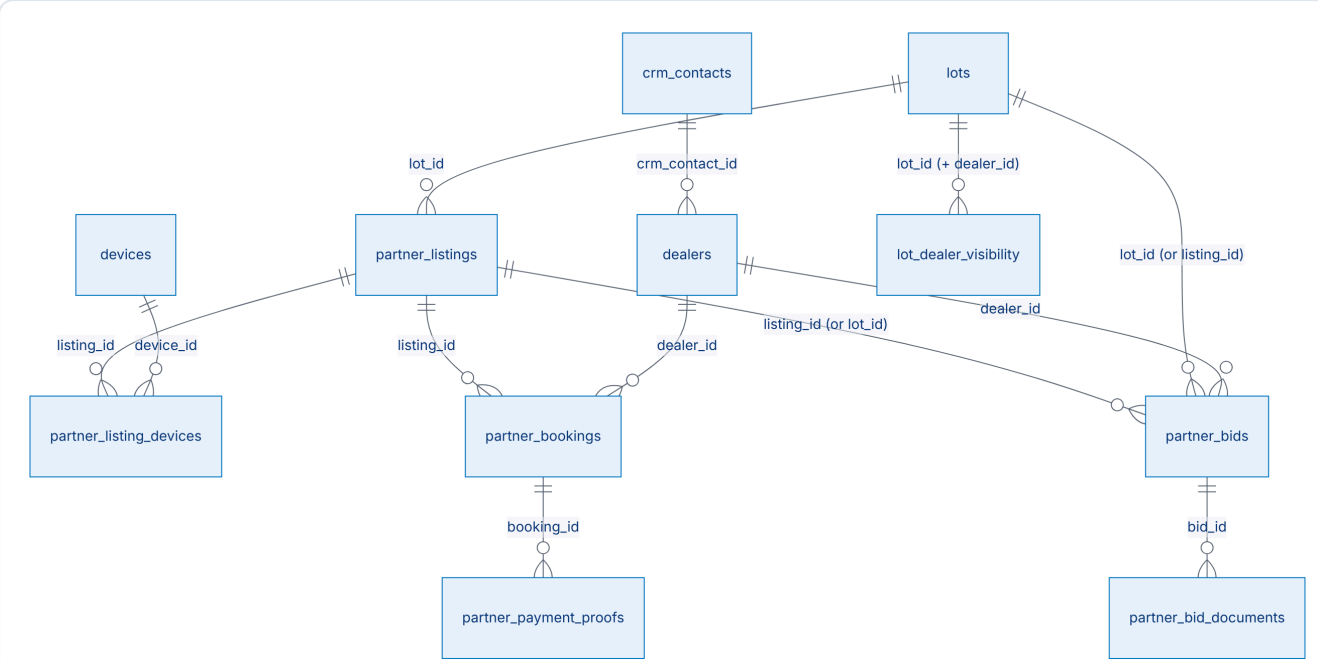
`crm_sales_opportunities.linked_sale_ids` is a comma-separated text list of sale UUIDs, not a FK (`models/crm.py`). Sources: `models/sales.py` , `models/company.py` , `models/dispatch_request.py` , `models/care.py` , `models/crm.py` , `models/dealers.py` .

3.8 Finance



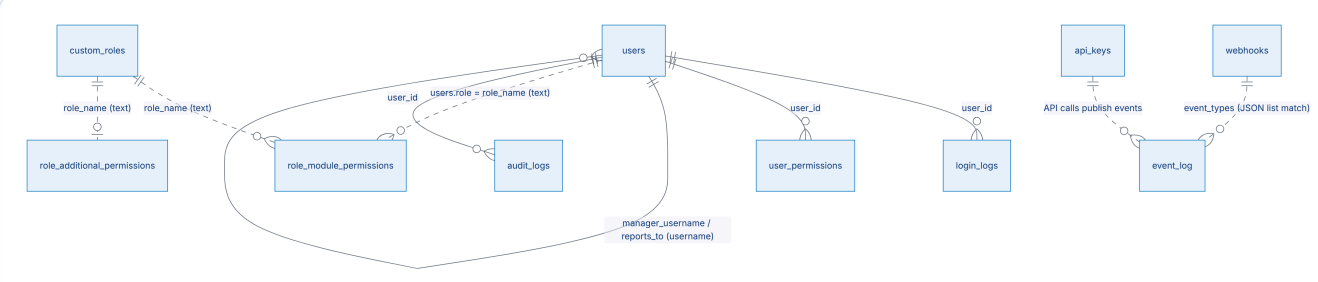
`cost_config` (key/value, `Numeric(10,2)`) and `business_pl_overrides` (year/month overrides) have no FKs. Sources: `models/crm.py`, `models/partner_payment.py`, `models/engines.py`, `models/cost_config.py`, `models/business_pl_override.py`.

3.9 Partner (Trade Partner portal)



`partner_bids.opportunity_id` points to `crm_sales_opportunities` on purpose without a FK ("a hard cross-context FK is exactly what the schema rules warn against", `models/partner.py`).

3.10 Admin and audit



Sources: `models/user.py`, `models/role_permissions.py`, `models/engines.py` (`AuditLog`), `models/event_log.py`, `models/webhook.py`, `models/api_key.py`.

4. Conventions

4.1 Naming

Convention	Detail	Source
Table names	snake_case, mostly plural (`devices` , `lots`). Exceptions: `parts_grn` , `device_costing` , `device_aging` , `event_log` , `cost_config` , `scrap_for_sale` , `partner_floor_config` , `attendance`	`models/*.py`
Primary key	`id` UUID. Exceptions: `stress_test_results.id` is an auto-increment Integer; `app_settings.key` and `partner_settings.key` are string PKs	`models/stress.py` , `models/settings.py` , `models/partner.py`
Tag Number	Stored as `devices.barcode` (String 100, unique). UI calls it "Tag Number"	`models/device.py`
Business numbers	`lot_number` ; `sale_number` = `SALE-####` from Postgres sequence `sale_number_seq` ; `work_id` 12 digits; `grn_imports.grn_number` 12 digits; `bucket_number` = `BKT` + 8 digits; `block_id` = `DDMMYYYY-CONTAINER-####` ; `CARE-0001` , `TPL-0001` , `TPB-0001` , `BID-0001` , `PO-2025-0001` , `SD-2025-0001` , `OPP-2025-0001` , `QT-2025-0001` , `CRM0001`	`routers/sales.py` (`nextval('sale_number_seq')`), `models/work_order.py` , `models/grn_import.py` , `models/bucket.py` , `models/block_tags.py` , `models/care.py` , `models/partner.py` , `models/crm.py`
"Who did it" columns	Usually a username string (`created_by` , `moved_by` , `sold_by` , `actioned_by`), not a FK to `users` . Some newer tables use real FKs (`block_records.created_by_user_id` , `audit_record_items.audited_by_user_id`)	`models/*.py`
Snapshots	Many tables copy values at the time of an action so later edits do not change history: `sales.company_*` , `block_record_items` device fields, `audit_record_items` device fields, `stock_transfers` device fields, `partner_bids.previous_amount/base_amount` , `partner_bookings.*_snapshot`	`models/sales.py` , `models/block_tags.py` , `models/audit_record.py` , `models/stock_transfer.py` , `models/partner.py`
Status columns	Mostly `String` with allowed values in a code comment, validated in routers/services. Only a few are real Postgres enums: `devicestage` , `devicegrade` , `repairstatus` , `blockitemstatus` , `zonetype` , `unittype` , `movementdirection` , plus location audit enums	`models/device.py` , `models/repair.py` , `models/block_tags.py` , `models/location.py`
Roles	`users.role` is free text via `RoleType` (String 60) so custom roles work; `UserRole` enum lists the 12 built-in roles	`models/user.py`

4.2 Soft delete

There is no single soft-delete pattern. Each table uses its own.

Table	Columns	Rule	Source
`lots`	`is_trashed` (Boolean, NOT NULL, default false), `trashed_at`	Filter with `is_trashed.isnot(True)` (project rule)	`models/lot.py` , `CLAUDE.md`
`devices`	`is_trashed` , `trashed_at` (Trash page) **and** `is_active` , `deleted_at` (deactivation)	Two separate flags. API v1 and stress save filter `is_active == True`	`models/device.py` , `routers/stress_api.py`

Table	Columns	Rule	Source
`spare_parts`, `parts_grn`	`is_trashed`, `trashed_at`		`models/spare_parts.py`, `models/parts_grn.py`
`scrap_for_sale`	`is_trashed`		`models/scrap_for_sale.py`
`crm_contacts`	`is_trashed`		`models/crm.py`
`grn_imports`	`is_deleted`, `deleted_at`	"GRN docs are kept + audit-logged, just hidden"; `grn_lots` rows are kept	`models/grn_import.py`
`dealers`	`trashed_at`, `trashed_by`	`trashed_at IS NULL` means live	`models/dealers.py`
`api_keys`, `webhooks`	`deleted_at` (+ `is_active`)		`models/api_key.py`, `models/webhook.py`
`device_pna_parts`	`is_active`	Unmarking sets false, never deletes	`models/pna_part.py`
`block_record_items`, `audit_record_items`, `lot_line_items`, `scrap_sale_items`, `as_is_lot_prices`	none	Hard-deleted with parent (`ondelete="CASCADE"` or ORM `delete-orphan`)	model files

Rule from `CLAUDE.md`: write an audit log entry before any `db.delete()`, because the row is gone after commit.

4.3 Money and decimals

Topic	Detail	Source
Column type	`Numeric(p, 2)`. Unit prices usually `Numeric(10,2)` or `(12,2)`; totals `Numeric(14,2)`; GST amounts on `lots` `Numeric(12,2)`, on `parts_grn` `Numeric(10,2)`	`models/lot.py`, `models/parts_grn.py`, `models/crm.py`
Currency	No currency column anywhere. All amounts are Indian Rupees by convention (₹ in templates, "Rs" in PDFs)	`templates/invoices/print.html`, `services/po_pdf.py`
Percentages	`Numeric(5,2)` / `(6,2)` (`min_margin_pct`, `floor_pct`)	`models/crm.py`, `models/partner.py`
Bid rounding	`Decimal`, rounded **up** to the whole rupee	`models/partner.py` `min_next_bid()`
Invoice GST maths	Done in Python `float` with `round(..., 2)`, not `Decimal`	`routers/invoices.py` `_compute_gst()`
PO PDF	Formats with `float(x)`	`services/po_pdf.py` `_rs()`
GST rates	`cost_config` keys `gst_rate_intra` / `gst_rate_inter`, default 18	`routers/invoices.py` `_fetch_gst_rates()`
Derived, not stored	`returns` balance, partner `balance_due`, `audit_records.difference`, partner payment status	`models/partner.py`, `models/audit_record.py`, `models/partner_payment.py`

The project's own skill note says priced documents should use "server-side Decimal totals" (`CLAUDE.md`). Invoice GST does not yet follow it.

4.4 Timestamps and time zone

Topic	Detail	Source
Default	`Column(DateTime, default=app_now)`, a **naive** datetime in the app time zone	`utils/timezone.py`
App time zone	`Asia/Kolkata` by default; changed at startup from `app_settings` and from Company Settings	`utils/timezone.py`
Time-zone-aware exceptions	`notifications.created_at` (`DateTime(timezone=True)`, `server_default=func.now()`) and `stress_test_results.run_at`	`models/notification.py`, `models/stress.py`
Date-only columns	`Date` for business dates: `grn_imports.grn_date/purchase_date`, `block_records.block_date`, `audit_records.report_date`, `supplier_payments.payment_date`, `devices.flag_*_date`	model files
Dates stored as text	`grn_imports.invoice_date` is `String(40)` (parsed from PDFs, "may be free-form")	`models/grn_import.py`
`updated_at`	Set by ORM `onupdate=app_now`; bulk `update()` statements do not trigger it	SQLAlchemy behaviour; models use `onupdate`

If the admin changes the time zone, older rows keep the old wall-clock values because no offset is stored.

5. Data dictionary

"Required" means `nullable=False` (or primary key) in the model. "App-required" means the column is nullable but the form or router insists on it. Types are SQLAlchemy types; `String(n)` is `varchar(n)`.

5.1 Lot — `lots` (`models/lot.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK	Row id	
lot_number	String(100)	Yes	Lot code, e.g. LOT-042	Unique, indexed. Test lots start `EPT-` (`models/master.py`)
supplier_name	String(200)	Yes	Supplier	
grn_system_number	String(50)	No	System GRN ref, e.g. GRN-178	
grn_number_new	Integer	No	Numeric GRN	
grn_date	DateTime	No	Material received date	
invoice_date	DateTime	No	Supplier invoice date	
invoice_no	String(100)	No	Supplier invoice no.	
invoice_value	Numeric(14,2)	No	Total invoice value	
taxable_amount	Numeric(14,2)	No	Taxable amount	
sgst / cgst / igst	Numeric(12,2)	No	GST amounts	
vehicle_number	String(30)	No	Vehicle	
e_way_bill	String(50)	No	Inbound e-way bill number (typed in)	Text only, not validated

Field	Type	Required	Meaning	Notes
po_number	String(100)	No	Our PO	
vendor_name	String(200)	No	Vendor from PO	May differ from supplier
buying_price	Numeric(12,2)	Yes	Total lot cost	
qty	Integer	Yes	Units in lot	
purchase_date	DateTime	Yes	Purchase date	
condition	String(30)	No	New / Refurbished / Used	
selling_price	Numeric(12,2)	No	Planned selling price	
notes	Text	No		
created_by	String(50)	No	Username	
created_at	DateTime	No		default `app_now`
is_trashed	Boolean	Yes	Soft delete	default false
trashed_at	DateTime	No	When trashed	
is_restricted	Boolean	Yes	Hide from partner catalog unless granted	default false; grants in `lot_dealer_visibility`

Child `lot_line_items`: `lot_id` (FK, required), `sub_category` (required), `brand`, `model`, `cpu`, `generation`, `ram_gb`, `has_ram`, `storage_gb`, `storage_type`, `has_storage`, `screen_size`, `grade` (A/B/C/D), `unit_price` (Numeric(12,2), required), `qty` (required, default 1), `notes`, `created_at`.

5.2 Device / Tag — `devices` (`models/device.py`)

One row per physical unit ("Tag Number").

Field	Type	Required	Meaning	Notes
id	UUID	PK		
barcode	String(100)	Yes	Tag Number	Unique, indexed
lot_id	UUID FK → lots	Yes	Source lot	
lot_line_item_id	UUID FK → lot_line_items	No	Invoice line it came from	
sub_lot_number	String(50)	No	As-Is sub-lot	Bulk-set by Change Floor / GRN Post-IQC
entity	String(50)	No	Owning company entity	e.g. OxyPC Computers / Renew Circuits; widened 30→50 on 2026-10-21
brand, model	String(50) / String(100)	No		brand indexed
device_type	String(30)	No		indexed
sub_category	String(20)	No	Laptop / Desktop / TFT	indexed
serial_no	String(100)	No		

Field	Type	Required	Meaning	Notes
invoice_number, po_number	String(100)	No	Invoice / PO on the tag	Replaced by sale invoice on sale (commit `eb48f29`, `253854f`)
grn_number	String(50)	No	GRN ref	indexed; text link to GRN
cpu, cpu_make, generation	String	No	Processor	
ram_gb, ram_type, ram_speed, ram_make, ram_summary	Integer / String	No	Memory	`ram_summary` packs several sticks
storage_gb, storage_type, hdd_capacity_gb, hdd_type, hdd_speed, hdd_summary	Integer / String	No	Storage	
total_ram_count, total_ram_size, total_hdd_count, total_hdd_size	String(50)	No	IQC counts	
battery_health_pct	Integer	No	0–100	
screen_size, color	String	No		
bios_password	Boolean	No		
grade	Enum `devicegrade`	No	Final grade	A, B, C, D, E, scrap
current_stage	Enum `devicestage`	Yes	Workflow stage	See enum below; default `iqc`; indexed
final_qc_status	String(10)	No	pass / fail	indexed
fqc_failure_reason, fqc_pass_notes, fqc_final_notes	Text	No	Final QC notes	
touch_type	String(10)	No	touch / non_touch	blank shown as NT
flag_authorized_by, flag_request_date, flag_target_date, flag_reason, flagged_at, flagged_by	String / Date / Text / DateTime	No	Urgent flag	Cleared when tag enters `stock_in` (event listener)
floor	String(50)	No	Zone value	
warehouse	String(100)	No	Legacy free text	Mirrors location name
location_id	UUID FK → storage_locations	No	Precise location	
bin_location	String(50)	No	Bin under location	Free text
bucket_id	UUID	No	Current bucket	**No FK**
assigned_to_user_id	UUID FK → users	No	Current owner employee	
l1l2_status	String(30)	No	New / Repair Started / Requested to L3/L4	default "New"
l34_status	String(30)	No	blank / Repair Started / Completed / Normal Scrap / Replacement Scrap	
l34_activity	String(100)	No	L3/L4 activity	Master Data list

Field	Type	Required	Meaning	Notes
repair_notes, stress_notes, notes	Text	No	Notes	
device_price	Numeric(12,2)	No	Unit buying price	
min_selling_price, max_selling_price	Numeric(12,2)	No	As-Is lot price band	
qty	Integer	No	Units this row covers	server default 1
return_status	Boolean	Yes	Returned via Process Return	default false
tag_return_status	String(120)	No	Return for Repair / Replaced by / Return for Credit Note	
replaced, replace_with_barcode	String	No	Replacement links	Tag text, no FK
scrap_verified	Boolean	Yes		default false
is_blocked	Boolean	Yes	Open Block Tags hold	Sell refused while true
partner_listed	Boolean	Yes	Backs a live partner listing	
is_active, deleted_at	Boolean / DateTime	Yes / No	Deactivation	
is_trashed, trashed_at	Boolean / DateTime	Yes / No	Trash	
created_at, updated_at	DateTime	No		`updated_at` indexed

`DeviceStage` values (in order): `grn`, `iqc`, `stock_in`, `l1`, `l2` (legacy, hidden from dropdowns), `l3`, `trc_production`, `qc_check` (labelled "Stress Test"), `cosmetic_received`, `cleaning`, `putty`, `dry_sanding`, `masking`, `painting`, `water_sanding`, `cosmetic_completed`, `final_qc`, `final_qc_pass_hold`, `final_qc_fail_hold`, `ready_to_sale`, `sold`, `returned`, `scrapped`, `scrap_for_sale`. `sold` is also hidden from generic stage pickers; only the sale flow sets it (`DROPDOWN_STAGES`).

Child `stage_movements`: `device_id` (FK, required), `from_stage`, `to_stage` (required, indexed), `moved_by`, `moved_at`, `exited_at`, `notes`.

5.3 IQC inspection — `iqc_inspections` (`models/iqc_inspection.py`)

One per device (`device_id` unique). Almost every check is `String(40)` holding a label.

Field	Type	Required	Meaning	Notes
id	UUID	PK		
device_id	UUID FK → devices	Yes	Inspected tag	Unique
inspector_name	String(100)	No		
inspected_at	DateTime	No		
power_on, bios_password, all_ok	String(40)	No	Functional status	Yes / No
status	String(50)	No	Power On / No Display / No Power	

Field	Type	Required	Meaning	Notes
screen_dot, screen_line, screen_functional, screen_discoloration, screen_broken, screen_flickering, screen_loose, screen_missing, screen_hinge_broken, screen_colour_spread, screen_keyboard_mark, screen_hard_press	String(40)	No	Screen checks	Yes / No
screen_patch	String(40)	No		No / Minor / Major
screen_scratch	String(40)	No		No / Minor Scratch / Major Scratch
touch_screen	String(40)	No		Yes / No / Yes - Faulty
panel_a_ , *panel_b_*, panel_c_, *panel_d_*	String(40)	No	Panels A (lid), B (base), C (bezel), D (palmrest): scratch, broken, missing, dent, colour_fade, rubber_cut (B only)	No / Yes / Major ...
keyboard_working, keyboard_colour_fade, keyboard_key_missing, keyboard_hard_press	String(40)	No	Keyboard	
speaker_status	String(50)	No		Both working / Both faulty / Left faulty / Right faulty
touchpad_working, touchpad_click_working, touchpad_scratch, touchpad_colour_fade, touchpad_missing, touchpad_logicboard	String(40)	No	Touchpad	
port_hdmi, port_usb_working, port_audio_jack	String(40)	No	Ports	Yes / Not Working / No
usb_a_ports, usb_c_ports, ethernet_ports	Integer	No	Port counts	
wifi_status	String(40)	No		Working / Faulty / Not Checked
webcam_status	String(40)	No		Ok / Faulty / Not Checked
hdd_connector, hdd_casing, battery_present, battery_cable, charging_port, dvd_drive	String(40)	No	Components	
cover_ram, cover_dvd, cover_storage, hinge_condition, hinge_cover	String(40)	No	Covers / hinge	
storage_health_pct, fan_sound_dba	Integer	No		
fan_working	String(40)	No		
r2v3_grade_category	String(40)	No	R2v3 category	C0 / C3 / C4 / C5
remarks	Text	No		
stress_report	Text	No	JSON from OxyQC stress run	Raw JSON string

5.4 GRN — `grn_imports`, `grn_lots`, `grn_stocked_tags` (`models/grn_import.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
grn_number	String(12)	Yes	12-digit GRN id	Unique, auto-generated
lot_number	String(60)	No	Lot text parsed from PDF	
lot_id	UUID FK → lots	No	First mapped lot	Legacy; `grn_lots` is source of truth since 2026-10-21
entity	String(50)	No	Owning entity	Backfilled to "OxyPC Computers" once
location_id	UUID FK → storage_locations	No	Default location for tags	Added 2026-10-21
invoice_number	String(100)	No		
invoice_date	String(40)	No		**Text**, not a date
sender_name	String(200)	No	"Vendor Name"	
quantity	Integer	No	Invoice quantity	
amount	Numeric(14,2)	No	Invoice price	
purchase_date, grn_date	Date	No		
po_number, vehicle_number, e_way_bill	String(50)	No		
notes	Text	No		
validated	Boolean	No	Validated via modal	
received_qty	Integer	No	Counted at validation	
validation_ref	String(100)	No		
validation_notes	String(500)	No		
source	String(20)	No	`invoice` / `post_iqc` / Manage Lots test value	
file_name, file_path	String	No	Uploaded invoice PDF	Under `uploads/grn`
file_hash	String(64)	No	SHA-256	indexed
created_by, created_at		No		
is_deleted, deleted_at	Boolean / DateTime	Yes / No	Soft delete	

`grn_lots`: `grn_id` (FK, required), `lot_id` (FK, required), unique (`grn_id`, `lot_id`), `created_by`, `created_at`. `grn_stocked_tags`: `grn_id`, `device_id`, `barcode_key` (upper-cased tag), `matched_invoice`, `source` (`invoice_match` / `grn_mapped` / `audit_recovery`), `created_at`; unique (`grn_id`, `barcode_key`); add-only.

5.5 Bucket — `buckets` (`models/bucket.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
bucket_number	String(30)	Yes	`BKT########`	Unique
name, location, category	String	No		`category` holds brand for normal buckets
status	String(20)	Yes		`stock_in` / `trc_pending` / `validated`
received_qty	Integer	No		
location_id	UUID FK → storage_locations	No		
assigned_to_production, _by, _at	Boolean / String / DateTime	Yes / No	Hand-off to production	
dept_assigned, _by, _at	Boolean / String / DateTime	Yes / No	Production Manager assigned	
fail_reason	String(30)	No	Final QC fail reason locked on first fail	
fail_engineer_user_id	UUID FK → users	No	Engineer for fail bucket	
fail_engineer_username, fail_engineer_name	String	No	Snapshot	
is_customer_return	Boolean	Yes	Created from Return Stock	
created_by, created_at, updated_at		No		

5.6 Work order / WorkID — `work_orders` (`models/work_order.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
work_id	String(12)	Yes	12-digit WorkID	Unique (enforced by `migrate_work_orders_unique_constraint.py`)
device_id	UUID FK → devices	Yes		
barcode	String(100)	No	Snapshot	
stage	String(5)	Yes	`l1` / `l2` / `l3`	
assigned_role	String(30)	No	e.g. `l1_engineer`	
assigned_user_id	UUID FK → users	No		
assigned_username, assigned_name	String	No	Snapshot	
status	String(20)	Yes		`pending` / `in_progress` / `completed`

Field	Type	Required	Meaning	Notes
requested_by_name	String(100)	No	L1/L2 engineer who requested L3/L4	
source_transfer_id	UUID FK → stock_transfers	No	Transfer that created it	
assigned_at	DateTime	Yes		
started_at, completed_at	DateTime	No		
paused_since	DateTime	No	Timeline paused (part out of stock)	
paused_days_total	Integer	Yes	Days paused so far	default 0
created_by, created_at		No		

Related `repair_jobs` (`models/repair.py`): `device_id`, `stage` (String(5)), `engineer_id` (FK users), `status` (enum `repairstatus`: `open` / `in_progress` / `completed`), plus L1/L2 fields (`faults`, `dust_cleaning`, `cmos_battery_change`, `thermal_paste`, `final_status` = Completed / PNA / Escalate to L4 / Scrap / Lot / Repair, `ram_*`, `hdd_*`) and L3 fields (`action_taken`, `problem_observed`, `scrap_reason`, `received_from`, `customer_internal`).

5.7 Part request — `part_requests` (`models/part_request.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
work_order_id	UUID FK → work_orders	No		
work_id	String(12)	No	Snapshot	
device_id	UUID FK → devices	Yes		
barcode	String(100)	No		
stage	String(5)	No	Page that raised it	`l1` / `l2` / `l3`
part_id	UUID FK → spare_parts	No		
part_name	String(150)	Yes	Label snapshot	
part_category, part_capacity, part_type, part_make, part_model	String	No		
request_type	String(20)	Yes		`new` / `replace` / `downgrade`
downgrade_type	String(20)	No	Only for downgrade	`normal` / `replace`
requested_by, engineer_name	String	No		
qty_requested	Integer	Yes		default 1
qty_handed_over	Integer	Yes		default 0

Field	Type	Required	Meaning	Notes
status	String(20)	Yes		`requested` / `handed_over` / `not_in_stock` / `procure` / `scrapped`
created_at, actioned_at, actioned_by		No		

`part_sourcing_requests` (same file): `status` `open` / `closed`; `source` `procure` / `production`; `verified`, `reupload_requested`, `confirmed` flags with `_at` / `_by`; `lot_id`, `estimate_id` FKs; `source_deal_id` is a String, not a FK.

5.8 PNA (Part Not Available) — `device_pna_parts` (`models/pna_part.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
device_id	UUID FK → devices	Yes		Unique with `part_name`
barcode	String(100)	No		indexed
part_name	String(150)	Yes		
part_category	String(100)	No		
part_id	UUID FK → spare_parts	No		
source	String(30)	Yes	Screen	`parts_consumption` / `l3l4`
marked_by, marked_at		No		
cleared_by, cleared_at		No		
is_active	Boolean	Yes	Still PNA	false = cleared, row kept

5.9 Spare part and Parts GRN — `spare_parts`, `parts_grn`, `parts_grn_line_items` (`models/spare_parts.py`, `models/parts_grn.py`)

`spare_parts`:

Field	Type	Required	Meaning	Notes
id	UUID	PK		
part_code	String(20)	Yes		Unique
name	String(150)	Yes		
category	String(30)	Yes		Options from `IQC_PART_CATEGORIES`
unit_price	Numeric(10,2)	Yes		default 0
qty_in_stock	Integer	Yes		Kept in sync with `spare_parts_ledger`
min_stock_alert	Integer	Yes		default 5
sold_qty	Integer	Yes	Sold as parts	Sellable = stock − handed over − sold
supplier, notes, source		No		

Field	Type	Required	Meaning	Notes
crate_number, make, model, part_lot	String	No		`part_lot` is free text, not a FK
part_type	String(20)	No	New / Replace / Upgrade / Downgrade	
created_at	DateTime	No		
is_trashed, trashed_at		Yes / No	Soft delete	

`parts_grn`: `grn_number` (String(20), unique, required); PO (`po_number`, `po_date`, `po_file`); invoice (`invoice_number`, `invoice_date`, `invoice_file`); e-way bill (`eway_bill_number`, `eway_bill_date`, `eway_bill_file`); vehicle (`vehicle_number`, `vehicle_seal_file`, `vehicle_image_file`); `status` (`in_plan` / `in_trc`, required); `date_received`, `vendor_name`; money (`invoice_value` Numeric(12,2), `sgst` / `cgst` / `igst` / `tax_amount` Numeric(10,2), `total_amount_invoice`); quantities (`total_po_qty`, `total_invoice_qty`, `total_physical_qty`); `created_by`, `created_at`, `is_trashed`, `trashed_at`.

`parts_grn_line_items`: `grn_id` (FK, required); `part_id` String(8) required (**not** a FK to `spare_parts.id`); `lot_number`, `po_number`, `grn_number`, `vendor_name`, `invoice_number`, `product_description`, `item_name`, `part_brand`, `part_model`, `part_name`, `invoice_qty`, `physical_qty`, `price` Numeric(10,2), `main_category`, `category`, `vehicle_number`, `invoice_ref`, `remarks`, `is_harvest` (Boolean, required), `created_at`.

5.10 QC / Stress report — `stress_test_results`, `qc_checks` (`models/stress.py`, `models/qc.py`)

`stress_test_results`:

Field	Type	Required	Meaning	Notes
id	Integer	PK		Auto-increment (not UUID)
barcode	String(100)	Yes	Tag Number	indexed; **no FK** to devices
brand	String(100)	No		
model_name	String(200)	No		
run_at	DateTime(tz)	Yes		Time-zone aware
duration	String(20)	No		
overall_status	String(30)	No		
results_json	JSON	No	Checklist `{item: "pass"/"fail"/"na"}` or automated run results	Items in `STRESS_CHECKLIST_ITEMS` (27 items); "Touch Screen" may be `na`
pdf_path	Text	No	Generated PDF	`static/stress_reports/` (`routers/stress_api.py`)
run_by	String(100)	No		

`qc_checks` (legacy scoring, still written by `routers/qc.py`): `device_id` (FK), `inspector_id` (FK users), `battery_score` / `screen_score` / `keyboard_score` / `body_score` (0–10), `total_score` (0–100), `result` (String(10), required, pass / fail), `grade` (A/B/C/D/S), `attempt_number` (required), `checked_at`, `notes`, `issues_found`, `send_to_stage`.

5.11 Cosmetic flow — `cosmetic_flow_rows` (`models/cosmetic_flow.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
label	String(100)	No	e.g. "Team A"	
cleaning_user_id, putty_user_id, dry_sanding_user_id, masking_user_id, painting_user_id, water_sanding_user_id	UUID FK → users	No	Person per stage	
created_by, created_at, updated_by, updated_at		No		

Not yet used to drive behaviour (model docstring). Only Admin and Cosmetic Manager can edit.

5.12 Final QC and grade — columns on `devices` (`models/device.py`)

Field	Type	Meaning	Notes
current_stage	Enum	`final_qc` → `final_qc_pass_hold` or `final_qc_fail_hold` → `ready_to_sale`	
final_qc_status	String(10)	`pass` / `fail`	
grade	Enum `devicegrade`	A, B, C, D, E, scrap	
touch_type	String(10)	`touch` / `non_touch`	
fqc_pass_notes	Text	Pass notes	
fqc_failure_reason	Text	Fail reason	Also locked onto `buckets.fail_reason`
fqc_final_notes	Text	Raw "Final Notes"	

There is no history table for Final QC decisions; only the latest value is kept. History is only in `stage_movements` and `audit_logs`.

5.13 Block Tag — `block_records`, `block_record_items` (`models/block_tags.py`)

`block_records`:

Field	Type	Required	Meaning	Notes
id	UUID	PK		
block_id	String(40)	Yes	`DDMMYYYY-CONTAINER-####`	Unique
block_date	Date	Yes		
container	Enum `unittype`	Yes		rack / crate / shelf / trolley / cabinet / floor
box_number	String(50)	No	Physical box	
location_zone	Enum `zonetype`	No		showroom, ground_floor, first_floor, second_floor, workshop, dispatch, warehouse, holding
block_for_user_id	UUID FK → users	No	Held for	

Field	Type	Required	Meaning	Notes
customer_name	String(150)	No		
created_by_user_id	UUID FK → users	Yes		
created_at	DateTime	No		

`block_record_items`: `block_record_id` (FK, cascade), `device_id` (FK), snapshot (`barcode` required, `lot_number`, `model`, `cpu`, `ram_gb`, `storage_gb`), `status` (enum `blockitemstatus`: `blocked` / `released`, required), `status_changed_by_user_id`, `status_changed_at`, `notes`, `created_at`. A tag sold from a block is closed as `released` with `notes` starting `"Sold — "` + sale number. While open, `devices.is_blocked` is true.

5.14 Sale — `sales` (`models/sales.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
sale_number	String(20)	Yes	`SALE-####`	Unique, from `sale_number_seq`
device_id	UUID FK → devices	Yes	The one tag sold	One row per tag
sale_price	Numeric(12,2)	Yes	GST-inclusive price	Invoice back-calculates tax
customer_name	String(100)	No		
customer_phone	String(20)	No		
customer_state	String(100)	No	Decides CGST+SGST vs IGST	
customer_address	Text	No		
invoice_no	String(50)	No	Typed invoice no.	Falls back to `sale_number` on print
payment_mode	String(50)	No	From Master Data `payment_mode`	
payment_reference	String(100)	No	Cheque / UTR	
sold_by	String(50)	No	Operator username	indexed
sales_person	String(100)	No	Person credited	Free text
sold_at	DateTime	No	Sale date	Admin can change (commit `8a0091b`)
notes	Text	No		
transport_mode	String(30)	No	courier / hand_delivery / self_pickup	
transport_via	String(100)	No	Courier name	

Field	Type	Required	Meaning	Notes
tracking_number	String(100)	No	AWB	
dispatch_date	DateTime	No		Set by Gate Pass exit
delivery_status	String(30)	No	pending / dispatched / delivered	
invoice_file_path	String(500)	No	Uploaded invoice PDF	`uploads/sale_invoices/`
warranty_type	String(20)	No	none / 30_days / 6_months / 1_year	default `none`
warranty_expires_at	DateTime	No	Computed server-side	`utils/warranty.py`
warranty_days	Integer	No	Running total after extensions	
sale_channel	String(20)	No	procurement / telecaller / showroom	
company_id	UUID FK → companies	No	Selling company	
company_name, company_address, company_gstin, company_state, company_state_code, company_phone, company_email	String	No	Snapshot for the legal invoice	Null on older sales (live fallback)

5.15 Gate Pass / Dispatch — no own table

Concept	Where it lives	Source
Telecaller request to sell	`telecaller_dispatch_requests`: `device_id` (FK, required), `barcode`, `telecaller_username`, `telecaller_name`, `qty_requested`, `qty_available`, `grade`, `status` (`requested` / `approved` / `rejected`), `rejected_notes/_at/_by`, `source` (`device` / `lot`), `model_name`, `model_make`, `created_at`, `approved_at`, `approved_by`	`models/dispatch_request.py`
Gate exit	`sales.delivery_status` set to `dispatched`, `sales.dispatch_date` stamped; audit actions `GATE_EXIT_CONFIRMED` / `GATE_EXIT_BULK_CONFIRMED`	`routers/gate_pass.py`
Delivery Challan	Rendered on demand from `sales` + `devices` + `lots`; nothing stored	`routers/invoices.py` `/invoices/waybill/{sale_id}`

5.16 Return / Credit Note — `returns`, `dealer_credit_notes` (`models/sales.py`, `models/dealers.py`)

`returns`:

Field	Type	Required	Meaning	Notes
id	UUID	PK		
sale_id	UUID FK → sales	Yes		

Field	Type	Required	Meaning	Notes
device_id	UUID FK → devices	Yes		
return_date	DateTime	No		
reason, condition_on_return	Text / String(50)	No		
action_taken	String(30)	No	restock / scrap / credit	
reentered_stage	String(50)	No	Stage the tag re-entered	Model comment: "iqc (default)"
processed_by	String(50)	No		
refund_amount	Numeric(12,2)	No	Paid back to customer	
notes	Text	No		
approval_status	String(20)	No	pending / approved / rejected	default pending
approved_by, approved_at, rejection_reason		No		
return_type	String(20)	No	customer / dealer	
serial_captured	String(100)	No		
warranty_status	String(20)	No	in_warranty / out_of_warranty / no_warranty	Server-computed
complaint_text	Text	No		
repair_cost, labour_cost	Numeric(12,2)	No	After repair	
customer_email, customer_name, customer_phone, customer_state, customer_address	String / Text	No	Also used as "Sender Details" for debit notes	
cn_number	String(50)	No	Old single-tag CN	No longer written
cn_stage	String(30)	No	Credit Note workflow stage	`Return Received` → `Debit Note Verified` → `Verify Payment` → `Payment Invoice Done` → `Restock Pending` → `CN Complete` (`CN_STAGES`)
debit_note_number	String(50)	No		
debit_note_amount	Numeric(12,2)	No		
payment_invoice	String(100)	No		

`dealer_credit_notes` : `credit_number` (unique, required), `dealer_id` (FK, required), `order_id` (FK), `credit_date` , `amount` (Numeric(14,2), required), `reason` , `items_description` , `notes` , `created_by` , `created_at` , `is_applied` (required), `applied_at` , `applied_to_order_id` (FK).

5.17 Payment — `supplier_payments`, `customer_receipts`, `partner_bid_payments` (`models/crm.py`, `models/partner_payment.py`)

`supplier_payments` (money out):

Field	Type	Required	Meaning	Notes
id	UUID	PK		
contact_id	UUID FK → crm_contacts	No	Supplier	
lot_id	UUID FK → lots	No		
po_id	UUID FK → crm_purchase_orders	No		
sourcing_deal_id	UUID FK → crm_sourcing_deals	No		
payment_date	Date	Yes		default today
amount	Numeric(14,2)	Yes		
payment_mode	String(20)	No	cash / upi / neft / cheque / rtgs	
reference_no	String(100)	No	UTR / cheque	
is_advance	Boolean	No		
notes, invoice_path, payment_photo_path		No	Files under `uploads/`	
created_by, created_at		No		

`customer_receipts` (money in): same shape with `dealer_id`, `sale_id`, `dealer_order_id`, `opportunity_id` FKs, `receipt_date` (Date, required), `amount` (required).

`partner_bid_payments`: `bid_id` (FK, required), `payment_date`, `payment_mode` (`bank_transfer` / `upi`), `payment_utr`, `payment_amount` (Numeric(14,2)), `notes`, `submitted_by`, `submitted_at`, `verified` (required), `verified_by`, `verified_at`, `verify_notes`. Status shown to users is derived, never stored: PO Pending → Payment Pending → Payment Verifying → Payment Done (`payment_status()`).

`crm_purchase_orders` and `crm_quotes` also carry `payment_status` (pending / paid), `utr_number`, `payment_snapshot_path`, `invoice_path`.

5.18 Warranty — `sales` warranty columns and `care_warranties` (`models/sales.py`, `models/care.py`)

Two warranty sources exist.

Source	Fields	Notes
`sales`	`warranty_type`, `warranty_expires_at`, `warranty_days`	Durations in `utils/warranty.py`: none, 30 days, 182 days, 365 days. Used by Extended Warranty page

Source	Fields	Notes
`care_warranties`	`device_id` (FK, required), `sale_id` (FK), `policy_id`, `coverage_type` (main_device / battery / adapter / accessory), `start_event` (invoice_date / delivery_date / replacement_dispatch), `start_date`, `expiry_date`, `battery_expiry_date`, `status` (required; `not_started`, `active`, `expiring`, `expired`, `void`, `suspended`, `replaced`, `returned`, `bought_back`), `extension_reference`, `replacement_reference`, `void_reason`, `created_at`, `updated_at`	Read by `services/care_service.py` `resolve_warranty()`. No code in `routers/` or `services/` creates rows (grep for `CareWarranty(` finds none)

5.19 User — `users` (`models/user.py`)

Field	Type	Required	Meaning	Notes
id	UUID	PK		
username	String(50)	Yes	Login	Unique (`uq_users_username`)
full_name	String(100)	Yes		
password_hash	String(200)	Yes		
role	RoleType (String(60))	Yes	Built-in or custom role	default `sales`
status	Boolean	No	Active	`is_active` property
email	String(150)	No		
whatsapp_number	String(20)	No	Former WhatsApp module	
designation	String(100)	No	Job title	
reports_to	String(50) FK → users.username	No	Org chart	
manager_username	String(50) FK → users.username	No	Attendance / team rollups	
entity	String(50)	No (app-required)	Company entity	Added 2026-10-17
tenant	String(50)	No	default `oxypc_internal`	Multi-tenant placeholder
created_at, created_by, last_login		No		

Built-in roles (`UserRole`): admin, inventory_manager, iqc_inspector, l1_engineer, l2_engineer, l3_engineer, qc_inspector, sales, spare_parts_manager, telecaller, sales_manager, cosmetic_manager.

5.20 Role / Permission — `role_module_permissions`, `custom_roles`, `role_additional_permissions`, `user_permissions` (`models/role_permissions.py`, `models/user.py`)

Table	Fields	Notes
`role_module_permissions`	`role_name` (String(60), required), `module` (String(60), required), `can_enable`, `can_add`, `can_edit`, `can_upload`, `updated_at`, `updated_by`	Unique (`role_name`, `module`). Admin bypasses all checks
`custom_roles`	`role_name` (unique), `display_name`, `created_by`, `created_at`	
`role_additional_permissions`	`role_name` (unique) + `can_upload`, `can_download`, `can_export`, `can_print`, `can_add_new_data` (legacy), `can_view_pricing`, `can_set_price`, `can_add_lot`, `can_add_iqc`, `can_add_grn`, `can_edit_devices`, `can_edit_iqc`, `can_edit_lot`, `can_edit_grn`, `updated_at`, `updated_by`	All Boolean, default true
`user_permissions`	`user_id` (FK), `permission` (String(100)), `granted`, `granted_by`, `granted_at`	Per-user flags

Permissions are cached in memory (`_PERM_CACHE`, `_ADDITIONAL_PERM_CACHE`) and refreshed when an admin saves the matrix.

5.21 Audit record and event log — `audit_logs`, `event_log`, `audit_records` (`models/engines.py`, `models/event_log.py`, `models/audit_record.py`)

`audit_logs` (the audit trail):

Field	Type	Required	Meaning	Notes
id	UUID	PK		
user_id	UUID FK → users	No		
username	String(50)	No		
action	String(50)	Yes	e.g. STAGE_MOVED, SALE_CREATED, GATE_EXIT_CONFIRMED	
table_name	String(50)	No		
record_id	String(50)	No		
old_value, new_value	Text	No	JSON snapshots	
ip_address	String(45)	No		
notes	Text	No		
timestamp	DateTime	Yes		indexed

Written by `services/audit_engine.py` `audit()`. The model says "No UPDATE or DELETE permitted", but this is a convention; no database trigger or grant enforcing it was found.

`event_log` (integration events): `event_type` (String(50), required), `payload` (JSON, required), `source_module`, `published_at` (required), `webhook_attempts` (required), `last_attempt_at`, `last_status_code`. Written by `services/webhook_dispatcher.py`.

`audit_records` / `audit_record_items` are **physical stock counts**, not an audit trail. `audit_records`: `report_date` (required), `my_stage` (enum), `my_location_id`, `category`, `current_count`, `physical_count`, `notes`. Items snapshot the device and compare `system_stage` / `system_location_id` with `my_stage` / `my_location_id`, plus `audited_by_user_id` (required).

Other audit-like tables: `care_audit_logs` (Care module), `login_logs`, `partner_login_logs`, `stage_movements`, `device_location_logs`, `fieldops_admin_audit`.

---

6. Gaps and inconsistencies

6.1 `docs/schema.dbml` versus models

`docs/schema.dbml` was generated on 2026-04-29 and last changed in commit `fac334c`. `docs/databasetablesoxypcapp.pdf` is a jsPDF export dated 29 Apr 2026 of the same diagram. Both are well behind the code.

Gap	Detail
Tables missing from dbml (58)	`api_keys`, `as_is_lot_prices`, `attendance_groups`, `attendance_group_members`, `audit_records`, `audit_record_items`, `block_records`, `block_record_items`, `buckets`, `business_pl_overrides`, `cosmetic_flow_rows`, `cost_config`, `crm_contact_locations`, `crm_contact_team_members`, `crm_leads`, `crm_lead_groups`, `crm_lead_calls`, `custom_roles`, `deals`, `deal_items`, `dealer_quotations`, `dealer_quotation_items`, `device_pna_parts`, `event_log`, `grn_imports`, `grn_lots`, `grn_stocked_tags`, `hw_ram`, `hw_storage`, `model_requests`, `notifications`, `part_estimates`, `part_estimate_lines`, `part_estimate_checklist_lines`, `part_estimate_open_lines`, `part_sale_requests`, `part_sales`, `partner_bids`, `partner_bid_documents`, `partner_bid_payments`, `parts_grn_line_items`, `pm_products`, `pm_dropdown_values`, `pm_competitive_lists`, `pm_competitor_prices`, `pm_calc_uploads`, `pm_calc_rows`, `role_module_permissions`, `role_additional_permissions`, `scrap_for_sale`, `scrap_sales`, `scrap_sale_items`, `stock_validations`, `stress_test_results`, `tag_number_movements`, `telecalling_assignments`, `tickets`, `webhooks`
Tables with column drift (26)	Largest: `devices` (38 columns missing from dbml, e.g. `entity`, `bucket_id`, `is_blocked`, `location_id`, `fqc_*`, `flag_*`), `sales` (20, incl. all `company_*` snapshot and warranty columns), `returns` (20, incl. the whole Credit Note workflow), `crm_contacts` (19 KYC/consent columns), `dealer_calls` (15), `iqc_inspections` (14), `telecalling_records` (13), `part_sourcing_requests` (13), `parts_grn` (10), `spare_parts` (9), `crm_purchase_orders` (8), `stock_transfers` (7), `part_requests` (7), `users` (6), `dealers` (6), `lots` (5, incl. `is_trashed`)
Columns in dbml but not in models	None found
Header count	dbml header says "Tables: 57" but contains 89 `Table` blocks

Several model comments say a change was "approved via dbdiagram" (for example `grn_lots`, `grn_stocked_tags`, `grn_imports.entity`, `grn_imports.location_id` on 2026-10-21, `users.entity` on 2026-10-17), but the approved diagram was not saved back to `docs/schema.dbml`. `CLAUDE.md` requires dbdiagram approval for schema changes, so the repo copy should be refreshed after each approval.

6.2 Missing or soft foreign keys

Column	Points to	Problem
`devices.bucket_id`	`buckets.id`	Plain UUID, no FK (`models/device.py`)
`stress_test_results.barcode`	`devices.barcode`	Text match only (`models/stress.py`)
`parts_grn_line_items.part_id`	spare part	`String(8)` , not a FK (`models/parts_grn.py`)
`part_sourcing_requests.source_deal_id`	`crm_sourcing_deals.id`	String(50) (`models/part_request.py`)
`crm_activities.deal_id`	sourcing deal or sales opportunity	Polymorphic, typed by `deal_type` (`models/crm.py`)
`crm_sales_opportunities.linked_sale_ids`	`sales.id`	Comma-separated text (`models/crm.py`)
`partner_bids.opportunity_id`	`crm_sales_opportunities.id`	Deliberately no FK (`models/partner.py`)
`devices.grn_number` , `devices.invoice_number`	`grn_imports`	Text match (`models/device.py` , `services/grn_stocked.py`)
`devices.replaced` , `replace_with_barcode`	other device	Tag text
`role_module_permissions.role_name`	`users.role` / `custom_roles.role_name`	Text
Most `*_by` columns	`users.username`	Text, survives user deletion but not renames

6.3 Other inconsistencies

Topic	Detail	Source
Three stage lists	`DeviceStage` has 24 values. `STAGE_SEQUENCE` / `DEFAULT_TRANSITIONS` in `models/stage_control.py` and `CANONICAL_STAGES` / `CANONICAL_TRANSITIONS` in `db_validator.py` each list fewer (both omit `putty` , `final_qc_pass_hold` , `final_qc_fail_hold` , `scrap_for_sale` ; `stage_control.py` also omits `grn` , `cosmetic_received` , `cosmetic_completed`)	`models/device.py` , `models/stage_control.py` , `db_validator.py`
Grade scales	`devices.grade` A–E + scrap; `qc_checks.grade` A/B/C/D/S; `lot_line_items.grade` A/B/C/D; CRM `GRADES` A, B, C, D, Mix, As-Is, Scrap	`models/device.py` , `models/qc.py` , `models/lot.py` , `models/crm.py`
Two soft-delete flags on devices	`is_trashed` and `is_active` mean different things; queries must check the right one	`models/device.py`
`CLAUDE.md` says `lots.is_trashed` is nullable	Model has `nullable=False, server_default='false'` . The `isnot(True)` rule is still safe	`models/lot.py` , `CLAUDE.md`
Two warranty stores	`sales.warranty_*` and `care_warranties` ; the second has no writer	`models/sales.py` , `models/care.py`
Legacy columns kept	`crm_contact_numbers.contact_id` , `returns.cn_number` , `role_additional_permissions.can_add_new_data` , `devices.warehouse` , `grn_imports.lot_id` , unused System Stage/Location columns on `audit_records`	model comments

Topic	Detail	Source
Repair stage field width	`work_orders.stage` and `repair_jobs.stage` are `String(5)` ; they hold only `l1` / `l2` / `l3` , while `devices.current_stage` uses the full enum. Any future repair stage name longer than 5 characters (e.g. `trc_production`) would not fit	`models/work_order.py` , `models/repair.py`
Indexes	`migrate_perf_indexes.py` adds `ix_device_location_logs_location_id` and drops the duplicate `idx_devices_location_id` . Other indexes come from `index=True` , Alembic `20260426_0940_..._add_missing_fk_indexes.py` , `20260427_1811_..._add_composite_indexes_sprint18.py` , `20260925_1200_add_dashboard_perf_indexes.py` , and model `__table_args__` (for example the partial index `ix_dealer_orders_overdue`)	listed files

6.4 Recent schema changes (newest first)

Date (from code comments / revision names)	Change	Source
2026-10-21	`grn_lots` (many-to-many GRN↔Lot), `grn_stocked_tags` , `grn_imports.entity` , `grn_imports.location_id` ; `devices.entity` widened to 50	`models/grn_import.py` , `models/device.py`
2026-10-17	`users.entity`	`models/user.py`
2026-10-16	`devices.bin_location`	`models/device.py`
2026-10-13	`dealers.dealer_sub_category` , `website` , `google_cid`	`alembic/versions/20261013_1200_dealers_sub_category_website_cid.py`
2026-10-10	`devices.touch_type` , `devices.flag_*` , `block_records.box_number` ; stress checklist items added	`alembic/versions/20261010_*` , `models/stress.py`
2026-10-07	`as_is_lot_prices`	`alembic/versions/20261007_1000_add_as_is_lot_prices.py`
2026-10-01	`block_records` , `block_record_items` ; `audit_records.my_*`	`alembic/versions/20261001_*`
2026-09-30	`audit_records` , `audit_record_items`	`alembic/versions/20260930_1200_add_physical_audits.py`
2026-09-22 to 09-24	Credit Note workflow on `returns` , `devices.tag_return_status/sub_lot_number` , `min/max_selling_price` , `stock_transfers.source` , `users.designation/reports_to` , unique `work_orders.work_id`	`migrate_credit_note_workflow.py` , `migrate_return_creditnote_sublot_fields.py` , `migrate_as_is_lot_selling_price.py` , `migrate_stock_transfer_source.py` , `migrate_user_designation_reports_to.py` , `migrate_work_orders_unique_constraint.py`

PART C

Integrations

This document lists every way data enters or leaves the OxyPC Inventory Lifecycle app: printed documents, machine APIs, desktop agents, partner and mobile front ends, file imports and exports, and messaging. Each section says what is live, what is partial, and what is missing. Every claim names the file it comes from.

Owner in this document means the role that the code lets operate the interface. No technical owner is named anywhere in the code; the only named person is Pankaj (Director, gives feature requests) in `CLAUDE.md`. Assigning a technical owner per interface is an open action.

1. Summary

#	Interface	Direction	Status
2.1	Sale Tax Invoice (HTML print)	Out (document)	Live, partial (see gaps)
2.2	Delivery Challan (HTML print)	Out (document)	Live
2.3	E-way bill generation	Out (government portal)	**Not implemented**
2.4	GST e-invoice (IRN / QR)	Out (government portal)	**Not implemented**
2.5	Gate Pass (exit confirmation)	Internal	Live
2.6	Ready to Dispatch / telecaller requests	Internal	Live
2.7	Purchase Order and Quote PDFs	Out (document)	Live
2.8	Invoice PDF upload and parsing (GRN, Sale)	In (file)	Live, best-effort
3.1	Session JSON API `/api/*`	Browser ↔ server	Live
3.2	Machine API `/api/v1/*` (API keys)	In / out	Live
3.3	Outbound webhooks + `event_log`	Out	Live
4.1	OxyQC IQC machine API `/iqc/api/*`	In	Live (client source not in repo)
4.2	OxyQC Diagnose agent (`127.0.0.1:8765`)	Station → browser	Live
4.3	Stress test runs and on-device results	In	Live
4.4	Customer Care Agent API `/care/api/v1/*`	In / out	Partial (pilot; agent not shippable)
5.1	Trade Partner portal `/partner/*`	In / out	Live
5.2	Partner payment verification `/partner-payments`	Internal	Live
5.3	Mobile telecalling PWA `/m/telecalling`	In / out	Live
5.4	FieldOps standalone app `/fieldops`	Separate app	Live only when `FIELDOPS_DATABASE_URL` is set

#	Interface	Direction	Status
6.1	WhatsApp bridge (`wa-service`)	Out	**Removed** (residual call left in Care)
6.2	`wa.me` deep links	Out (user's phone)	Live
6.3	Email / SMTP	Out	**Not found**
7.1	Bulk CSV upload `/bulk-upload`	In (file)	Live
7.2	Other CSV / Excel imports	In (file)	Live
7.3	Report exports (CSV / Excel)	Out (file)	Live
7.4	Daily Stock Report (scheduled files)	Out (file)	Live
7.5	File uploads and `/uploads` static mount	In / out	Live, with a security gap

2. Documents, tax and dispatch

2.1 Sale Tax Invoice

Item	Detail
Purpose	Printable "TAX INVOICE" for one sale
Direction	Out (HTML page the user prints from the browser)
Protocol / format	HTTP GET, HTML (`templates/invoices/print.html`)
Endpoint	`GET /invoices/print/{sale_id}` (`routers/invoices.py`)
Auth	Staff session cookie (`get_current_user` , `auth/dependencies.py`)
Data fields	From `sales` : `sale_number` , `invoice_no` (falls back to `sale_number`), `sale_price` , `customer_name/phone/state/address` , `company_*` snapshot. From `devices` : `brand` , `model` , `cpu` , `ram_gb` , `storage_gb` , `storage_type` , `barcode` , `serial_no` . From `lots` . GST rates from `cost_config` keys `gst_rate_intra` / `gst_rate_inter` (default 18)
Tax logic	`sale_price` is treated as GST-inclusive. Same state (customer vs company, string compare) → CGST + SGST split in half; otherwise IGST (`_compute_gst()`). Missing `customer_state` defaults to `"Delhi"`
Error handling	Unknown sale → redirect to `/sales?error=Sale+not+found`
Owner	Any logged-in user can open it; sales are created by sales roles (`routers/sales.py`)
Status	**Live, partial**

Gaps found in the code:

Gap	Source
One tag per invoice: the template prints a single line with qty 1. Multi-tag sales share an `invoice_no` but print separately	`templates/invoices/print.html` , `models/sales.py` (`device_id` per row)
HSN code hard-coded to `8471`	`templates/invoices/print.html`
GST calculated with Python `float` , not `Decimal`	`routers/invoices.py` `_compute_gst()`
`invoice_no` is free text typed by the user; no sequential, gap-free invoice series per financial year	`routers/sales.py` (`invoice_no: str = Form("")`)

Gap	Source
No PDF is generated or stored; the user prints the HTML. A separately uploaded invoice PDF can be attached (`sales.invoice_file_path`)	`routers/sales.py` `/sales/parse-invoice-pdf`
No IRN, no signed QR code, no e-invoice fields	grep for `irn` / `einvoice` finds nothing
Older sales without a company snapshot use the live "oldest active company"	`routers/invoices.py` `_sale_company_snapshot()` , `routers/settings.py` `get_company_settings`

2.2 Delivery Challan (waybill)

Item	Detail
Purpose	"DELIVERY CHALLAN" that travels with the goods; used by Gate Pass
Direction	Out (HTML print)
Endpoint	`GET /invoices/waybill/{sale_id}` (`routers/invoices.py` , `templates/invoices/waybill.html`)
Auth	Staff session
Data fields	Same sale / device / lot / company data as the invoice, plus signature boxes for dispatcher and receiver
Error handling	Unknown sale → redirect with error
Owner	Sales and Gate Pass users
Status	**Live**

2.3 E-way bill

Item	Detail
Purpose	Government e-way bill for moving goods worth over the threshold
Direction	Should be outbound to the NIC e-way bill system
What exists	Inbound e-way bill **numbers and files only**, typed in or uploaded when goods arrive: `lots.e_way_bill` (`models/lot.py`), `grn_imports.e_way_bill` (`models/grn_import.py`), `parts_grn.eway_bill_number / eway_bill_date / eway_bill_file` (`models/parts_grn.py` , `routers/parts_grn.py`), `crm_sourcing_deals.eway_bill_path` (`routers/crm_sourcing.py`), `e_way_bill` column in the lots CSV template (`routers/bulk_upload.py`)
What is missing	Nothing generates, validates, extends or cancels an e-way bill for outbound sales or stock transfers. `sales` has no e-way bill column (`models/sales.py`). No NIC / GSP client, credentials or API calls exist (grep for `eway` , `nic.in` , `gsp` in `.py` finds only the inbound fields above)
Status	**Not implemented**

What would be needed to build it:

1. A decision on route: direct NIC e-way bill API (needs GSTIN-level API credentials) or a GSP (GST Suvidha Provider) account.
2. Data the app does not hold today: HSN per item (only hard-coded `8471`), transporter ID / name, transport document number and date, vehicle number for outbound, distance in km, supply type and sub-type, document type, recipient GSTIN and PIN code. `sales` has `customer_state` and

`customer_address` but no customer GSTIN or PIN; `crm_contacts` has `gstin` and `pincode` but sales are not linked to a contact (`models/sales.py`, `models/crm.py`).

3. A multi-line invoice model (one invoice, many tags) so one e-way bill covers one consignment.
4. New columns or a new table for the e-way bill number, date, validity, status and cancel reason, approved on dbdiagram first (`CLAUDE.md`).
5. Secure credential storage (today secrets come from env / `config.ini`, `config.py`), retries and an audit trail (`services/audit_engine.py`).
6. Same need for internal stock transfers between entities (`stock_transfers`, `tag_number_movements`), which today only store an uploaded invoice PDF (`tag_number_movements.invoice_pdf_path`, `routers/entity_movement.py`).

2.4 GST e-invoice (IRN)

Item	Detail
Status	**Not implemented** (no IRN, IRP, signed QR or acknowledgement fields; grep finds none)
What would be needed	Registration with an IRP / GSP; a proper invoice header + line model with `Decimal` totals; HSN and UQC per line; buyer GSTIN; storage for IRN, ack number, ack date, signed QR; cancellation within the allowed window. The invoice template would need to print the QR. Same data gaps as 2.3

2.5 Gate Pass

Item	Detail
Purpose	Security gate confirms that sold tags physically left the building
Direction	Internal (user action → database)
Protocol / format	HTML page + server-side DataTables JSON; form POSTs; CSV/XLSX tag upload
Endpoints	`GET /gate-pass`, `GET /gate-pass/data`, `GET /gate-pass/ids`, `POST /gate-pass/resolve-tags` (file or text list, read-only), `POST /gate-pass/{sale_id}/exit`, `POST /gate-pass/bulk-exit` (`routers/gate_pass.py`)
Auth	Staff session + CSRF (`verify_csrf`); roles admin, sales_manager, sales, inventory_manager (`require_roles`)
Data written	`sales.delivery_status = "dispatched"`, `sales.dispatch_date` (if empty); audit `GATE_EXIT_CONFIRMED` / `GATE_EXIT_BULK_CONFIRMED`
Error handling	404 for unknown sale; already dispatched/delivered rows are skipped in bulk; bad upload → 400 "Could not read the file — upload a CSV or XLSX"
Owner	Sales / inventory roles
Status	**Live**

There is no separate gate pass number or table; the Delivery Challan (2.2) is the gate document.

2.6 Ready to Dispatch and telecaller requests

Item	Detail
Purpose	Telecaller asks to sell a Ready-to-Sale tag or a model group; manager approves, which enables the Sell button
Direction	Internal

Item	Detail
Endpoints	`GET /dispatch`, `POST /dispatch/request`, `POST /dispatch/{req_id}/approve`, `POST /dispatch/bulk-approve`, `POST /dispatch/{req_id}/reject`, `GET /inventory-requests`, `POST /inventory-requests/upload-tags` (read-only tag file match) (`routers/dispatch.py`)
Auth	Session + CSRF. View: admin, inventory_manager, sales, sales_manager, telecaller. Approve: any signed-in user, limited by permission matrix module `inventory_requests` / `edit`
Data	`telecaller_dispatch_requests` (`models/dispatch_request.py`)
Status	**Live**

2.7 Purchase Order and Quote PDFs

Item	Detail
Purpose	Downloadable PO to suppliers and quotation PDF to buyers
Format	PDF built with fpdf2; latin-1 only, so ₹ prints as "Rs " (`services/po_pdf.py`)
Endpoints	`GET /crm/sourcing/{deal_id}/download-po` (`routers/crm_sourcing.py`); quote PDF via `_quote_pdf()` in `routers/crm_quotes.py`; HTML print `GET /crm/quotes/{quote_id}/print`
Auth	Session + CSRF
Status	**Live**

2.8 Invoice PDF upload and parsing

Item	Detail
Purpose	Read invoice number, date, sender, quantity and amount from a supplier or sale invoice PDF to pre-fill forms
Direction	In (file)
Endpoints	`POST /grn/upload` (`routers/grn.py`, parser `services/invoice_parser.py` using `pypdf`); `POST /sales/parse-invoice-pdf` (`routers/sales.py`, uses `pdfplumber` if installed)
Storage	`uploads/grn/` with SHA-256 `file_hash`; `uploads/sale_invoices/inv_<epoch>_<filename>`
Error handling	Extraction is best-effort; failures leave fields blank for the user to fill (`services/invoice_parser.py` docstring)
Status	**Live, best-effort**

3. Application APIs

3.1 Session JSON API (`/api`)

Item	Detail
Purpose	Auto-fill and lookups for the web UI
Endpoints	`GET /api/lot-line-item/{item_id}`, `GET /api/lot-line-items-by-lot/{lot_id}`, `GET /api/lot-meta`, `GET /api/port-profile`, `GET /api/dealers/search`, `GET /api/device/{device_id}/next-stages` (`routers/api.py`)
Auth	Staff session cookie `access_token` (JWT HS256, `auth/dependencies.py`)

Item	Detail
Errors	404 JSON when not found
Status	**Live** (internal use only)

3.2 Machine-to-machine API (`/api/v1`)

Item	Detail
Purpose	Let other systems read inventory and register devices / move stages
Protocol	REST, JSON, Pydantic schemas in `schemas/`
Auth	`Authorization: Bearer ok_live_<64 hex>` ; only the SHA-256 hash is stored (`models/api_key.py`); scopes checked by `require_scope()` (`auth/api_key.py`). Valid scopes: `devices:read/write` , `lots:read/write` , `sales:read/write` , `iqc:read/write` , `dealers:read` , `spare_parts:read` , `intelligence:read` , `api_keys:manage`
Endpoints	`GET /api/v1/health` ; `GET /api/v1/devices` , `GET /api/v1/devices/{barcode}` , `PATCH /api/v1/devices/{barcode}/stage` ; `GET /api/v1/lots` , `/lots/{lot_number}` ; `GET /api/v1/sales` , `/sales/{sale_number}` ; `GET /api/v1/spare-parts` , `/spare-parts/{part_code}` ; `POST /api/v1/iqc/register` , `GET /api/v1/iqc/lookup` ; key admin `POST/GET/DELETE /api/v1/api-keys` (admin session); webhook admin (3.3); telecalling (5.3) (`routers/api_v1/`)
Events published	`STAGE_MOVED` (stage PATCH), `DEVICE_REGISTERED` (iqc register) (`routers/api_v1/devices.py` , `routers/api_v1/iqc.py`)
Errors	401 "Missing or malformed Authorization: Bearer header"; 401 "Invalid or revoked API key"; 403 "API key missing required scopes: ..." (`auth/api_key.py`)
CORS	Origins from `OXYPC_ALLOWED_ORIGINS` , default localhost dev ports (`config.py` , `main.py`)
Owner	Admin creates keys
Status	**Live**. No consumer of these keys is identified in the repo

3.3 Outbound webhooks and event log

Item	Detail
Purpose	Push business events to subscribers
Direction	Out
Protocol	HTTP POST, JSON, 10-second timeout (`services/webhook_dispatcher.py`)
Headers	`X-OxyPC-Signature: sha256=<hmac>` , `X-OxyPC-Event: <type>`
Signing	HMAC-SHA256 with key = the stored `secret_hash` (SHA-256 of the raw secret), not the raw secret. A receiver must hash its secret first (`_sign()`)
Event types	`DEVICE_REGISTERED` , `LOT_CREATED` , `QC_PASSED` , `SALE_COMPLETED` , `STAGE_MOVED` , `API_KEY_CREATED` , `API_KEY_REVOKED` , telecalling `CALL_LOGGED` , `CALL_ORDER_PLACED` , `CALL_QUOTE_SENT` , `CALL_DNC_SET` (`services/event_bus.py` , `services/call_service.py`)
Publishers found	`routers/sales.py` and `routers/stock.py` (`SALE_COMPLETED`), `routers/stock.py` (`LOT_CREATED`), `routers/api_v1/*` , `services/call_service.py` . No publisher of `QC_PASSED` was found
Endpoints (admin)	`POST/GET /api/v1/webhooks` , `DELETE /api/v1/webhooks/{id}` , `GET /api/v1/webhooks/{id}/events` (`routers/api_v1/webhooks.py`)
Delivery log	One `event_log` row per event: `webhook_attempts` , `last_attempt_at` , `last_status_code` (`models/event_log.py`)

Item	Detail
Error handling	Runs after the response (FastAPI background task). Failures and timeouts return `None` and are logged; there is **no retry** or dead-letter queue
Status	**Live**

4. Desktop agents and stations

4.1 OxyQC IQC machine API (`/iqc/api`)

Item	Detail
Purpose	Standalone OxyQC desktop app (PyInstaller, offline-capable per `CLAUDE.md`) registers devices and IQC results
Direction	In
Endpoints	`GET /iqc/api/health` , `GET /iqc/api/lot/{prefix}` (lot from first 4 chars of barcode), `GET /iqc/api/users` , `GET /iqc/api/check/{barcode}` , `POST /iqc/api/submit` (`routers/iqc_api.py`)
Auth	Static shared key in header `X-OxyQC-Key` , from env `OXYPC_API_KEY` or `config.ini [oxyqc] api_key` ; default value `oxyqc-default-key-change-me` (`config.py`)
Data in	`IQCSubmitPayload` : `barcode` , `lot_id` , `sub_category` , `device_type` , `brand` , `model` , `serial_no` , `grn_number` , `cpu` , `generation` , `ram_gb` , `storage_gb` , `storage_type` , `hdd_capacity_gb` , `screen_size` , `battery_health_pct` , `color` , `grade` , `floor` , `warehouse` , `notes` , `inspector_name` , `status` , `r2v3_grade_category` , IQC check fields, `charger_wattage` (stored inside a ports note, no column), `stress_report` (JSON string)
Writes	`devices` , `iqc_inspections` , `stage_movements` in one transaction
Errors	401 bad key; 404 lot not found; 409 "Barcode ... already registered"; 422 bad `lot_id`
Status	**Live**. The OxyQC desktop client source is **not in this repo** (only the server side and `tests/test_barcode_case_insensitive_creation.py` reference the endpoint)

4.2 OxyQC Diagnose agent (station hardware auto-fill)

Item	Detail
Purpose	Fill the IQC form from the hardware of the station PC, which a browser cannot read
Direction	Station agent → browser on the same PC → form fields (nothing goes to the server until the user saves)
Protocol	HTTP on loopback `127.0.0.1:8765` ; CORS `*` and `Access-Control-Allow-Private-Network: true`
Endpoints	`GET /diagnose` (or `/`), `/ping` , `/smart` , `/stress` , `/quit` (`oxyqc-agent/oxyqc_agent.py`)
Caller	`templates/iqc/form.html`
Data returned	`brand` , `model` , `serial_no` , `cpu` , `cpu_make` , `generation` , `ram_gb` , `ram_summary` , `total_ram_count/size` , `storage_gb` , `storage_type` , `hdd_capacity_gb` , `hdd_summary` , `total_hdd_count/size` , `storage_health_pct` , `sub_category` , `device_type` , `screen_size` , `battery_health_pct` , `battery_wh` , `battery_present` , `battery_cable` , port and component checks (`port_hdmi` , `port_usb_working` , `port_audio_jack` , `usb_a_ports` , `usb_c_ports` , `ethernet_ports` , `wifi_status` , `webcam_status` , `speaker_status` , `keyboard_working` , `touchpad_*` , `fan_working` , `fan_rpm` , `charging_port` , `dvd_drive` , `hdd_connector` , `hdd_casing` , `power_on` , `status` , `screen_functional`), `notes` / `_summary`

Item	Detail
Auth	None (loopback only)
Platforms	Windows (WMI/CIM via PowerShell) and macOS; agent version `2026-07-29.2`
Distribution	`GET /iqc/agent-exe` is **unauthenticated** so PXE first-boot scripts can download it (`routers/iqc.py`, `oxyqc-agent/pxe/README_PXE.txt`). Exe is unsigned (`README_AGENT.txt`)
Status	**Live**

4.3 Stress test

Item	Detail
Purpose	Run and record stress / checklist tests before cosmetic and Final QC
Endpoints	`POST /stress/{barcode}/run`, `GET /stress/{barcode}/status`, `POST /stress/{barcode}/stop`, `POST /stress/{barcode}/save`, `POST /stress/{barcode}/save-agent` (results JSON produced by the local agent on a PXE stress station, forwarded by the browser), `GET /stress/{barcode}/report` (PDF), `GET /stress/results/{barcode}`, `POST /stress/{barcode}/fail`, `POST /stress/{barcode}/complete-to-paint`, `GET /stress/has-results` (`routers/stress_api.py`)
Auth	Staff session + CSRF
Data	`stress_test_results` (`results_json`, `overall_status`, `pdf_path`); PDFs in `static/stress_reports/`; `devices.stress_notes` on fail
Errors	422 "No results in payload"; 404 "Device not found" (only active devices)
Status	**Live**

4.4 Customer Care Agent

Item	Detail
Purpose	After-sale warranty view, support tickets, diagnostics and offers from an agent installed on the customer's PC
Direction	In / out
Public API	`/care/api/v1`: `POST /pair`, `GET /device`, `GET /warranty`, `GET /offers`, `GET/POST /tickets`, `GET /tickets/{ticket_number}`, `POST /diagnostics`, `POST /heartbeat`, `POST /token/rotate`, `POST /deactivate` (`routers/care_api.py`)
Staff APIs	`/care/internal/pairings`, `/care/internal/pairings/{id}/revoke`, `/care/internal/dispatch-readiness/{device_id}`, `/care/internal/dispatch-exceptions` (`routers/care_internal.py`); admin UI `/care-support/*` (`routers/care_admin.py`)
Auth	One-time provisioning token (30-minute TTL) exchanged at `/pair` for a device token; `Authorization: Bearer <device-token>`; tokens stored as SHA-256 only; IPs stored hashed (`models/care.py`, `auth/care_auth.py`, `services/care_service.py`). Staff routes use session + permission module `care_support`
Format	Envelope `{success, data, request_id, server_time}`; errors with codes such as `CARE_MISSING_TOKEN`, `CARE_INVALID_TOKEN`, `CARE_TOKEN_EXPIRED`, `CARE_DEVICE_MISMATCH`, `CARE_TICKET_NOT_FOUND`
Data in	Diagnostics allowlist only: `bios_serial`, `manufacturer`, `model`, `cpu`, `ram_gb`, `storage_summary`, `battery_health_pct`, `battery_cycle_count`, `smart_status`, `os_version`, `hardware_warning_summary`, `system_error_summary` (max 2000 chars each). Tickets: category from a fixed set, description ≤ 2000 chars, `idempotency_key` for offline retries
Rate limits	`/pair` 10/min per IP; ticket create 10/min; diagnostics 20/min; token rotate 5/min (`limiter`)

Item	Detail
Client	`care_agent_windows/` (Windows service + tray, DPAPI credential store, offline queue)
Status	**Partial.** Server side is built and registered (`main.py`). The agent README says "not a shipped product": no code signing, no installer, no update framework, no uninstall script. `care_warranties` has no writer in the code, so `/warranty` relies on the fallback in `resolve_warranty()`

5. Partner, mobile and side apps

5.1 Trade Partner portal (`/partner`)

Item	Detail
Purpose	Dealer-facing B2B catalog: browse listings and visible lots, bid, book, upload payment proof
Endpoints	`GET/POST /partner/login` , `POST /partner/logout` , `GET /partner/catalog` , `GET /partner/listings/{id}` , `POST /partner/listings/{id}/bid` , `/custom-price` , `/book` ; `GET /partner/lots/{lot_id}/models` , `POST /partner/lots/{lot_id}/bid` , `/custom-price` , `/book` , `/withdraw` ; `GET /partner/bookings` , `/bookings/{id}` , `POST /bookings/{id}/proof` , `GET /bookings/{id}/proof-file/{proof_id}` , `POST /bookings/{id}/cancel` ; `GET /partner/bids` , `GET /bids/{bid_id}/document/{doc_id}` , `POST /bids/{bid_id}/payment` ; `GET /partner/profile` ; `GET /partner/sw.js` (`routers/partner_portal.py`)
Auth	Separate JWT cookie `partner_token` , 12-hour session, own CSRF cookie `partner_csrf_token` ; login by phone + bcrypt password on `dealers.portal_phone` / `portal_password_hash` ; lockout after 5 failed attempts; `portal_password_version` invalidates old tokens (`auth/partner_auth.py` , `models/dealers.py`)
Data boundary	Must never expose `lots.buying_price` , `supplier_name` , margins or `cost_basis` (module docstring)
Data in	Bids (`partner_bids`), bookings, payment proofs (UTR, amount, screenshot), bid payments (date, mode `bank_transfer` / `upi` , UTR, amount)
File rules	Proofs: `.jpg` , `.jpeg` , `.png` , `.pdf` , max 5 MB (`services/partner_service.py`)
Staff side	`routers/partner_admin.py` (listings, Manage Lots, bids, Mark Won → creates Buyer Deal)
Status	**Live**

5.2 Partner payment verification

Item	Detail
Endpoints	`GET /partner-payments` , `GET /partner-payments/{bid_id}/details` , `POST /partner-payments/{bid_id}/verify` (`routers/partner_payments.py`)
Auth	Session + CSRF; admin, sales_manager
Data	Sets `partner_bid_payments.verified` , `verified_by` , `verified_at` , `verify_notes`
Status	**Live**. No bank or payment-gateway integration; verification is manual against the UTR

5.3 Mobile telecalling PWA

Item	Detail
Purpose	Telecallers log calls and follow-ups from a phone

Item	Detail
Endpoints	Pages under `/m/telecalling` (`routers/m_telecalling.py`); JSON under `/api/v1/telecalling` : `GET /session/today` , `GET /assigned-leads` , `POST /assignments` , `POST /calls` , `GET /calls` , `GET /calls/{call_id}/timeline` , `GET /followups/today` , `GET /kpi/dashboard` , `GET /kpi/team` , `GET /lots/{lot_id}/skus` (`routers/api_v1/telecalling.py`)
Auth	Staff session (`get_current_user`), not API keys
Data	`telecalling_records` (incl. `latitude` , `longitude` , `idempotency_key`), `telecalling_assignments` (`models/telecalling.py`)
Status	**Live**

5.4 FieldOps (Reliance asset app)

Item	Detail
Purpose	Separate field-asset app hosted in the same process; shares nothing with OxyPC data
Endpoints	`/fieldops/login` , `/fieldops/setup` , `/fieldops/api/auth/*` , `/fieldops/api/me` , `/fieldops/api/admin/users` , `/fieldops/api/admin/export` , `/fieldops/api/admin/import` , `/fieldops/api/admin/audit` , `POST /fieldops/api/sync` (offline change push, capped per push), `/fieldops/api/status` , static app `fieldops_app/` (`routers/fieldops.py`)
Auth	Own users and own session cookie (`fieldops_auth.py`); OxyPC session gives no access
Database	`FIELDOPS_DATABASE_URL` ; tables `fieldops_users` , `fieldops_records` , `fieldops_admin_audit` (`fieldops_db.py`)
Status	**Live when configured**; startup prints "FIELDOPS_DATABASE_URL not set — /fieldops disabled" otherwise (`main.py`)

6. Messaging

6.1 WhatsApp bridge (`wa-service`)

Item	Detail
Purpose (historic)	Per-user WhatsApp sessions via whatsapp-web.js: QR link, send message, groups, broadcasts
Protocol	Node service on `localhost:3001` : `GET /status` , `/qr` , `/sessions` , `/groups` , `/group-messages/:id` ; `POST /connect` , `/disconnect` , `/send` , `/send-group` , `/sync-group-messages` (`wa-service/index.js` , `start_wa.bat`)
Status	**Removed.** `main.py` comments out both the import and `app.include_router(whatsapp_router)` with "WhatsApp integration removed — router registration disabled (routes unreachable)". `routers/whatsapp.py` , `wa-service/` and tables `whatsapp_sessions` , `whatsapp_messages` , `whatsapp_groups` , `whatsapp_broadcasts` remain (`models/whatsapp.py`)
Residual	`services/care_service.py` `send_offer_whatsapp()` still POSTs to `http://localhost:3001/send` when a Care offer uses channel `whatsapp` or `both` . With the bridge gone this returns status 0 and an error, recorded as a failed `care_offer_deliveries` row
QA notes	`QA_UAT_REPORT.md` G-08 noted the module could not be tested without a real account; `docs/uat_results_2026-04-27.md` flagged the live dependency on wa-service. Both predate the removal

6.2 `wa.me` deep links

Item	Detail
Purpose	Open WhatsApp on the user's own phone with a pre-filled message
Where	`templates/trade_partner/my_desk.html`, `templates/trade_partner/bookings.html`, `templates/partner/booking_detail.html`, `templates/partner/listing_detail.html`, `templates/m/telecalling/call.html`
Auth / data	None server-side; phone and text are put in the URL
Status	**Live** (no server integration involved)

6.3 Email / SMTP

Item	Detail
Status	**Not found.** No `smtplib`, `aiosmtplib`, SendGrid or similar in application code. `users.email`, `crm_contacts.email` and `returns.customer_email` are stored but nothing sends mail. The return receipt and invoices are printed or downloaded only

7. Files: imports, exports, uploads

7.1 Bulk CSV upload (`/bulk-upload`)

Item	Detail
Endpoints	`GET /bulk-upload`, `GET /bulk-upload/template/{upload_type}`, `POST /bulk-upload/lots`, `/devices`, `/spare-parts`, `/leads`, plus duplicate handling `POST /bulk-upload/duplicates/approve-move`, `/update`, `/update-all`, `/add-lot` (`routers/bulk_upload.py`)
Format	CSV. Encoding detected by `utils/csv_decode.py` `decode_csv_bytes()` : UTF-16/32 only with a BOM, then UTF-8, then Windows-1252, then latin-1 (never fails)
Templates	`lots_template.csv` (lot_number, supplier_name, buying_price, qty, purchase_date, grn_*, invoice_no, invoice_value, taxable_amount, sgst, cgst, igst, po_number, vehicle_number, e_way_bill, notes); `devices_template.csv` and `devices_test_partner_template.csv` (`DEVICE_CSV_HEADERS`); `spare_parts_template.csv` (part_code, name, category, unit_price, min_stock_alert, supplier, notes); `leads_template.csv`
Auth	Session + CSRF; upload rights through `can_upload` in the permission matrix (`models/role_permissions.py`)
Validation	Stage moves go through `services/control_engine.validate_transition` ; writes are audit-logged (`services/audit_engine.py`)
Process note	Project skill `bulk-csv-import` requires asking local vs production and reporting exact skip counts (`CLAUDE.md`)
Status	**Live**

7.2 Other CSV / Excel imports

Module	Format	Source
Dealers (incl. Google Maps export, dedupe on `google_cid`)	CSV, XLSX	`routers/dealers.py`, `models/dealers.py`
Grade price matrix	CSV, XLSX	`routers/crm_price_matrix.py`
Block Tags (up to 5000 rows)	XLSX	`routers/block_tags.py`
Product Master / Pricing Calculator	XLSX	`routers/product_master.py`
Gate Pass tag list	CSV, XLSX	`routers/gate_pass.py`
Inventory Requests tag list	CSV	`routers/dispatch.py`
CRM contacts, devices, spare parts, sales	CSV	`routers/crm_contacts.py`, `routers/devices.py`, `routers/spare_parts.py`, `routers/sales.py`
FieldOps data	JSON import/export	`routers/fieldops.py`

7.3 Report exports

Item	Detail
Endpoints	`/reports/daily-stock/export/overall`, `/tags`, `/location`, `/daily-stock/export/daily-report` (XLSX, three sheets), `/reports/daily-stock/report/{date}/workbook`, `/reports/export/lot-pl`, `/reports/lot-pl/{lot_id}/export`, `/reports/business-pl/export-month`, `/reports/business-pl/export-fy`, `/reports/export/sales`, `/reports/overdue/csv` (`routers/reports.py`)
Format	CSV, and XLSX via `openpyxl` (`_daily_report_xlsx()`)
Other Excel writers	`routers/part_estimation.py`, `routers/dealers.py`, `routers/crm_price_matrix.py`, `routers/block_tags.py`, `routers/product_master.py`, `routers/gate_pass.py`
Auth	Session; export rights via `can_export` (`models/role_permissions.py`)
Status	**Live**

7.4 Daily Stock Report (scheduled)

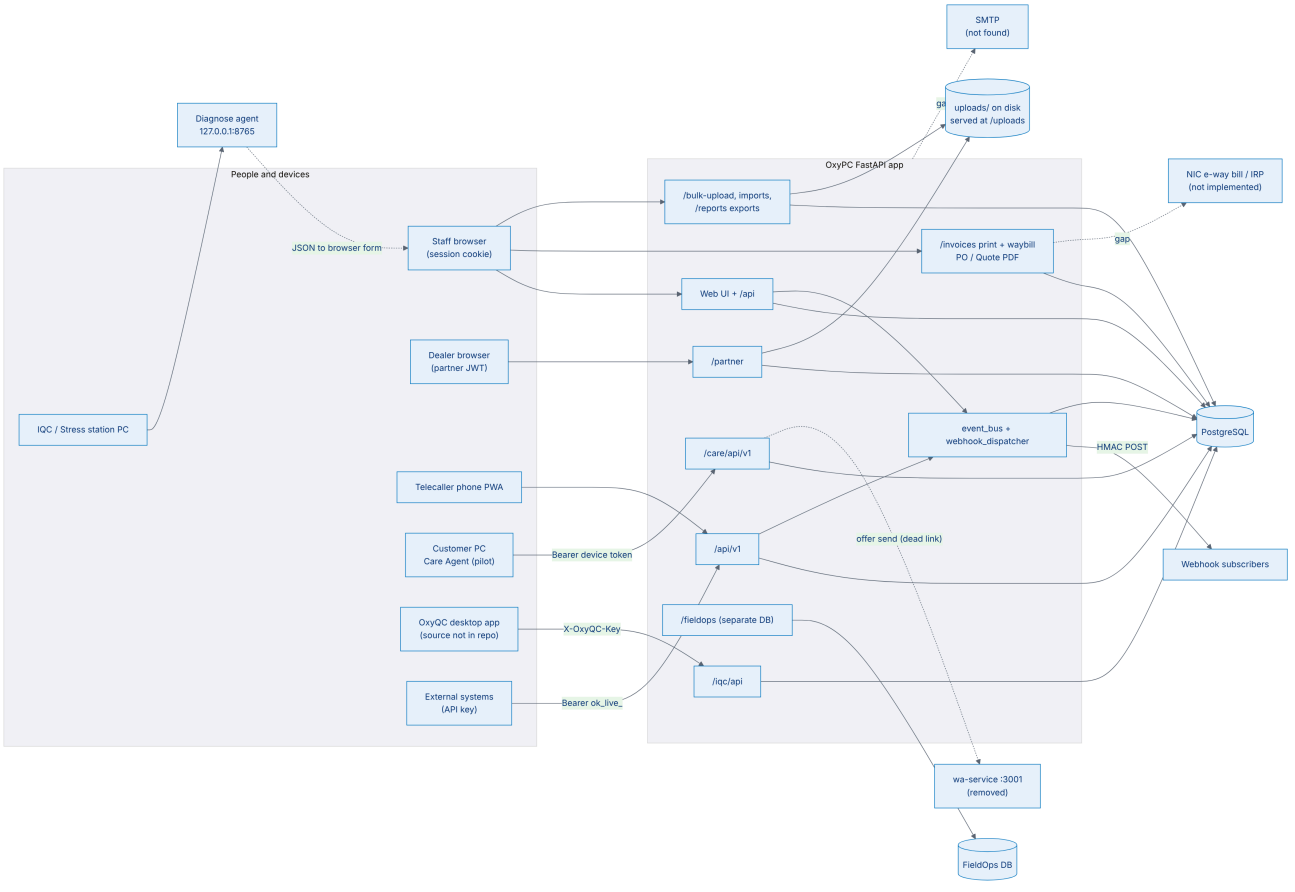
Item	Detail
Purpose	Snapshot of Overall / Tag / Location stock every day at 9:00 PM
Mechanism	Background loop started at startup (`main.py` → `services/daily_stock_reports.scheduler_loop`)
Output	`uploads/daily_stock_reports/<YYYY-MM-DD>/{overall,tags,location}.csv`; no database table
Status	**Live**

7.5 File uploads and the `/uploads` mount

Item	Detail
Storage	Local disk under `uploads/` (`config.py` `UPLOADS_DIR`): `grn/`, `sale_invoices/`, `crm/`, `partner/`, `partner/bids/`, `part_estimates/`, `entity_movement_invoices/`, `tickets/`, `daily_stock_reports/`, plus parts GRN files; stress PDFs under `static/stress_reports/`

Item	Detail
Validation	Only some routes check type/size: support ticket photos (PNG/JPEG/GIF/WEBP, 5 MB, `routers/tickets.py`) and partner proofs (JPG/PNG/PDF, 5 MB, `services/partner_service.py`). Parts GRN uploads only strip `/` and `\` from the filename (`routers/parts_grn.py` `_save_upload()`)
Serving	`app.mount("/uploads", StaticFiles(...))` in `main.py`
Security gap	The mount is plain `StaticFiles` with no auth check, so anyone who can reach the server and knows or guesses a path can download KYC documents (`crm_contacts.*_doc_path`), payment screenshots, invoices and daily stock files, bypassing the authenticated download routes (`/grn/download/{grn_id}`, `/partner/bookings/{id}/proof-file/{proof_id}`). The server is described as LAN-only (`CLAUDE.md`, `routers/iqc.py`), but `main.py` also contains Railway HTTPS-redirect handling for an internet-facing deployment
Status	**Live, needs hardening**

8. Integration map



PART D

Architecture Decision Records

These records explain the main technical decisions behind the OxyPC Inventory Lifecycle app. They were **reconstructed** from the code, the plans and specs in `docs/superpowers/`, and git history. No formal ADRs existed before this file.

How to read the history evidence: the git history in this repo starts with one large import commit, `fac334c` (22 Aug 2026, 789 files). Decisions made before that date are dated from Alembic file names (`alembic/versions/2026MMDD_…`) and sprint plans (`docs/superpowers/plans/2026-MM-DD-…`). Where a decision is older than both, the date is shown as "before 22 Aug 2026".

Status values: **Accepted** (in force), **Accepted with drift** (in force, but code and rule disagree), **Superseded**, **Deprecated**.

Index

ADR	Decision	Status	Date (best evidence)
001	Server-rendered Jinja2 pages, not a single-page app	Accepted	Before 22 Aug 2026
002	PostgreSQL with async SQLAlchemy 2 + asyncpg	Accepted	31 Mar 2026 (Alembic baseline)
003	Schema provisioned at startup by `db_validator`; Alembic secondary	Accepted with drift	Before 22 Aug 2026
004	Soft delete with `is_trashed` and a Trash page	Accepted with drift	3 May / 26 May 2026
005	Immutable audit log, written before delete	Accepted	Before 22 Aug 2026
006	In-process event bus with outbound webhooks	Accepted	30 Apr 2026
007	Custom roles and an admin-managed Permission Matrix	Accepted with drift	26 Aug 2026 onward
008	One "Global Table" component for heavy lists	Accepted	2 Sep 2026
009	Per-station localhost agent for IQC hardware diagnosis	Accepted	Before 22 Aug 2026
010	WhatsApp integration disabled	Accepted (code retained)	Before 22 Aug 2026
011	Whole app rendered at 90 % zoom	Accepted	Before 22 Aug 2026
012	Containerise the app only; one worker; poll-based auto-deploy	Accepted (supersedes Windows/bare-metal hosting)	1 Oct 2026

ADR-001 · Server-rendered Jinja2 pages, not a single-page app

Context. A small team builds many operational screens (242 templates) for floor staff on a LAN. The director gives feature requests directly (`CLAUDE.md` "People"), and the git log shows frequent page-level

changes (420 commits between 22 Aug and 21 Oct 2026). Staff use shared floor PCs and barcode scanners (`README.md` "Barcode Scanner").

Decision. Every page is rendered on the server by FastAPI routes with Jinja2 templates, styled with Bootstrap 5 and enhanced with jQuery, DataTables and small vanilla JS files. All libraries are stored in `static/vendor/` (no CDN). When other systems needed data, a separate JSON API (`/api/v1/`) was added **alongside** the HTML routes rather than converting the UI ("Parallel routes — all existing HTML routes remain unchanged", `docs/superpowers/plans/2026-04-29-sprint17a-ecosystem-foundation.md`).

Consequences. - One codebase and one deploy; permissions are enforced in the same place pages are built (`templates_config.py` globals such as `has_perm` , `role_allowed`). - Fast to add a page; no build step for the front end. - Interactivity needs extra patterns: AJAX partial refresh with `DOMParser` swaps (`.claude/skills/ajax-partial-refresh`), form autosave (`static/js/form-autosave.js`). - Big tables can produce multi-MB HTML (`deploy/nginx-oxypc-internal.conf` comment), which led to ADR-008. - Pages are sent `Cache-Control: no-store` and assets are cache-busted so deploys show at once (`main.py` `_no_store_pages` , `templates_config.py` `ASSET_VERSION`).

Source. `main.py` , `templates_config.py` , `templates/base.html` , `static/vendor/` , sprint 17a plan, design system checklist "No CDN links".

ADR-002 · PostgreSQL with async SQLAlchemy

Context. 30+ users write at the same time (stage moves, IQC entries, sales). Data is relational (lots → devices → stage movements → sales) and needs transactions, enums and UUID keys.

Decision. Use PostgreSQL with SQLAlchemy 2.0 in async mode and the asyncpg driver. One `AsyncSession` per request, committed after the handler and rolled back on error. Standard query form: `await db.execute(select(...))` then `.scalar_one_or_none()` / `.scalars().all()` .

Consequences. - Good concurrency on a single async worker (ADR-012). - Every handler must `await` DB calls; blocking calls (for example `subprocess.run` in `POST /admin/backup-now` , `routers/admin.py`) stall all users. - Pool size must stay under PostgreSQL `max_connections` (`database.py` comment: 8 + 4 per process). - Raw SQL is still used for reports and migrations (`text(...)` in `db_validator.py` , `migrate_*.py`).

Source. `database.py` , `requirements.txt` (`sqlalchemy[asyncio]==2.0.36` , `asyncpg==0.30.0`), `CLAUDE.md` "Key pattern", `alembic/versions/20260331_1600_ba22dd09a6a3_initial_baseline.py` .

ADR-003 · Schema is provisioned at startup by `db_validator` ; Alembic is secondary

Context. New columns ship often. Missing columns in production caused "silent 500s" (`db_validator.py` header). Alembic revisions were not always applied to production.

Decision. On every start, `db_validator.validate_and_fix()` compares ORM models to the database, creates missing tables, adds missing columns (`ADD COLUMN IF NOT EXISTS`), reseeds stages and transitions, and aborts startup if something cannot be fixed. Alembic (36 revisions) and one-off `migrate_*.py` scripts remain for indexes, type changes and data moves. New tables or columns need dbdiagram.io approval first.

Consequences. - Deploys "just work" for additive changes; the Docker auto-deploy needs no migration step (`deploy/oxypc-autodeploy.sh` comment). - Non-additive changes (rename, drop, type change, index) are not covered and need manual scripts. - Alembic history no longer describes production: `20260925_1200_add_dashboard_perf_indexes.py` found indexes from an April revision "never actually present on the live schema". - Three overlapping mechanisms (validator, Alembic, 51 `migrate_*.py`) make the true schema hard to know.

Source. `db_validator.py`, `main.py` startup, `upgrade_db.py` header, `.github/workflows/deploy.yml` comment ("evolves the schema via the additive-column auto-provisioner … NOT hand-written Alembic migrations"), `CLAUDE.md` Critical Rules.

ADR-004 · Soft delete with `is_trashed` and a Trash page

Context. Devices, lots and parts carry financial and audit history. Hard deletes broke reports and made mistakes unrecoverable.

Decision. Mark rows with `is_trashed = true` instead of deleting them; filter them out of lists; restore from a Trash page. Devices also have `is_active`. The single remaining `db.delete(device)` was replaced (sprint 21).

Consequences. - Mistakes can be undone; history stays intact. - Every query must remember the filter (187 `is_trashed` references in routers/services/models). - Rule drift: `CLAUDE.md` says the column is nullable and must be filtered with `isnot(True)`. The models now declare it `nullable=False` with a `false` default (`models/device.py` l.248, `models/lot.py` l.47), and most code uses `== False` (126 vs 10). Both are correct today; the rule text should be updated. - 36 `db.delete()` calls still exist for other records, so ADR-005 matters.

Source. `alembic/versions/20260503_0900_soft_delete_devices.py`, `alembic/versions/20260526_1400_add_trash_to_lots_devices.py`, `docs/superpowers/plans/2026-05-08-sprint21-pagination-softdelete.md`, `routers/trash.py`, commit `7e72210` ("Trash duplicates").

ADR-005 · Immutable audit log, written before delete

Context. The director needs to know who changed what (stage moves, prices, deletions). After `db.delete()` and commit, the record's values are gone.

Decision. A single helper, `services/audit_engine.audit()`, writes an `AuditLog` row (user, IP, action, table, record id, old and new values, notes). It does not commit; it joins the caller's transaction, so the audit row and the change succeed or fail together. Rule: call it **before** `db.delete()`. The table is documented as immutable ("No UPDATE or DELETE permitted").

Consequences. - 242 call sites; a full audit-log page for admins (`routers/admin.py`). - Immutability is by convention, not enforced by database permissions or triggers. - No retention policy; the table will grow without limit. - Some startup recoveries rely on it (for example `recover_from_audit_once` in `services/grn_stocked.py`).

Source. `services/audit_engine.py`, `models/engines.py` (`AuditLog`), `CLAUDE.md` Critical Rules, `alembic/versions/20261001_1000_audit_records_my_fields.py`.

ADR-006 · In-process event bus with outbound webhooks

Context. Other "ecosystem" apps (Customer Portal, AI layer, ESG, Finance) need to hear about key events in real time (`config.py` CORS comment; sprint 17a/17b plans).

Decision. A lightweight in-process pub/sub (`services/event_bus.py`). Routes call `publish()` after commit; handlers run as FastAPI `BackgroundTasks` after the response. At startup the webhook dispatcher subscribes to 7 event types and sends HMAC-signed HTTP calls to registered webhooks; deliveries are logged in `event_log` .

Consequences. - No message broker to run; simple to operate. - Events are lost if the process dies between response and delivery; there is no retry queue in the bus itself. - Only 10 `publish(EventType…)` calls exist, so coverage of business events is partial. - Works only within one process; more workers would each run their own handlers (fine for webhooks, but noted).

Source. `docs/superpowers/plans/2026-04-30-sprint17b-event-system.md` , `services/event_bus.py` , `services/webhook_dispatcher.py` , `main.py` startup (subscribe loop), `alembic/versions/20260430_0900_add_webhooks_event_log.py` .

ADR-007 · Custom roles and an admin-managed Permission Matrix

Context. The original fixed roles (admin, inventory_manager, sales, iqc_inspector, l1/l2/l3_engineer, qc_inspector — `CLAUDE.md`) did not match how teams actually split work (e.g. `trc_manager` , cosmetic leads, External Partner pages). Every change needed a code edit.

Decision. Admins create custom roles and tick modules per role in Master Data → Module Permissions. The sidebar and the backend both read this matrix (cached, refreshed every 60 s). `require_roles()` lets custom roles through non-admin gates and lets the matrix grant admin-only modules explicitly. In templates, `role_allowed()` mirrors this so buttons match the backend. A `sub_admin` role is seeded at startup with no permissions.

Consequences. - New roles need no deploy. - Two layers (built-in allow-lists + matrix) make "why is this 403?" harder to answer; `_SEGMENT_MODULE_ALIASES` exists to close gaps where the nav and routes disagree. - Drift: 15 template checks still hard-code `current_user.role.value` , so custom roles can miss buttons. The `custom-role-permissions` skill exists to stop new cases.

Source. `auth/dependencies.py` (`require_roles` , `_matrix_grants_path` , `require_module_perm`), `templates_config.py` (`_role_allowed`), `models/role_permissions.py` , `main.py` (sub_admin seed, cache refresh), `.claude/skills/custom-role-permissions` , commits `5a0b33e` (real Edit/Add permissions), `d862fa7` (External Partner split into 8 pages).

ADR-008 · One "Global Table" component for heavy lists

Context. Inventory Search holds "~19-23k rows — the heaviest table in the app" (commit `ae43e3f`). Each page had its own DataTables setup, so headers wrapped, columns squashed and behaviour varied.

Decision. One module, `static/js/global-table.js` , applied to server-side DataTables: single-line headers, horizontal scroll with a frozen first column (second too if the first is a checkbox) using CSS

`position: sticky`, 12 rows by default, 32-character clamp with tooltip, a standard toolbar (page view, row count, search/scan, pagination) and optional scan-to-select. It is part of the cache-busting asset list.

Consequences. - Consistent tables across 49 templates that reference it; 13 use full server-side paging. - One file is now critical: a change affects every big list, so `regression-sweep` is required when shared UI changes. - A cross-browser bug (2026-09-03) came from caching, not rendering, and was fixed by adding the file to `_VERSIONED_ASSETS` (`tests/test_asset_version_cache_busting.py`).

Source. `static/js/global-table.js` header, commits `ae43e3f`, `097c736`, `108a742`, `5583f89`, `8aa8032`, `templates_config.py`.

ADR-009 · Per-station localhost agent for IQC hardware diagnosis

Context. IQC inspectors type CPU, RAM, storage and serial details for every device. "A browser cannot read the hardware of the PC it runs on" (`oxyqc-agent/README_AGENT.txt`). Earlier, a standalone OxyQC EXE submitted inspections through a machine API.

Decision. Each inspection station runs a small agent (`Diagnose_Device_Agent.exe`, from `oxyqc-agent/oxyqc_agent.py`) that listens only on `127.0.0.1:8765`. The IQC page's JavaScript calls it to auto-fill hardware fields. It self-installs per user with no admin rights. The separate OxyQC app keeps using `/iqc/api/*` with an `X-OxyQC-Key` header. A GitHub Action rebuilds the agent EXE when its source changes. PXE-deployed station images (IQC line and Stress Test / Final QC) download the latest agent from `<SERVER_URL>/iqc/agent-exe` on each boot (`oxyqc-agent/pxe/README_PXE.txt`).

Consequences. - No inbound firewall rules; the agent is never exposed on the LAN. - One more piece of software to install and update on every station; the EXE is committed to git. - Hardware quirks need agent fixes (commit `8835f10`: agent "dies silently on some hardware's blank SMBIOS data"). - The OxyQC API key has an insecure default in `config.py` if not set.

Source. `oxyqc-agent/README_AGENT.txt`, `oxyqc-agent/`, `routers/iqc_api.py`, `config.py` (`OXYQC_API_KEY`), `.github/workflows/build-agent.yml`, commit `8835f10`.

ADR-010 · WhatsApp integration disabled

Context. A WhatsApp integration existed: a Node service (`wa-service/`), a router and model (`routers/whatsapp.py`, `models/whatsapp.py`), templates (`templates/whatsapp/`) and a start script (`start_wa.bat`).

Decision. Disable it by commenting out the router import and registration, so its routes are unreachable. The code stays in the repo.

Consequences. - No WhatsApp messaging from the ERP; account-level WhatsApp CSV columns were also dropped from CRM bulk upload (commit `12388b1`). Manuals omit WhatsApp pages (commit `0c496bf`). - Dead code remains (router, model, ~1.7 GB local `wa-service/` install excluded via `.dockerignore`, docker spec §9). - The Care Agent plan still mentions WhatsApp offers (`docs/care-agent/phase5-offers-whatsapp.md`), so the topic may return. - The reason for disabling is not recorded in code or commits.

Source. `main.py` lines 176 and 266-267 ("WhatsApp integration removed — router registration disabled (routes unreachable)"), present since `fac334c`; commits `12388b1`, `0c496bf`.

ADR-011 · Whole app rendered at 90 % zoom

Context. Pages show wide tables and dense forms. The reason for the global zoom is not written down; the most likely aim is to fit more on screen without horizontal scrolling (inferred, see Open questions).

Decision. Scale the whole UI with `body { zoom: var(--app-zoom) }`, `--app-zoom: 0.9`. Anything sized with `vh` must use `--full-vh` (`100vh / 0.9`). The design system documents all sizes "before that zoom".

Consequences. - More data fits on screen on every PC with no per-user setup. - Side effects need fixes: modal backdrop sized with `100%` instead of viewport units (`static/css/app.css` l.23-30); Chart.js hover misaligned until `static/js/chart-zoom-fix.js` (commit `eb8b4e2`); fixed-position dropdowns rendered ~10 % off because Chromium applies the zoom twice to `position: fixed` elements (commit `8ef7df4`). - Effective table text is about 12.6 px, which may hurt readability (see NFR-13, NFR-15). - Depends on CSS `zoom`, which is non-standard in older Firefox versions.

Source. `static/css/app.css` l.10-20, `qa_docs/design_system/design_system.html` (typography and tokens), commits `eb8b4e2`, `8ef7df4`.

ADR-012 · Containerise the app only; one worker; poll-based auto-deploy

Context. The app first ran on a Windows laptop (`192.168.1.100:8000`, scheduled tasks, NSSM, watchdog) and then on a Linux Internal Server as a bare-metal venv with systemd. The bare-metal setup had "no environment reproducibility", "no rollback primitive" and "no blast-radius isolation" (docker spec §1). The server has a private IP that GitHub cannot reach.

Decision. 1. Run the app in one Docker container (`python:3.12-slim`, non-root uid 1000) with host networking; keep PostgreSQL native on the host for now ("the one component where a mistake is genuinely hard to undo", spec §3). 2. Run **one** uvicorn worker, because permission/transition caches and the rate limiter live in process memory. 3. Deploy by a systemd timer that polls `origin/main` every 2 minutes, refuses to deploy over local edits, rebuilds the image, polls `/health`, and does **not** auto-roll back. 4. Keep the disabled `oxypc.service` as the rollback path.

Consequences. - Cutover took ~8 s of downtime; rollback is "run the previous commit/image". - Uploads/backups are bind mounts, so data survives rebuilds; they needed a one-time `chown` to uid 1000. - Files written inside the container but not mounted (e.g. `errors.log`) are lost on each deploy. - Single worker means any blocking call freezes all users; scaling out needs a shared cache/limiter first. `deploy/oxypc.service` still claims 4 workers are safe, which conflicts with the `Dockerfile`. - `.github/workflows/deploy.yml` (self-hosted, `systemctl restart oxypc`) predates this decision and may now be obsolete. - Postgres containerisation is deferred as "Phase 2".

Supersedes. Windows hosting scripts (`run_server.ps1`, `setup_autostart.ps1`, `watchdog_server.ps1`, `install_service.bat`) and bare-metal `deploy/oxypc.service` for production.

Source. `docs/superpowers/specs/2026-10-01-docker-containerization-design.md`, `Dockerfile`, `docker-compose.yml`, `deploy/oxypc-autodeploy.sh`, `deploy/oxypc-autodeploy.timer`, commits `992255e`,

`efc7388`, `e3a90c9`, `8feadf0`.

Open questions

1. Why was the WhatsApp integration disabled (cost, policy, reliability)? Should the code be deleted or revived for the Care Agent offers?
2. Who decided on the 90 % zoom, and was it tested on the smallest floor monitors?
3. Should Alembic become the only migration path (ADR-003), or should the validator stay the main mechanism?
4. Is moving to multiple workers (ADR-012) planned? It needs a shared rate-limit store and a single owner for scheduled jobs.
5. Should audit-log immutability (ADR-005) be enforced in the database (revoke UPDATE/DELETE)?
6. Is the standalone OxyQC app still used, or has the browser IQC page with the localhost agent fully replaced it (ADR-009)?
7. Git history before 22 Aug 2026 is not in this repository. Is there an older repository (for example `TheCodeOrbit/oxypc`, named in `installer.py`) with the original decision history?



OXYPC INVENTORY LIFECYCLE

OPERATIONS

Operations & Testing

Deploying, running and recovering the app, its quality targets, and how releases are tested and signed off.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Operations & Testing

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

In this document: Part A — Deployment & Environments · Part B — Non-Functional Requirements · Part C — Runbook & Disaster Recovery · Part D — UAT Plan & Sign-off

PART A

Deployment & Environments

This document explains where the app runs, how it is started, how configuration and secrets are supplied, how the database schema changes, and how a change reaches production. It cites the file behind each statement. Secret **values** are never shown; only variable names.

Key point. Production is no longer a Windows laptop. Since 1 Oct 2026 it is a Linux "Internal Server" (`10.199.206.109`) running the app in a Docker container behind nginx, with PostgreSQL installed natively on the same box (`docs/superpowers/specs/2026-10-01-docker-containerization-design.md` §9). The Windows scripts in the repo root describe the earlier laptop setup and the standalone UAT installer. They are kept here as "legacy / local" paths.

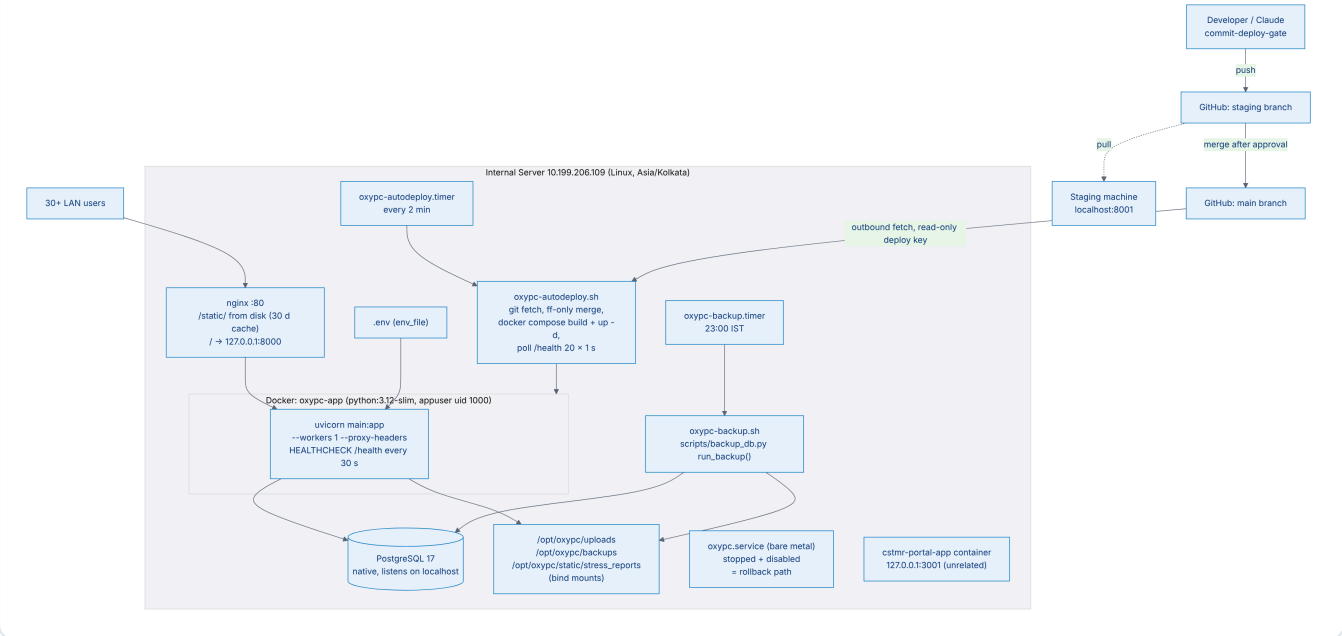
1. Environments

Environment	URL	Host and runtime	Code source	Database	Source
Production	`http://10.199.206.109` (plain HTTP, LAN only)	Linux Internal Server, `/opt/oxypc` ; nginx :80 → Docker container `oxypc-app` (uvicorn, 1 worker, host network, port 8000)	GitHub `main` (`Bluemonk123/oxypc-inventory`), pulled every 2 min	PostgreSQL 17, native on the same host	`deploy/nginx-oxypc-internal.conf` , `docker-compose.yml` , `Dockerfile` , `deploy/oxypc-autodeploy.sh` , design system release table
Staging	`http://localhost:8001`	Staging machines run `RUN_STAGING.bat` (Windows) or `START_STAGING.command` (Mac)	`staging` branch	Not stated	`qa_docs/design_system/design_system.html` (Environments row and "From change to production" step 3)
Local / developer	`http://localhost:8000`	`python main.py` or `uvicorn main:app`	Any branch	Local PostgreSQL (`oxypc_db`)	`README.md` Quick Start, `.claude/launch.json`
Legacy Windows server	`http://192.168.1.100:8000`	Windows laptop, uvicorn via scheduled task / NSSM service / watchdog	Local folder `C:\Users\Pankaj.sehgal\Claude\Oxypc\oxypc-inventory`	Local PostgreSQL	`CLAUDE.md` , `open_firewall.ps1` , `run_server.ps1`

Environment	URL	Host and runtime	Code source	Database	Source
Standalone UAT installer	`http://localhost:8000` (+ optional Cloudflare tunnel)	`OxyPC_UAT_Setup.exe` bundling Python 3.11.9 + PostgreSQL 16.6 (port 5433) + cloudflared	Build machine	Bundled PostgreSQL	`oxypc_setup.iss` (version `1.0-UAT`), `prepare_bundle.bat`, `BUILD_ME.bat`
Public VPS (planned)	`https://erp.oxypc.in` (placeholder)	Ubuntu VPS, nginx + certbot, systemd	—	—	`deploy/DEPLOY.md` ("Replace erp.oxypc.in with your real subdomain")

`RUN_STAGING.bat` and `START_STAGING.command` are **not** in the repository. See Open questions.

2. Production deployment diagram



Sources: `deploy/oxypc-autodeploy.{timer,service,sh}`, `deploy/oxypc-backup.{timer,service,sh}`, `deploy/nginx-oxypc-internal.conf`, `docker-compose.yml`, `Dockerfile`, docker spec §9 (other container, rollback path, PostgreSQL 17 noted in `Dockerfile`).

3. How the server runs

3.1 Production (Linux + Docker)

Piece	What it does	Source
`Dockerfile`	Two-stage build on `python:3.12-slim`; installs `postgresql-client-17` so the in-app "Run Backup Now" button has `pg_dump`; runs as non-root `appuser` (uid 1000); `HEALTHCHECK` calls `/health`; `CMD uvicorn main:app --host 0.0.0.0 --port 8000 --workers 1 --proxy-headers`	`Dockerfile`, commit `6b6bf96`

Piece	What it does	Source
`docker-compose.yml`	One service `oxypc-app`, `restart: unless-stopped`, `network_mode: host`, `env_file: .env`, bind mounts for `uploads/`, `backups/`, `static/stress_reports/`	`docker-compose.yml`
`.dockerignore`	Keeps local cruft (venvs, `wa-service/`, agent build output, Office files) out of the image	`.dockerignore`, docker spec §9
nginx	LAN vhost, no TLS; deliberately does **not** set `X-Forwarded-Proto` (otherwise the app's HTTPS redirect would break plain-HTTP access); serves `/static/` from disk with gzip and 30-day cache; 50 MB upload limit; 120 s proxy timeout	`deploy/nginx-oxypc-internal.conf`
Why one worker	In-memory caches (`_PERM_CACHE`, transitions) and the in-memory rate limiter are per process	`Dockerfile` comment, `main.py` `__main__` comment
Rollback path	`oxypc.service` (bare-metal venv) kept installed but disabled	docker spec §6 and §9

Conflict to resolve: `deploy/oxypc.service` and `services/oxypc-inventory.service` say 4 workers are safe because of the 60-second cache refresh. `Dockerfile` says one worker "is a hard requirement". Production uses one worker (Docker). See Open questions.

3.2 Windows scripts (legacy server, local and UAT)

All Windows scripts hard-code the path `C:\Users\Pankaj.sehgal\Claude\Oxypc\oxypc-inventory` and port 8000.

Script	Purpose	Notes	Source
`start_server.bat`	Endless loop: start uvicorn on `127.0.0.1:8000`, restart 5 s after exit; logs to `server_service.log`	Binds to localhost only, so not reachable from LAN	file
`start_oxypc.bat`	One-shot uvicorn on `0.0.0.0:8000`, 1 worker, logs to `server.log`	Used by the watchdog	file
`run_server.ps1`	Supervisor loop around uvicorn (`C:\Python314\python.exe`), restart after 3 s	Launched by the `OxyPC_ERP` scheduled task	file
`setup_autostart.ps1`	Registers scheduled task `OxyPC_ERP` at logon, restart up to 999 times	Run elevated once	file
`watchdog_server.ps1`	Pings `http://127.0.0.1:8000/auth/login`; if down, kills the port-8000 listener and starts `start_oxypc.bat`	Logs to `watchdog.log`	file
`setup_watchdog.ps1`	Registers `OxyPC_ERP_Watchdog` task: at logon and every 30 min; **removes** the `OxyPC_ERP` task	So watchdog and supervisor are alternatives, not both	file
`install_service.bat`	NSSM service `OxyPC_ERP` running `C:\Python313\python.exe main.py`; logs rotated at 10 MB	Needs `nssm.exe` on PATH	file
`scripts/install_service.ps1`	Downloads NSSM 2.24, installs service `OxyPCInventory` wrapping `scripts/start_server.bat` (venv, `0.0.0.0:8000`)	Second, different service name	file

Script	Purpose	Notes	Source
`setup_server_ip.bat`, `open_firewall.ps1`	Open firewall port 8000; force static IP `192.168.1.100`, DNS `8.8.8.8`	Changes network settings	files
`stop_server.bat`, `oxypc_stop.bat`	Kill uvicorn / ngrok / cloudflared; `stop_server.bat` kills **every** `python.exe`	Use with care	files
`launcher.py`, `oxypc_launcher.bat`	Runs the app plus a Cloudflare tunnel; expects PostgreSQL already running		`launcher.py` header
`build_all.bat`, `BUILD_ME.bat`, `prepare_bundle.bat`, `build_installer.bat`, `oxypc_setup.iss`, `oxypc_setup_light.iss`	Build `dist\OxyPC_UAT_Setup.exe` with Inno Setup 6; bundle Python 3.11.9, PostgreSQL 16.6-1 (port 5433), cloudflared	Version `1.0-UAT`; Windows 10 1809+ (`MinVersion=10.0.17763`)	files
`build.spec`	PyInstaller spec → `dist/OxyPC_Inventory.exe`		`README.md` "Build EXE"
`installer.py`	Browser-based Linux installer on port 8080; clones `TheCodeOrbit/oxypc` into `/opt/oxypc`	Points at a different GitHub repo	`installer.py` header

3.3 Linux bare metal (pre-Docker, kept for rollback)

`deploy/oxypc.service` runs `/opt/oxypc/venv/bin/uvicorn main:app --host 127.0.0.1 --port 8000 --workers 4` as `www-data`, with `Restart=always`. `deploy/DEPLOY.md` is the full VPS runbook (PostgreSQL, venv, systemd, nginx + certbot, hardening list).

---

4. Configuration and secrets

4.1 How settings are resolved

`config.py` loads `.env` (python-dotenv), then reads each setting in this order: **environment variable → `config.ini` → hard-coded default**. On first run `main.py` writes a `config.ini` if none exists (`write_default_config`), and that file contains the secret key and API key in plain text.

`.env` and `config.ini` must never be committed (`config.py` comment). Templates: `.env.example`, `deploy/.env.example`.

4.2 Environment variables (names only)

Variable	Used for	Read in
`OXYPC_DATABASE_URL`	Main DB URL (`postgresql+asyncpg://…`)	`config.py`, `scripts/backup_db.py`
`OXYPC_SECRET_KEY`	JWT signing key	`config.py`
`OXYPC_API_KEY`	OxyQC machine API key	`config.py`
`OXYPC_TOKEN_EXPIRE_MINUTES`	**No longer read.** Session length is fixed at 1440 min in code (commit `d8bc498`)	`.env.example` only

Variable	Used for	Read in
`OXYPC_REFRESH_DAYS`	Refresh token days (default 7)	`config.py`
`OXYPC_PORT`, `OXYPC_HOST`	Bind port/host for `python main.py`	`config.py`
`OXYPC_COOKIE_SECURE`	Secure flag on auth cookies (needs HTTPS)	`config.py`
`OXYPC_ALLOWED_ORIGINS`	CORS origins for `/api/v1/*`	`config.py`
`OXYPC_DB_POOL_SIZE`, `OXYPC_DB_MAX_OVERFLOW`	Pool size (default 8 + 4)	`database.py`
`OXYPC_AUTO_FIX`	Let `db_validator` fix schema at startup (default on)	`main.py`
`OXYPC_DEBUG`	Show exception detail on error pages (LAN dev only)	`main.py`
`OXYPC_OPEN_BROWSER`	Open a browser when run via `python main.py`	`main.py`
`OXYPC_FIELDOPS_DATABASE_URL` / `FIELDOPS_DATABASE_URL`	FieldOps DB; unset → `/fieldops` disabled	`fieldops_db.py`
`FIELDOPS_ADMIN_USERNAME`, `FIELDOPS_ADMIN_PASSWORD`, `FIELDOPS_DEMO_PASSWORD`, `FIELDOPS_DB_POOL_SIZE`, `FIELDOPS_DB_MAX_OVERFLOW`	FieldOps accounts and pool	`fieldops_db.py`
`OXYPC_CARE_API_BASE`	Care Agent → server base URL	`care_agent_windows/`
`OXYPC_BACKUP_PRUNE`	`1` lets the nightly job prune old backups (default off)	`deploy/oxypc-backup.sh`
`OXYPC_DB_HOST`, `OXYPC_DB_PORT`, `OXYPC_DB_NAME`, `OXYPC_DB_USER`, `OXYPC_BACKUP_DB_PASS`	Root-level `backup_db.py` (older script)	`backup_db.py`
`PGPASSWORD`, `OXYPC_BACKUP_DIR`, `OXYPC_RETENTION_DAYS`	Shell backup/restore scripts	`scripts/backup.sh`, `scripts/restore.sh`

Risky defaults in code (values not repeated here): `config.py` has a fallback DB URL with a password, and a fallback OxyQC API key marked "change-me". If `OXYPC_SECRET_KEY` is unset and no `config.ini` exists, a random key is generated and written to `config.ini`. `deploy/DEPLOY.md` "MUST-HARDEN" lists rotating all of these.

5. Database schema changes

Three mechanisms exist side by side. Production relies mainly on the first.

Mechanism	What it does	When it runs	Source
`db_validator.validate_and_fix()`	Compares ORM models to the live DB. Creates missing tables, `ADD COLUMN IF NOT EXISTS` for missing columns, reseeds `stage_master` and `allowed_transitions`. Raises (and aborts startup) if something cannot be fixed	Every app start (so every deploy) when `OXYPC_AUTO_FIX=1` (default). Also `python db_validator.py` to diagnose only	`db_validator.py` header, `main.py` startup, `deploy/oxypc-autodeploy.sh` comment

Mechanism	What it does	When it runs	Source
Alembic	36 revisions from `20260331_1600_ba22dd09a6a3_initial_baseline` to `20261013_1200_dealers_sub_category_website_cid`. `alembic/env.py` reads the URL from `config.py`	Manual `alembic upgrade head`. Not run by the Docker auto-deploy	`alembic/versions/`, `alembic.ini`
One-off scripts	51 `migrate_*.py` (e.g. `migrate_perf_indexes.py` creates/drops indexes), 8 `backfill_*.py`, `upgrade_db.py` (adds enum values and columns, `create_all`, stamps Alembic), `setup_db.py` (first install)	Manual, one at a time	repo root
Startup backfills	Idempotent one-time data fixes (GRN-Lot links, tag backfills, size units, Product Master)	Every app start	`main.py` startup

Known consequences:

- `db_validator` only **adds**. It does not rename, drop, change types, or add indexes. Those still need Alembic or a `migrate_*.py` script.
- Alembic is not reliably applied in production. `20260925_1200_add_dashboard_perf_indexes.py` says indexes "declared in an earlier migration (f991834630d1) ... [were] never actually present on the live schema".
- `CLAUDE.md` rule: no schema change without dbdiagram.io approval (`docs/schema.dbml`; design system step 1).

---

6. Release process

6.1 Branches

Branch	Role	Evidence
`staging`	Work lands here first; staging machines load it at `localhost:8001`	Design system step 3; `origin/staging` = `55ce58a`
`main`	Production. Auto-deploy polls `origin/main`	`deploy/oxypc-autodeploy.sh`; `origin/main` = `bfca61d` (behind staging at time of writing)

6.2 Steps (from `qa_docs/design_system/design_system.html` "From change to production")

#	Step	Gate	Source
1	Database approval	New table/column drafted as `.dbml`, approved in dbdiagram.io	design system, `CLAUDE.md`
2	Build and verify	`verify-feature` skill (compile, Jinja parse, `db_validator.py`, httpx script); `regression-sweep` if `base.html`, `app.css`, a macro or a Jinja global changed	`.claude/skills/verify-feature`, `.claude/skills/regression-sweep`
3	Staging	Push to `staging`; test at `localhost:8001` with Ctrl+F5	design system
4	Approval	Explicit "push" for **this** batch; an earlier yes never carries over	`.claude/skills/commit-deploy-gate/SKILL.md`

#	Step	Gate	Source
5	Production	Merge to `main`. Auto-deploy picks it up within ~2 min (design system says 2-5 min)	`deploy/oxypc-autodeploy.timer`, design system
6	Docs	Update the design system HTML and rebuild the PDF in the same change	design system

6.3 What auto-deploy does

From `deploy/oxypc-autodeploy.sh`:

1. `git fetch origin main`. Exit quietly if nothing new.
2. **Refuse** to deploy if the server working tree has uncommitted changes.
3. Save the current SHA to `/var/lib/oxypc-last-good-sha`.
4. `git merge --ff-only origin/main`.
5. `docker compose build` then `docker compose up -d`.
6. Poll `http://127.0.0.1:8000/health` once a second for up to 20 s.
7. On failure, print the rollback command (`git reset --hard <last good>` + `docker compose up -d --build`). It does **not** roll back automatically, by design.

Watch it with `journalctl -u oxypc-autodeploy -f`; pause it with `systemctl stop oxypc-autodeploy.timer` (`deploy/oxypc-autodeploy.service` header).

6.4 Health checks

Check	What it tests	Source
`GET /health`	Runs `SELECT 1`. 200 `{"status":"ok","db":"ok",…}` or 503 `degraded`. Exempt from HTTPS redirect	`routers/health.py`, `main.py` `_force_https`
Docker `HEALTHCHECK`	`/health` every 30 s, 3 retries	`Dockerfile`, `docker-compose.yml`
`GET /api/v1/health`	API-layer health	`routers/api_v1/health.py`
`GET /iqc/api/health`	OxyQC connectivity	`routers/iqc_api.py` header
`GET /admin/backup-status`	Age and size of the newest backup file (admin only)	`routers/admin.py` l.613
`/admin/system/health`	**Not found in the code.** No route with this path exists on staging @ 55ce58a	grep of `routers/`, `templates/`

7. CI/CD in GitHub

Workflow	Trigger	What it does	Status note
`.github/workflows/build-agent.yml`	Push to `main` touching `oxyqc-agent/oxyqc_agent.py` or its `.spec`	Builds `Diagnose_Device_Agent.exe` on `windows-latest` with Python 3.12 and commits it	Active per file

Workflow	Trigger	What it does	Status note
`.github/workflows/deploy.yml`	Push to `main`	On a **self-hosted** runner: pip install into `/opt/oxypc/venv`, rsync, `alembic upgrade head`, force-run `db_validator`, `systemctl restart oxypc`, curl `/health`	Targets the bare-metal service that was disabled at the Docker cutover. May be stale

There is no CI job that runs `pytest` (no workflow calls it).

Open questions

1. Where are `RUN_STAGING.bat` and `START_STAGING.command`? Which database does staging use (a copy of production, or seed data)? Who owns the staging machines?
2. Is `.github/workflows/deploy.yml` still connected to a self-hosted runner? If yes, it would try to restart the disabled `oxypc.service` on every push to `main`.
3. Workers: should production stay at 1 worker (`Dockerfile`) or move to 4 (`deploy/oxypc.service`)? Moving needs a shared rate-limit store and a single owner for the daily report job.
4. Is Alembic `alembic_version` on production at head? What is the policy: Alembic for every change, or `db_validator` plus scripts?
5. Is there a public or Railway deployment (`README.md` names `https://app.oxypc.com/fieldops`; Railway is mentioned in `main.py`, `limiter.py`, `deploy/oxypc-autodeploy.timer`)? If so, what is its database and release flow?
6. Is `/admin/system/health` a planned page? It is not in the code.
7. Does the nightly backup still run with `/opt/oxypc/venv/bin/python` on the host after the Docker cutover? (`deploy/oxypc-backup.sh` assumes the host venv exists.)
8. Who holds the server SSH keys and the GitHub read-only deploy key, and where are `.env` values stored outside the server?

PART B

Non-Functional Requirements

This document lists the quality requirements for the OxyPC Inventory Lifecycle app. Each requirement has an ID (NFR-xx), the **current state** found in the code, a **target**, and the **gap**. Targets marked *(proposed)* are not written down anywhere in the repo; they are suggestions for the business to confirm. Unknown numbers are not invented; they are listed under Open questions.

1. Summary scorecard

ID	Area	Status
NFR-01	Large-list performance	Partly met
NFR-02	Page and dashboard response time	Unmeasured
NFR-03	Database indexing	Partly met
NFR-04	Concurrency and capacity	Partly met
NFR-05	Availability and auto-restart	Met for app; single server
NFR-06	Backup	Partly met
NFR-07	Restore and recovery	Not tested in repo
NFR-08	Authentication and sessions	Partly met
NFR-09	Authorisation	Met, with drift
NFR-10	Transport and data protection	Not met (LAN HTTP)
NFR-11	Audit trail	Met
NFR-12	Browser support	Met (desktop)
NFR-13	Display scale (90 % zoom)	Met
NFR-14	Phone and tablet use	Partly met
NFR-15	Accessibility	Not defined
NFR-16	Logging and monitoring	Partly met
NFR-17	Maintainability	Partly met
NFR-18	Testability and release safety	Partly met
NFR-19	Data integrity (soft delete, schema)	Met, with drift

Status legend: **Met** = code meets the stated target. **Partly met** = some evidence, clear gaps. **Not met / Not defined** = no evidence or no target.

2. Performance

NFR-01 · Large lists stay fast

Requirement	Lists with thousands of devices, tags or sales load and filter without freezing the browser or the server.
Current state	13 heavy list pages use server-side DataTables (`serverSide: true`), e.g. `templates/devices/list.html` , `templates/iqc/list.html` , `templates/sales/list.html` , `templates/grn/records.html` , `templates/gate_pass/list.html` . `static/js/global-table.js` adds the shared UX on top (12 rows default, sticky first column, cell clamping). Repair and QC lists got `page` / `page_size` pagination (`docs/superpowers/plans/2026-05-08-sprint21-pagination-softdelete.md`). CSV/Excel report exports are capped at `MAX_EXPORT_ROWS = 5_000` (`routers/reports.py` l.40). The LAN nginx has larger proxy buffers because "some list pages render multi-MB HTML responses" (`deploy/nginx-oxypc-internal.conf`).
Target *(proposed)*	Every list that can exceed ~1,000 rows uses server-side paging; no page sends more than ~1 MB of HTML.
Gap	229 of 242 templates do not use server-side paging; some still render full tables in HTML. No inventory of which pages exceed the size budget. The 5,000-row export cap can silently truncate (a warning exists at `routers/reports.py` l.882 for one report).

NFR-02 · Page and dashboard response time

Requirement	Common pages respond quickly for floor staff.
Current state	Dashboard counts were batched into one grouped query (PERF-3, `docs/superpowers/plans/2026-05-14-sprint22-dashboard-perf-report-limits-role-checks.md`). Sprint 18 plan aimed to cut dashboard load "from 3–8 s to under 1 s" (`docs/superpowers/plans/2026-04-27-sprint18-20-performance-smartux.md`); no measured result is recorded. Static files are served by nginx with gzip and a 30-day cache; measured on `10.199.206.109` : `bootstrap.min.css` 30.3 → 10.7 ms, `chart.umd.min.js` 40.1 → 18.4 ms, `app.js` 8.0 → 0.8 ms (`deploy/nginx-oxypc-internal.conf`). App responses are gzip-compressed above 1,000 bytes (`main.py`). A stress-test module exists (`routers/stress_api.py` , `stress_runner.py`).
Target *(proposed)*	95th percentile under 2 s for list and dashboard pages at 30 concurrent users.
Gap	No agreed target. No request-time logging or APM, so there is no baseline.

NFR-03 · Database indexing

Requirement	Hot filter, join and group-by columns are indexed.
Current state	Index migrations: `20260426_0940_f991834630d1_add_missing_fk_indexes.py` (9 indexes), `20260427_1811_b62e6ba33486_add_composite_indexes_sprint18.py` (4), `20260925_1200_add_dashboard_perf_indexes.py` (6), plus `migrate_perf_indexes.py` (adds `ix_device_location_logs_location_id` , drops a duplicate on `devices.location_id`).
Target	Every index declared in models or migrations exists in production.
Gap	The September migration states the April indexes were "never actually present on the live schema". Production index state is not checked by any tool; `db_validator.py` checks tables and columns only.

NFR-04 · Concurrency and capacity

Requirement	30+ concurrent LAN users (`CLAUDE.md`) without errors or 429s.

Current state	Production runs **one** uvicorn worker (async) in Docker (`Dockerfile`). DB pool 8 + 4 overflow per process (`database.py`). Rate limit 2,000 requests/min per session, 20/min per IP on login (`limiter.py` , `routers/auth.py` l.55); keyed per session so one NAT'd office IP is not one shared bucket. Per-process caches refreshed every 60 s (`main.py`).
Target *(proposed)*	50 concurrent users with headroom; no request blocked by another user's long task.
Gap	Any blocking call stalls everyone on a single worker. Example: `POST /admin/backup-now` runs `subprocess.run(... timeout=300)` synchronously inside an async route (`routers/admin.py` l.711-727), so the app can freeze for the whole backup. Scaling to more workers needs a shared rate-limit store and a single owner for the 9 PM report job (`services/daily_stock_reports.py`).

3. Availability and recovery

NFR-05 · Availability and auto-restart

Requirement	The app restarts itself after a crash or reboot, during working hours and after power loss.
Current state	Production: `restart: unless-stopped` and a Docker `HEALTHCHECK` on `/health` every 30 s (`docker-compose.yml` , `Dockerfile`). Auto-deploy waits up to 20 s for `/health` and reports unhealthy deploys, without auto-rollback (`deploy/oxypc-autodeploy.sh`). Rollback path: the disabled `oxypc.service` (docker spec §6). Legacy Windows: `watchdog_server.ps1` every 30 min, `setup_autostart.ps1` at logon, NSSM services (`install_service.bat` , `scripts/install_service.ps1`).
Target *(proposed)*	Available during business hours; unplanned downtime under 30 min per incident.
Gap	No uptime target or monitoring outside the box (`routers/health.py` mentions Uptime Robot but none is configured in the repo). One physical server holds app, database and backups. `restart: unless-stopped` restarts a crashed container but Docker does not restart a container that is merely "unhealthy".

NFR-06 · Backup

Requirement	Database and uploaded files are backed up daily and kept off the server.
Current state	Nightly systemd timer at 23:00 IST with `Persistent=true` (`deploy/oxypc-backup.timer`), runs `scripts/backup_db.py run_backup()` → `backups/oxypc_YYYYMMDD_HHMMSS.sql.gz` (plain SQL, gzipped). Pruning is **off** unless `OXYPC_BACKUP_PRUNE=1` ; a dump is "~2 MB" (`deploy/oxypc-backup.sh`). Admins can run, list, download and delete backups (`routers/admin.py` l.613-727, audited).
Target *(proposed)*	Daily DB **and** `uploads/` backup, with at least one copy off the server, checked daily.
Gap	Only the database is dumped; `uploads/` (invoices, payment photos, KYC, IQC images) has no backup job. Off-site copy is only "recommended" (`docs/backup-runbook.md` , `deploy/DEPLOY.md` §7). Three schedules are documented (02:00 in `docs/backup-runbook.md` and `scripts/setup_backup_task.bat` , 23:00 in `deploy/oxypc-backup.timer` and `deploy/DEPLOY.md` , 23:00 in root `backup_db.py`), and root `backup_db.py` writes `oxypc_db_YYYY-MM-DD_…` names that the admin panel's filename pattern does not list.

NFR-07 · Restore and recovery

Requirement	A tested way to restore to a known point within an agreed time.
Current state	`scripts/restore.sh` pipes a dump into `psql` after typing `RESTORE`. `docs/backup-runbook.md` describes a monthly test restore into `oxypc_test`.
Target	RPO and RTO agreed and tested (see 08 Runbook & DR).
Gap	No RPO/RTO defined. No record of a test restore. The dump is made without `--clean`, so restoring into a non-empty database fails or duplicates; `restore.sh` says it "will DROP all tables" but does not.

4. Security (summary)

Full detail lives in 04 Security & Access Design.

NFR-08 · Authentication and sessions

Current state	JWT in an `httponly`, `samesite=strict` cookie (`routers/auth.py` l.115). Session fixed at 1 day (`config.py`, commit `d8bc498`). bcrypt 5.0.0 password hashing (`auth/dependencies.py`). Lockout after 5 failed logins for 15 min (`routers/auth.py` l.37-38). Login 20/min per IP.
Target *(proposed)*	As now, plus MFA for admins (`deploy/DEPLOY.md` hardening item 9).
Gap	`secure` flag is off unless `OXYPC_COOKIE_SECURE=1`, which needs HTTPS (not on the LAN). No MFA.

NFR-09 · Authorisation

Current state	Built-in roles + admin-created custom roles + Permission Matrix (`auth/dependencies.py` `require_roles`, `_matrix_grants_path`, `require_module_perm`). Partner, care-agent and API-key auth are separate (`auth/partner_auth.py`, `auth/care_auth.py`, `auth/api_key.py`).
Gap	15 template checks still hard-code `current_user.role.value` instead of `role_allowed()` (e.g. `templates/crm/contacts/list.html` l.52), so custom roles may not see buttons their backend allows.

NFR-10 · Transport and data protection

Current state	Production is plain HTTP on the LAN (`deploy/nginx-oxypc-internal.conf`). nginx sets `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`. CSRF double-submit on all writes (`verify_csrf`). PostgreSQL trusts only `127.0.0.1` (docker spec §9).
Gap	No TLS, so passwords and cookies cross the LAN in clear text. `/uploads` is mounted as public static files with no login check (`main.py` l.136); file names are UUIDs (`routers/accounts.py` l.43) but anyone with the link can open them. Code holds default credentials as fallbacks (`config.py`).

NFR-11 · Audit trail

Current state	`services/audit_engine.audit()` called 242 times in routers/services; records user, IP (from `X-Forwarded-For`), table, record, old/new values. Rule: audit **before** `db.delete()` (`CLAUDE.md`). Admin audit-log page (`routers/admin.py`).

Gap	No retention policy for audit rows. No check that every `db.delete()` (36 call sites) is preceded by an audit call.

5. Usability, devices and accessibility

NFR-12 · Browser support

Current state	`static/js/global-table.js` (point 8) states the table works "identically in Chrome, Edge, Safari, and Firefox", and fixed a cross-browser cache issue on 2026-09-03 via `ASSET_VERSION` cache busting (`templates_config.py` , `tests/test_asset_version_cache_busting.py`). All libraries are vendored (`static/vendor/`), so pages work without internet. HTML is sent `no-store` so a deploy is seen at once (`main.py` `_no_store_pages`).
Target *(proposed)*	Latest Chrome and Edge on Windows as primary; Safari and Firefox best effort.
Gap	No written support matrix or minimum versions.

NFR-13 · Display scale (90 % app zoom)

Current state	`body { zoom: 0.9 }` via `--app-zoom` (`static/css/app.css` l.12-20). Viewport-height layout uses `--full-vh = 100vh / 0.9` . Modal backdrop and Chart.js hover were fixed for zoom (`static/css/app.css` l.23-30, `static/js/chart-zoom-fix.js` , commit `eb8b4e2`).
Gap	CSS `zoom` behaves differently across engines (Firefox only gained support recently); new components must remember `--full-vh` . Effective table text is 14 px × 0.9 ≈ 12.6 px (`global-table.js` point 6), which may be small for some users.

NFR-14 · Phone and tablet use

Current state	`base.html` has a responsive viewport tag; `app.css` has one `@media (max-width: 768px)` block. Dedicated mobile PWAs exist for telecalling (`static/m/telecalling/` , with offline queue), trade partners (`static/partner/`), and Reliance FieldOps (`fieldops_app/`). Gate Pass supports scan auto-select (commit `170c51f`).
Target *(proposed)*	Security-gate staff can confirm exits on a phone; IQC and stock pages usable on a tablet.
Gap	Main staff pages are desktop-first; there is no mobile layout for Gate Pass or scanning pages and no phone test in the UAT plan. Whether gate staff actually use phones is not recorded.

NFR-15 · Accessibility

Current state	About 200 `aria-` attributes across templates (grep). Bootstrap 5 defaults. Global Table puts the full text of clamped cells in `title=` tooltips.
Target *(proposed)*	WCAG 2.1 AA for colour contrast, keyboard use and form labels on core floor pages.
Gap	No accessibility target or audit. Colour-only status pills and small effective font size are likely issues.

6. Operations and engineering quality

NFR-16 · Logging and monitoring

Current state	Unhandled 500s go to console and to `errors.log` next to `main.py` (`main.py` l.112-120). Startup prints a status line per step. In Docker, console output is visible via `docker compose logs` . Auto-deploy and backup runs are in the systemd journal.
Target *(proposed)*	Errors and slow requests retained for 30 days on persistent storage; alert when `/health` fails or no backup in 26 h.
Gap	In the container `errors.log` is written to `/app/errors.log` , which is **not** a bind mount, so it is lost on every rebuild (every deploy). No log rotation is configured for `errors.log` or Docker's log driver (`docker-compose.yml`). No access log with timings, no alerting.

NFR-17 · Maintainability

Current state	Clear layering (routers / models / services / templates), project skills that encode rules (`.claude/skills/`), and a design system document (`qa_docs/design_system/design_system.html`).
Gap	`main.py` is 868 lines with many one-time backfills in startup. The repo root holds 51 `migrate_*.py` , 8 `backfill_*.py` , 12 `verify_*.py` , 9 `seed_*.py` , 7 `_*.py` scratch scripts, Office documents and logs (`server_log.txt` , `test_results.txt`). Three ways to change the schema (see 06 §5). Two Windows service names (`OxyPC_ERP` , `OxyPCInventory`). Stray `routers/repair.py.bak` . Dead WhatsApp code kept (`routers/whatsapp.py` , `models/whatsapp.py` , `wa-service/`).

NFR-18 · Testability and release safety

Current state	172 pytest files (`tests/` , `pytest.ini`). Mandatory pre-release skills: `verify-feature` , `regression-sweep` , `commit-deploy-gate` . Staging at `localhost:8001` before `main` .
Gap	No CI runs the tests (`.github/workflows/` has only deploy and agent build). Release relies on manual discipline. No automatic rollback on failed health check (deliberate, `deploy/oxypc-autodeploy.sh`).

NFR-19 · Data integrity

Current state	Soft delete with `is_trashed` (NOT NULL on main tables), Trash page for restore (`routers/trash.py`). Startup schema validator aborts on unfixable drift (`db_validator.py`). Server-side Decimal money totals (`.claude/skills/server-side-financial-snapshot`). Transaction per request (`database.py`).
Gap	`CLAUDE.md` still says `is_trashed` is nullable; rule and code differ (see 01 §6). No database-level check that soft-deleted rows are excluded from every report.

Open questions

1. What response time and uptime does the business expect (business hours, days, peak user count)?
2. What RPO (data loss) and RTO (time to recover) are acceptable? None are written down.
3. Do security-gate staff use phones for Gate Pass today? Which phone browsers?
4. Is there an accessibility requirement (for example, staff with low vision)?

5. How long must audit logs, backups and uploaded documents be kept (tax/GST rules)?
6. Is moving the LAN app to HTTPS (internal certificate) planned?
7. What is the current production data size (rows in `devices`, `sales`; size of `uploads/`)? The only figure in the repo is "~2 MB" per gzipped dump (`deploy/oxypc-backup.sh`).
8. Which browsers and versions are installed on floor PCs?

PART C

Runbook & Disaster Recovery

This is the operator's guide for the production OxyPC Inventory Lifecycle server. It covers daily checks, how to start and stop the app, common incidents, backup and restore, and disaster recovery. It builds on `docs/backup-runbook.md`, `deploy/DEPLOY.md` and `docs/superpowers/specs/2026-10-01-docker-containerization-design.md`.

Production setup at a glance (from 06 Deployment): Linux Internal Server `10.199.206.109`, app folder `/opt/oxypc`, Docker container `oxypc-app` on port 8000, nginx on port 80, PostgreSQL 17 native on the same box, database `oxypc_db`. Commands below assume an SSH session on that server in `/opt/oxypc`.

Rule: any restart or deploy follows `.claude/skills/commit-deploy-gate` — get an explicit go-ahead for this specific action first.

1. Daily checks (5 minutes, start of shift)

#	Check	How	Healthy result	Source
1	App and DB up	`curl -s http://127.0.0.1:8000/health` (or open `http://10.199.206.109/health`)	`{"status":"ok","db":"ok",…}`, HTTP 200	`routers/health.py`
2	Container healthy	`docker compose ps`	`oxypc-app` … `(healthy)`	`docker-compose.yml` healthcheck
3	Last night's backup ran	`journalctl -u oxypc-backup --since yesterday` or Admin → Database Backup panel (`/admin/backup-status`)	A file `oxypc_YYYYMMDD_23xxxx.sql.gz` under 26 h old	`deploy/oxypc-backup.service`, `routers/admin.py` l.613
4	Backup timer armed	`systemctl list-timers oxypc-backup.timer`	Next run tonight 23:00 IST	`deploy/oxypc-backup.timer`
5	Auto-deploy healthy	`journalctl -u oxypc-autodeploy --since today`	Silence, or `DEPLOYED OK` lines. No `REFUSING TO DEPLOY` / `DEPLOY UNHEALTHY`	`deploy/oxypc-autodeploy.sh`
6	Disk space	`df -h /opt /var/lib/docker /var/lib/postgresql`	Under 80 % used (proposed threshold)	docker spec §9 (44 % on 2026-10-01)
7	New errors	`docker compose logs --since 24h oxypc-app | grep -E "500 |Traceback"`	None, or known	`main.py` `generic_exception_handler`
8	Daily Stock report made	Reports page / daily report list	Yesterday's 9 PM file exists	`services/daily_stock_reports.py`

Note: `oxypc-backup.service` is `Type=oneshot`, so `systemctl is-active` showing "inactive" after a good run is **normal** (`deploy/oxypc-backup.service` header).

2. Start, stop, restart

2.1 Production (Docker)

Action	Command	Notes
Status	`docker compose ps`	
Live logs	`docker compose logs -f oxypc-app`	Console output includes startup steps and 500 tracebacks
Restart app	`docker compose restart oxypc-app`	~10 s; worst cold start seen ~9 s (`deploy/oxypc-autodeploy.sh`)
Stop app	`docker compose stop oxypc-app`	Users see nginx 502
Start app	`docker compose up -d`	Runs `db_validator` and startup backfills
Rebuild + start (deploy current checkout)	`docker compose up -d --build`	What auto-deploy does
Pause auto-deploy	`sudo systemctl stop oxypc-autodeploy.timer`	Do this before any manual change or cutover
Resume auto-deploy	`sudo systemctl start oxypc-autodeploy.timer`	
Deploy now	`sudo systemctl start oxypc-autodeploy.service`	
nginx	`sudo nginx -t && sudo systemctl reload nginx`	Config: `deploy/nginx-oxypc-internal.conf`
PostgreSQL	`sudo systemctl status postgresql` / `restart postgresql`	App reconnects (`pool_pre_ping=True` , `database.py`)

2.2 Roll back a bad deploy

1. `sudo systemctl stop oxypc-autodeploy.timer` (stop it redeploying).
2. Read the last good SHA: `cat /var/lib/oxypc-last-good-sha` .
3. `cd /opt/oxypc && git reset --hard <sha> && docker compose up -d --build` (command printed by `deploy/oxypc-autodeploy.sh` on failure).
4. Check `/health` . Then fix forward on `main` , because the timer will redeploy `main` when resumed.

Emergency fallback to bare metal (docker spec §6): `docker compose down` then `sudo systemctl enable --now oxypc.service` . Only if the host venv is intact (see Open questions).

2.3 Legacy Windows server (if ever used again)

Action	Command	Source
Start	`start_oxypc.bat` or scheduled task `OxyPC_ERP` / `OxyPC_ERP_Watchdog`	`start_oxypc.bat` , `setup_autostart.ps1` , `setup_watchdog.ps1`
Service	`nssm start|stop|status OxyPC_ERP` (or `OxyPCInventory`)	`install_service.bat` , `scripts/install_service.ps1`
Stop	`oxypc_stop.bat` (kills port 8000 listener). Avoid `stop_server.bat` : it kills **every** `python.exe`	files

3. Common incidents

Each row: **Symptom → Check → Fix**. Commands are for production unless noted.

3.1 Server down (browser cannot load the app)

Symptom	Check	Fix
Browser "can't reach site"	`ping 10.199.206.109` ; `sudo systemctl status nginx`	Network/power issue → escalate to IT. nginx stopped → `sudo systemctl start nginx`
nginx "502 Bad Gateway"	`docker compose ps` ; `curl -s 127.0.0.1:8000/health`	Container stopped → `docker compose up -d` . Restarting in a loop → read `docker compose logs --tail=100 oxypc-app` (often a schema error, see 3.5)
Page redirects to `https://` and fails	Was `X-Forwarded-Proto` added to the nginx vhost?	Remove it. The LAN vhost must not set it (`deploy/nginx-oxypc-internal.conf` , `main.py` `_force_https`)
New code not live after push	`journalctl -u oxypc-autodeploy -n 50`	`REFUSING TO DEPLOY: working tree has uncommitted changes` → inspect `git status` on server, commit or discard deliberately. `fetch failed` → network or deploy key
Users see old page after deploy	Ctrl+F5	HTML is `no-store` ; static files are cache-busted by `ASSET_VERSION` (`main.py` , `templates_config.py`). If a JS file changed but is not in `_VERSIONED_ASSETS` , add it

3.2 Database connection errors

Symptom	Check	Fix
`/health` returns 503 `"db":"error: …"`	`sudo systemctl status postgresql` ; `sudo -u postgres psql -c "select 1"`	Start PostgreSQL. Then `docker compose restart oxypc-app` if errors persist
"too many connections"	`sudo -u postgres psql -c "select count(*), state from pg_stat_activity group by state"`	Pool per process is 8 + 4 (`database.py`). Look for idle-in-transaction sessions or extra processes (old `oxypc.service` , scripts). Terminate stuck sessions; restart app
Connection refused from container	`.env` `OXYPC_DATABASE_URL` host; `pg_hba.conf`	Container uses host network and must use `localhost` / `127.0.0.1` . PostgreSQL only trusts `127.0.0.1` (docker spec §9)
Password failure after rotation	`.env` vs DB role	Update `.env` , `docker compose up -d` (env is read at container start)

3.3 403 "Access Denied" for a user

Symptom	Check	Fix
User sees 403 on a page their role should open	Master Data → Module Permissions for that role (the 403 page says this, `main.py` `_ERROR_INFO`)	Tick the module for the role. Changes reach every cache within 60 s (`main.py` cache refresh)
Custom role (e.g. `trc_manager`) can open the page but buttons are missing	Template uses `current_user.role.value == …` instead of `role_allowed()`	Code fix per `.claude/skills/custom-role-permissions`

Symptom	Check	Fix
Built-in role blocked on an admin-only page	`require_roles(UserRole.admin)` on the route	Grant the matching module in the matrix; `_matrix_grants_path` honours explicit grants (`auth/dependencies.py`)
User bounced to login on form submit	CSRF cookie missing/stale = session expired	Log in again. Sessions last 1 day (`config.py`)
User cannot log in at all	Lockout: 5 failures in 15 min (`routers/auth.py`); or user `status` inactive	Wait 15 min or have admin check User Management
"429 Too Many Requests"	Login is limited to 20/min per IP	Wait one minute

3.4 Slow pages

Symptom	Check	Fix
Whole app slow for everyone at once	Is a manual backup running? (`/admin/backup-now` blocks the single worker up to 300 s, `routers/admin.py`)	Wait for it to finish; avoid manual backups in working hours
	`docker stats oxypc-app` ; `top` for postgres	High CPU in postgres → find slow query: `select pid, now()-query_start, query from pg_stat_activity where state='active' order by 2 desc`
One list page slow	Is it a server-side DataTables page? (`serverSide: true` in template)	Check indexes exist: `\di` in psql vs `alembic/versions/20260925_1200_add_dashboard_perf_indexes.py` and `migrate_perf_indexes.py` . Re-run missing index scripts
Large export slow or cut off	Reports are capped at 5,000 rows (`routers/reports.py`)	Narrow the date range

3.5 Failed schema change / migration

Symptom	Check	Fix
Container restarts in a loop after deploy	`docker compose logs oxypc-app | grep -A20 Schema`	`db_validator` raised `RuntimeError` for an unfixable mismatch (it can add tables/columns, not change types or rename). Roll back (2.2), then fix with an Alembic revision or a `migrate_*.py` script
500 on a page with `UndefinedColumn`	Was `OXYPC_AUTO_FIX` set to 0?	Run `docker compose exec oxypc-app python db_validator.py` to diagnose; restart with auto-fix on
`alembic upgrade head` fails	`alembic current`	Production is often not stamped at head (see `20260925_1200_add_dashboard_perf_indexes.py` note). Check the revision's SQL by hand; most revisions use existence checks
Before any manual migration	—	Take a backup first (`migrate_perf_indexes.py` header: "Backup taken first")

3.6 Disk full

Symptom	Check	Fix
Uploads fail, backup fails, Postgres errors "No space left on device"	`df -h` ; `du -sh /opt/oxypc/backups /opt/oxypc/uploads /var/lib/docker`	1) `docker builder prune -f` (reclaimed 4.48 GB on 2026-10-01, docker spec §9). 2) Old images: `docker image ls` , remove unused. 3) Backups are never pruned by default: copy old ones off-server, then delete via Admin panel or set `OXYPC_BACKUP_PRUNE=1` for one run. 4) Container logs: `docker inspect --format='{{.LogPath}}' oxypc-app` and truncate if huge
Upload "Permission denied" (not disk)	`ls -ln /opt/oxypc/uploads`	Must be owned by uid 1000: `sudo chown -R 1000:1000 uploads backups static/stress_reports` (docker spec §9)

4. Backup

4.1 What is backed up today

Item	Backed up?	How	Where	Source
Database `oxypc_db`	Yes, nightly 23:00 IST (+ up to 2 min jitter; missed runs fire on boot)	`oxypc-backup.timer` → `oxypc-backup.sh` → `scripts/backup_db.py run_backup()` (`pg_dump --format=plain` , gzip)	`/opt/oxypc/backups/oxypc_YYYYMMDD_HHMMSS.sql.gz`	`deploy/oxypc-backup.*`
Database (on demand)	Yes	Admin → "Run Backup Now" (`POST /admin/backup-now` , never prunes)	same folder	`routers/admin.py` l.711, commit `6b6bf96`
`uploads/` (invoices, payment photos, documents)	**No**	—	—	no job found
`.env` and server config	**No**	—	—	no job found
FieldOps database	**No** job in repo	—	—	`fieldops_db.py`
Off-server copy	**No** (recommended only)	—	—	`docs/backup-runbook.md` , `deploy/DEPLOY.md` §7

Retention: everything is kept unless `OXYPC_BACKUP_PRUNE=1` ; then files older than 30 days are deleted (`deploy/oxypc-backup.sh` , `scripts/backup_db.py`).

4.2 Recommended additions *(proposed, not yet built)*

1. Nightly copy of the newest dump **and** a tar of `uploads/` to a second machine or cloud (`rclone` example in `docs/backup-runbook.md`).
2. Weekly backup of `/opt/oxypc/.env` to a secure password store (not git).
3. A check that alerts if the newest backup is older than 26 h.

5. Restore

Always test in a separate database first (`docs/backup-runbook.md`). The dump is plain SQL **without** `--clean` , so it must be loaded into an **empty** database. Restoring on top of the live DB will fail or duplicate data, even though `scripts/restore.sh` says it drops tables.

5.1 Test restore (monthly, from `docs/backup-runbook.md` , adapted for production)

```
cd /opt/oxypc
sudo -u postgres createdb oxypc_test
gunzip -c backups/oxypc_YYYYMMDD_HHMMSS.sql.gz | sudo -u postgres psql -d oxypc_test
sudo -u postgres psql -d oxypc_test -c "SELECT COUNT(*) FROM devices;" -c "SELECT COUNT(*)
FROM sales;"
sudo -u postgres dropdb oxypc_test
```

Record the date, file used, row counts and time taken.

5.2 Production restore

1. Get go-ahead (commit-deploy-gate). Announce downtime to users.
2. `sudo systemctl stop oxypc-autodeploy.timer`
3. `docker compose stop oxypc-app`
4. Take a safety dump of the current DB: `sudo -u postgres pg_dump -Fc oxypc_db > /opt/oxypc/backups/pre_restore_$(date +%Y%m%d_%H%M).dump`
5. Rename the current DB instead of dropping it: `sudo -u postgres psql -c "ALTER DATABASE oxypc_db RENAME TO oxypc_db_old_$(date +%Y%m%d);"`
6. Create an empty DB owned by the app role: `sudo -u postgres createdb -O <app_db_user> oxypc_db` (role name is in `OXYPC_DATABASE_URL`).
7. Load the dump: `gunzip -c backups/<file>.sql.gz | sudo -u postgres psql -d oxypc_db`
8. `docker compose up -d` — startup runs `db_validator` and adds any tables/columns newer than the dump.
9. Optional: `docker compose exec oxypc-app alembic upgrade head` (per `docs/backup-runbook.md`).
10. Check `/health` , log in, open Devices and Sales lists, compare counts with step 7 of the test restore.
11. `sudo systemctl start oxypc-autodeploy.timer`
12. Keep `oxypc_db_old_*` until the business confirms, then drop it.

Uploaded files are not in the dump. If `uploads/` was lost, restore it separately from whatever copy exists (none is automated today).

6. Disaster recovery

6.1 Scenarios

Scenario	Impact	Recovery today	Gap
App crash / bad deploy	App down	Docker auto-restart; rollback per 2.2	No auto-rollback (deliberate)

Scenario	Impact	Recovery today	Gap
PostgreSQL corruption	Data loss since last dump	Restore last nightly dump (section 5)	Up to ~24 h of data lost
Server disk or hardware failure	App, DB **and backups** lost together	Rebuild a Linux box per `deploy/DEPLOY.md` + Docker files; restore DB from an off-server copy	**No off-server copy exists**, so full loss is possible
Ransomware / accidental mass delete	Data damaged	Restore an older dump (all are kept by default)	Backups sit on the same server and could be hit too
Office network down	Users cannot reach the LAN app	None	No remote access path documented

6.2 Rebuild on new hardware (outline)

1. Install Ubuntu, PostgreSQL 17, nginx, Docker Engine + Compose (`deploy/DEPLOY.md` §2-3; versions from docker spec §9 and `Dockerfile`).
2. Create DB and role; restore the latest dump (section 5.2 steps 6-7).
3. Clone the repo to `/opt/oxypc` with the read-only deploy key; restore `.env` .
4. `mkdir uploads backups static/stress_reports && chown -R 1000:1000 …` ; restore `uploads/` if a copy exists.
5. `docker compose up -d --build` ; install nginx vhost `deploy/nginx-oxypc-internal.conf` .
6. Install timers: `oxypc-backup.*` , `oxypc-autodeploy.*` (headers of each file list the commands).
7. Point users at the new IP (or move `10.199.206.109` to the new box).

6.3 RPO and RTO

Measure	Defined?	What the setup implies today
RPO (max data loss)	**No**	Up to 24 h (one nightly dump); total loss if the server disk dies
RTO (max downtime)	**No**	Minutes for app faults; unknown (likely a day or more) for hardware loss, as no standby or spare exists

These must be agreed with the business. See Open questions.

7. Contacts

Role	Name	Phone / email	When to call
Business owner / Director	Pankaj Sehgal (`CLAUDE.md`)	*TBD*	Approval for restarts, restores, data fixes
App development team	*TBD*	*TBD*	Code faults, deploys, rollbacks
Server / IT admin (Internal Server)	*TBD*	*TBD*	Hardware, network, OS, SSH, disk
Database admin	*TBD*	*TBD*	Restores, performance
Internet / network vendor	*TBD*	*TBD*	LAN or WAN outage

Role	Name	Phone / email	When to call
Escalation if nobody answers	*TBD*	*TBD*	

Open questions

1. What RPO and RTO does the business accept? Is a 24-hour data-loss window acceptable?
2. Where should the off-server backup copy go (NAS, Google Drive, S3), and who owns it?
3. Does the host still have a working `/opt/oxypc/venv` for `oxypc-backup.sh` and for the bare-metal rollback? The docker spec says nightly backups run "as root", while `deploy/oxypc-backup.service` says `User=www-data`.
4. Is there a monitoring tool watching `/health` from outside the server (the docstring mentions Uptime Robot/Grafana)?
5. Who has SSH access to `10.199.206.109` and sudo rights? Who holds the GitHub deploy key?
6. Is the FieldOps database backed up, and by whom?
7. Has a full test restore ever been done? When, and how long did it take?
8. What is the plan if the office LAN or the server room loses power for a day?

PART D

UAT Plan & Sign-off

Purpose: one repeatable User Acceptance Test (UAT) cycle per release, run on **staging**, signed by a business owner for each of the 12 personas. This plan reuses the existing test assets. It does not replace them.

1. Existing assets this plan reuses

Asset	What it holds	How this plan uses it
`OxyPC_UAT_Scenarios_v1.0.docx` (Mar 2026)	18 modules, IDs `UAT-AUTH-001` … `UAT-E2E-003`, plus a UAT Sign-Off Sheet	Source of step-level scenarios. Section 5 maps each persona to these IDs.
`OxyPC_UAT_Test_Cases.xlsx`	~80 test cases in 16 module sheets (`TC-AUTH-001` , `TC-LOT-001` , `TC-IQC-001` , `TC-L1-001` , `TC-QC-001` , `TC-COS-001` , `TC-SALE-001` , `TC-RET-001` , `TC-PARTS-001` , `TC-RAM-001` , `TC-BULK-001` , `TC-MASTER-001` , `TC-ADMIN-001` , `TC-RPT-001` , `TC-SEC-001` …), Execution Tracker, Defect Log, Test Data	Execution tracker for each cycle. Copy per release, fill "Executed By / Date / Pass-Fail / Defect Ref".
`docs/OxyPC_UAT_TestPlan.xlsx` (27 Apr 2026)	Phase P1–P3 route checks, Security Tests, Showstoppers, All Auto Results (48 auto tests, generated by `run_uat_excel.py`)	Automated smoke on staging before manual UAT. Showstoppers re-checked (Roadmap N5).
QA Dashboard `/qa` (`routers/qa_uat.py` , `models/qa_uat.py`)	Requirements, Test Cases, Executions, Defects, UAT Scenarios (with `sign_off_by` / `sign_off_at`), Releases, RTM	System of record. Every defect found in UAT is logged at `/qa/defects` ; every scenario result at `/qa/uat` ; the release moves `In UAT → UAT Done → Approved` at `/qa/releases` .
`seed_uat_users.py`	One UAT login per built-in role (`uat_admin` , `uat_invmgr` , `uat_iqc` , `uat_l1` , `uat_l2` , `uat_l3` , `uat_qc` , `uat_sales` , `uat_spares`)	Test accounts. Add custom-role accounts (Production Manager, Cosmetic Lead, Final QC, Accounts, TRC Manager) before the cycle.
`e2e_uat_test.py` , `tests/` (173 test files, `pytest.ini`)	Automated workflow and unit tests	Must pass before UAT entry.

2. Scope

In scope	Out of scope
Device lifecycle: Lot → GRN → IQC → Stock In → Production → L1/L2, L3/L4 → Stress → Cosmetic (6 stages) → Final QC → Ready to Sale → Sold → Return / Credit Note / Replacement	CRM sourcing, telecalling, dealer calling (own UAT)
Parts: request, handover, verify, Not In Stock, Procure, PNA, Parts GRN (As New), Harvest, warranty part cycle	Reliance FieldOps (`/fieldops`) — own 147 checks

In scope	Out of scope
Sales: Ready to Sale pricing, block tags, sale, invoice print, delivery challan, gate pass, transport update, returns, credit note, extended warranty	Trade Partner portal
Accounts: customer receipts, supplier payments, sales list, daily stock	WhatsApp service
Management: Overall Dashboard, Business P&L, Lot P&L, Daily Stock, Stock Aging, Receivables, Overdue, Physical Audits, Tickets	Performance / load testing (separate)
Permissions: each persona sees only its pages; negative tests from `UAT-RBAC-001…006`	
Any PRD in the release (PRD-01…04) — scenarios added from its acceptance criteria	

3. Environments

Environment	Purpose	Data	Who deploys	Notes
Local / Dev	Developer checks (`verify-feature` , `regression-sweep` skills)	Seed data (`seed_dummy_data.py`)	Developer	Not used for sign-off
Staging	**UAT and sign-off**	Copy of production restored from the nightly backup (`deploy/oxypc-backup.sh`), customer names / phones / GSTIN masked	App Development Team	**To be provisioned** (Roadmap N1). Today `deploy/oxypc-autodeploy.sh` deploys `origin/main` straight to production; there is no staging host in the repo. Use the `staging` branch; record "Staging" in the QA Dashboard environment field.
Production	Live use	Real	Auto-deploy after approval	Deploy only after Exit criteria are met and release status is `Approved`

Staging set-up checklist:

#	Item	Done
1	Separate DB and `.env` (no production secrets)	☐
2	`db_validator.py` passes on start-up	☐
3	UAT users created (`seed_uat_users.py` + custom-role users)	☐
4	Diagnose / OxyQC agent points at staging, not production	☐
5	Outbound WhatsApp / email / partner webhooks disabled	☐
6	Banner or different sidebar colour so testers know it is staging	☐

4. Entry and exit criteria

Entry (all must be true before testers start)

#	Criterion	Evidence
E1	Release built from the `staging` branch and deployed to staging	Release row at `/qa/releases` with status `In UAT`
E2	`pytest` green on the release commit	CI / console log attached to the release
E3	Full template parse: 0 errors (`regression-sweep` Step 1)	Log
E4	Route scan at baseline (`regression-sweep` Step 2: exactly 9 known flagged routes). Note: the skill calls `_scan_routes.py`, which is not in this repo checkout — locate it before the cycle.	Log
E5	`run_uat_excel.py` auto-smoke run on staging; no new failures vs last run	`OxyPC_UAT_TestPlan.xlsx` copy
E6	No open **Critical** defects for the release at `/qa/defects`	Filter: severity = Critical, status ≠ Closed
E7	Permission matrix reviewed for each persona's role (custom roles included)	Screenshot of `/admin/master/permissions` per role
E8	Release notes list every change and the personas affected	Release description

Exit (all must be true to sign off)

#	Criterion
X1	100% of in-scope scenarios executed; ≥ 95% Pass
X2	0 open Critical and 0 open High defects
X3	Medium / Low defects open only with an agreed fix date and owner
X4	End-to-end journey E2E-01 passed with the same tag from Lot to Credit Note
X5	Dashboard and report figures reconcile for the test tags (see 05)
X6	Sign-off form (Section 9) signed by every persona owner in scope
X7	Release status moved to `UAT Done`, then `Approved` by Managers/Admin

5. Roles and sign-off owners

#	Persona	UAT login (built-in or to create)	Sign-off owner (business)	Existing scenario IDs
1	IQC Inspector	`uat_iqc`	IQC lead	UAT-IQC-001...005, TC-IQC-*
2	Inventory Manager	`uat_invmgr`	Inventory Manager	UAT-LOT-001...004, UAT-STK-001...002, UAT-TRF-001...004, UAT-DEV-001...005
3	Production Manager	create custom-role user	Production Manager	(new — Section 6.3)

#	Persona	UAT login (built-in or to create)	Sign-off owner (business)	Existing scenario IDs
4	Repair Engineer L1/L2, L3/L4	`uat_l1` , `uat_l3`	Repair supervisor	UAT-L1R-001…006, UAT-L3R-001…004 (UAT-L2R-* legacy)
5	QC Inspector (Stress, OS install, diagnose)	`uat_qc`	QC lead	UAT-QC-001…004, TC-QC-*
6	Cosmetic Lead	create `cosmetic_manager` user	Cosmetic Lead	UAT-COS-001…006, TC-COS-*
7	Final QC Inspector	user with `cosmetic_finalqc` edit	Final QC lead	UAT-FQC-001…003
8	Spare Parts Manager	`uat_spares`	Spare Parts Manager	UAT-SPR-001…004, TC-PARTS-, *TC-RAM-*
9	Sales Executive	`uat_sales` + a `sales_manager` user	Sales lead	UAT-SAL-001…004, TC-SALE-, *TC-RET-*
10	Accounts	create custom-role user	Accounts head	(new — Section 6.10)
11	Managers / Admin	`uat_admin`	Director / Admin	UAT-RPT-001…003, UAT-ADM-001…004, UAT-RBAC-001…006
12	TRC Manager	create `trc_manager` user	TRC Manager	(new — Section 6.12)

6. Test scenarios

Format: **ID · Scenario · Expected result**. Steps for existing IDs live in `OxyPC_UAT_Scenarios_v1.0.docx`. New scenarios below are written from the current code; add them to `/qa/uat` before the cycle.

6.0 End-to-end device journey (run first, one tag, all personas)

ID	Step	Persona	Expected result (from code)
E2E-01.1	Create lot with supplier, qty, buying price, e-way bill	Inventory Manager	Lot number generated (MAX-based, collision-safe); `lots.e_way_bill` saved
E2E-01.2	GRN the lot; map GRN post-IQC	Inventory Manager	Tag counted in pipeline "GRN" (has `grn_number`)
E2E-01.3	IQC the tag, mark a part Required	IQC Inspector	Tag at `iqc` ; spec saved; dashboard IQC +1
E2E-01.4	Move to Stock In, then to Production	Inventory Manager	Stage `stock_in` → `trc_production` ; urgent flag cleared on entering Stock In
E2E-01.5	Allocate to an L1/L2 engineer	Production Manager	WorkID created; tag at `l1` ; engineer sees it in "assigned to me"
E2E-01.6	Start repair; raise part request (RAM 8GB)	Repair Engineer	`part_requests.status = requested`
E2E-01.7	Mark a second part PNA	Repair Engineer	`device_pna_parts` active row; Production PNA tile +1; L1/L2 Mark Complete blocked while PNA open

ID	Step	Persona	Expected result (from code)
E2E-01.8	Hand over the part	Spare Parts Manager	status `handed_over` ; Device Detail shows Verify button
E2E-01.9	Verify part; clear PNA; Complete to Stress	Repair Engineer	status `received` ("Part Changed"); tag at `qc_check`
E2E-01.10	Stress test **Fail** with checklist items	QC Inspector	Tag back at `l1` , assigned engineer notified; `stress_notes` filled
E2E-01.11	Fix and Complete to Stress again; Stress **Pass** (complete-to-paint)	Repair Eng., QC Inspector	Tag enters cosmetic chain
E2E-01.12	Cleaning → Putty → Dry Sanding → Masking → Painting → Water Sanding → Cosmetic Completed	Cosmetic Lead	One `StageMovement` per step; WorkIDs assigned on move
E2E-01.13	Final QC **Fail — Cosmetic**, then Move	Final QC Inspector	Tag to `final_qc_fail_hold` , then routed to Cosmetic Received by `FAIL_REASON_TO_DEPT`
E2E-01.14	Redo cosmetic; Final QC **Pass**, grade B	Final QC Inspector	Tag at `final_qc_pass_hold` ; `device_price` finalised once (base + parts + changed parts); grade saved
E2E-01.15	Move to Inventory (Ready to Sale)	Final QC Inspector / Inventory Manager	Tag at `ready_to_sale` ; dashboard Ready +1
E2E-01.16	Set min/max price; block tag for a customer	Sales Executive	Audit `TAG_PRICE_SET` ; `is_blocked = true` ; sale by other users refused
E2E-01.17	Sell from Block Tags with invoice number, warranty 6 months	Sales Executive	Sale row; tag `sold` ; company snapshot on sale; warranty expiry computed
E2E-01.18	Print invoice and delivery challan; update transport; gate-pass exit	Sales Executive	`/invoices/print/{id}` shows GST split intra/inter-state, HSN 8471; `/invoices/waybill/{id}` titled "Delivery Challan"
E2E-01.19	Record customer receipt	Accounts	Receipt at `/accounts/customer-receipts` ; sales list shows it
E2E-01.20	Process a return for credit note; approve	Sales Executive, Managers	`tag_return_status = 'Return for Credit Note'` ; Lot P&L revenue net of return
E2E-01.21	Check numbers	Managers/Admin, TRC Manager	Dashboard, Business P&L, Lot P&L, Daily Stock all reflect the tag consistently

Variants: **E2E-02** desktop and **E2E-03** TFT (forms differ — see `docs/OXYPC - Desktop-TFT - Fields.xlsx`); **E2E-04** L3/L4 path with Replacement Scrap.

6.1 IQC Inspector

ID	Scenario	Expected result
IQC-01	Register tag via OxyQC / diagnose agent	Specs auto-filled; tag at `iqc`
IQC-02	Duplicate tag or wrong lot	Live duplicate check blocks save
IQC-03	Bulk IQC upload with bad rows	Count alerts: inserted vs errors, exact tag list
IQC-04	Negative: open Sales page	Access denied / not in sidebar

6.2 Inventory Manager

ID	Scenario	Expected result
INV-01	Create, edit, trash lot	`is_trashed` set; lot gone from counts
INV-02	Stage transfer of a batch	Only `AllowedTransition` moves allowed; rejected moves listed
INV-03	Flag items urgent, then move to Stock In	Flag cleared automatically
INV-04	Change Floor / As-Is Lot sub-lot	`sub_lot_number` written to every selected tag

6.3 Production Manager

ID	Scenario	Expected result
PM-01	Summary tiles match their CSV	Same tags (shared `_prod_tile_criteria`)
PM-02	Allocate tags / buckets to engineers	WorkIDs created, notifications sent
PM-03	PNA summary CSV	Lists L1/L2 and L3/L4 tags with active PNA parts, marked by / at
PM-04	Hardware Final QC fail with no engineer	Tag parked in Production (`trc_production`)

6.4 Repair Engineer (L1/L2, L3/L4)

ID	Scenario	Expected result
REP-01	Start, pick, complete L1/L2 job	WorkID status changes; Mark Complete blocked by open PNA
REP-02	Request L3/L4 with repair note	Tag to `l3`; note shown as Repair Notes
REP-03	Multi / bulk part request	One request per tag; duplicate open request refused
REP-04	Faulty changed part	New request raised, first change kept in history
REP-05	L3/L4 Normal Scrap / Replacement Scrap	Tag scrapped or replaced; `replaced` text set

6.5 QC Inspector (Stress Test, OS install, diagnose)

ID	Scenario	Expected result
QC-01	Run stress via agent; save results	Report at `/stress/{barcode}/report`
QC-02	Fail with checklist items + note	Tag to `l1`, engineer chosen, notification; Stress Notes shown on L1/L2
QC-03	Complete to paint	Tag enters cosmetic chain
QC-04	Negative: fail to a non-L1/L2 user	Refused: "Selected user is not an active L1/L2 engineer"

6.6 Cosmetic Lead

ID	Scenario	Expected result
COS-01	Move one tag through all six stages	Each stage logs `StageMovement`
COS-02	Bulk Assign at Cosmetic Received	Selected tags advance to Cleaning and are assigned
COS-03	Bulk Assign at Painting	Pure reassignment, stage unchanged
COS-04	Fail at Cleaning / Water Sanding / Completed	Tag to `l1`, Repair Notes = " Failed — assigned to ..."

ID	Scenario	Expected result
COS-05	Skip Cosmetic → Final QC	Allowed from any cosmetic stage

6.7 Final QC Inspector

ID	Scenario	Expected result
FQC-01	Pass with grade, touch type, bucket name	Pass Hold; price finalised once
FQC-02	Fail Hardware / Software / Cosmetic, then Bulk Move	Routed to L1/L2 engineer / Stress / Cosmetic Received; unrecognised reason skipped and reported
FQC-03	Pass the same tag a second time after rework	Price **not** added again
FQC-04	Required-field validation on fail	Form refuses without reason

6.8 Spare Parts Manager

ID	Scenario	Expected result
SPM-01	Handover / Not In Stock / Procure	Status changes; Procure creates open sourcing request
SPM-02	Sourcing closed by Sales Manager then Verify	Stock credited once; request back to `requested`
SPM-03	Parts GRN (As New) with e-way bill file	Line items added; stock up
SPM-04	Add Harvest part	`source = harvest` ; Harvest count up
SPM-05	Warranty send / receive	`warranty_sent` → `warranty_replaced` , restocked

6.9 Sales Executive

ID	Scenario	Expected result
SAL-01	Ready to Sale table: auto min price	Unit price + other rate + margin (`services/ready_pricing.py`)
SAL-02	Block tags, release, sell from block	Blocked tags unsellable except from their own block
SAL-03	Multi-tag sale with price groups	One Sale row per tag, shared invoice number
SAL-04	Below-cost sale	Warning shown, not blocked
SAL-05	Mark sold via generic stage move	Not possible (`sold` excluded from `DROPDOWN_STAGES`)
SAL-06	Delivery challan and gate-pass exit (single and bulk)	Gate pass records exit
SAL-07	Returns: return for repair, paid repair, replacement, credit note	Correct `tag_return_status` ; credit note flow advances; approval required
SAL-08	Rentals	**Not found in code — mark Blocked and raise question**

6.10 Accounts

ID	Scenario	Expected result
ACC-01	Customer receipt (single and bulk upload)	Receipts saved; template download works
ACC-02	Supplier payment against PO; upload invoice	Payment complete; file stored
ACC-03	Invoice print for an inter-state sale	IGST only; intra-state shows CGST + SGST
ACC-04	Sales list export with filters	CSV matches screen
ACC-05	Daily Stock for yesterday; download saved Daily Report	Opening + movements = Closing; files present from 9 PM run

6.11 Managers / Admin

ID	Scenario	Expected result
MGR-01	Dashboard tiles vs drill-down pages	Each pipeline tile equals its `/devices?stage=…` count (except L3/L4, see 05 D5)
MGR-02	Financial Summary vs Business P&L for the same FY	Equal
MGR-03	Business P&L override for a month	FY totals and dashboard change accordingly; audit logged
MGR-04	User tickets raise / close	Ticket status Open → Closed
MGR-05	RBAC negative tests UAT-RBAC-001…006	Refused with 403 / hidden

6.12 TRC Manager

ID	Scenario	Expected result
TRC-01	Extended warranty: set and update	New expiry shown; audit logged
TRC-02	Physical audit: look up tags, submit, export stage / location / tag summaries	Record saved; CSVs match
TRC-03	Daily Stock with "Exclude Admin User"	Admin / test-user moves excluded
TRC-04	Oversight across Production, Part Master, WorkID Status	All pages open for `trc_manager` via matrix

7. Defect severity definitions

Values match `models/qa_uat.py::DefectSeverity` and `DefectPriority`.

Severity	Definition	Examples	Default priority	Fix before sign-off?
Critical	A core flow is blocked or data is wrong / lost / exposed, no workaround	Tag cannot move to Ready; sale saved with wrong price; P&L visible to engineers; 500 on a main page	P1 – Immediate	Yes

Severity	Definition	Examples	Default priority	Fix before sign-off?
High	A core flow works only with a hard workaround, or a number shown to managers is wrong	Fail reason lost; dashboard tile ≠ report; bulk action silently skips tags	P2 – High	Yes
Medium	A secondary feature fails or a core one has an easy workaround	Export column wrong; filter ignored; wrong label	P3 – Medium	No, if dated and owned
Low	Cosmetic or wording issue; no effect on data	Alignment, colour not `#0F7A37`, typo	P4 – Low	No

Status flow in the QA Dashboard: `New → Assigned → In Progress → Fixed → Retest → Closed` (or `Reopened`, `Won't Fix`).

8. Regression checklist (every release, before and after UAT fixes)

#	Check	How	Pass rule
1	Unit + workflow tests	`pytest` (`pytest.ini`, `tests/`)	All green
2	Template parse	`regression-sweep` Step 1 (now 242 templates)	0 errors
3	Route scan	`regression-sweep` Step 2	Exactly the 9 known flagged routes
4	Schema check	`python db_validator.py`	No drift
5	Auto smoke	`run_uat_excel.py` on staging	No new failures
6	Stage rules	Try one disallowed move per stage group	Refused by `validate_transition`
7	Sell guard	Try to sell a blocked tag and a non-ready tag	Refused by `validate_sale_allowed`
8	Dashboard reconcile	MGR-01, MGR-02	Equal
9	Daily Stock	ACC-05	Reconciles
10	RBAC	UAT-RBAC-001...006 + one custom role (e.g. `trc_manager`)	Correct allow / deny
11	CSRF	POST without token	403 (re-check Showstopper SEC-01)
12	Global Table	Pagination, search, sticky header on Devices, Ready to Sale, Sales	Matches Design System §07
13	Exports	Devices, Sales, Lot P&L, Daily Stock CSV / XLSX	Columns align with headers
14	Audit log	Price set, sale, return, part handover produce rows at `/admin/audit-log`	Rows present

9. Sign-off form (copy one per persona per release)

```
OxyPC Inventory Lifecycle — UAT Sign-off
-----------------------------------------------------------------
Release / version (QA Dashboard):  v_____   Build commit: ________
Environment: Staging   URL: _____________________   Dates: __/__/____ to __/__/____
Persona: _____________________   App role / login used: _____________

Scenarios in scope (IDs): ____________________________________________
Executed: ____   Passed: ____   Failed: ____   Blocked: ____

Open defects (ID / severity / agreed fix date / owner):
  1. ____________________________________________________________
  2. ____________________________________________________________

Conditions or notes: ____________________________________________________

Decision:  [ ] Accept   [ ] Accept with conditions   [ ] Reject

Business owner: _____________________  Signature: ___________  Date: __/__/____
QA lead:        _____________________  Signature: ___________  Date: __/__/____
App Dev lead:   _____________________  Signature: ___________  Date: __/__/____
-----------------------------------------------------------------
Record in QA Dashboard: /qa/uat → scenario → sign_off_by / sign_off_at
```

Release approval (once all persona forms are in):

Persona	Decision	Signed by	Date
IQC Inspector			
Inventory Manager			
Production Manager			
Repair Engineer			
QC Inspector			
Cosmetic Lead			
Final QC Inspector			
Spare Parts Manager			
Sales Executive			
Accounts			
Managers / Admin			
TRC Manager			
Release approved for production (Managers/Admin)			

10. Cycle timeline (per release)

Day	Activity
D-2	Deploy to staging; entry checks E1–E8
D1	E2E-01 walkthrough with all personas together
D2–D4	Persona scenarios; defects logged at `/qa/defects`
D5	Fixes deployed to staging; retest; regression checklist
D6	Sign-off forms; release status → `UAT Done` → `Approved`
D7	Production deploy (`commit-deploy-gate` skill); smoke on production

11. Open questions

#	Question	Owner
1	Who hosts staging and who restores the masked copy each cycle?	Admin
2	Where is `_scan_routes.py` ? Referenced by the `regression-sweep` skill but not in this checkout.	App Development Team
3	Rentals (SAL-08): in scope or not?	Sales lead



OXYPC INVENTORY LIFECYCLE

PRODUCT

Product Requirements (PRDs)

Requirements for the top improvement opportunities, plus the template for new ones.

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

App Development Team · Bluemonk

Product Requirements (PRDs)

Last updated: 25 Oct 2026 · Source: staging branch @ 55ce58a

In this document: Part A — PRD-01 · Part Request & PNA Status · Part B — PRD-02 · Fail Reasons & Rework Routing · Part C — PRD-03 · Grading-Driven Pricing · Part D — PRD-04 · Invoice & E-way Bill from Sale · Part E — PRD Template

PART A

PRD-01 · Part Request & PNA Status

Field	Value
Status	Draft
Roadmap item	N2 (Now, Q4 2026) — 02_Roadmap.md
Business owner	Spare Parts Manager (with Repair supervisor)
Personas	Repair Engineer (L1/L2, L3/L4), Spare Parts Manager, Production Manager, TRC Manager, Sales Manager (closes sourcing)
Related	Proposed KPIs P3 PNA wait time, P4 Parts fill rate (05); Roadmap X3 Harvested parts first

1. Problem

An engineer who needs a part raises a request and then has to walk to Stores or open each Device Detail page to learn what happened to it. If the part cannot be found, the engineer marks it "PNA" (Part Not Available) in a different place, and nothing ties that mark to the request. Stores cannot see how long tags have been waiting, and no one is told when a part is handed over, sourced or declared not in stock. Tags sit in L1/L2 with an open PNA and cannot be completed (`routers/repair.py::l1l2_complete_to_stress` blocks Mark Complete while PNA is open), but the reason and age are hard to see in one place.

2. Personas

Persona	Need
Repair Engineer	See every request's status on my queue; be told when my part is ready
Spare Parts Manager	One list of open requests and PNA tags, oldest first, with what is blocking each
Production Manager	Know which tags are stalled on parts, and for how long
TRC Manager	PNA count and age across the floor
Sales Manager	Know which sourcing requests are urgent because tags are waiting

3. Goals and non-goals

Goals	Non-goals
One visible status per part request, on the engineer's queue and Device Detail	Changing how sourcing deals are closed in the CRM
PNA linked to the request it came from (when there is one)	Automatic purchase orders
Notify the engineer on handover / not in stock / sourced	Changing stock-ledger arithmetic
Keep every step's time so wait time and fill rate can be measured	Harvest-first suggestion (Roadmap X3, separate PRD)
Keep PNA history across repeated mark / clear cycles	

4. Current behaviour (from code)

Area	What happens today	Source
Raise request	From Device Detail Parts Consumption (single or multi), L1/L2 Bulk Part Request, L3/L4 Request Parts modal. Duplicate open request (requested / handed_over / procure) for same part is refused in bulk.	`routers/part_requests.py::create_part_request`, `create_part_requests_batch`; `routers/repair.py::bulk_part_request`
Request statuses	`requested`, `handed_over`, `received` ("Part Changed"), `not_in_stock`, `procure` ("Sent for Sourcing"), `scrapped`, `warranty_sent`, `warranty_replaced`, `cancelled`	`models/part_request.py::PartRequest`, `routers/part_requests.py`
Timestamps	One `actioned_at` / `actioned_by` pair, overwritten by every step (handover, verify, not in stock ...). Step times are lost.	`routers/part_requests.py::handover_part`, `validate_receiving`, `not_in_stock`
Spare Parts actions	Handover (qty = requested), Not In Stock, Procure (creates `PartSourcingRequest` `open`), Scrap, Warranty send / receive, Bulk action	`routers/part_requests.py`
Sourcing loop	Sales Manager closes the deal in CRM; Spare Parts Manager verifies → stock credited via ledger; originating request goes back to `requested` (shown as "Available")	`close_sourcing`, `verify_sourcing`
Engineer visibility	Status badges only on Device Detail Parts Consumption	`templates/devices/detail.html` (badges for `not_in_stock`, `procure`, `received`)
Notifications	None in the part-request flow	`routers/part_requests.py` (no `create_notification` call)
PNA mark	Any signed-in user can mark / unmark a part PNA for a tag; one row per (tag, part name); unmark sets `is_active = false`; re-mark reopens the same row, so only the latest cycle is kept; "Remove All PNA" clears all	`models/pna_part.py::DevicePNAPart`, `routers/part_requests.py::mark_part_pna`, `clear_all_pna`
PNA ↔ request	No link. `not_in_stock` does not create PNA; PNA does not change the request.	both models
PNA effect	Blocks L1/L2 Mark Complete; counted on Production "PNA" tile (L1/L2 only); PNA summary CSV (L1/L2/L3); PNA column on WorkID Status; `pna_map` on L3/L4	`routers/repair.py`, `routers/stock.py::trc_production_list`, `trc_production_pna_summary`, `routers/workid_status.py`, `services/pna_lookup.py`

Area	What happens today	Source
Audit	Every step audited (`PART_HANDOVER` , `PART_RECEIVED` , `PART_NOT_IN_STOCK` , `PART_PROCURE` , `PART_PNA_MARKED` , …)	`routers/part_requests.py`

5. User stories and acceptance criteria

US-01 — See part status on my queue

As a Repair Engineer, I want a Parts column on my L1/L2 and L3/L4 queue, so that I know which tags I can work on now.

#	Given	When	Then
AC-01.1	A tag on my L1/L2 queue has 2 open requests (one `handed_over` , one `procure`)	I open `/repair/l1`	The Parts column shows "1 ready to verify · 1 sourcing" with coloured pills
AC-01.2	A tag has an active PNA part	I open my queue	The Parts column shows a red "PNA · n days" pill; hovering lists part names, who marked, when
AC-01.3	A tag has no requests and no PNA	I open my queue	The Parts column shows "—"
AC-01.4	I filter the queue by "Waiting on parts"	I apply the filter	Only tags with an open request or active PNA are listed, and the count matches the Production PNA tile for L1/L2

US-02 — Be told when my part moves

As a Repair Engineer, I want a notification when my part is handed over, declared not in stock or becomes available after sourcing, so that I do not have to chase Stores.

#	Given	When	Then
AC-02.1	I raised a request	Spare Parts Manager clicks Handover	I get an in-app notification "RAM 8GB handed over for — verify on Device Detail" linking to the tag
AC-02.2	I raised a request	Spare Parts Manager clicks Not In Stock	I get a notification, and the request shows "Not In Stock" on my queue
AC-02.3	The request was sent for sourcing	Spare Parts Manager verifies the sourcing	I get "Part now available" and the request shows "Available"

US-03 — Not In Stock creates PNA

As a Spare Parts Manager, I want "Not In Stock" to mark the part PNA on the tag (linked to the request), so that the tag is visibly blocked without a second step.

#	Given	When	Then
AC-03.1	An open request for "RAM" on tag T	I click Not In Stock	A PNA row for (T, "RAM") is active with `source = not_in_stock` and `part_request_id` set; audit `PART_PNA_MARKED` written
AC-03.2	T already has an active PNA for "RAM" (marked by the engineer)	I click Not In Stock	The existing PNA row is linked to the request; no duplicate

#	Given	When	Then
AC-03.3	A linked PNA exists	The part is later handed over	The PNA is cleared automatically (`cleared_by` = Spare Parts Manager) and the history keeps both events

US-04 — One list of what is blocked

As a Spare Parts Manager, I want a "Blocked tags" tab on Part Master listing every active PNA and every open request older than 1 working day, oldest first, so that I work the longest waits first.

#	Given	When	Then
AC-04.1	There are active PNA rows and open requests	I open `/spare-parts` → Blocked tags	A Global Table lists Tag, Stage, Part, Status, Linked request, Sourcing status, Age (days), Engineer, sorted by Age desc
AC-04.2	I tick rows and choose Handover	I submit	Same rules as Bulk action; result alert lists moved and skipped tags

US-05 — Keep the whole PNA story

As a TRC Manager, I want every mark and clear of a PNA kept, so that I can explain a delay even after the part was re-marked.

#	Given	When	Then
AC-05.1	Part was marked, cleared, marked again	I open Device Detail → PNA history	Both cycles are listed with who / when
AC-05.2	The WorkID Status PNA history column	I export	Both cycles are in the export

6. Functional requirements

ID	Requirement	Priority
FR-01	Record a timestamp and user for each step of a part request (requested, handed over, received, not in stock, procure, available, scrapped, cancelled, warranty). Do not overwrite earlier steps.	Must
FR-02	Show a derived **Part status** per request with these labels: Requested · Handed over (verify) · Changed · Not in stock · Sourcing · Available · PNA · Scrapped · Cancelled · Warranty.	Must
FR-03	Add a Parts column and a "Waiting on parts" filter to `/repair/l1` and `/repair/l3l4`.	Must
FR-04	Send an in-app notification (`services/notifications.py::create_notification`) to the requesting engineer on Handover, Not In Stock and Sourcing Verified.	Must
FR-05	Not In Stock creates or links an active PNA row for the same tag and part; Handover or Verify of a linked request clears it.	Must
FR-06	Keep every PNA mark / clear cycle as history (no overwrite on re-mark).	Must
FR-07	Add "Blocked tags" tab on Part Master (Global Table, oldest first, CSV export).	Should
FR-08	Show PNA age (working days) wherever PNA is shown (queues, Production tile tooltip, WorkID Status).	Should

ID	Requirement	Priority
FR-09	Include L3/L4 tags in the Production "PNA" tile (today L1/L2 only) or rename the tile "PNA (L1/L2)".	Should
FR-10	Daily digest notification to Spare Parts Manager: count of requests open > 1 working day and PNA > 5 working days.	Could

7. Data and permission changes (Proposed)

Schema (dbdiagram approval required; additive):

Table	Change	Notes
`part_request_events` (new)	`id`, `part_request_id` FK, `from_status`, `to_status`, `by_username`, `at`, `notes`	Append-only step log. Backfill one row per existing request from `created_at` and current `actioned_at`.
`part_requests`	add `handed_over_at`, `received_at` (nullable)	Fast reads for P4 fill rate; also written to the events table
`device_pna_parts`	add `part_request_id` FK (nullable)	Link PNA ↔ request
`device_pna_events` (new)	`id`, `pna_part_id` FK, `action` (marked / cleared), `by_username`, `at`, `source`	Keeps every cycle; existing row stays the "current state"

Audit actions: reuse existing ones; add `PART_PNA_LINKED` (FR-05) and `PART_PNA_AUTO_CLEARED`.

Permissions: No new module. The Parts column is visible to anyone who can see the repair queue. "Blocked tags" tab follows the existing Part Master view permission. PNA mark stays open to any signed-in user (current rule).

8. UX notes

- Parts column pills: Handed over = `bg-primary`; Changed = `bg-primary` outline; Sourcing = `bg-info` `text-dark` (as today on Device Detail); Not in stock / PNA = `bg-danger`; Available = success `#0F7A37`.
- PNA pill text "PNA · 6d"; tooltip lists parts, marked by, marked at (same pattern as WorkID Status `pna_tip`).
- "Blocked tags" tab is a Global Table card with Sky Blue header; toolbar holds Bulk Handover / Not In Stock / Export.
- Bulk results use the Design System multi-line `alert-info` ("12 handed over, 2 skipped — already actioned: ...").
- Notifications use the existing bell; type `info` for handover / available, `warning` for not in stock.

9. Metrics

Metric	Baseline	Target	Source
PNA wait time (median, mark → clear)	baseline: to be measured	≤ 5 working days *(proposed)*	`device_pna_events`
Parts fill rate (handover ≤ 1 working day)	baseline: to be measured	≥ 85% *(proposed)*	`part_requests.handed_over_at − created_at`

Metric	Baseline	Target	Source
Tags blocked by PNA (count, daily)	baseline: to be measured	−30% in 3 months *(proposed)*	Production PNA tile
Engineer visits to Device Detail just to check part status	baseline: to be measured (access log)	−50% *(proposed)*	server log

10. Rollout

Phase	Scope	Gate
1	FR-01, FR-02, FR-03, FR-04, FR-06 + backfill	Staging UAT: REP-01…03, SPM-01…02 and new US-01/02/05 scenarios
2	FR-05 (Not In Stock ↔ PNA link)	Spare Parts Manager agrees auto-PNA rule
3	FR-07…FR-10	KPI baselines (Roadmap N6)

Setting: `AppSetting` key `pna_auto_from_not_in_stock` (default on) so FR-05 can be switched off. Manual page update in `utils/manual_content.py` (Part Master, L1/L2 Repair).

11. Open questions

#	Question	Owner
1	Should Not In Stock always create PNA, or only when no sourcing is planned?	Spare Parts Manager
2	Working-day calendar: which days and holidays count?	TRC Manager
3	Should Sales Manager get a notification when a sourcing request is blocking > N tags?	Sales Manager
4	Should engineers be allowed to cancel their own request? Today `cancelled` is set only via bulk action.	Repair supervisor

PART B

PRD-02 · Fail Reasons & Rework Routing

Field	Value
Status	Draft
Roadmap item	N3 phase 1 (Q4 2026) and X6 phase 2 (Q1 2027) — 02_Roadmap.md
Business owner	Final QC lead (with QC lead and Cosmetic Lead)
Personas	QC Inspector (Stress), Cosmetic Lead, Final QC Inspector, Repair Engineer, Production Manager, TRC Manager
Related	Proposed KPI P2 First-pass yield (05); `tests/test_final_qc_fail_assign.py`, `tests/test_cosmetic_fail_button.py`, `tests/test_failed_fqc_fail_note_tooltip.py`

1. Problem

A tag can fail at Stress Test, at four cosmetic steps and at Final QC. Each fail is written into a single "current value" field on the device, and the next event clears or overwrites it. When the tag reaches the next engineer, the earlier fail story is gone from the screen; it survives only as free text in the stage-movement log. Stress and Cosmetic fails always go to L1/L2, while Final QC routes by reason. Engineers repeat diagnosis, repeat fails are invisible, and first-pass yield cannot be measured cleanly.

2. Personas

Persona	Need
Repair Engineer	See every past fail on the tag (what, where, who, when) before I start
QC Inspector	Record a reason category, not only checklist items, so the tag goes to the right place
Cosmetic Lead	Fail a tag back for hardware or for rework inside the paint line
Final QC Inspector	See if this tag failed before and why; route failed tags in bulk
Production Manager / TRC Manager	Count rework loops per tag, per stage, per engineer

3. Goals and non-goals

Goals	Non-goals
Every fail is kept as a structured event on the tag	Changing the Stress Test checklist itself
The open (unresolved) fail is shown on the receiving queue	Changing who can pass / fail (permissions stay)

Goals	Non-goals
One reason-category list and one routing rule for Stress, Cosmetic and Final QC	Automatic engineer scoring or penalties
Rework count visible per tag	Replacing `StageMovement` as the movement log

4. Current behaviour (from code)

Area	What happens today	Source
Stress fail	Fail modal with failed / N/A checklist items + note; engineer must be L1/L2; tag → `l1`, new L1 WorkOrder, notification. `Device.stress_notes = "Failed: <items> — <note>"`. Checklist kept in `StressTestResult` (`results_json`).	`routers/stress_api.py::stress_fail_assign`, `_record_checklist_result`
Stress pass	`stress_notes` set to `None` on Complete to paint	`routers/stress_api.py::stress_complete_to_paint`
Cosmetic fail	Allowed at Cosmetic Received, Cleaning, Water Sanding, Cosmetic Completed; engineer must be L1/L2; tag → `l1`; `Device.repair_notes` **overwritten** with " Failed — assigned to . "	`routers/cosmetic.py::cosmetic_fail_assign`
Other `repair_notes` writes	Overwritten by Request L3/L4 and by L3/L4 Complete	`routers/repair.py::request_l3l4`, `l3l4_complete`
Final QC fail	Reason from Master Data `qc_failure_reason` + required Final Notes; tag → `final_qc_fail_hold`; `fqc_failure_reason`, `fqc_final_notes` set (single value); bucket gets the latest reason / engineer	`routers/cosmetic.py::advance_stage`, `templates/cosmetic/final_qc.html`
Final QC routing	"Move" / "Bulk Move" routes by `FAIL_REASON_TO_DEPT`: Hardware → L1/L2 (to last engineer, else parked in `trc_production`), Software → Stress Test, Cosmetic → Cosmetic Received. Any other Master Data reason is **skipped** ("No recognized Failure Reason").	`routers/cosmetic.py::fqc_move_failed`, `FAIL_REASON_TO_DEPT`, `_resolve_fail_engineer`
Final QC edit	Edit clears `final_qc_status`, `fqc_failure_reason`, `fqc_final_notes`, `fqc_pass_notes` and returns tag to `final_qc`	`routers/cosmetic.py::fqc_edit_revert`
History	Free text in `StageMovement.notes` ("Final QC Failed — ...", " Failed — ...")	`models/device.py::StageMovement`
Where shown	L1/L2 queue: Repair Notes and Stress Notes columns; Devices Failed table: failure reason, final notes; Device Detail notes	`templates/repair/*.html`, `templates/cosmetic/final_qc.html`, `templates/devices/detail.html`
Audit	Cosmetic fail audited (`COSMETIC_FAIL_ASSIGNED`); Stress / Final QC fail rely on StageMovement + WorkOrder	as above

5. User stories and acceptance criteria

US-01 — Fail history on the tag

As a Repair Engineer, I want a Fail history panel on Device Detail and an "open fail" line on my queue, so that I start from what already failed.

#	Given	When	Then
AC-01.1	Tag T failed Stress (items: Keyboard, Wi-Fi) and later Final QC (Hardware: "hinge loose")	I open Device Detail for T	Fail history lists both, newest first: stage, category, items, note, failed by, failed at, routed to, resolved at
AC-01.2	T's latest fail is open	I open `/repair/l1`	T shows "Open fail: Final QC · Hardware · hinge loose" and a "Rework #2" badge
AC-01.3	T passes Stress after the fix	The pass is saved	The Stress fail event gets `resolved_at` / `resolved_by`; Stress Notes column no longer shows it as open, but history keeps it

US-02 — Same reasons everywhere

As a QC Inspector or Cosmetic Lead, I want to pick a reason category (Hardware / Software / Cosmetic / Other) when I fail a tag, so that routing and reports work the same way as Final QC.

#	Given	When	Then
AC-02.1	Fail modal on Stress or Cosmetic	I open it	A required Reason category dropdown appears (Master Data `qc_failure_reason`), plus the existing checklist / note
AC-02.2	I submit without a category	—	The modal keeps my values and shows `alert alert-danger small`: "Choose a fail reason"

US-03 — Route by rule (phase 2)

As a Final QC Inspector, I want every failed tag to route by a rule I can see, so that nothing is skipped because its reason is not one of three hard-coded words.

#	Given	When	Then
AC-03.1	Admin maps each `qc_failure_reason` value to a destination in Master Data	I Bulk Move failed tags	Each tag goes to its mapped destination; the result alert lists moved and skipped tags with reasons
AC-03.2	A reason has no mapping	I Bulk Move	Tag is skipped and the alert says "No route for reason '' — ask Admin to map it"
AC-03.3	Stress fail with category Software	QC Inspector submits	Tag stays at Stress for OS reinstall (no L1 hand-off) *(proposed rule; confirm in Open questions)*

US-04 — See rework loops

As a Production Manager, I want rework count per tag and fail counts per stage and reason, so that I can fix root causes.

#	Given	When	Then
AC-04.1	Fail events exist	I open the Flow KPIs page (Roadmap X5)	First-pass yield per stage and top 5 reasons per stage are shown, matching the events table

6. Functional requirements

ID	Requirement	Priority
FR-01	Write one fail event for every fail at Stress, Cosmetic (all fail points) and Final QC, in the same transaction as the stage move.	Must
FR-02	Mark the open fail event resolved when the tag next passes the stage where it failed (Stress complete-to-paint; cosmetic advance past that stage; Final QC pass).	Must
FR-03	Show Fail history on Device Detail and an "Open fail" + "Rework #n" on L1/L2, L3/L4, Stress, Cosmetic and Final QC queues.	Must
FR-04	Keep `stress_notes`, `repair_notes`, `fqc_failure_reason` working as today (backward compatible) but stop relying on them as history.	Must
FR-05	Reason category required on Stress and Cosmetic fail (Master Data `qc_failure_reason`).	Should (phase 2)
FR-06	Routing map from reason to destination stage held in Master Data / AppSetting, used by Final QC Move and by Stress / Cosmetic fail; replaces hard-coded `FAIL_REASON_TO_DEPT`.	Should (phase 2)
FR-07	Final QC "Edit" (revert decision) marks the fail event `voided`, not deleted.	Must
FR-08	Backfill fail events from `StageMovement.notes` patterns ("Final QC Failed —", "Failed — assigned to", Stress moves to `l1`) and `StressTestResult` FAIL rows.	Should
FR-09	Audit `DEVICE_FAIL_RECORDED`, `DEVICE_FAIL_RESOLVED`, `DEVICE_FAIL_VOIDED`.	Must

7. Data and permission changes (Proposed)

Schema (dbdiagram approval required; additive):

Table	Columns	Notes
`device_fail_events` (new)	`id`, `device_id` FK, `barcode`, `stage` (DeviceStage at fail), `reason_category`, `failed_items` (JSON), `notes`, `routed_to_stage`, `routed_to_user_id`, `failed_by`, `failed_at`, `resolved_at`, `resolved_by`, `status` (open / resolved / voided), `rework_seq`, `stress_result_id` (nullable FK)	Index on (`device_id`, `status`)
Master Data	new category `fail_reason_route` (reason → destination)	Phase 2

Permissions: unchanged. Fail actions keep their gates (`cosmetic_finalqc` edit for Final QC; stage module edit for Cosmetic; Stress page permission). Fail history is read-only for anyone who can open Device Detail.

8. UX notes

- Device Detail: a "Fail history" card (Global Table style, Sky Blue header) under Stage History; open fail row highlighted with `table-warning`; resolved rows show a success `#0F7A37` tick.
- Queues: one compact line under the tag: red badge "Rework #2" + "Final QC · Hardware · hinge loose" (tooltip shows full note, same pattern as the existing Final QC fail-note tooltip).
- Fail modals: Reason category first, then checklist, then note; errors per Design System §09 (modal stays open, values kept).
- Bulk Move result: multi-line `alert-info` with moved and skipped tags in `font-monospace`.

9. Metrics

Metric	Baseline	Target	Source
First-pass yield at Final QC	baseline: to be measured (from StageMovement)	≥ 85% *(proposed)*	`device_fail_events`
First-pass yield at Stress	baseline: to be measured	to be agreed	`device_fail_events`
Tags with ≥ 2 rework loops	baseline: to be measured	−30% in 2 quarters *(proposed)*	`rework_seq`
Final QC fails skipped at Move for unrecognised reason	baseline: to be measured	0	Move results

10. Rollout

Phase	Scope	Gate
1 (Q4 2026)	FR-01…04, FR-07…09; backfill	UAT: QC-02, COS-04, FQC-02, FQC-03 + US-01
2 (Q1 2027)	FR-05, FR-06 routing map	Final QC lead approves the map; UAT US-02, US-03

11. Open questions

#	Question	Owner
1	Should a Stress "Software" fail stay in Stress (OS reinstall) instead of going to L1/L2?	QC lead
2	Should Cosmetic be able to fail back to an earlier cosmetic step (e.g. Painting → Putty) instead of only to L1/L2?	Cosmetic Lead
3	Which Master Data `qc_failure_reason` values exist in production today, beyond Hardware / Software / Cosmetic?	Admin
4	Do we count a Final QC "Edit" revert as a fail for yield? (Proposed: no — voided.)	Final QC lead

PART C

PRD-03 · Grading-Driven Pricing

Field	Value
Status	Draft
Roadmap item	X1 (Next, Q1 2027) — 02_Roadmap.md
Business owner	Sales lead (with Final QC lead and Director)
Personas	Final QC Inspector, Sales Executive, Managers/Admin, Inventory Manager
Related	BRD v6 objective 4 "Grade-driven pricing" (`docs/generate_brd_v6.js`); audit "Commercial 3.5/10" and P-4 `expected_sale_value` (`docs/audits/2026-04-29-claude-md-full-audit.md`); proposed KPI P5 Grade mix

1. Problem

Final QC gives every passed tag a grade, but the grade plays no part in its selling price. Ready to Sale computes a minimum price from cost plus a margin, so two identical laptops — one A grade, one C grade — get the same floor. Sales sets real prices by hand, one tag or one selection at a time. The CRM already holds a Grade Price Matrix, but nothing on the refurbishment side reads it. The BRD has promised grade-driven pricing since v6.

2. Personas

Persona	Need
Final QC Inspector	Know the grade I give changes the price, so grading is consistent
Sales Executive	See a suggested price per tag that reflects grade, and a floor I should not go below
Managers/Admin	Maintain one grade price list; see margin by grade
Inventory Manager	Know expected value of stock by grade

3. Goals and non-goals

Goals	Non-goals
A suggested min / max selling price per tag from grade + device type (+ optional make / generation)	Dynamic market pricing from external sites
Keep the cost-based floor; warn when the grade price is below it	Blocking a sale (stays a warning, as today)
Sales can override with a reason; every change audited	Changing how As-Is lots are priced (phase 2)
One grade scale used by Final QC, Sales and CRM	Re-grading historical sold tags

4. Current behaviour (from code)

Area	What happens today	Source
Grade capture	Final QC Pass form offers A / B / C ("— No change —" keeps the old grade); only saved on Pass	`templates/cosmetic/final_qc.html`, `routers/cosmetic.py::advance_stage`
Grade enum	`DeviceGrade` = A, B, C, D, E, scrap	`models/device.py::DeviceGrade`
CRM grades	`GRADES = A, B, C, D, Mix, As-Is, Scrap`	`models/crm.py`
Cost on pass	Final QC Pass sets `device_price = base + consumed parts + changed parts`, once per tag	`routers/cosmetic.py::advance_stage` ("price finalized")
Ready to Sale min price (auto)	`unit price + other rate (CostConfig cosmetic_rate) + item margin (amount or %)`; unit price = `device_price` or lot buying price ÷ qty	`services/ready_pricing.py::tag_min_price`, `unit_price`, `load_ready_rates`
Manual price	Set price / Bulk set price write `Device.min_selling_price`, `max_selling_price`; audited `TAG_PRICE_SET` / `TAG_PRICE_BULK_SET`; role check `_role_can_set_price`	`routers/sales.py::set_ready_tag_price`, `bulk_set_ready_tag_price`
As-Is lot price	Lot total + as-is margin; stored overrides in `AsIsLotPrice`	`services/ready_pricing.py::as_is_min_price`, `routers/sales.py` As-Is routes
Sale guard	Below-cost warning only (does not block)	`routers/sales.py::create_sale` (`below_cost_warning_for`)
Grade Price Matrix	Table `crm_grade_price_matrix`: device_type, grade, material_type, brand, min/max buy price, `target_sell`, `min_margin_pct` (default 15). Used only by its own CRUD + CSV upload pages.	`models/crm.py::GradePriceMatrix`, `routers/crm_price_matrix.py`
Expected sale value	`expected_sale_value` is never populated, so the auto-scrap / scrap-warning rule never fires	audit Apr-29 P-4; `services/cost_engine.py`

5. User stories and acceptance criteria

US-01 — Suggested price from grade

As a Sales Executive, I want each Ready to Sale tag to show a Grade Price (min–max) next to the Cost Floor, so that I quote the right price quickly.

#	Given	When	Then
AC-01.1	Tag T: Laptop, grade B, a price row exists for (Laptop, B) with min ₹18,000 / max ₹21,000	I open `/sales/ready`	T shows Grade Price ₹18,000–21,000 and Cost Floor (today's `tag_min_price`)
AC-01.2	T has no grade, or no matching price row	I open the page	Grade Price shows "—" and a tooltip "No grade price for Laptop / "
AC-01.3	Grade min < Cost Floor	I open the page	Grade Price shows an amber warning icon "Below cost floor by ₹x"
AC-01.4	A more specific row exists (Laptop, B, make Dell, gen 8)	Price is resolved	The most specific match wins: make + generation > make > device type only

US-02 — Apply and override

As a Sales Executive, I want to apply grade prices to selected tags in one action and override any one with a reason.

#	Given	When	Then
AC-02.1	I select 40 graded tags	I click "Apply grade price"	`min_selling_price` / `max_selling_price` set from the grade row for each; tags with no row are skipped and listed in the result alert
AC-02.2	I set a manual price different from the grade price	I save	A reason is required; audit `TAG_PRICE_OVERRIDE` stores grade price, new price, reason
AC-02.3	A tag already has a manual price	Grade prices are re-applied	Manual price is kept unless I tick "Overwrite manual prices"

US-03 — Maintain the grade price list

As Managers/Admin, I want to maintain grade prices with effective dates and upload them by CSV.

#	Given	When	Then
AC-03.1	I add (Laptop, A, from 1 Jan 2027, min, max)	I save	Older row for (Laptop, A) is closed the day before; history kept
AC-03.2	I upload a CSV with a bad grade value	I submit	Count alerts: rows inserted vs rows with errors, with the row numbers

US-04 — Grading is consistent

As a Final QC Inspector, I want the grade list on the Final QC form to be the agreed sales grade scale, so that every grade has a price.

#	Given	When	Then
AC-04.1	The agreed scale is (example) A / B / C / D	I pass a tag	Only those values are offered; Grade is **required** on Pass

6. Functional requirements

ID	Requirement	Priority
FR-01	Grade price resolution service (server-side, `Decimal`): input device type (`sub_category`), grade, optional make, CPU generation; output min, max, source row id.	Must
FR-02	Ready to Sale Tag table: add Grade Price and Cost Floor columns and a below-floor warning.	Must
FR-03	"Apply grade price" bulk action and per-tag override with mandatory reason; audited.	Must
FR-04	Grade price list maintenance with effective dates and CSV upload (Managers/Admin).	Must
FR-05	One grade scale in Master Data used by Final QC form, Ready to Sale filters, CRM matrix. Grade required on Final QC Pass.	Must
FR-06	On Final QC Pass, store the resolved grade price as `expected_sale_value` (`models/engines.py::DeviceCosting`) so cost-engine scrap warnings can fire.	Should

ID	Requirement	Priority
FR-07	Report: margin by grade (sale price − device cost) per month.	Should
FR-08	As-Is lot suggested price from grade mix of its tags.	Could (phase 2)

7. Data and permission changes (Proposed)

Schema (dbdiagram approval required). Two options; recommendation is **Option A**.

Option	Change	Pros / cons
A	Extend `crm_grade_price_matrix` : add `min_sell` , `max_sell` , `cpu_generation` , `effective_from` , `effective_to`	Reuses an existing table and UI; CRM and refurbishment share one list
B	New `grade_sell_price` table	Clean separation; a second list to maintain

Also: `devices.grade_price_min` , `devices.grade_price_max` snapshot columns (nullable) written at Final QC pass and on Apply, so later list edits do not change history.

Audit actions: `GRADE_PRICE_APPLIED` (bulk), `TAG_PRICE_OVERRIDE` , `GRADE_PRICE_ROW_SAVED` .

Permissions: Maintain grade prices — Managers/Admin (the Price Matrix page permission in the Module Permission matrix; module key to confirm). Apply / override — roles that pass `_role_can_set_price` today. Pricing columns visible only where `can_view_pricing` is true.

8. UX notes

- Ready to Sale: two new Global Table columns after Unit Cost — "Grade Price" (`₹18,000–21,000`) and "Cost Floor". Below-floor uses `text-warning` icon with tooltip.
- Toolbar button "Apply grade price" next to "Bulk Set price"; result uses the multi-line `alert-info` (applied, skipped no-grade, skipped no-row, kept-manual).
- Override modal: price fields + required "Reason" textarea; errors per Design System §09.
- Grade badge colours: A success `#0F7A37` , B primary, C warning, D/E secondary.

9. Metrics

Metric	Baseline	Target	Source
Ready tags with a grade price	0%	100% of graded tags	`devices.grade_price_min`
Sales below grade min (count / month)	baseline: to be measured	to be agreed	Sale vs snapshot
Gross margin by grade	baseline: to be measured	to be agreed per grade	FR-07 report
Time from Ready to Sold (P6)	baseline: to be measured	−15% *(proposed)*	05 P6

10. Rollout

Phase	Scope	Gate
0	Agree grade scale (Open question 1); load first price list	Director + Sales lead sign
1	FR-01…05 (suggest only, no auto-apply)	UAT: SAL-01, FQC-01 + US-01…04
2	FR-06, FR-07; FR-08 As-Is	One month of override data reviewed

11. Open questions

#	Question	Owner
1	Which grade scale is the sales scale: A/B/C (Final QC form), A–E (`DeviceGrade`) or A/B/C/D/Mix (CRM)?	Final QC lead + Sales lead
2	Price key: device type + grade only, or also make, model, CPU generation, RAM / SSD?	Sales lead
3	Should Sales be able to sell below grade min without manager approval? (Today: warning only on below cost.)	Director
4	Who updates the list, and how often?	Sales Manager

PART D

PRD-04 · Invoice & E-way Bill from Sale

Field	Value
Status	Draft
Roadmap item	X4 phase 1 (Next, Q1 2027) and L2 phase 2 (Later, Q2 2027) — 02_Roadmap.md
Business owner	Accounts head (with Sales lead)
Personas	Sales Executive, Accounts, Managers/Admin, gate staff (Gate Pass)
Related	Project skill `server-side-financial-snapshot`; `docs/product-architecture/02-architecture/05_Integration_Specs.md` (planned in the folder README); Vision open question Q4 (e-invoice)

This PRD does not give tax advice. Every GST rule, HSN code and threshold named below must be confirmed by Accounts before build.

1. Problem

A sale in the app records tags, price, customer and an invoice number that someone types in or reads from an uploaded PDF. The app can print a tax invoice and a delivery challan, but **one per tag**: a 40-laptop sale prints 40 single-line invoices. The invoice number has no series or check for gaps. The e-way bill — needed to move goods above the statutory value — is captured only for goods coming **in** (lots, GRN, Parts GRN), never for goods going **out**. Accounts re-keys the sale into the billing tool to make the real invoice and the e-way bill, and the gate lets goods out without checking either.

2. Personas

Persona	Need
Sales Executive	Finish a sale and print one correct invoice and challan for the whole order
Accounts	A gap-free invoice series per company and FY, correct GST split, e-way bill recorded against the invoice
Gate staff	See that the invoice and (when needed) e-way bill exist before confirming exit
Managers/Admin	Sales list and P&L that match invoices

3. Goals and non-goals

Goals	Non-goals
One invoice document per sale event, with all tags as lines	Replacing the accounting software / GL

Goals	Non-goals
Server-generated invoice number series per selling company and FY	Credit-note document redesign (separate flow exists)
E-way bill number, date, vehicle, transporter captured on the invoice; printed on challan and gate pass	Inbound e-way bill changes (already captured on lots / GRN)
Warn (configurable: block) gate exit when e-way bill is required but missing	Phase 1 does not call any government API
Phase 2: generate e-way bill (and e-invoice IRN, if required) through a GST provider	

4. Current behaviour (from code)

Area	What happens today	Source
Sale creation	One form; `barcode` may be a comma list or `price_groups` JSON; **one `Sale` row per tag**; invoice number, customer, state, address, payment mode shared across the rows	`routers/sales.py::create_sale`
Sale number	From PostgreSQL sequence `sale_number_seq` (self-heals if missing)	`routers/sales.py::ensure_sale_number_seq`
Invoice number	Free text `Sale.invoice_no`, optional; can be auto-filled by parsing an uploaded PDF (`pdfplumber`, best effort). On sale, the tag's `Device.invoice_number` is replaced by the sale invoice number.	`routers/sales.py::parse_invoice_pdf`, `create_sale`
Company snapshot	Selling company name, GSTIN, address, state, state code, phone, email frozen onto each Sale	`models/sales.py::Sale` (`company_*`), `routers/invoices.py::_sale_company_snapshot`
Tax invoice print	`/invoices/print/{sale_id}` — one Sale = one device = one line. GST back-calculated from the inclusive sale price; intra-state (CGST + SGST) when customer state equals company state, else IGST. Rates from CostConfig `gst_rate_intra` / `gst_rate_inter`, default 18%. **Customer state defaults to "Delhi" when blank.** HSN printed as **8471 for every device** (TFT monitors included). Invoice no. shown = `invoice_no` or `sale_number`.	`routers/invoices.py::print_invoice`, `_compute_gst`, `_fetch_gst_rates`, `templates/invoices/print.html`
Delivery challan	`/invoices/waybill/{sale_id}` titled "Delivery Challan", one per Sale	`routers/invoices.py::print_waybill`, `templates/invoices/waybill.html`
Transport	Transport mode, courier, tracking no., dispatch date, delivery status on Sale	`routers/sales.py::sale_update_transport`
Gate pass	Exit confirm (single / bulk) sets `delivery_status = dispatched`, dispatch date; audited `GATE_EXIT_CONFIRMED`; no invoice / e-way bill check	`routers/gate_pass.py::confirm_exit`, `bulk-exit`
E-way bill (inbound only)	`Lot.e_way_bill`; GRN form field; `PartsGRN.eway_bill_number`, `eway_bill_date`, `eway_bill_file`	`models/lot.py`, `routers/grn.py`, `models/parts_grn.py`, `routers/parts_grn.py`
E-way bill (outbound)	**None.** No field on `sales`.	`models/sales.py`

Area	What happens today	Source
Accounts	Customer receipts (single / bulk CSV), supplier payments, PO / quote invoice upload	`routers/accounts.py`

5. User stories and acceptance criteria

US-01 — One invoice per sale

As a Sales Executive, I want one invoice for all tags sold together, so that the customer gets one correct document.

#	Given	When	Then
AC-01.1	I sell 3 laptops at ₹20,000 and 2 monitors at ₹6,000 in one submit	The sale saves	One invoice header is created with 5 lines (or 2 grouped lines by model / price — see Open question 3), totals computed server-side in `Decimal`
AC-01.2	The invoice exists	I open Print Invoice from any of the 5 Sale rows	The same single invoice opens
AC-01.3	Customer state is blank	I try to save	Save is refused: "Customer state is required for GST" (no silent "Delhi")

US-02 — Invoice number series

As Accounts, I want the app to issue invoice numbers from a series per selling company and FY, so that there are no gaps or duplicates.

#	Given	When	Then
AC-02.1	Series for "OxyPC Computers", FY 2026-27, prefix configured	A sale saves	Invoice number = next in series (e.g. `<prefix>/26-27/00042`), unique per company + FY
AC-02.2	Two sales save at the same moment	—	Different numbers, no gap (DB sequence / row lock per series)
AC-02.3	An invoice must be cancelled	Accounts cancels it with a reason	Number is kept as "Cancelled" (not reused); audit logged; linked tags handled by the Return / Credit Note flow
AC-02.4	Legacy flow (typed invoice number) is still enabled	Sales types a number	Allowed only when setting `invoice_numbering = manual` ; duplicates per company + FY refused

US-03 — Record the e-way bill (phase 1)

As Accounts, I want to record the e-way bill on the invoice, so that it prints on the challan and the gate can check it.

#	Given	When	Then
AC-03.1	Invoice value is above the configured threshold	I open the invoice	A banner says "E-way bill required" until number + date are saved
AC-03.2	I enter EWB number, date, vehicle number, transporter	I save	Values stored, audited `EWAY_BILL_RECORDED` , printed on Delivery Challan and Gate Pass

#	Given	When	Then
AC-03.3	Value is below the threshold	—	No banner; fields optional

US-04 — Gate checks before exit

As gate staff, I want the gate pass to show invoice and e-way bill status, so that goods do not leave without papers.

#	Given	When	Then
AC-04.1	E-way bill required but missing; setting `gate_requires_ewb = warn`	I confirm exit	Exit allowed with a warning; audit notes "exit without EWB"
AC-04.2	Same, setting `gate_requires_ewb = block`	I confirm exit	Refused: "E-way bill missing for invoice "
AC-04.3	Bulk exit of 10 sales, 2 missing EWB, block mode	I submit	8 exit, 2 listed as skipped in the result alert

US-05 — Generate e-way bill from the app (phase 2)

As Accounts, I want to generate the e-way bill from the invoice, so that I do not re-key it on the government portal.

#	Given	When	Then
AC-05.1	GST provider credentials configured	I click Generate EWB	Request built from invoice data; returned number / date / validity stored; failure message shown verbatim
AC-05.2	E-invoice (IRN) is required for the company (Accounts decision)	Invoice is finalised	IRN and QR obtained first, then EWB

6. Functional requirements

ID	Requirement	Priority
FR-01	Create a sale invoice header for each sale submit; link every Sale row of that submit to it.	Must
FR-02	Print invoice from the header: all lines, per-line HSN by category (table in Master Data), taxable value, CGST / SGST / IGST, totals; company and customer snapshot (reuse `_sale_company_snapshot`).	Must
FR-03	Server-generated invoice number per (company, FY) series; prefix and start number set by Admin; cancellation keeps the number.	Must
FR-04	Customer state (and GSTIN for B2B) required; remove the "Delhi" default in `_compute_gst`.	Must
FR-05	E-way bill fields on the invoice header (number, date, valid-until, vehicle no., transporter, transporter ID, distance) with threshold rule in `AppSetting`.	Must
FR-06	Delivery Challan and Gate Pass print from the header and show EWB.	Must
FR-07	Gate exit check with setting off / warn / block.	Should

ID	Requirement	Priority
FR-08	Backfill: group existing Sale rows by (`invoice_no` , `company_id` , `sold_at` date, customer) into headers; leave ungroupable rows as single-line headers.	Should
FR-09	Sales list and exports show invoice header number and EWB status.	Should
FR-10	Phase 2: GST provider integration for EWB (and IRN if required), with retry and error log.	Could (phase 2)

7. Data and permission changes (Proposed)

Schema (dbdiagram approval required; follow `server-side-financial-snapshot` — totals stored as columns, computed in `Decimal`):

Table	Columns	Notes
`sale_invoices` (new)	`id` , `invoice_number` (unique per company + FY), `company_id` + snapshot columns, `fy` , `invoice_date` , customer snapshot (name, GSTIN, state, state code, address), `taxable_total` , `cgst` , `sgst` , `igst` , `grand_total` , `status` (issued / cancelled), `cancel_reason` , `ewb_number` , `ewb_date` , `ewb_valid_until` , `vehicle_no` , `transporter_name` , `transporter_id` , `irn` (nullable), `created_by` , `created_at`	Header
`sales`	add `sale_invoice_id` FK (nullable), `hsn_code` , `taxable_value` , `tax_rate`	Each Sale row becomes an invoice line
`invoice_series` (new)	`company_id` , `fy` , `prefix` , `next_number`	Row-locked on issue
Master Data	`hsn_by_category` (Laptop, Desktop, TFT Monitor, ...)	Values confirmed by Accounts
`AppSetting`	`invoice_numbering` (auto / manual), `ewb_threshold_amount` , `gate_requires_ewb` (off / warn / block)	

Audit actions: `INVOICE_ISSUED` , `INVOICE_CANCELLED` , `EWAY_BILL_RECORDED` , `EWAY_BILL_GENERATED` , `GATE_EXIT_WITHOUT_EWB` .

Permissions: Issue invoice — whoever can create a sale today (`require_module_perm("sales", "add")`). Cancel invoice, edit series, record / generate EWB — Accounts (custom role via matrix) and Admin. Gate exit unchanged.

8. UX notes

- After Save Sale: success alert (`#0F7A37`) "Sale saved · Invoice · 5 tags" with buttons Print Invoice, Print Challan.
- Sale Detail: an "Invoice" card (Global Table style) listing lines; EWB status badge: Not needed (secondary) / Required (warning) / Recorded (success).
- EWB modal: fields in Design System form layout; errors keep values (§09).
- Gate Pass Global Table: new columns Invoice No. and EWB; blocked rows show a red badge and are excluded from Bulk Exit with the standard multi-line result alert.
- Print templates stay A4, one invoice may run to several pages; repeat header on each page.

9. Metrics

Metric	Baseline	Target	Source
Sales with an invoice number	baseline: to be measured (`sales.invoice_no` null rate)	100%	`sale_invoices`
Invoices per multi-tag sale	n tags = n invoices today	1	`sale_invoices`
Outbound consignments above threshold with EWB	0% recorded in app	100%	`sale_invoices.ewb_number`
Gate exits without EWB (warn mode)	baseline: to be measured	0 after 1 month	audit `GATE_EXIT_WITHOUT_EWB`
Accounts re-keying time per sale	baseline: to be measured (time study)	−70% *(proposed)*	time study

10. Rollout

Phase	Scope	Gate
1a (Q1 2027)	FR-01…04, FR-08 backfill; `invoice_numbering = manual` first, then switch to auto at FY or month start	Accounts reconciles one month of invoices vs billing tool on staging
1b (Q1 2027)	FR-05…07, FR-09; gate in `warn` mode	UAT ACC-03, SAL-06 + US-03/04
2 (Q2 2027)	FR-10 GST provider integration; gate `block` mode	Provider contract; Accounts sign-off

11. Open questions

#	Question	Owner
1	Is e-invoice (IRN) mandatory for each selling company / entity (the dashboard entity list shows OxyPC Computers, Renew Circuits, Deshwal)?	Accounts
2	E-way bill threshold and rules for intra-state movement — confirm value and any state-specific rule.	Accounts
3	Invoice lines: one per tag (serial on line) or grouped by model and price with tag list annexure?	Accounts + Sales lead
4	HSN per category — the current template prints 8471 for all devices; confirm correct codes for monitors and others.	Accounts
5	Which GST provider / GSP, and who holds credentials?	Accounts
6	Do replacement sales (₹0, "Replaced with …") need a delivery challan only, not an invoice?	Accounts

PART E

PRD Template

Field	Value
Status	Draft / In review / Approved / Built / Shipped
Roadmap item	02_Roadmap.md>
Business owner	
Personas	
Related	/qa/requirements)>

Rule: describe **current behaviour only from code** and cite `file::function`. Mark everything new as **Proposed**.

1. Problem

What hurts, for whom, how often. One short paragraph and the evidence (screens, counts, code).

2. Personas

Persona	What they need from this feature

3. Goals and non-goals

Goals	Non-goals

4. Current behaviour (from code)

Area	What happens today	Source
		`path/file.py::function`

5. User stories and acceptance criteria

US-01 —